

OFFICIAL REFERENCE MANUAL

ModelScript Engineering Toolchain

ModelScript

The Polyglot Cyber-Physical Modeling, Simulation & Formal Verification Environment

Architecture, Languages & Algorithmic Foundations

Edition & Target
2026 Comprehensive Edition
WebAssembly (WASM) & Node.js ≥ 24

Open Source License
GNU AGPL v3.0
github.com/modelscript/modelscript

Documentation Portal
docs.modelscript.org
Compiler & Engine Core

Table of Contents

Part I: Getting Started & User Guide

1. Introduction
2. Installation & Requirements
3. Getting Started
4. Embedding & React SDK
5. Platform Administration

Part II: Compiler & Pipeline Architecture

6. Compiler Pipeline Overview
7. WASM GLR Native Parser
8. Salsa Query Engine
9. Linear DAE Arena
10. Triple Graph Grammars
11. Nelson-Oppen Coordinator
12. Simulation Solvers
13. SIL & Real-Time HIL Runtime

Part III: Polyglot Languages & Formats

14. Language Support Overview
15. ModelScript (.msx) Native Language
16. Modelica Physical Modeling
17. SysML v2 & KerML
18. MATLAB Simulink
19. AUTOSAR Architecture
20. ReqIF Requirements
21. SPICE Electronics
22. SIMPACK Multibody Dynamics
23. GENSYS Railway Dynamics
24. MSC Adams MBD
25. URDF & Xacro Robotics
26. PSCAD Power Systems
27. PSS/E Power Flow (.raw)
28. OLGA Multiphase Flow
29. PipelineStudio (.lps)
30. EASY5 Simulation
31. GT-SUITE / GT-POWER
32. SIMA / SIMO / RIFLEX
33. NASA ROCETS Propulsion
34. STEP CAD (ISO 10303)
35. OWL2 Ontologies
36. OpenSCAD CSG Geometry
37. SSP System Packaging
38. CFD (SU2 & OpenFOAM)
39. FEA Structural Analysis
40. CSV Telemetry
41. CCSDS Space Telemetry
42. EnergyPlus Building Thermal
43. EPANET Water Networks
44. OpenDRIVE Road Networks
45. PLCopen Industrial Automation
46. Verilog-A Compact Modeling
47. Fortran Legacy Integration

Part IV: Comprehensive Algorithms Catalog

48. Algorithms Overview & Map
49. Pantelides Index Reduction
50. Tarjan BLT Partitioning
51. Cellier-Elmqvist Tearing
52. Acausal Connector Balancing
53. Automatic Differentiation
54. Numerical ODE/DAE Solvers
55. Optimization Algorithms
56. Nelson-Oppen & SMT Solvers
57. CHC & IC3 Verification
58. Abstract Interpretation Domains
59. Graph Rewriting & E-Graphs
60. Linear Memory Structures
61. Scientific Computing & Surrogates
62. SDP, BVP & Geometric CSG
63. Diagram Layout & Routing

Part V: Command-Line Interface (msx)

64. CLI Overview & Flags
65. Command Reference

Part VI: Reference & Diagnostics

66. Compiler Linter Rules
67. Standards & Export Formats
68. MCP Server & IDE Workbench

PART I

Getting Started & User Guide

CHAPTER 1

Introduction

Welcome to **ModelScript**, the open-source **computable digital thread** platform and polyglot engineering compiler designed for multi-domain physical modeling, formal verification, and AI-assisted systems engineering.

What is the Computable Digital Thread?

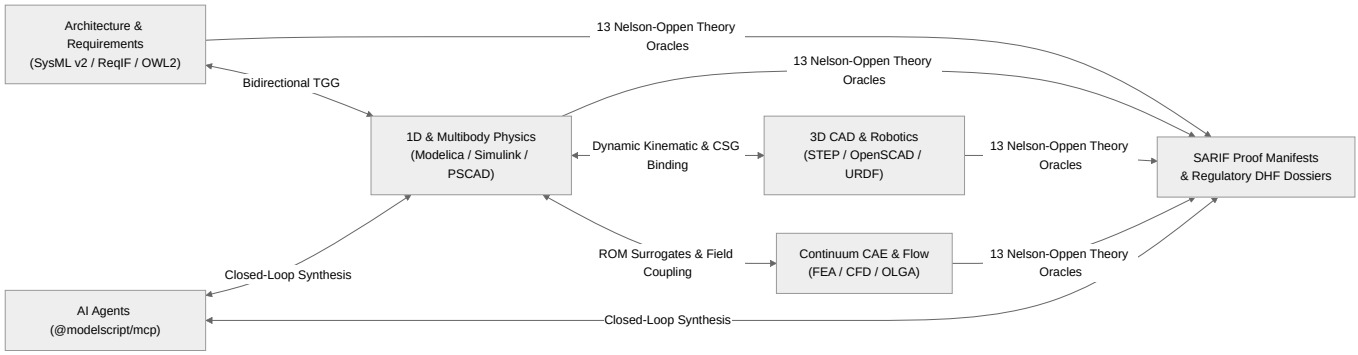
Modern cyber-physical systems — from electric vehicles, utility power grids, and satellites to surgical robots and subsea flowlines — require tight coordination across **44 engineering languages and file formats** spanning **12 industry verticals**:

- **System Architecture, Requirements & Ontologies** (`SysML v2` , `KerML` , `ReqIF` , `OSLC 3.0` , `OWL2` , `AAS / DDP`)
- **Multi-Domain Acausal & Block-Diagram Simulation** (`Modelica 3.x` , `ModelScript .msx` , `MATLAB Simulink` , `FMI 2.0/3.0 FMU` , `SSP 1.0/2.0`)
- **Power Grids, Electronics & Embedded Control** (`PSCAD EMT` , `Siemens PSS/E` , `SPICE` , `IBIS` , `Touchstone` , `Gerber` , `PLCopen IEC 61131-3` , `AUTOSAR ARXML` , `CAN DBC` , `ANSI C`)
- **3D CAD Geometry, Multibody & Robotics** (`STEP AP214/AP242` , `OpenSCAD CSG` , `ROS URDF / Xacro` , `SIMPACK` , `MSC Adams` , `GENSYS`)
- **Continuum CAE, Process, Thermal & Fluid Networks** (`CalculiX / Abaqus FEA` , `SU2 / OpenFOAM CFD` , `Gmsh` , `VTK` , `OLGA` , `PipelineStudio` , `Aspen Plus` , `EnergyPlus` , `EPANET` , `EPA SWMM` , `SBML` , `THERM` , `RADIANCE`)

Traditionally, these disciplines operate in disconnected silos. Enterprise "digital threads" have historically been **passive and descriptive** — composed of static document links, heavyweight PLM databases, and manual spreadsheets that drift out of sync the moment a parameter changes.

ModelScript makes the digital thread *computable*.

Instead of passive hyperlinks, ModelScript treats every engineering artifact as **executable, Git-versioned source code** compiled into a shared WebAssembly representation and continuously checked by graph rewriting engines and mathematical solvers:



Traditional Passive PLM vs. Computable Digital Thread

Dimension	Traditional Passive PLM / MBSE	ModelScript Computable Digital Thread
Artifact Storage	Proprietary binary blobs in centralized SQL/PLM vaults	Plain-text, human-readable source files versioned in Git
Cross-Domain Links	Manual, untyped hyperlinks that silently go stale	Typed `@satisfies` / `@verifies` bindings & bidirectional Triple Graph Grammars (TGG)
Static Analysis	Checked manually during design reviews	**790 static lint rules** across **43 languages** evaluated on every keystroke and CI push
Requirement Verification	Post-hoc spreadsheet sign-offs	Automated $\mathrm{CDCL}(\mathrm{T})$ SMT, IC3/PDR, and Taylor-model reachability proofs
Code Review & Diffs	Opaque binary attachments or screenshot exports	Structural AST/DAE diffs and visual schematic PR diffs (`msx pr-diff`)
AI Copilot Integration	Unverified text generation prone to physical hallucinations	Closed-loop MCP server (`@modelscript/mcp`) grounded in DAE solvers and formal counterexamples

---

Core Pillars of ModelScript

1. Code-First Engineering Across 44 Languages

Every model, schematic, requirement, netlist, and mesh configuration lives as clean text in Git. Engineers diff, review, merge, and gate physical hardware designs using standard pull requests, SARIF code scanning, and reproducible CI/CD pipelines.

2. Zero-Allocation WebAssembly CST & DAE Arenas

Each language grammar compiles ahead-of-time via `@modelscript/dsl` (`buildParser`) into a native WebAssembly GLR parser (`parser.wasm`). Concrete syntax trees and flattened Differential-Algebraic Equations (`DAEBuilder`) reside in Struct-of-Arrays (SoA) linear memory with zero JavaScript object wrapper overhead, delivering consistent compiler performance across the CLI, VS Code, and browser WebWorkers.

3. Bidirectional Triple Graph Grammars (TGG) & Critical Pair Analysis

Keep system architecture, electronics, CAD, and physics in permanent synchronization. Declarative Double-Pushout (DPO) Triple Graph Grammar rules propagate architectural changes (`SysML v2`) forward into physical simulation models (`Modelica` , `SPICE` , `STEP`), while back-annotating numerical sizing results back into system requirements — verified for confluence via static Critical Pair Analysis (CPA).

4. Nelson-Oppen Multi-Theory Formal Verification (13 Theory Oracles)

Cross-domain verification is orchestrated by a $\mathrm{CDCL}(\mathrm{T})$ Nelson-Oppen coordinator (`@modelscript/runtime`) that exchanges equality facts, interval contractors, and conflict clauses across **13 specialized domain theory oracles**:

1. **Ontology Theory (`OntologyTheoryOracle`)** — OWL2 Description Logic ($\mathcal{SROIQ}$) subsumption and disjointness
2. **Interval Constraint Theory (`ConstraintTheoryOracle`)** — SysML v2 / KerML HC4 revise and δ-complete contractors
3. **Abstract Interpretation (`AbstractDomainOracle`)** — Relational Zone (Difference-Bound Matrix) and Octagon invariants
4. **Continuous Safety (`ContinuousSafetyOracle`)** — Sum-of-Squares (SOS) barrier certificates and Picard Taylor-model flowpipes
5. **Dynamic Simulation (`DynamicSimulationOracle`)** — DAE trajectory evaluation and Signal Temporal Logic (STL) robustness $\rho(\phi, \mathbf{x}, t)$
6. **3D Spatial Physics (`SpatialPhysicsOracle`)** — OpenCASCADE B-Rep minimum clearance, collision, and mass-property bounds
7. **CAE Contracts (`CaeContractOracle`)** — Structural FEA von Mises yield margins and CFD aerodynamic envelope contracts
8. **Causalization & Structural Matching (`CausalizationOracle`)** — Hopcroft-Karp matching, Pantelides DAE index, and algebraic loops
9. **Dimensional Analysis (`DimensionalOracle`)** — 7-base SI dimension exponent vector algebra ($\mathbb{Q}^7$) and affine unit conversions
10. **Assume-Guarantee Refinement (`EntailmentOracle`)** — Compositional contract refinement ($A_2 \subseteq A_1 \wedge G_1 \subseteq G_2$)
11. **Flow Conservation (`FlowAlgebraOracle`)** — Kirchhoff and fluid stream conservation ($\sum f_i = 0$) across connector graphs
12. **GD&T Tolerance Stack-Up (`ToleranceStackOracle`)** — Worst-case and RSS geometric tolerance stack-up from STEP AP242 PMI
13. **Uncertainty Quantification (`UqOracle`)** — Split conformal prediction intervals and First-Order Reliability Method (FORM) β indices

5. Unified Multi-Language LSP & Incrementally Memoized Salsa Engine

A single background worker pool (`@modelscript/lsp`) indexes polyglot workspaces through an incremental Salsa query engine (`fetch → deep_verify → execute → backdate`). Engineers get cross-language go-to-definition, hover equations, unit checking, schematic rendering, and suspect-link diagnostics in VS Code, the browser IDE, and the native Tauri desktop studio.

6. AI-Native Closed-Loop Engineering (`@modelscript/mcp`)

Large language models struggle with physical engineering when left to guess differential equations in free text. `@modelscript/mcp` exposes **31 deterministic compiler, simulation, CAD, TGG, and formal verification tools** over the Model Context Protocol (MCP). AI agents iteratively draft code, run `modelica_lint` , integrate stiff DAEs with `modelica_simulate` , falsify requirements, and calibrate parameters via `modelica_diff_calibrate` until every formal proof passes.

Ready to Get Started?

- **Installation** — Set up the zero-install Browser Playground, VS Code extension, `msx` CLI, or `@modelscript/mcp` AI agent server.
- **Getting Started Tutorial** — Build, verify, bind to 3D CAD, simulate, and export your first end-to-end computable digital thread.
- **AI Agents & MCP Workflows** — Connect Claude Code, Cursor, or custom LLM orchestrators to the deterministic compiler and solver loop.
- **CI/CD & GitHub Actions** — Gate pull requests with polyglot linting, SARIF formal verification reports, visual schematic diffs, and FDA/ISO compliance dossiers.

CHAPTER 2

Installation

ModelScript runs from the same WebAssembly compiler core across browsers, desktop IDEs, headless CI runners, and AI agent hosts. Choose the surface that matches your workflow.

For Users & Engineers

1. Zero-Install Browser Playground & Web IDE

To explore ModelScript without installing anything locally:

- **Interactive Playground (`/playground`)**: Try curated multi-domain examples, inspect live GLR syntax trees, run stiff DAE simulations, and view rendered schematics directly in your browser tab.
- **Full Web Studio (`https://ide.modelscript.org`)**: Multi-file workspace explorer, dockable 2D schematic and 3D OpenCASCADE CAD viewports, requirements traceability matrix, and in-browser WebWorker compilation.

2. Command-Line Interface (`msx`)

The `@modelscript/cli` package installs both the `modelscript` binary and its shorthand alias `msx` for local development, simulation sweeps, formal verification, and CI/CD automation (requires **Node.js ≥ 22**):

```
# Run any command via npx without global installation
npx @modelscript/cli --help

# Or install globally for system-wide access
npm install -g @modelscript/cli
msx --version
```

Accelerating Repeated CLI Runs with `--daemon`

When running frequent interactive checks or pre-commit hooks, start the persistent background compiler daemon so `msx` reuses warm WebAssembly parser instances and incremental Salsa caches over a local Unix socket (`~/.modelscript/daemon.sock`):

```
# Start the background daemon and inspect its status
msx daemon start
msx daemon status

# Pass --daemon to any command to reuse warm workspace caches
msx lint ./models --daemon
```

3. VS Code Extension

The official extension turns VS Code or Cursor into a complete multi-domain modeling studio:

1. Open VS Code and open the Extensions view (`Ctrl+Shift+X` or `Cmd+Shift+X`).
2. Search for **`ModelScript`** (`modelscript.modelscript-vscode`) and click **Install**.

The extension bundles the native WebAssembly Language Server (`@modelscript/lsp`), real-time diagnostics across all 44 supported formats, interactive schematic and Bode plot panels, the 3D WebGL CAD viewer, and the DAP time-travel simulation debugger.

4. AI Coding Assistants & MCP Server (`@modelscript/mcp`)

Connect AI agents (Claude Code, Claude Desktop, Cursor, Windsurf, or custom orchestrators) to ModelScript's **31 deterministic engineering tools** over the Model Context Protocol:

```
# Register @modelscript/mcp in Claude Code with a single command
claude mcp add modelscript -- npx -y @modelscript/mcp
```

For **Cursor** (`.cursor/mcp.json`), **Claude Desktop**, or **VS Code**, add the `stdio` server entry:

```
{
  "mcpServers": {
    "modelscript": {
      "command": "npx",
      "args": ["-y", "@modelscript/mcp"]
    }
  }
}
```

See the AI Agents & MCP Workflows Guide for closed-loop synthesis recipes and remote SSE gateway configuration.

5. Native Desktop Studio (Tauri v2)

For air-gapped networks, classified labs, or hardware-in-the-loop (HIL) benches, download the standalone native desktop application (`@modelscript/desktop`) from GitHub Releases:

- **Linux**: Standalone `.AppImage` or Debian/Ubuntu `.deb` packages.
- **macOS**: Universal Apple Silicon / Intel `.dmg` disk image.
- **Windows**: 64-bit `.msi` installer or portable `.exe` .

The desktop studio embeds offline WebAssembly GLR parsers, local SQLite artifact caching, and native serial/UDP bridges for real-time HIL testing.

6. Containerized Enterprise Hub & Simulation Server

Deploy the self-hosted ModelScript Hub, package registry, REST/GraphQL/SPARQL APIs, and multi-tenant MCP gateway via Docker:

```
# Pull official multi-architecture containers
docker pull ghcr.io/modelscript/api:latest
docker pull ghcr.io/modelscript/web:latest

# Launch backend simulation cluster and web IDE
docker compose up -d
```

See the Platform Administration Guide for Kubernetes Helm charts, Slurm HPC worker provisioning, and PostgreSQL audit-ledger setup.

For Contributors & Compiler Developers

To build and hack on the ModelScript monorepo locally, install **Node.js ≥ 24** (matching `.nvmrc`) and `npm` .

1. Clone & Install

```
git clone https://github.com/modelscript/modelscript.git
cd modelscript
npm install
```

2. Build Core Compiler & Language Runtimes

ModelScript compiles its native WebAssembly GLR parsers (`parser.wasm`), AssemblyScript flattening kernels, and TypeScript packages via Nx:

```
npm run build:libs    # Build core compiler packages and language runtimes
npm run build         # Build all monorepo packages and applications
```

3. Launch Local Development Servers

```
npm run dev           # Start API backend, LSP workers, and Web IDE
```

Next Steps

- Follow the **Getting Started Tutorial** to build, verify, simulate, and export your first connected multi-domain model.
- Explore **CLI Overview** for global flags, environment variables, and all 41 `msx` commands.

CHAPTER 3

Getting Started

This tutorial walks you through building a complete **computable digital thread** in ModelScript across five steps:

1. Authoring and linting a **Modelica** physical dynamics model (`BouncingBall.mo`)
2. Linking **SysML v2** safety requirements (`BouncingBallReq.sysml`) and running **formal verification** (`msx verify`)
3. Binding physical state trajectories to **3D CAD geometry** (`msx csg`)
4. Running **time-domain simulation**, rendering **SVG schematics**, and exporting an **FMI 3.0 Co-Simulation FMU** (`msx simulate` , `msx render` , `msx fmu`)
5. Closing the loop with an **AI Agent** via `@modelscript/mcp`

Step 1: Create a Workspace & Physics Model

Create a new directory and initialize a `modelscript.json` workspace manifest:

```
mkdir bouncing-ball-thread && cd bouncing-ball-thread
msx init -y
```

Create `BouncingBall.mo` with a hybrid continuous-discrete impact model:

```
model BouncingBall
  parameter Real e = 0.7 "Coefficient of restitution (0 < e < 1)";
  parameter Real g = 9.81 "Gravitational acceleration [m/s2]";
  parameter Real radius = 0.05 "Ball radius [m]";
  Real h(start = 1.0, fixed = true) "Center height above ground [m]";
  Real v(start = 0.0, fixed = true) "Vertical velocity [m/s]";
equation
  der(h) = v;
  der(v) = -g;
  when h <= radius and v < 0 then
    reinit(v, -e * pre(v));
  end when;
end BouncingBall;
```

Run the static analyzer to verify syntax, types, and equation balance (`2 variables, 2 equations`):

```
msx lint BouncingBall.mo
```

Step 2: Add SysML v2 Requirements & Formal Verification

Now create `BouncingBallReq.sysml` next to `BouncingBall.mo` to capture architectural requirements and bind them to the physics model:

```
package BouncingBallArchitecture {
  attribute def Meter;
  attribute def RestitutionCoeff;

  requirement def <'REQ-ENERGY-01'> PassiveReboundRequirement {
    doc /* The coefficient of restitution must remain strictly passive (0 < e <= 1)
         * so rebound height never exceeds initial drop height. */
    attribute e : RestitutionCoeff;
    require constraint { e > 0.0 and e <= 1.0 }
  }

  requirement def <'REQ-CLEARANCE-02'> PeakAltitudeBound {
    doc /* Peak rebound altitude after the first impact shall not exceed 0.60 m. */
    attribute hMax : Meter;
    require constraint { hMax <= 0.60 }
  }

  part def BouncingBallSystem {
    attribute e : RestitutionCoeff = 0.7;
    attribute hMax : Meter = 0.49;
    satisfy PassiveReboundRequirement by self;
    satisfy PeakAltitudeBound by self;
  }
}
```

Lint both files together and run the unified formal verifier to check all requirements and emit a **SARIF 2.1.0** report:

```
# Lint the polyglot workspace (Modelica + SysML v2)
msx lint BouncingBall.mo BouncingBallReq.sysml

# Formally verify requirements and contracts, emitting a SARIF proof report
msx verify BouncingBall BouncingBall.mo --format sarif --report results.sarif
```

Step 3: Bind 3D CAD Geometry to Physical States

In ModelScript, physical states can directly drive 3D Constructive Solid Geometry (CSG) or external STEP/glTF assets. Add a `CAD` annotation to `h` in `BouncingBall.mo`:

```
model BouncingBall
  parameter Real e = 0.7 "Coefficient of restitution (0 < e < 1)";
  parameter Real g = 9.81 "Gravitational acceleration [m/s2]";
  parameter Real radius = 0.05 "Ball radius [m]";
  Real h(start = 1.0, fixed = true) "Center height above ground [m]" annotation(
    CAD(
      primitive = "sphere",
      radius = "radius",
      position = {0, 0, 0},
      dynamicPosition = "{0, h, 0}"
    )
  );
  Real v(start = 0.0, fixed = true) "Vertical velocity [m/s]";
equation
  der(h) = v;
  der(v) = -g;
  when h <= radius and v < 0 then
    reinit(v, -e * pre(v));
  end when;
end BouncingBall;
```

Extract and compile the 3D CSG topology from the command line, or open **ModelScript: Open 3D CAD Viewer** (`Ctrl+Shift+P`) in VS Code to watch the sphere rebound in 3D:

```
msx csg BouncingBall BouncingBall.mo
```

Step 4: Simulate, Render Schematics & Export an FMI 3.0 FMU

Simulate the hybrid DAE over $3.0\ \mathrm{s}$, render an SVG schematic diagram, and package the model into a standards-compliant **FMI 3.0 Co-Simulation FMU** with an embedded WebAssembly binary:

```
# Flatten to the linear-memory DAE arena and simulate for 3.0 s
msx simulate BouncingBall BouncingBall.mo --stop-time 3.0 --interval 0.005 --format csv > trajectory.csv

# Render a publication-ready SVG schematic diagram
msx render BouncingBall BouncingBall.mo -o schematic.svg

# Export an FMI 3.0 Co-Simulation FMU with embedded C and WebAssembly binaries
msx fmu BouncingBall BouncingBall.mo --fmi-version 3 --type cs --wasm --output BouncingBall.fmu
```

Step 5: Closed-Loop Calibration with an AI Agent (`@modelscript/mcp`)

Suppose experimental drop-test telemetry shows a lower rebound height than `e = 0.7` predicts. With `@modelscript/mcp` connected to your AI assistant (see AI Agents & MCP Workflows), the agent can invoke **`modelica_diff_calibrate`** or **`modelscript_closed_loop_synthesize`** to automatically fit `e` against your CSV data and re-verify `BouncingBallReq.sysml`:

```
{
  "tool": "modelica_diff_calibrate",
  "arguments": {
    "modelName": "BouncingBall",
    "sourceCode": "model BouncingBall ... end BouncingBall;",
    "parameters": [{ "name": "e", "initial": 0.7, "min": 0.1, "max": 0.99 }],
    "optimizer": "lbfgs",
    "maxIterations": 40
  }
}
```

Next Steps

- **Polyglot Languages Overview** — Explore all 44 supported engineering languages and cross-domain projections.
- **AI Agents & MCP Workflows** — Automate closed-loop synthesis, TGG reconciliation, and RTM auditing with AI agents.
- **CI/CD & GitHub Actions** — Gate pull requests with `msx lint`, `msx verify --format sarif`, `msx pr-diff`, and `msx dhf export`.
- **Triple Graph Grammar (TGG) Engine** — Learn how bidirectional synchronization keeps SysML v2 and Modelica aligned.

CHAPTER 4

Embedding Artifacts & Models

The **ModelScript Embed Service** allows you to embed interactive 2D schematics, 3D CAD/STEP models, simulation trajectory charts, and editable playgrounds directly into external websites, blogs, Notion, and documentation portals.

---

1. Quick Start: Web Component (`<modelscript-viewer>`)

The simplest and cleanest way to embed a ModelScript model in any modern web page or documentation framework is using the custom Web Component:

```
<script type="module" src="https://cdn.modelscript.org/embed/v1/viewer.js"></script>

<modelscript-viewer
  dialect="modelica"
  view="diagram"
  anchor="R1"
  code="model RLC
  Modelica.Electrical.Analog.Basic.Resistor R1(R=100);
  Modelica.Electrical.Analog.Basic.Capacitor C1(C=1e-6);
equation
  connect(R1.p, C1.n);
end RLC;"
>
</modelscript-viewer>
```

Supported Attributes

Attribute	Default	Description
`dialect`	`"modelica"`	Language dialect: `"modelica"`, `"sysml2"`, `"step"`, `"scad"`, `"pscad"`, `"olga"`, `"csv"`.
`view`	`"diagram"`	Initial visual mode: `"diagram"`, `"cad"`, `"plot"`, `"code"`, `"split-columns"`.
`anchor`	`undefined`	Target component element ID to visually highlight and auto-center (e.g. `"R1"` or `"chassis"`).
`controls`	`"none"`	Set to `"parameters"` to display an interactive live parameter slider ribbon below the model.
`zoom-mode`	`"ctrl-wheel"`	Scroll protection mode: `"ctrl-wheel"` (requires Ctrl/⌘ to zoom, preventing iframe page wheel-trapping), `"always"`, or `"click-to-interact"`.
`multi-file`	`false`	If set, the watermark pill routes the user to the **ModelScript IDE** instead of the Playground.
`artifact-id`	`undefined`	Numerical ID of a published artifact on the ModelScript Hub.

---

2. Zero-Config `<iframe>` Embed

For Markdown documents, Notion, or environments where custom scripts are restricted, use an `<iframe>` with a compressed `#m=...` permalink hash:

```
<iframe
  src="https://embed.modelscript.org/?view=diagram&theme=dark#m=<compressed-token>"
  width="100%"
  height="500"
  frameborder="0"
  allow="clipboard-write"
>
</iframe>
```

---

3. Programmatic JavaScript SDK (`@modelscript/embed`)

For full programmatic control in React, Vue, Svelte, or vanilla TypeScript applications, install the client package:

```
npm install @modelscript/embed
```

```
import { ModelScriptEmbed } from "@modelscript/embed";

const embed = new ModelScriptEmbed({
  container: "#viewer-container",
  dialect: "modelica",
  view: "diagram",
  anchor: { elementId: "R1" },
  code: `model Example ... end Example;`,
});

// React to user interactions inside the embed
embed.on("ANCHOR_CLICKED", ({ anchor }) => {
  console.log("User focused element:", anchor.elementId);
});

// Update model code or trigger simulation dynamically
embed.setCode(`model Updated ... end Updated;`);
embed.triggerSimulate({ stopTime: 10.0 });
```

---

4. React Component SDK (`@modelscript/react`)

For native React applications, `@modelscript/react` provides first-class components and hooks for engineering inspection:

```
npm install @modelscript/react
```

```
import { BlackboxInspector, StepResponseTestbench, useCWasmCompiler } from "@modelscript/react";

export function SimulationDashboard({ fmuBuffer }: { fmuBuffer: ArrayBuffer }) {
  const { compile, isCompiling } = useCWasmCompiler();

  return (
    <div className="dashboard-grid">
      {/* Live inspection of FMU variables, inputs, and outputs */}
      <BlackboxInspector
        buffer={fmuBuffer}
        onVariableChange={(name, val) => console.log(`Param ${name} updated to ${val}`)}
      />

      {/* Dynamic step response testbench with automatic settling time & overshoot metrics */}
      <StepResponseTestbench
        buffer={fmuBuffer}
        inputVariable="voltage_in"
        outputVariable="speed_out"
        stepMagnitude={10.0}
        stopTime={5.0}
      />
    </div>
  );
}
```

---

5. Accelerated Rehydration (`IndexedDB`)

When loading models that depend on standard libraries (such as the Modelica Standard Library or KerML), `@modelscript/embed` utilizes pre-baked binary symbol tables (`.sym.bin`):

```
import { PackageRehydrationCache } from "@modelscript/embed";

// Retrieves from local IndexedDB in <15ms or streams from CDN on cache miss
const symbolBuffer = await PackageRehydrationCache.loadOrFetch("Modelica", "4.0.0");
```

This bypasses parsing thousands of library classes on every page load, which substantially reduces startup time.

---

6. Live Parameter Slider Ribbon

Embeds can be made dynamically interactive without opening full code editors. Pass `controls="parameters"` or append `?controls=parameters` to any embed URL:

```
<modelscript-viewer
  dialect="modelica"
  view="plot"
  controls="parameters"
  code="model RC
  parameter Real R = 100 \"Resistance\";
  parameter Real C = 1e-4 \"Capacitance\";
  Real v(start=5.0);
equation
  der(v) = -v / (R * C);
end RC;">
</modelscript-viewer>
```

The viewer automatically extracts top-level parameters from the model code, renders sleek glassmorphic range sliders with real-time numeric inputs, and re-executes simulations upon user adjustments with built-in 300ms debouncing.

---

7. Universal oEmbed API & Rich Social Cards

ModelScript embeds implement the standard oEmbed specification (`GET /api/v1/oembed`), enabling drop-in interactive previews in Notion, Discord, Slack, and WordPress.

Auto-Discovery

Every ModelScript post (`https://modelscript.org/p/:id`) includes standard auto-discovery `<link>` tags in the HTML header:

```
<link
  rel="alternate"
  type="application/json+oembed"
  href="https://api.modelscript.org/api/v1/oembed?url=https%3A%2F%2Fmodelscript.org%2Fp%2F123&format=json"
  title="ModelScript Interactive Model"
/>
```

Direct oEmbed Query

Clients can query the oEmbed endpoint directly using either a post URL or a playground permalink hash (`#m=...`):

```
curl "https://api.modelscript.org/api/v1/oembed?url=https%3A%2F%2Fmodelscript.org%2F%23m%3DeJy...&maxwidth=800&format=json"
```

Response:

```
{
  "version": "1.0",
  "type": "rich",
  "provider_name": "ModelScript",
  "provider_url": "https://modelscript.org",
  "title": "Interactive Model Simulation",
  "width": 800,
  "height": 500,
  "html": "<iframe src=\"https://embed.modelscript.org/?embed=true#m=eJy...\" width=\"100%\" height=\"500\" frameborder=\"0\" l
}
```

8. WebGL Context Management & Mobile Scroll Protection

To guarantee rock-solid stability when multiple 3D CAD or WebGPU viewers exist on the same page:

- **Context Pooling (`CanvasContextPool`):** Restricts active WebGL contexts to 6 on mobile/Safari and 8 on desktop. Inactive canvases automatically sleep and display an ultra-crisp WebP raster snapshot with a glassmorphic `🧊 Click to re-activate 3D` overlay pill.
- **Scroll Hijacking Protection:** Embeds intercept wheel events and require holding `Ctrl` (Windows/Linux) or `⌘` (macOS) to zoom, displaying a non-intrusive badge hint if scrolling is attempted without the key.
- **Mobile Touch Passthrough:** Single-finger dragging vertically passes straight through to scroll the host webpage; two-finger pinch and drag gestures control 3D orbit and dolly zoom.

CHAPTER 5

Platform Administration & Enterprise Hub Guide

The **ModelScript Enterprise Hub** (`@modelscript/api` and `@modelscript/web`) provides a self-hostable or cloud-managed control plane for polyglot package management, Git Smart HTTP hosting, OMG SysML v2 / Semantic Web APIs, Slurm HPC simulation offload, operational digital twins, and cryptographically verifiable regulatory compliance.

---

1. Administrative Console & Role-Based Access Control (RBAC)

Access to the Platform Administration Console (`/admin`) and `/api/v1/admin/*` endpoints is strictly enforced via JWT authentication and role-based middleware (`requireAdmin`):

- **`admin`** : Full platform superuser privileges (user moderation, runtime feature flag mutation, system maintenance windows, compute wallet minting, and audit log verification).
- **`moderator`** : Read/moderation access to public registry packages, flagged comments, and community discussions.
- **`user` / Organization Roles (`owner` , `admin` , `member` , `billing` , `viewer`)**: Scoped access within `/api/v1/organizations/:org` namespaces.

User & Account Governance (`/api/v1/admin/users`)

Platform administrators can search, filter, and manage user accounts via the Admin Console or REST API:

- **Role Assignment (`PATCH /api/v1/admin/users/:id/role`)**: Promote or demote accounts between `user` , `moderator` , and `admin` . Self-demotion by the active administrator is blocked to prevent accidental lockout.
- **Account Suspension (`POST /api/v1/admin/users/:id/suspend` & `/unsuspend`)**: Immediately revokes active JWT refresh tokens, blocks package publishing, and halts running cloud HPC jobs while preserving historical audit trails.
- **Platform Telemetry (`GET /api/v1/admin/stats`)**: Real-time metrics covering total registered users, published package versions, active Co-Simulation sessions, cloud HPC queue depth, and object storage consumption.

---

2. Runtime Feature Flags & Progressive Rollouts

ModelScript includes an in-memory and database-backed **Feature Flag Engine** (`apps/api/src/services/feature-flag-service.ts`) that allows operators to toggle experimental compilers, cloud solvers, or UI capabilities at runtime without redeploying containers.

Flag Evaluation Precedence

1. **Environment Variable Override**: Setting `FEATURE_FLAG_<UPPER_SNAKE_KEY>=true|false` (for example, `FEATURE_FLAG_WEBGPU_CLOUD_SOLVER=true`) overrides database state for deterministic staging/CI configuration.
2. **Role Gate (`allowedRoles`)**: If a flag specifies `allowedRoles` (e.g., `["admin", "moderator"]`), only authenticated users holding one of those roles receive `true` .
3. **Deterministic Percentage Rollout (`rolloutPercentage`)**: When `0 < rolloutPercentage < 100` , the service computes a deterministic hash over `(flagKey, userId)` so a given user receives a consistent cohort experience across sessions.

Administrative Flag Endpoints

Method & Endpoint	Access	Description
`GET /api/v1/flags`	Public / User	Evaluate all feature flags for the current caller's `userId` and `role` .
`GET /api/v1/admin/flags`	`admin`	List all registered feature flags with full rollout metadata, role gates, and environment override states.
`PATCH /api/v1/admin/flags/:key`	`admin`	Dynamically update `enabled` (`boolean`), `rolloutPercentage` (`0` – `100`), `allowedRoles` (`string[]`), or `description` , and record an immutable admin audit event.

---

3. Enterprise Authentication, SSO, 2FA & Bot Accounts

The authentication subsystem (`/api/v1/auth` , `/api/v1/auth/sso` , `/api/v1/auth/2fa` , `/api/v1/bots`) supports defense-in-depth enterprise identity requirements:

- **OpenID Connect (OIDC) & SAML Enterprise SSO (`/api/v1/auth/sso`)**: Connect Okta, Azure AD / Entra ID, Keycloak, or Google Workspace identity providers with automatic Just-in-Time (JIT) organization provisioning and group-to-role mapping.
- **TOTP Two-Factor Authentication (`/api/v1/auth/2fa`)**: RFC 6238 Time-Based One-Time Password enrollment (QR code provisioning, backup recovery codes, and mandatory 2FA enforcement policies per organization).
- **Machine & CI/CD Bot Accounts (`/api/v1/bots`)**: Provision scoped, non-interactive service accounts bearing `ms_bot_*` API tokens with granular permissions (`package:read` , `package:publish` , `cloud:dispatch` , `mcp:invoke` , `dhf:export`) and automatic token rotation schedules.
- **Signed Outbound Webhooks (`/api/v1/webhooks`)**: Subscribe external CI/CD pipelines to workspace events (`package.published` , `simulation.completed` , `verification.failed` , `twin.anomaly_detected`) signed with HMAC-SHA256 (`X-ModelScript-Signature-256`).

---

4. Native Git Smart HTTP Server & Content-Addressable Registry

The Hub integrates both a **Git Smart HTTP Backend** and a **Signed Content-Addressable Package Registry**:

Native Git Smart HTTP Server (`/git/:namespace/:project` & `/api/v1/git-server`)

Engineers can clone, fetch, branch, and push polyglot model repositories using standard `git` clients over HTTPS:

```
git clone https://hub.example.com/git/aerospace/flight-control.git
```

- Supports `git-upload-pack` (`GET /info/refs?service=git-upload-pack`) and `git-receive-pack` (`POST /git-receive-pack`) authenticated via Basic Auth (`username` + PAT or `ms_bot_*` token).
- Pre-receive hooks optionally enforce zero-error `msx lint` checks and Digital Thread correspondence verification before accepting pushes to protected branches.

Cryptographic Package Signing & Multi-Lens Manifests (`/api/v1/packages`)

Every package published via `msx publish` is stored in a SHA-256 Content-Addressable Storage (CAS) blob store and signed with the publisher's **Ed25519** private key:

- **Integrity & Provenance**: `msx install --verify-signatures` validates both the SHA-256 tarball digest against `modelscript.lock` and the Ed25519 signature against the registry's public keyring.
- **Multi-Lens Manifest Projection (`GET /api/v1/packages/:name/:version?lens=<lens>`)**: The registry dynamically projects any `modelscript.json` package manifest into three industry interoperability schemas on the fly:
  - `?lens=npm` : Standard Node/NPM-compatible package metadata for JavaScript/TypeScript tooling.
  - `?lens=aas` : **IDTA Asset Administration Shell (AAS v3.0)** JSON submodel descriptor containing `TechnicalData` and `SimulationModels` (FMU) references.
  - `?lens=okh` : **DIN SPEC 3105 / OSHWA Open Know-How (OKH v1.0)** hardware manifest exposing CAD/ECAD bill-of-materials, manufacturing files, and SPDX license metadata.

---

5. Standards-Compliant Systems & Semantic Web APIs

The Hub natively implements open MBSE and Semantic Web specifications so external enterprise tools can query models without proprietary adapters:

API Surface	Base Path	Standard / Capabilities
OMG SysML v2 REST API	`/api/v1/sysml2` (and root `/` alias)	Implements the **OMG Systems Modeling API and Services v2.0** specification: `/projects`, `/projects/{id}/commits`, `/projects/{id}/branches`, `/projects/{id}/tags`, `/projects/{id}/commits/{cid}/elements`, and `/relationships`.
GraphQL Library API	`/api/v1/libraries/graphql`	Strongly typed GraphQL schema for querying packages, Modelica classes, ports, equations, physical units, and dependency graphs in a single round-trip.
W3C RDF Serialization	`/api/v1/libraries/rdf`	Exports package and workspace ontologies as `text/turtle`, `application/ld+json`, or `application/n-triples`.
W3C SPARQL 1.1 Endpoint	`/api/v1/libraries/sparql`	Executes SPARQL 1.1 `SELECT`, `CONSTRUCT`, and `ASK` queries with optional OWL 2 RL entailment reasoning over indexed engineering libraries.
OSLC 3.0 & TRS 3.0	`msx oslc serve` / `/api/v1/threads`	Exposes OASIS OSLC Requirements (`oslc_rm`) and Architecture (`oslc_am`) catalogs with Tracked Resource Set (TRS 3.0) incremental change logs for IBM DOORS Next and Polarion synchronization.

---

6. Cloud Slurm HPC Offload, CAE Sweeps & Compute Credit Billing

When engineers run `msx simulate --cloud` or `msx cloud dispatch`, the Hub orchestrates containerized batch execution across Slurm HPC clusters or Kubernetes GPU pools:

- **Cloud Simulation & CAE Endpoints (`/api/v1/cloud` & `/api/v1/cae`)**:
  - Dispatches single-run stiff DAE simulations, multi-thousand-run Monte Carlo ensembles, SU2 CFD runs, and CalculiX FEA jobs to isolated worker nodes.
  - Streams live progress events and solver logs via WebSockets (`/ws`) and persists output trajectories (`CSV`, `Parquet`, `VTK`, `FRD`) to S3-compatible object storage.
- **Compute Credit Wallet & Stripe Billing (`/api/v1/billing`)**:
  - Each user or organization maintains an ACID transactional **Compute Credit Wallet**.
  - Job submission places a pre-authorization hold (`reserveCredits`) based on the requested hardware profile (`standard`, `high-cpu`, `gpu-a100`, `slurm-mpi`); upon completion or cancellation (`msx cloud cancel`), exact wall-clock consumption is settled and unused credits are refunded.
  - Supports self-service credit top-ups via Stripe Checkout webhooks (`/api/v1/billing/webhook`) or manual enterprise quota allocation by platform admins.

---

7. Digital Thread Hash-Chained Audit Ledger & Operational Digital Twins

For safety-critical industries (aerospace DO-178C, automotive ISO 26262, medical FDA 21 CFR Part 11 / 820, and nuclear IEC 61513), the Hub maintains a tamper-evident digital thread and live twin telemetry pipeline:

Cryptographic Hash-Chained Audit Ledger (`/api/v1/threads/audit-log`)

Every mutation to a requirement, model equation, TGG alignment link, verification proof, or administrative setting is appended to a PostgreSQL audit ledger where each record k stores:

$$H_k = \text{SHA256}(H_{k-1} \parallel \text{timestamp}_k \parallel \text{actorId}_k \parallel \text{action}_k \parallel \text{payloadHash}_k)$$

Operators and external auditors can verify the unbroken integrity of the entire historical chain at any time via the CLI or REST API:

```
# Verify cryptographic hash-chain integrity across the PostgreSQL audit ledger
msx db verify

# Export and verify a signed Design History File (DHF) compliance archive
msx dhf export ./workspace -o release_v1.dhf.md
msx dhf audit release_v1.dhf.md
```

Operational Digital Twins (`/api/v1/twins` & `/api/v1/instances`)

- **Asset Instance Registry (`/api/v1/instances`)**: Tracks serialized physical assets in the field (e.g., tail numbers, inverter serial IDs, or battery packs) bound to specific immutable commits of their ModelScript digital twin.
- **Live Telemetry & Drift Detection (`/api/v1/twins`)**: Ingests time-series sensor telemetry via REST/WebSockets, executes an online Extended Kalman Filter (EKF) / moving-horizon observer against the compiled WASM DAE arena, and triggers automated alerts when physical parameter drift or residual anomalies exceed safety thresholds.

---

8. Hosted MCP SSE Gateway (`/api/v1/mcp`)

Enterprise teams can connect cloud AI agents and internal copilot orchestrators directly to the Hub's **Hosted Model Context Protocol (MCP) Gateway**:

- **`GET /api/v1/mcp/sse`** : Authenticates via Bearer token (`JWT` or `ms_bot_*`) and opens a server-sent events transport session.
- **`POST /api/v1/mcp/messages`**: Dispatches any of the 31 MCP engineering tools inside sandboxed worker isolates with automatic compute credit metering and full audit-ledger logging of every AI tool invocation.

PART II

Compiler & Pipeline Architecture

CHAPTER 6

Architecture Overview

ModelScript is built on a **data-oriented, WebAssembly-first compiler architecture**. Rather than allocating deep hierarchies of garbage-collected JavaScript AST wrapper objects, every stage of the compiler—from GLR parsing to DAE flattening, sparse linear algebra, and formal verification—operates on contiguous **Struct-of-Arrays (SoA)** arenas in WebAssembly linear memory.

End-to-End Compilation Pipeline



Core Architectural Pillars

1. Zero-Wrapper Linear Memory CST & DAE Arena

Equations, expressions, and statements are lowered directly from 32-bit linear-memory CST node pointers (`u32`) into `DAEBuilder` integer IDs (`VarId` , `ExprId` , `EqId` , `StmtId`). This eliminates garbage-collection pauses even when flattening multi-thousand-equation Modelica Standard Library (MSL) models.

2. Demand-Driven Incremental Compilation (Salsa)

Inspired by the Rust compiler (`rust-analyzer`), `QueryEngine` implements the **Salsa** incremental recomputation algorithm (`fetch → deep_verify → execute → backdate`). When a user edits a parameter in the IDE, only queries whose transitive input hashes actually changed are recomputed.

3. Polyglot Synchronization via Triple Graph Grammars (TGG)

Cross-language traceability is enforced algebraically via **Double-Pushout (DPO)** graph rewriting (`packages/dsl/src/tgg.ts`) with **Critical Pair Analysis (CPA)** confluence verification (`packages/dsl/src/cpa.ts`). Changes in SysML v2 requirements, Modelica parameters, STEP CAD assemblies, or ModelScript `@unmanaged` structs propagate bidirectionally without losing comments or formatting.

4. Multi-Domain Theory Coordinator (CDCL(T))

Thirteen specialized **Domain Theory Oracles**—spanning OWL2 Description Logic (`OntologyTheoryOracle`), SysML v2 intervals (`ConstraintTheoryOracle`), Zones/Octagons (`AbstractDomainOracle`), Barrier Certificates & Taylor Flowpipes (`ContinuousSafetyOracle`), DAE trajectories (`DynamicSimulationOracle`), 3D B-Rep clearance (`SpatialPhysicsOracle`), FEA/CFD contracts (`CaeContractOracle`), DAE structural matching (`CausalizationOracle`), SI dimensional analysis (`DimensionalOracle`), Assume-Guarantee refinement (`EntailmentOracle`), Kirchhoff/stream conservation (`FlowAlgebraOracle`), AP242 GD&T stack-up (`ToleranceStackOracle`), and Conformal UQ (`UqOracle`)—exchange variable bounds and Nelson-Oppen equalities until a global fixpoint or minimal unsatisfiable core (`UnsatCore`) is reached.

Monorepo Package Matrix

Package	Path	Primary Responsibility
`@modelscript/dsl`	`packages/dsl`	Grammar combinators, Ahead-of-Time WASM GLR parser compiler (`buildParser`), TGG/DPO engine, Critical Pair Analysis, E-graphs
`@modelscript/runtime`	`packages/runtime`	Linear-memory `DAEBuilder` , `ArenaDAEPrinter` , Salsa `QueryEngine` , `WorkspaceIndex` , BLT/Pantelides solver, 13 Formal Theory Oracles
`@modelscript/simulate`	`packages/simulate`	Numerical integrators (SUNDIALS CVODE/IDA, Radau IIA, DOPRI5, Dommel EMT), WebGPU batched solver, FNO surrogates, Monte Carlo
`@modelscript/cad`	`packages/cad`	OpenCascade (OCCT) B-Rep kernel, CSG evaluator, STEP AP214/AP242 serializer, 3D tetrahedral FEA solver, Gerber ECAD parser
`@modelscript/diagram`	`packages/diagram`	Polyglot schematic layout engine, orthogonal Manhattan connection router, SVG/Canvas renderer, dark-mode color inversion
`@modelscript/exchange`	`packages/exchange`	FMI 2.0/3.0 FMU import/export, SSP 1.0/2.0 container toolkit, Co-Simulation Master Orchestrator, WebSerial/AudioWorklet HIL
`@modelscript/lsp`	`packages/lsp`	Multi-language LSP server, semantic tokens, hover, go-to-definition, code actions, background worker pool
`@modelscript/mcp`	`packages/mcp`	Unified Model Context Protocol server exposing 31 core and polyglot compilation, simulation, CAD, and verification tools to AI agents
`@modelscript/ide-client`	`packages/ide`	Reusable WebAssembly IDE client components, Monaco/CodeMirror language bindings, and dockable engineering viewports
`@modelscript/examples`	`packages/examples`	Curated multi-domain reference workspaces (CubeSat, EV Battery Pack, IEEE 39-Bus Grid, Robotic Arm, Aero-Structural Wing)
`modelscript` (`msx`)	`apps/cli`	Unified command-line interface (`flatten` , `simulate` , `verify` , `align` , `polyglot` , `fmu` , `csg` , `surrogate` , `mc` , `hil`)
`@modelscript/api`	`apps/api`	REST, GraphQL, SPARQL, and WebSocket simulation backend server
`@modelscript/web`	`apps/web`	Browser-native WebAssembly Polyglot Playground and visual modeling environment

CHAPTER 7

WebAssembly GLR Incremental Parser

ModelScript avoids external runtime parser dependencies. Instead, every supported language in `languages/` is parsed using a custom, ahead-of-time (AOT) compiled **Generalized LR (GLR)** parser executing directly in WebAssembly (`parser.wasm`).

The parser compiler is implemented in `@modelscript/dsl` under `packages/dsl/src/codegen/` .

Why GLR in WebAssembly?

1. **Handling Complex Engineering Grammars**: Engineering modeling languages like Modelica and SysML v2 possess ambiguities that cannot be parsed by strict deterministic LR(1) or LALR parsers. The GLR algorithm handles non-deterministic grammar paths by branching parse stacks and pruning invalid branches dynamically.
2. **Zero-Allocation Memory Footprint**: Rather than allocating JavaScript AST object wrappers for every identifier and operator, tokens and CST nodes are laid out in a flat WebAssembly linear memory arena.
3. **Web-Native Performance**: The compiled `parser.wasm` executes at near-native speeds in both Node.js server environments and client-side browser contexts (such as the IDE and the Web Playground).

Parser Initialization

In TypeScript or AssemblyScript code, parsers are loaded and registered via `createWasmParser` :

```
import { createWasmParser } from "@modelscript/modelica/parser";
import { Context } from "./src/context.js";

// Load the compiled WebAssembly parser binary
const modelicaWasmPath = require.resolve("@modelscript/modelica/parser.wasm");
const { parser } = await createWasmParser(modelicaWasmPath);

// Register file extension mapping in the compilation Context
Context.registerParser(".mo", parser);
```

Linear CST Memory Layout

When a source file is parsed, the parser allocates a continuous buffer of node headers in linear memory. Each CST node is referenced by a 32-bit integer pointer (`u32` node offset) rather than an object reference:

```
┌──────────────────────────────────────────────────┐
│                  Linear CST Node Arena           │
├─────────────┬────────────┬──────────────┬────────┤
│ Field       │ Offset     │ Type         │ Description     │
├─────────────┼────────────┼──────────────┼────────┤
│ typeId      │ 0          │ uint16       │ Node type code  │
│ flags       │ 2          │ uint16       │ Semantic flags  │
│ startOffset │ 4          │ uint32       │ Start byte pos  │
│ endOffset   │ 8          │ uint32       │ End byte pos    │
│ firstChild  │ 12         │ uint32       │ Offset to child │
│ nextSibling │ 16         │ uint32       │ Offset to sib   │
└─────────────┴────────────┴──────────────┴────────┘
```

Direct Field Accessors (`Cst`)

High-performance accessors read typed node properties directly from the linear buffer:

```
import { Cst } from "@modelscript/modelica/language";

// Accessing child fields without constructing intermediate wrappers
const comp = Cst.ComponentClause.componentDeclarationList(node);
for (const decl of comp) {
  const nameNode = Cst.ComponentDeclaration.declaration(decl);
  const identifier = db.ast.text(nameNode);
}
```

Incremental Re-Parsing

When editing code in the Language Server Protocol (LSP) or Web IDE, the GLR parser performs incremental tree re-use:

- Text edits pass line and character delta offsets.
- Unaffected subtrees in linear memory remain unchanged and are spliced into the new parse tree.
- Syntax diagnostics are generated during parsing and stored in the diagnostics arena.

CHAPTER 8

Salsa Incremental Query Engine

ModelScript's semantic analysis and symbol resolution pipeline is driven by an incremental query computation engine inspired by the open-source Salsa incremental query framework.

The engine lives in `@modelscript/runtime` and provides memoized, demand-driven evaluation across multi-file engineering workspaces.

The Query Life Cycle

Every query executed in ModelScript follows a 4-phase lifecycle to ensure maximum cache re-use and correct invalidation:



The 4 Stages

1. **`fetch`**: Checks if the query result exists in the memoization table.
2. **`deep_verify`**: If the workspace revision has advanced, transitively checks whether any input queries or files that contributed to this result have actually changed.
3. **`execute`**: Re-runs the query definition if inputs have genuinely changed, tracking any newly accessed queries.
4. **`backdate`**: If re-executing the query produces an output value identical to the previous result, the query's revision is backdated to prevent cascading invalidation down the dependency tree.

Key Compiler Queries

The query engine handles semantic resolution, type checking, and specialization:

Query	Scope	Description
`instantiate`	Class	Resolves all direct components, inherited elements, and nested declarations of a class.
`classInstance`	Component	Specializes a generic class definition with specific modifier expressions.
`resolveSimpleName`	Scope	Performs hierarchical lexical scoping and imports resolution.
`variability`	Component	Computes effective variability: `constant`, `parameter`, `discrete`, or `continuous`.
`causality`	Component	Computes signal causality: `input`, `output`, or `internal`.
`effectiveModification`	Hierarchy	Resolves and merges class modification clauses across multi-level inheritance graphs.

Salsa & Linear CST Synergy

AST wrapper nodes are **never** stored inside the Salsa cache. Storing heavy AST objects would duplicate memory and defeat garbage collection guarantees.

Instead:

- The Salsa cache stores semantic integer handles and primitive metadata (e.g. `SymbolId`, resolved `TypeInfo`, variability enums).
- Concrete CST nodes are retrieved on-demand from linear memory using `db.cstNode(symbolId)` only when lowering or evaluating equations.

CHAPTER 9

Linear Memory DAE Arena & Lowering

The Differential Algebraic Equations (DAE) representation in ModelScript is implemented via `DAEBuilder` in `@modelscript/runtime` (`packages/runtime/src/dae/wasm_dae.ts`).

Unlike traditional compilers that construct linked expression trees using heap-allocated objects, `DAEBuilder` operates strictly on **flat, contiguous typed arrays** (Struct-of-Arrays) in WebAssembly linear memory.

Arena Struct-of-Arrays (SoA) Layout

The arena maintains three primary contiguous arrays:

```
┌──────────────────────────────────────────────────────────┐
│                    DAEBuilder Linear Memory              │
├──────────────┬──────────────┬────────────────────────────┤
│ Buffer       │ Element Size │ Contents                   │
├──────────────┼──────────────┼────────────────────────────┤
│ varData      │ 32 bytes     │ Name, type, variability,   │
│              │              │ causality, start value, flags│
├──────────────┼──────────────┼────────────────────────────┤
│ exprData     │ 16 bytes     │ OpKind, data1, leftExprId, │
│              │              │ rightExprId                │
├──────────────┼──────────────┼────────────────────────────┤
│ eqData       │ 24 bytes     │ EqKind, lhsExprId,         │
│              │              │ rhsExprId, flags           │
└──────────────┴──────────────┴────────────────────────────┘
```

Every variable, expression, and equation is assigned a persistent 32-bit integer handle:

- `VarId` : Handle to a variable in `varData`
- `ExprId` : Handle to an expression node in `exprData`
- `EqId` : Handle to an equation in `eqData`

Lowering Pipeline

Flattening a high-level Modelica model into linear DAE form proceeds in two coordinated layers:

Layer 1: Components & Hierarchy

Reads Salsa queries (`instantiate` , `classInstance` , `variability`) to allocate variables and parameters in `DAEBuilder` :

- Scalar variables (`Real` , `Integer` , `Boolean` , `String`)
- Array dimensions and indices
- Start values, nominals, and min/max boundaries

Layer 2: Equations & Statements

Walks the linear CST of the class body (`db.cstNode(classId)`):

- Lowers binary, unary, and call expressions directly into `ExprId` nodes in linear memory.
- Emits equality equations (`lhs = rhs`), initial equations, and algorithm statements.

Expression Lowering Rules

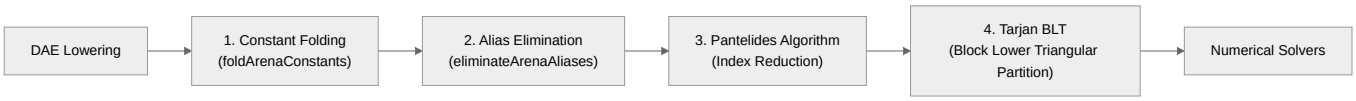
To match OpenModelica's canonical output, the flattener applies specific normalization rules:

1. **Real Coercion in Binary Operations**: If one operand of an arithmetic operator is `Real` and the other is `Integer` , the integer operand is automatically cast to Real using `castToRealExpr()` .

2. **Negation Distribution**: Expressions of the form `-(a * b)` are rewritten canonically to `(-a) * b`.
3. **Binding Literal Kind Matching**: When a binding expression is lowered, `addIntLiteral()` is used for `VarType.Integer` and `addRealLiteral()` for `VarType.Real` (e.g. `parameter Real R = 100` becomes `100.0`).

Optimization & Transformation Passes

Once lowered to the arena, equations pass through four high-performance transformations:



1. **Constant Folding (`foldArenaConstants`)**: Recursively simplifies compile-time constant arithmetic expressions directly in the arena.
2. **Alias Elimination (`eliminateArenaAliases`)**: Detects trivial equality equations ($x = y$) generated by connectors and replaces all references with a single canonical variable handle.
3. **Pantelides Index Reduction**: Differentiates high-index algebraic constraints to reduce the DAE index to 1 or 0 for stable numerical integration.
4. **Tarjan BLT Partitioning**: Constructs the bipartite incidence graph between variables and equations, computes maximum matching, and finds Strongly Connected Components (SCC) using Tarjan's algorithm.

CHAPTER 10

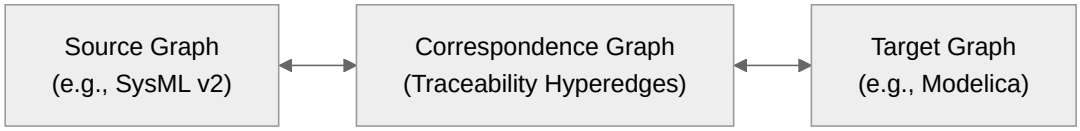
Polyglot Triple Graph Grammars (TGG)

ModelScript provides declarative, ahead-of-time (AOT) compiled **Triple Graph Grammars (TGG)** and Double-Pushout (DPO) algebraic graph rewriting via `@modelscript/dsl/tgg`.

TGG serves as the foundational synchronization mechanism across the **computable digital thread**, enabling bidirectional transformations between disparate engineering models (such as architectural SysML v2 specifications and 1D Modelica physical simulations).

What is a Triple Graph Grammar?

A Triple Graph Grammar synchronizes three distinct graphs:



- **Source Graph (G_S)**: Represents domain concepts in the source language (e.g. SysML v2 Part Definitions, Ports, Requirements).
- **Target Graph (G_T)**: Represents concepts in the target language (e.g. Modelica Models, Connectors, Equations).
- **Correspondence Graph (G_C)**: Explicitly records semantic trace links between elements in the source and target models.

Rather than writing ad-hoc imperative glue code, engineers write declarative rules specifying how elements correspond.

Declarative TGG Rules

A TGG rule specifies patterns in the source, correspondence, and target graphs simultaneously:

```
import { tggRule } from "@modelscript/dsl/tgg";

export const ComponentToModel = tggRule({
  name: "ComponentToModel",
  source: (s) => ({
    part: s.node("PartUsage", { name: s.var("compName") }),
    port: s.node("PortUsage", { parent: s.nodeRef("part") }),
  }),
  target: (t) => ({
    model: t.node("ModelDefinition", { name: t.var("compName") }),
    pin: t.node("PinConnector", { parent: t.nodeRef("model") }),
  }),
  correspondence: (c) => ({
    tracePart: c.link("part", "model"),
    tracePort: c.link("port", "pin"),
  }),
});
```

AOT Compilation to WebAssembly

Unlike interpreted graph transformation engines, `@modelscript/dsl/tgg` compiles rules into deterministic AssemblyScript dispatch routines executing directly in WebAssembly:

- `tgg_forward_dispatch`: Incremental forward propagation from source changes to target models.
- `tgg_backward_dispatch`: Reverse propagation of parameter sizing or component changes back to architectural definitions.
- `tgg_propagate_all_stale`: High-throughput batch reconciliation across dirty workspace subgraphs.

Critical Pair Analysis (CPA)

A notorious problem in graph transformation is rule conflict and non-determinism. ModelScript includes an automated **Critical Pair Analysis (CPA)** verification pass:

1. **Confluence Verification**: Analyzes whether overlapping rule applications could cause divergent target states.
2. **Termination Guarantees**: Proves that cyclic dependencies cannot cause infinite rewrite loops during synchronization.
3. **Completeness Checking**: Flags unmapped source elements that lack corresponding target constructors.

CHAPTER 11

Multi-Domain Theory Coordinator (Nelson-Oppen & CDCL(T))

The **Multi-Domain Theory Coordinator** (`packages/runtime/src/formal/theory_coordinator.ts`) is ModelScript's cross-language formal verification engine. It combines **Nelson-Oppen equality sharing** with **Conflict-Driven Clause Learning Modulo Theories (**$\mathrm{CDCL(T)}$**)** to verify system-wide invariants across ontological, structural, geometric, physical, causal, and dynamic simulation domains simultaneously.

Architectural Motivation

In traditional Model-Based Systems Engineering (MBSE), each engineering tool operates in an isolated semantic silo:

- A **SysML v2** requirement specifies `maxBatteryTemp <= 333.15 [K]`.
- A **STEP / OpenSCAD** 3D CAD enclosure defines the heat-sink surface area A_{sink}.
- A **CFD** configuration computes the convective heat transfer coefficient $h(v_\infty)$.
- A **Modelica** thermal-electrical DAE integrates $C_{\text{th}}\dot{T} = I^2 R_{\text{int}} - hA_{\text{sink}}(T - T_{\text{amb}})$.

No single solver understands all four representations. The **Theory Coordinator** bridges these silos by treating each engineering domain as a decidable or semi-decidable **Theory Oracle** (T_i) sharing a common set of interface variables (V_{shared}).

The 13 Domain Theory Oracles

`packages/runtime/src/formal/theory_coordinator.ts` and `packages/runtime/src/formal/oracles/index.ts` coordinate 13 specialized domain theory oracles implementing the `TheoryOracle` interface:



#	Oracle Class	Domain (`TheoryDomain`)	Decision / Propagation Procedure
1	`OntologyTheoryOracle`	`ontology`	$\mathcal{SROIQ}(\mathbb{D})$ Tableau reasoning over OWL 2 axioms, class disjointness, and property domains
2	`ConstraintTheoryOracle`	`constraints`	Interval arithmetic and relational constraint propagation over SysML v2 requirements
3	`AbstractDomainOracle`	`abstract_interpretation`	Floyd-Warshall Difference Bound Matrix (DBM) closure over **Zone** ($x_i - x_j \le c$) and **Octagon** ($\pm x_i \pm x_j \le c$) domains
4	`ContinuousSafetyOracle`	`continuous_safety`	Control Barrier Function (CBF) Lie-derivative positivity ($L_f B + L_g B \cdot u + \alpha(B) \ge 0$) and Taylor-model flowpipe safety
5	`DynamicSimulationOracle`	`simulation`	Sensitivity-guided DAE trajectory integration (CVODE/IDA) with exact zero-crossing root finding
6	`SpatialPhysicsOracle`	`spatial`	3D CAD bounding-box, B-Rep minimum clearance distance, mass budget, and interference checks
7	`CaeContractOracle`	`cae`	FEA structural safety factor ($\sigma_{\text{von Mises}} \le \sigma_{\text{yield}}/\text{SF}$) and CFD aerodynamic/thermal drag polar bounds
8	`CausalizationOracle`	`causalization`	Hopcroft-Karp maximum bipartite matching ($O(E\sqrt{V})$), Tarjan BLT decomposition, and Pantelides DAE index reduction
9	`DimensionalOracle`	`dimensional`	Gaussian elimination over the 7-base SI unit exponent matrix $[\text{m}, \text{kg}, \text{s}, \text{A}, \text{K}, \text{mol}, \text{cd}]$
10	`EntailmentOracle`	`entailment`	Assume-Guarantee contract compatibility ($\bigwedge A_i = \bot$) and compositional refinement ($A_P \Rightarrow A_C \wedge G_C \Rightarrow G_P$)
11	`FlowAlgebraOracle`	`flow_algebra`	Acausal connection graph conservation ($\sum i_k = 0$ for flow variables, $v_1 = v_2 = \cdots = v_n$ for potential variables)
12	`ToleranceStackOracle`	`tolerance`	STEP AP242 GD&T 1D/3D tolerance chain stack-up via Worst-Case ($\sum t_i$) and Root-Sum-Square ($\sqrt{\sum t_i^2}$)
13	`UqOracle`	`uq`	Latin Hypercube Sampling (LHS), Polynomial Chaos Expansion (PCE) Sobol indices, and distribution-free conformal prediction bounds

---

The Verification Loop

1. **Initialization**: Shared variables (V_{shared}) are initialized to $[-\infty, +\infty]$ or extracted from SysML v2 requirement bounds and `.ssv` parameter sets.
2. **Domain Propagation (`propagate`)**: Each registered `TheoryOracle` inspects the current shared interval store and tightens bounds monotonically:

$$I^{(k+1)}(x) = I^{(k)}(x) \cap \bigcap_{m=1}^{M} \text{Oracle}_m.\text{propagate}(I^{(k)})(x)$$

3. **Nelson-Oppen Equality Deduction**: Whenever $I^{(k+1)}(x) = [c, c]$ and $I^{(k+1)}(y) = [c, c]$ (or linear relational equalities $x - y \le 0 \wedge y - x \le 0$ hold in the Octagon DBM), the coordinator deduces the inter-theory equality $x = y$ and broadcasts it to all oracles.
4. **Conflict & Unsat Core Extraction (CDCL(T))**: If any oracle detects $I(x) = \emptyset$ (i.e., $\text{lo} > \text{hi}$) or an axiomatic contradiction, it emits a minimal **Unsatisfiable Core (`UnsatCore`)** pinpointing the exact offending AST nodes across files (e.g., `battery.mo:42` , `enclosure.step:118` , `thermal_req.sysml:15`).
5. **CEGAR Counterexample Guided Refinement**: If all static/algebraic oracles return `SAT` or `UNKNOWN` , `DynamicSimulationOracle` synthesizes worst-case corner parameters from the narrowed interval box and executes a numerical DAE simulation to either confirm satisfaction or extract a concrete counterexample trajectory $\mathbf{x}(t)$.

```
# Run multi-domain formal verification across Modelica, SysML v2, and STEP artifacts
npx msx verify SystemRequirement model.mo architecture.sysml chassis.step
```

CHAPTER 12

Simulation Solvers & Numerical Integrators

ModelScript embeds a multi-tier suite of numerical solvers in `@modelscript/simulate` to support both desktop simulations and massive batched cloud/WebGPU parameter sweeps.

Supported Solvers



---

1. SUNDIALS CVODE & IDA Integrators

For high-precision industrial models, ModelScript integrates Lawrence Livermore National Laboratory's **SUNDIALS** suite compiled to WebAssembly:

- **CVODE**: Solves Initial Value Problems (IVPs) for stiff and non-stiff systems of ordinary differential equations (ODEs). Uses Variable-Order Variable-Step-size Adams-Moulton methods for non-stiff systems and Backward Differentiation Formulas (BDF) for stiff systems.
- **IDA**: Solves Initial Value Problems for systems of differential-algebraic equations (DAEs) of index up to 1:

$$F(t, y, y') = 0$$

  Includes dense and sparse linear solvers with adaptive step-size control.

---

2. WebGPU Batched Parallel Solvers

When performing Monte Carlo uncertainty analysis or hyperparameter sweeps, sequential simulation is a bottleneck.

ModelScript compiles linearized DAE blocks into WebGPU compute shaders (`@modelscript/simulate/gpu`):

- Runs thousands of parameter combinations in parallel on GPU hardware.
- Scales efficiently across client GPUs in browsers and headless datacenter instances.
- Returns statistical distributions, mean trajectories, and variance envelopes directly.

---

3. Direct Collocation Optimal Control

ModelScript formulates dynamic optimization and optimal control problems directly from Modelica models:

$$\min_{u(t)} \int_0^T L(x(t), u(t), t)\, dt$$

$$\text{subject to } F(t, x, \dot{x}, u) = 0, \quad u_{\min} \le u(t) \le u_{\max}$$

- **Transcription**: Transcribes the continuous system into non-linear programming (NLP) problems using Radau or Lobatto collocation points.
- **Solvers**: Solves the resulting sparse NLP using interior-point optimization techniques.

4. Surrogate Reduced Order Models (ROM)

High-fidelity 3D simulations (such as CFD or FEA) are often too computationally expensive for real-time control loops or web deployment.

ModelScript's surrogate modeling pipeline (`msx surrogate`):

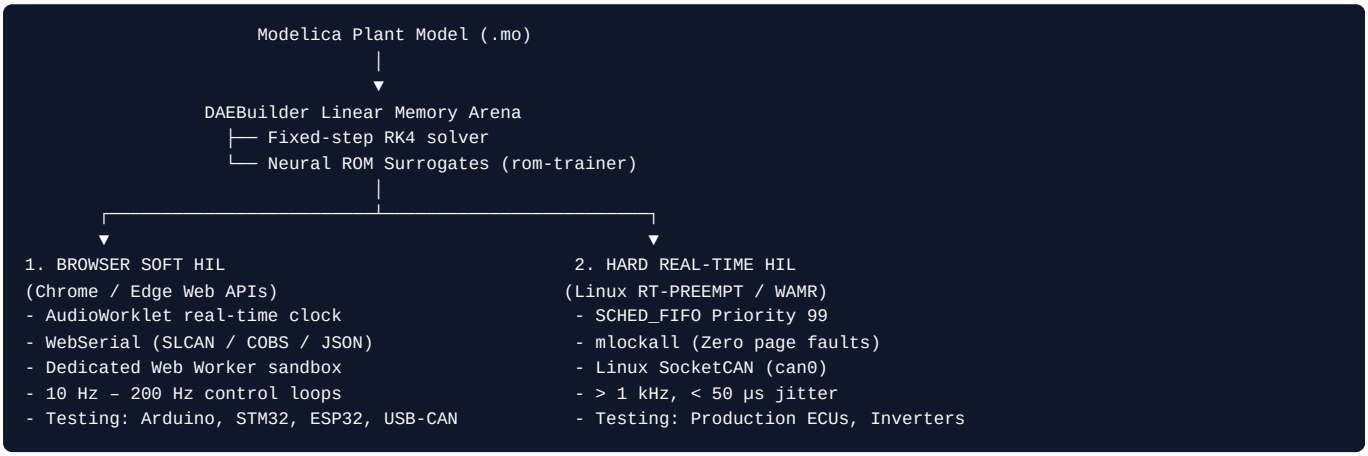
1. **Sampling**: Generates experimental design samples (Latin Hypercube, Sobol sequences).
2. **Execution**: Evaluates high-fidelity runs across parameter spaces.
3. **Training**: Trains compact surrogate models using:
   - Polynomial Chaos Expansion (PCE)
   - Gaussian Process Regression (Kriging)
   - Neural Ordinary Differential Equations (Neural ODEs)
4. **Embedding**: Exports the trained surrogate as a standardized FMI Functional Mock-up Unit (FMU) or standalone WebAssembly module.

CHAPTER 13

Software-in-the-Loop (SIL) & Hardware-in-the-Loop (HIL) with ModelScript

This guide documents ModelScript's unified SIL and HIL architecture based on **eFMI-style deterministic code generation**, **WebAssembly**, **FMI Layered Standards (FMI-LS-BUS)**, and **browser-native Soft HIL**.

Architecture Overview



---

1. FMI-LS-BUS (Layered Standard for Network Communication)

ModelScript implements the **Modelica Association FMI-LS-BUS** standard:

- `FmiLsBusManifest`: Describes CAN, CAN-FD, LIN, and Ethernet buses and frames.
- `generateFmiLsBusXml`: Generates standard `fmi-ls-bus.xml` bundled inside the FMU archive.
- `FmiLsBusCodec`: High-performance bit-level encoder and decoder supporting Intel (little-endian) and Motorola (big-endian) bit alignments, linear scaling (`physical = raw * factor + offset`), signed/unsigned integers, and float32 data types.

Example: Defining CAN Frames in TypeScript

```
import { FmiLsBusFrame, FmiLsBusCodec, generateFmiLsBusXml } from "@modelscript/exchange";

const engineFrame: FmiLsBusFrame = {
  name: "EngineStatus",
  id: 0x120, // CAN ID 288
  length: 8,
  cycleTime: 0.01, // 10 ms (100 Hz)
  signals: [
    {
      name: "engineSpeed",
      valueReference: 1,
      startBit: 0,
      bitLength: 16,
      byteOrder: "littleEndian",
      factor: 0.25, // 1 LSB = 0.25 rpm
      offset: 0,
      unit: "rpm",
    },
    {
      name: "throttlePosition",
      valueReference: 2,
      startBit: 16,
      bitLength: 8,
      byteOrder: "littleEndian",
      factor: 0.392, // 0-100%
      offset: 0,
      unit: "%",
    },
  ],
};
```

---

2. In-Browser "Soft HIL" (WebSerial, WebUSB, WebHID)

For desktop and browser-based testing, ModelScript provides the `WebSerialParticipant` (`packages/exchange/src/cosim/participants/web-hardware.ts`):

Supported Hardware Adapters & Microcontrollers

- **USB-to-CAN Adapters (SLCAN)**: CANable, candleLight, USBtin, Zubax Babel.
- **Microcontrollers (USB-UART / CDC)**: STM32 Nucleo, ESP32, Arduino, Teensy, Raspberry Pi Pico.
- **Framing Protocols**:
  - `slcan` : Standard ASCII CAN protocol (`t12080102030405060708\r`).
  - `cobs` : Consistent Overhead Byte Stuffing for binary frames.
  - `json` : Line-delimited JSON objects (`{"rpm": 3200, "throttle": 45}\n`).

Using WebSerialParticipant in Co-Simulation

```
import { CoSimOrchestrator, CoSimSession, WebSerialParticipant } from "@modelscript/exchange";

// Create hardware participant
const hwParticipant = new WebSerialParticipant({
  id: "ecu_serial",
  baudRate: 115200,
  protocol: "slcan",
  busFrames: [engineFrame],
});

// Add to co-simulation session
session.addParticipant(hwParticipant);
```

Unthrottled Browser Pacing via `AudioWorkletClock`

Browsers throttle `setInterval` to 1 Hz when tabs are backgrounded. ModelScript's `AudioWorkletClock` (`packages/exchange/src/cosim/audio-worklet-clock.ts`) leverages the high-priority OS audio thread (44.1 kHz) to drive the simulation loop, so it is not subject to background-tab timer throttling.

---

3. Hard Real-Time HIL on Linux RT-PREEMPT

For hard real-time execution with physical ECUs over CAN:

1. **Compile Model to Standalone C / WASM**: ModelScript's C generator produces a static memory model with an embedded zero-allocation `integrate_rk4` solver.
2. **Compile Neural ROM Surrogates to C**: Use `exportROMToC(trainedRom)` from `@modelscript/simulate` to generate zero-dependency static C arrays for surrogate evaluation in < 20 µs.
3. **Deploy with Standalone Runner**: Use `hil-runner-template.c` (`packages/exchange/src/fmu/hil-runner-template.c`):
   - Compiles with `gcc -O3 hil-runner-template.c model.c -lm -o model_hil` .
   - Runs with `sudo ./model_hil can0` under `SCHED_FIFO` priority 80 with `mlockall` .

---

4. Shared-Memory IPC & Zero-Overhead Local Agent Proxy

For multi-rate co-simulation between compiled FMUs, ModelScript provides a high-throughput **Shared-Memory IPC Protocol** (`packages/exchange/src/cosim/ipc/fmu-shm-protocol.ts`):

- **Zero-Copy Memory-Mapped Ring Buffers**: Uses POSIX shared memory (`/dev/shm` on Linux) and named shared memory segments on Windows with cache-line-aligned (64-byte) atomic variables.
- **Spinlock & Futex Synchronization**: State exchanges achieve > 500,000 steps/sec with sub-2µs latency, bypassing TCP/loopback network stack overheads.
- **Local Agent Proxy (`local-agent-proxy.ts`)**: Couples proprietary or legacy simulation executables into ModelScript's `CoSimOrchestrator` via shared-memory participants with dead-reckoning extrapolation and interface Jacobian negotiation.

PART III

Polyglot Languages & Formats

CHAPTER 14

Polyglot Language Ecosystem

ModelScript is not a single-language compiler—it is a **polyglot systems engineering framework**. All engineering languages share the same WebAssembly GLR parser compiler (`@modelscript/dsl`), the same `SymbolIndex` and `WorkspaceIndex` architecture, and the same Triple Graph Grammar (TGG) synchronization engine.

Explore by Domain

Search by name, file extension, or engineering domain, or filter by industry. See Industries for domain-specific workflows.

Language Capability Matrix

Filter by platform group, industry, or search across all 44 engineering languages to inspect simulation engines, linear-memory DAE lowering, and polyglot projections:

Unified Polyglot Pipeline

Every language plugin implements the `LanguageDefinition` interface (`packages/dsl/src/language.ts`):

1. **Grammar DSL (`defineGrammar`)**: Compiled Ahead-of-Time into a compact `.wasm` binary parser table.
2. **Symbol Indexer (`SymbolIndexer`)**: Walks the linear-memory CST and populates `SymbolEntry` declarations into the global `WorkspaceIndex` .
3. **Salsa Queries (`queries`)**: Memoized semantic analyses (type resolution, unit inference, structural validation).
4. **Compiler Lints (`lints`)**: Real-time diagnostics surfaced identically in the CLI (`msx lint`), LSP server, and Web IDE.
5. **TGG Rules (`tggRules`)**: Declarative bidirectional synchronization rules linking AST nodes across languages (e.g., `SysMLv2 PartUsage` ↔ `Modelica Component` ↔ `STEP Product` ↔ `ModelScript Struct`).

CHAPTER 15

ModelScript (`.msx` , `.modelscript`)

`@modelscript/modelscript` (`languages/modelscript`) is the native **ModelScript** systems modeling and query language. It combines zero-cost linear-memory data layout control (`@unmanaged struct`), WebAssembly/C++ native compilation, and declarative XQuery-inspired **FLWOR** (`for` – `let` – `where` – `order by` – `return`) queries over polyglot engineering graphs.

Supported Extensions & Dialects

Extension	Description
`.msx`	Canonical ModelScript source module
`.modelscript`	Long-form ModelScript source module

Language Features & Grammar

The native WebAssembly GLR parser (`languages/modelscript/src/language.ts`) parses `.msx` modules into a compact Concrete Syntax Tree (CST) with five primary syntactic constructs:

1. Module Imports

Import declarations bind symbols from external ModelScript modules or polyglot engineering artifacts:

```
import { ThermalNode, Conductance as G } from "./thermal_kernel.msx";
```

2. Unmanaged Linear-Memory Structs (`@unmanaged struct`)

Structs annotated with `@unmanaged` are packed directly into 32-bit WebAssembly linear memory without garbage-collected wrapper allocation:

```
@unmanaged struct StateVector {
  id: u32;
  temperature: f64;
  pressure: f64;
  active: bool;
}
```

3. Type System

- **Primitive Types**: `i32` , `u32` , `i64` , `u64` , `f32` , `f64` , `bool` , `string` , `usize` , `void`
- **Generic & Pointer Types**: `Array<T>` , `Slice<T>` , `ptr<T>`
- **User-Defined Structs**: Any declared `struct` identifier

4. Functions, Statements & Control Flow

Functions declare strongly typed parameters and return types, supporting `let` and `const` variable bindings, `if` / `else` branches, `while` loops, C-style `for` loops, and member/index expressions:

```
const BOLTZMANN: f64 = 1.380649e-23;

function computeHeatFlux(k: f64, area: f64, dT: f64, dx: f64): f64 {
  if (dx <= 0.0) {
    return 0.0;
  }
  let flux: f64 = -k * area * (dT / dx);
  return flux;
}
```

5. Declarative FLWOR Queries

ModelScript embeds first-class **FLWOR** (`for` – `let` – `where` – `order by` – `return`) expressions directly in the expression grammar for querying collections, CST nodes, and simulation trajectories:

```
let criticalNodes =
  for node in nodes
  let margin := node.limit - node.temperature
  where margin < 15.0 && node.active == true
  order by margin asc
  return { id: node.id, margin: margin };
```

Multi-Target Compiler (`ModelScriptCompiler`)

`ModelScriptCompiler` (`languages/modelscript/src/index.ts`) lowers the linear-memory CST across three code generation backends:

Backend	Emitter Class	API Methods	Target Output
JavaScript	`JsEmitter`	`compileToJs(source)` / `executeJs(source, context)`	Executable ES module with lowered FLWOR pipelines (`.filter().sort().map()`)
WebAssembly	`WasmEmitter`	`compileToWat(source)` / `compileToWasm(source)` / `executeWasm(source, fn, args)`	Textual WAT S-expressions and binary `.wasm` modules with `@unmanaged` linear-memory offsets
Native Binary	`NativeEmitter`	`compileToNative(source, { outPath, entryFn?, cc?, cflags? })`	WASM lowered to C via `wasm2c` , then compiled with the host C compiler (`$CC` , default `clang`)

```
import fs from "node:fs";
import { ModelScriptCompiler } from "@modelscript/modelscript";

const compiler = new ModelScriptCompiler();
await compiler.init();

const source = fs.readFileSync("kernel.msx", "utf8");
const { js } = compiler.compileToJs(source); // ES module source
const wasm = await compiler.compileToWasm(source); // Uint8Array
const result = await compiler.executeWasm(source, "main", []);
const binary = await compiler.compileToNative(source, { outPath: "build/kernel" }); // requires wasm2c + clang
```

Command Line Interface (`msx compile` & `msx lint`)

Compile, lint, and run ModelScript source modules directly from the terminal:

```
# Lint ModelScript modules for syntax, type errors, and unmanaged struct layout
msx lint kernel.msx

# Compile a ModelScript module into flattened DAE or WebAssembly bytecode
msx compile MainModule kernel.msx --flattener=wasm --timing
```

Salsa Query Engine & Linear-Memory Struct Layout

`ModelScriptQueryEngine` (`languages/modelscript/src/queries.ts`) provides memoized semantic queries over the CST:

- **`structLayout(rawStructNode)`** : Computes deterministic 32-bit WebAssembly linear-memory byte layouts (`StructLayout`) for every `struct_declaration` :
  - `bool` : 1 byte (alignment 1)
  - `i32` , `u32` , `f32` , `usize` , `string` , `ptr<T>` , `Array<T>` , `Slice<T>` : 4 bytes (alignment 4)
  - `i64` , `u64` , `f64` : 8 bytes (alignment 8)
  - Automatically inserts natural alignment padding (`alignOffset`) between fields and aligns the total struct byte size to the maximum field alignment (`maxAlign`).
- **`inferExpressionType(rawNode, scope)`** : Infers static types across literals, binary/unary expressions, struct field access, function calls, and FLWOR `return` clauses (`Array<ReturnType>`).

Semantic Linter & Error Codes

`ModelScriptLinter` (`languages/modelscript/src/linter.ts` and `languages/modelscript/src/errors.ts`) enforces 10 static analysis rules across syntax, name resolution, type checking, FLWOR clauses, and struct/function calls:

Polyglot TGG Rules

`@modelscript/modelscript` registers two Triple Graph Grammar (TGG) synchronization rules in `languages/modelscript/src/language.ts` for bidirectional projection with Modelica:

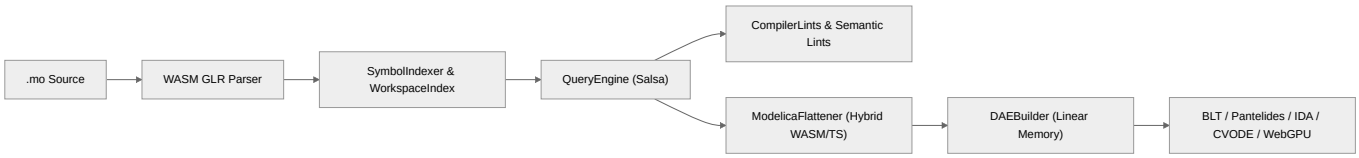
Rule Name	Source Pattern	Target Pattern (`modelica`)	Semantics
`ModelScriptStructToModelicaRecord`	`StructDeclaration`	`ClassDefinition` (`record`)	Synchronizes a `@unmanaged struct` declaration with a Modelica `record` definition
`ModelScriptParamToModelicaMod`	`VariableDeclaration`	`ComponentClause` (`parameter`)	Synchronizes a top-level constant or variable binding with a Modelica component parameter modification

CHAPTER 16

Modelica 3.x (`.mo`)

`@modelscript/modelica` (`languages/modelica`) is the flagship physical modeling language of ModelScript. It provides a complete Modelica 3.x compiler pipeline—from a native WebAssembly GLR parser and Salsa query engine to an AssemblyScript/TypeScript arena flattener (`ModelicaFlattener`) and the full Modelica Standard Library (MSL 4.0.0).

Compilation Pipeline



Core Language Coverage

- **Class Specializations**: `model` , `block` , `connector` , `expandable connector` , `record` , `type` , `package` , `function` , `operator` , `operator record` , `operator function`
- **Inheritance & Redeclarations**: Multiple `extends` , `redeclare` , `replaceable` , `constrainedby` , `inner` / `outer` dynamic scoping, and deep hierarchical class modifications
- **Equations & Algorithms**: Differential equations (`der(x)`), `connect(a, b)` (both potential/flow and `stream` connectors), `if` / `for` / `when` equations, `initial equation` , and imperative `algorithm` / `initial algorithm` sections
- **Synchronous Clocks (Modelica 3.3+)**: `Clock` , `sample` , `hold` , `previous` , `subSample` , `superSample` , `shiftSample` , `backSample`

Polyglot & Physical Extensions (`languages/modelica/src/extensions/`)

Beyond standard Modelica 3.x, `@modelscript/modelica` includes first-class compiler extensions for 3D CAD geometry, multiphysics FEA/CFD studies, formal verification, and localization:

1. **Embedded 3D CSG & STEP Export (`shape-flattener.ts` , `csg.ts` , `cad.ts`)**:
   - `ShapeFlattener` extracts `Modelica.Mechanics.MultiBody.Visualizers.FixedShape` components, parametric dimensions, and `Placement` transformations directly from flattened models.
   - Compiles visualizers into parametric OpenSCAD scripts (`npx msx csg ModelName file.mo`) or exports watertight ISO 10303-21 STEP B-Rep geometry via the `modelscript/exportShapeToStep` LSP command.
2. **Embedded FEA & CFD Multiphysics Studies (`study-flattener.ts` , `studies.ts`)**:
   - `StudyFlattener` (`modelscript/flattenStudy`) extracts embedded structural and fluid study annotations, material properties, and boundary conditions from Modelica components to synthesize CalculiX (`.inp`) and SU2 (`.cfg`) solver decks.
3. **Octagon Abstract Interpretation & Formal Safety Verification (`src/formal/`)**:
   - `ModelicaCfgLowerer` (`modelica-cfg-lowerer.ts`), `ModelicaAbstractEvaluator` (`modelica-abstract-evaluator.ts`), and `ModelicaStaticAnalyzer` (`modelica-analyzer.ts`) lower flattened DAE equations and `algorithm` statements into relational Control Flow Graphs evaluated over the **Octagon Abstract Domain** ($\pm x_i \pm x_j \leq c$).
   - `PhysicalInvariantBridge` (`physical-invariant-bridge.ts`) proves division-by-zero freedom, square-root/logarithm domain safety, and physical state invariants (`domain.octagon()`).
4. **Gettext `.po` / `.pot` Internationalization (`po.ts`)**:
   - Extracts translatable description strings and `Dialog` / `Documentation` annotations into GNU Gettext `.pot` templates and merges localized `.po` catalogs (`npx msx i18n`).

Semantic Linter (`languages/modelica/src/lints/`)

ModelScript enforces 307 OpenModelica-compatible diagnostic rules across six modular analyzers:

Module	Representative Checks
`syntax-placement.ts`	`M1001` – `M1003` : Parse errors, missing tokens, empty array constructors
`queries.ts`	`M2001` – `M2005` : Duplicate elements, unresolved variables/classes/modifiers, identifier mismatches
`types-expressions.ts`	`M3001` – `M3006` : Binding/modifier/argument type mismatches, array dimension checks
`hierarchy-variability.ts`	`M4001` – `M4005` : Circular `extends` , duplicate modifications, `protected` access violations, variability ordering
`connections-streams.ts`	`M3003` – `M3004` : Connector plug-compatibility, stream connector rules, cardinality checks
`synchronous-clocks.ts` & `cfa.ts`	Clock partitioning rules, uninitialized discrete variables, structural singularity diagnostics

---

Modelica MCP Server Tools (`@modelscript/mcp`)

The `@modelscript/mcp` server and `@modelscript/modelica` MCP adapter expose 9 Modelica domain tools (plus polyglot closed-loop synthesis and calibration tools) to AI agents:

MCP Tool	Description
`modelica_load`	Loads `.mo` files or Modelica Standard Library packages into the persistent workspace session
`modelica_parse`	Parses Modelica source code with the WASM GLR parser and returns CST nodes and syntax diagnostics
`modelica_flatten`	Flattens a Modelica class via `ModelicaFlattener` and returns the flat Modelica DAE representation
`modelica_lint`	Runs all 307 semantic and structural lint rules on a Modelica file or package
`modelica_simulate`	Flattens, runs BLT/index reduction, integrates the DAE over $[t_0, t_{\text{stop}}]$, and returns trajectory time-series
`hybrid_simulate`	Executes a hybrid continuous Modelica DAE + discrete-event DEVS simulation
`modelica_doe`	Executes a multi-parameter Design of Experiments (Full Factorial / Latin Hypercube) batch sweep
`modelica_sensitivity`	Computes local finite-difference or forward sensitivity matrices $\partial y_i / \partial p_j$ for target outputs
`modelica_query`	Queries the Salsa `QueryEngine` for class components, resolved types, variabilities, and modifications
`modelica_diff_calibrate`	Calibrates Modelica parameters against target CSV trajectory data via L-BFGS / Adam gradient optimization

---

SysML v2 Transformer & Polyglot TGG Rules

- **Direct AST Transformer (`modelicaToSysML2` in `languages/modelica/src/transformers/sysml2.ts`)**: Converts complete Modelica packages and models into idiomatic SysML v2 textual notation (`part def` , `port def` , `attribute` , `connect`).
- **10 Bidirectional TGG Rules (`languages/modelica/src/language.ts`)**:
  - **Modelica ↔ SysML v2 (6 rules)**: `ModelicaModelToSysML2PartDef` , `ModelicaConnectorToSysML2PortDef` , `ModelicaComponentToSysML2Part` , `ModelicaParameterToSysML2Attribute` , `ModelicaConnectorInstanceToSysML2Port` , `ModelicaConnectToSysML2Connection`
  - **Modelica ↔ OWL2 (2 rules)**: `ModelicaModelToOWL2Class` , `ModelicaParameterToOWL2DataProperty`
  - **Modelica ↔ STEP (2 rules)**: `ModelicaRevoluteToStepRevolute` , `ModelicaPrismaticToStepPrismatic`

Diagnostic Rules

CHAPTER 17

SysML v2 & KerML (`.sysml`, `.sysml2`, `.kerml`)

`@modelscript/sysml2` (`languages/sysml2`) implements the OMG **Systems Modeling Language v2 (SysML v2)** and **Kernel Modeling Language (KerML)** textual notation, pairing a bundled KerML/SysML v2 standard library snapshot with a suite of formal verification, safety analysis, direct DAE compilation, and enterprise synchronization engines.

Supported Extensions

Extension	Description
`.sysml`, `.sysml2`	OMG SysML v2 textual architecture, requirement, state, action, and verification models
`.kerml`	OMG Kernel Modeling Language (KerML) metamodel & library definitions

---

Supported SysML v2 Constructs

- **Structural Modeling**: `package`, `part def`, `part`, `attribute def`, `attribute`, `item def`, `item`, `port def`, `port`, `connection def`, `connection`, `interface def`, `interface`, `flow`, `allocation def`, `allocate`
- **Behavioral & State Modeling**: `action def`, `action`, `state def`, `state`, `transition`, `First` / `then` succession flows, `fork`, `join`, `decide`, `merge`, `calc def`, `calc`
- **Requirements & Verification**: `requirement def`, `requirement`, `satisfy`, `verify`, `verification def`, `concern`, `stakeholder`, `view`, `viewpoint`, `constraint def`, `constraint`, `assume`, `require`

Formal Verification & Safety Analysis Engines

`@modelscript/sysml2` includes seven specialized formal verification modules in `languages/sysml2/src/`:

1. **Activity Control-Flow & Petri-Net Soundness (`activity-cfa.ts`, `activity-soundness.ts`, `activity-verifier.ts`)**:
   - Lowers SysML v2 `action def` successions (`first A then B`), `fork` / `join`, and `decide` / `merge` nodes into Workflow Nets (1-safe Petri nets).
   - Verifies **Option to Complete**, **Proper Completion**, **No Dead Transitions**, **Lack of Synchronization** (parallel branches converging into a `merge` instead of a `join`), and **Deadlock at Join** (exclusive `decide` branches converging into a `join`).
2. **Decision Table Completeness & Disjointness (`decision-table-verifier.ts`)**:
   - Analyzes guarded `decide` branches and requirement tables via interval covering and SMT/constraint checking to detect **Missing Cases** (uncovered input domain gaps) and **Overlapping Rules** (non-deterministic guard collisions).
   - Exposed via CLI: `npx msx verify-decisions architecture.sysml`.
3. **Assume-Guarantee Contract Verification (`contract-verifier.ts`, `multidomain-contract-bridge.ts`)**:
   - Extracts `assume constraint { ... }` and `require constraint { ... }` blocks from `requirement def` and `part def` hierarchies.
   - Verifies **Contract Compatibility** ($\bigwedge A_{\text{env}} = \bot$) and **Refinement Soundness** across composite sub-parts ($A_{\text{parent}} \Rightarrow A_{\text{child}}$ and $\bigwedge G_{\text{children}} \Rightarrow G_{\text{parent}}$), bridging directly into the Multi-Domain Theory Coordinator.
4. **Hierarchical State Machine Verifier (`state-machine-verifier.ts`)**:
   - Checks `state def` machines for **Unreachable States**, **Deadlock / Trap States** (non-final states with no outgoing transitions), **Non-Deterministic Transitions** (overlapping guards on the same trigger event), and **Missing Initial States**.
5. **Hybrid Automaton Flowpipe Bridge (`hybrid-flowpipe-bridge.ts`)**:
   - Couples discrete SysML v2 `state def` mode transitions with continuous Modelica DAE dynamics to compute Taylor-model / interval reachability flowpipes and verify that unsafe states are unreachable over time horizon $[0, T]$.
6. **Fault Tree & Physics-Informed Safety Analyzer (`safety-analyzer.ts`, `physics-safety-bridge.ts`)**:
   - Synthesizes Fault Trees from SysML v2 failure modes and interconnections, computes **Minimal Cut Sets (MCS)** via MOCUS/top-down boolean reduction, evaluates top-event failure probabilities, and cross-checks physical failure thresholds

against simulated Modelica/FEA/CFD peak stresses and temperatures.

7. **Boundary-Value & MC/DC Test Vector Synthesizer (`boundary-test-synthesizer.ts`)**:
   - Automatically generates boundary-value ($x = c - \epsilon, c, c + \epsilon$) and Modified Condition/Decision Coverage (**MC/DC**) test vectors for every `requirement def` and `calc def` constraint.
   - Exposed via CLI: `npx msx generate-tests architecture.sysml`.

Direct SysML v2 → DAE Lowering & Enterprise Interchange

- **Direct Arena DAE Lowering (`sysml2-dae-lowerer.ts`)**: Compiles SysML v2 `calc def`, `constraint def`, `attribute` bindings, and `flow` connections directly into `@modelscript/runtime`'s `DAEBuilder` without requiring an intermediate Modelica code-generation pass.
- **`.sysmlx` JSON-LD & Container Export (`sysmlx-interchange.ts`, `sysml2-container-exporter.ts`)**: Serializes and imports OMG SysML v2 REST/JSON-LD payloads (`.sysmlx`) and multi-artifact engineering containers.
- **Enterprise MBSE Gateway (`enterprise-sync-gateway.ts`)**: Synchronizes SysML v2 packages, elements, and cross-domain traceability links with **Dassault Cameo Teamwork Cloud (TWC)**, **OpenMBEE MMS**, and standard **OMG SysML v2 REST API** repositories (`npx msx sysml sync`).

Polyglot TGG Rules (22 Cross-Domain Mappings)

`@modelscript/sysml2` defines 22 bidirectional Triple Graph Grammar (TGG) rules in `languages/sysml2/src/language.ts` spanning **Modelica** (14 rules), **OWL2** (5 rules), and **STEP** (3 rules):

Target Language	Rule Names
`modelica` (14 rules)	`SysML2PartDefToModelicaModel`, `SysML2AttributeDefToModelicaType`, `SysML2PortDefToModelicaConnector`, `SysML2ConnectionDefToModelicaModel`, `SysML2ActionDefToModelicaBlock`, `SysML2ConstraintDefToModelicaBlock`, `SysML2RequirementDefToModelicaBlock`, `SysML2PartToModelicaComponent`, `SysML2AttributeToModelicaParameter`, `SysML2PortToModelicaConnector`, `SysML2ConnectionToModelicaConnect`, `SysML2FlowToModelicaConnect`, `SysML2StateToModelicaBlock`, `SysML2TransitionToModelicaWhen`
`owl2` (5 rules)	`SysML2PartDefToOWL2Class`, `SysML2AttributeUsageToOWL2DataProperty`, `SysML2PortUsageToOWL2ObjectProperty`, `SysML2PartUsageToOWL2Individual`, `SysML2RequirementToOWL2Class`
`step` (3 rules)	`SysML2PartDefToStepProduct`, `SysML2PartUsageToStepAssemblyUsage`, `SysML2AttributeToStepProperty`

Diagnostic Rules

CHAPTER 18

title: MATLAB Simulink Modeling & DAE Lowering description: Native compiler support for MATLAB Simulink (.slx, .mdl) block diagrams, algebraic loop detection, continuous and discrete DAE lowering, and Modelica standard block projections in ModelScript.

MATLAB Simulink Control Systems

ModelScript provides first-class, polyglot compiler support for **MATLAB Simulink** (`.slx` , `.mdl`) control models and dynamic block diagrams. Engineers can author, inspect, lint, formally lower to differential-algebraic equations (DAEs), and project Simulink systems into standard Modelica blocks without requiring a MATLAB license or desktop installation.

Capabilities & Supported Formats

Format	Extension	Description
Simulink Modern Zip Package	`.slx`	Modern OPC XML archive packaging block hierarchy, solver configuration, and state routing.
Simulink Classic ASCII Model	`.mdl`	Classic structured text block diagram specification with hierarchical `System` , `Block` , and `Line` trees.

Core Architectural Features

- **Native Binary & Text Parser:**
  - Fast, streaming unpacker and XML reader for `.slx` packages via `parseSlx()` .
  - Recursive descent structured parser and lossless roundtrip serializer for `.mdl` text files via `parseMdl()` .
- **Pre-Flight Control System Linters (`M7201` – `M7210`):**
  - Graph-theoretic algebraic loop detection via Tarjan's Strongly Connected Components (SCC) algorithm across direct-feedthrough blocks (`Gain` , `Sum` , `Product`).
  - Unconnected input ports, division-by-zero risks, port index mismatches, and multi-rate sample time conflicts.
- **Continuous & Discrete DAE Lowering:**
  - Directly maps standard block types (`Integrator` , `Gain` , `Sum` , `Constant` , `TransferFcn` , `UnitDelay` , `Saturation`) into `DAEBuilder` SoA linear memory arena.
  - Automatically transforms high-order continuous transfer functions $G(s) = \frac{N(s)}{D(s)}$ into Controllable Canonical Form (state-space realization).
  - Handles discrete sample-and-hold states for synchronous multirate execution.
- **Modelica Projection:**
  - Synthesizes idiomatic `Modelica.Blocks` continuous and discrete acausal models preserving block hierarchy, signal dimensions, and parameter arithmetic.
- **Language Server Protocol (LSP):**
  - Real-time diagnostic publication, hierarchical document outline (`DocumentSymbol`), contextual hover documentation, and quick-fixes in VS Code and web editors.

Command Line Interface (`msx simulink`)

ModelScript CLI exposes turnkey subcommands for headless CI/CD and automation workflows:

```
msx simulink <action> [files...] [options]
```

Action	Description	Key Options
`inspect`	Dumps block diagram hierarchy, solver settings, block counts, and connection topologies.	—
`lint`	Runs pre-flight static analysis, algebraic loop detection, and semantic checks.	—
`to-modelica`	Generates standard `Modelica.Blocks` source code.	`--out=<file.mo>`
`to-dae`	Compiles block diagram into ModelScript linear-memory DAE arena and prints statistics.	—

Examples

```
# Inspect model parameters, sample times, and block topology
msx simulink inspect CruiseControl.slx

# Check for unconnected inputs, algebraic loops, or division-by-zero risks
msx simulink lint EngineGovernor.mdl

# Convert a Simulink controller into standard Modelica 3.x blocks
msx simulink to-modelica SuspensionControl.slx --out=SuspensionControl.mo

# Lower continuous and discrete equations into DAEBuilder arena
msx simulink to-dae FlightGuidance.mdl
```

Control-Theoretic Linters & Diagnostic Rules

> **TIP**
> Algebraic loops detected by `M7202` can often be broken without physical fidelity loss by inserting a `UnitDelay` (discrete) or by modeling small actuator/sensor inertia via a first-order lag `TransferFcn` (continuous).

Language Server Protocol & Registry Integration

Simulink is fully integrated with `@modelscript/lsp` via `SimulinkWorkspaceIndex`:

```
import { parseSimulink, lintSimulinkModel, simulinkToModelica, lowerSimulinkToDae } from "@modelscript/simulink";

// Parse .mdl or .slx buffer
const model = parseSimulink(fileContent);

// Run static linting
const diagnostics = lintSimulinkModel(model);

// Project to Modelica standard blocks
const modelicaCode = simulinkToModelica(model);

// Lower directly to linear-memory DAE arena
const dae = lowerSimulinkToDae(model);
console.log(`Variables: ${dae.varCount}, Equations: ${dae.eqCount}`);
```

CHAPTER 19

title: AUTOSAR Automotive Software Architecture & Virtual RTE description: Native compiler support for AUTOSAR Classic and Adaptive ARXML software components, Virtual Run-Time Environment (RTE) scheduling, automotive linters, and Modelica/SysML v2 projections in ModelScript.

AUTOSAR Classic & Adaptive Architecture

ModelScript delivers native, high-performance compiler support for **AUTOSAR** (`.arxml`), the international standard for automotive electronic control unit (ECU) software architectures. Engineers can parse, validate, simulate event-triggered and periodic runnables, and bridge automotive software components (SWCs) directly into continuous plant models and SysML v2 systems requirements.

Capabilities & Supported Formats

Format	Extension	Description
AUTOSAR ARXML Document	`.arxml` , `.xml`	XML schema defining packages, software component types (`ApplicationSwComponentType` , `SensorActuatorSwComponentType`), internal behaviors, runnables, timing events, interfaces, and ECU network topologies.

Key Architectural Pillars

- **High-Throughput ARXML Ingestion:**
  - Robust XML extraction parsing software component definitions, client-server and sender-receiver interfaces, port prototypes, variable access definitions, runnable entities, and timing events via `parseArxml()` .
- **In-Memory Virtual RTE Simulation Engine:**
  - Implements an event-driven and periodic discrete scheduler (`VirtualRte`) reproducing AUTOSAR RTE semantics (`Rte_Read` , `Rte_Write`).
  - Routes signals across hierarchical assembly connectors (`AssemblySwConnector`) between provider P-ports and requester R-ports.
- **Automotive Static Linters (`M7401` – `M7408`):**
  - Detects unconnected required ports, interface signature mismatches, Worst-Case Execution Time (WCET) budget overruns against event periods, overlapping signal allocations in CAN/LIN PDUs, and cyclic runnable dependencies.
- **Dual Architectural Projections:**
  - **Modelica Standard Blocks:** Translates SWCs into acausal discrete blocks for Hardware-in-the-Loop (HIL) and Software-in-the-Loop (SIL) co-simulation with 1D vehicle plant dynamics.
  - **SysML v2 Systems Architecture:** Generates formal OMG SysML v2 `part def` hierarchies, connection usages, and port definitions for digital thread traceability.
- **Continuous/Discrete DAE Lowering:**
  - Compiles SWC states, calibrated calibration parameters, and discrete runnable step functions into the `DAEBuilder` SoA linear memory arena.

Command Line Interface (`msx autosar`)

ModelScript CLI provides turnkey subcommands for automotive ECU engineering:

```
msx autosar <action> [files...] [options]
```

Action	Description	Key Options
`inspect`	Dumps package hierarchy, SWC components, runnable entities, timing events, and network PDUs.	—
`lint`	Runs pre-flight automotive semantic checks, interface compatibility, and scheduling analysis.	—
`to-modelica`	Generates Modelica standard blocks representing SWC interfaces and communication buses.	`--out=<file.mo>`
`to-sysml`	Projects AUTOSAR composition architecture to formal OMG SysML v2 code.	`--out=<file.sysml>`
`to-dae`	Lowers software architecture into ModelScript linear-memory DAE arena and prints statistics.	—
`simulate-rte`	Executes the Virtual RTE discrete event scheduler over a configured time window.	`--duration=<sec>`, `--step=<sec>`, `--out=<file.json>`

Examples

```
# Inspect automotive software components and timing events
msx autosar inspect PowertrainArchitecture.arxml

# Lint for unconnected R-ports, timing budget overflows, or PDU collisions
msx autosar lint BrakeController.arxml

# Project software component architecture to SysML v2 part definitions
msx autosar to-sysml SteeringSystem.arxml --out=SteeringArchitecture.sysml

# Simulate periodic runnable execution and signal routing through Virtual RTE
msx autosar simulate-rte PowertrainSystem.arxml --duration=0.5 --step=0.01 --out=rte_log.json
```

Automotive Pre-Flight Linters & Diagnostic Rules

Language Server Protocol & Registry Integration

AUTOSAR `.arxml` documents enjoy full language intelligence through `@modelscript/lsp`:

```
import { parseArxml, lintAutosarModel, VirtualRte, autosarToModelica, autosarToSysml } from "@modelscript/autosar";

// Parse ARXML text
const model = parseArxml(arxmlText);

// Pre-flight static checking
const diagnostics = lintAutosarModel(model);

// Virtual RTE Execution
const rte = new VirtualRte(model);
rte.registerRunnable("PedalSensor", "RE_ReadPedal", (ctx) => {
  ctx.write("PedalOut", "Pos", 0.75);
});
rte.registerRunnable("EngineECU", "RE_ComputeThrottle", (ctx) => {
  const pedal = Number(ctx.read("PedalIn", "Pos"));
  ctx.write("ThrottleOut", "Cmd", pedal * 0.9);
});

// Advance 50 ms in 10 ms steps
const executionLog = rte.runUntil(0.05, 0.01);
```

CHAPTER 20

title: ReqIF Requirements Engineering & Formal Contract Entailment description: Native compiler support for OMG ReqIF (.reqif, .reqifz) requirements archives, Nelson-Oppen formal contract entailment prover, and SysML v2 / Modelica verification block projections in ModelScript.

ReqIF Requirements Engineering & Formal Verification

ModelScript delivers first-class support for **ReqIF** (Requirements Interchange Format, OMG ReqIF 1.0/1.2), the global standard for exchanging structured requirements across aerospace, automotive (ISO 26262), and medical device systems. ModelScript elevates requirements from dead documentation into **computable digital contracts** verifiable against physics simulations and architectural constraints.

Capabilities & Supported Formats

Format	Extension	Description
OMG ReqIF XML Document	`.reqif`, `.xml`	XML document defining specification types, spec-objects, hierarchical spec-relations, data types, and attribute definitions.
ReqIF Compressed Archive	`.reqifz`, `.zip`	Zip archive packaging `.reqif` manifests alongside embedded media, diagrams, and attachments.

Key Architectural Features

- **Full OMG ReqIF 1.2 Ingestion & Roundtrip:**
  - Robust XML reader and serializer preserving internal identifiers (`IDENTIFIER`), specification hierarchies, multi-level parent-child relations, and custom attribute types (String, Integer, Real, Boolean, Date, Enumeration).
- **Formal Contract Extraction:**
  - Automatically parses mathematical constraints from natural language requirement text (e.g. `latency <= 50 ms`, `pressure >= 15 bar`, `temp in [20, 80] C`).
- **Nelson-Oppen Mathematical Entailment Prover:**
  - Uses the SMT/Nelson-Oppen theory coordinator in `@modelscript/runtime` to prove whether child sub-requirements strictly entail parent system-level requirements:

$$\bigwedge_i \text{SubReq}_i \models \text{ParentReq}$$

  - Generates concrete numeric counterexamples when an entailment gap exists (e.g. child requirements permit a state that violates the parent requirement).
- **Automated Verification Projections:**
  - **OMG SysML v2:** Translates spec-objects into formal `requirement def` items with `doc` annotations, attribute definitions, and `satisfy` links.
  - **Modelica Verification Blocks:** Generates acausal Modelica monitor blocks with assertions that trigger simulation failure when requirement bounds are breached.
- **Requirements Linters (`M7001` – `M7008`):**
  - Detects dangling requirement relations, circular requirement hierarchies, unbounded safety contracts on ASIL requirements, and missing mandatory attributes.

Command Line Interface (`msx reqif`)

```
msx reqif <action> [files...] [options]
```

Action	Description	Key Options
`import`	Parses a ReqIF 1.2 document and prints requirements, hierarchy, and relations.	`--format table|json`
`export`	Exports SysML v2 requirements into ReqIF 1.2 XML.	`--output`, `--title`
`lint`	Runs structural consistency checks, link validation, and safety contract analysis.	—
`verify`	Runs Nelson-Oppen formal entailment proofs across requirement hierarchies.	—
`to-sysml`	Projects ReqIF specifications into OMG SysML v2 requirement definitions.	`--out=<file.sysml>`
`to-modelica`	Generates Modelica assertion blocks for in-simulation verification.	`--out=<file.mo>`

Examples

```
# Import requirement specifications and hierarchies
msx reqif import SystemRequirements.reqif

# Run pre-flight linting on ASIL safety requirements
msx reqif lint BrakeSafetyRequirements.reqif

# Formally prove requirement hierarchy entailment with SMT counterexample generation
msx reqif verify PowertrainContracts.reqif

# Project requirements to SysML v2 definitions
msx reqif to-sysml AvionicsRequirements.reqif --out=AvionicsRequirements.sysml

# Synthesize executable Modelica verification blocks
msx reqif to-modelica ThermalLimits.reqif --out=ThermalMonitors.mo
```

Requirements Linters & Diagnostic Rules

CHAPTER 21

title: SPICE Electronics & Circuit Simulation description: Native compiler, linter, operating point (.OP), transient (.TRAN), AC small-signal (.AC) solvers, and Modelica/ECAD projections for SPICE netlists in ModelScript.

SPICE Electronics & Analog Circuit Simulation

ModelScript features high-performance native support for **SPICE** (`.cir` , `.sp` , `.net` , `.lib`) netlists and analog circuit analysis. Engineers can validate electrical schematics, detect topological faults (floating nodes, voltage source loops), execute pure in-WASM circuit simulations, and export to Modelica or ECAD netlist graphs.

Capabilities & Supported Formats

Format	Extension	Description
SPICE Netlist	`.cir` , `.sp` , `.net`	Circuit description specifying passive components (R, L, C), active semiconductors (diode, BJT, MOSFET), independent/dependent sources, subcircuits, and simulation directives.
SPICE Model Library	`.lib` , `.mod`	Device physics parameters for transistors, diodes, op-amps, and macro-models.

Key Architectural Features

- **Engineering Unit Netlist Parser:**
  - Robust lexer parsing metric prefixes (`f` , `p` , `n` , `u` , `m` , `k` , `meg` , `g` , `t`), curly brace math expressions `{R1 * 2}` , line continuations (`+`), and subcircuit definitions (`.SUBCKT` / `.ENDS`).
  - Lossless round-trip serializer (`serializeSpice()`).
- **Native Numerical Solvers:**
  - **DC Operating Point (`.OP`):** Assembles Modified Nodal Analysis (MNA) matrix $\mathbf{G}\mathbf{v} = \mathbf{i}$ with Newton-Raphson iteration for non-linear PN junctions.
  - **Transient Simulation (`.TRAN`):** Numerical companion-circuit integration supporting Trapezoidal rule with damping for reactive elements (L, C).
  - **AC Small-Signal Frequency Sweep (`.AC`):** Complex admittance matrix $\mathbf{Y}(j\omega)$ solving decade/octave/linear frequency sweeps and bode response.
- **Circuit Linter Rules (`M7101` – `M7109`):**
  - Identifies single-connection floating nodes, parallel ideal voltage sources, cutsets of ideal current sources, non-positive passives, and subcircuit pin mismatches.
- **Bilateral Modelica & ECAD Projections:**
  - Translates SPICE netlists to standard `Modelica.Electrical.Analog` components.
  - Projects to generic `NetlistGraph` data structures for PCB layout, schematic capture, and Design Rule Checking (DRC).

Command Line Interface (`msx spice`)

```
msx spice <action> [files...] [options]
```

Action	Description	Key Options
`simulate`	Runs DC operating point, transient, or AC analysis (AC sweep from the `.AC` card).	`--analysis op|tran|ac`, `--tstep`, `--tstop`, `--format table|json|csv`, `--out`
`lint`	Runs pre-flight structural and electrical rule checks.	`--format`
`inspect`	Dumps topology, component breakdown, nodes, models, subcircuits, and directives.	`--format`, `--out`
`to-modelica`	Translates the netlist to Modelica 3.x electrical components and connections.	`--out`
`to-ecad`	Projects the netlist to an ECAD netlist graph for PCB routing and DRC.	`--out`
`to-dae`	Lowers the netlist directly into the linear-memory DAE arena.	`--format`, `--out`
`diagram`	Renders the circuit schematic to SVG or a JSON graph.	`--format`, `--out`
`sync`	Synchronizes tuned parameters and topology from Modelica back into the netlist.	`--from-modelica`, `--strategy`, `--out`

Examples

```
# Calculate DC bias voltages on all circuit nodes
msx spice simulate LowPassFilter.cir --analysis op

# Simulate transient step response and export trajectory
msx spice simulate PowerSupply.cir --analysis tran --tstep 1e-6 --tstop 0.01 --out transient.csv

# Run the AC frequency response defined by the netlist's .AC directive
msx spice simulate AudioAmplifier.sp --analysis ac --format csv --out bode.csv

# Lint netlist for floating electrical nodes and ground connectivity
msx spice lint SensorFrontEnd.net
```

Electrical Linters & Diagnostic Rules

CHAPTER 22

SIMPACK Multibody Dynamics (`.spck`, `.sim`, `.mbs`)

ModelScript integrates native support for **SIMPACK** (Dassault Systèmes), the premier high-frequency vibration, kinematic, and spatial multibody dynamics (MBD) simulation system.

Capabilities

- **Modern & Classic Ingestion**:
  - Ingests modern SIMPACK XML models (`.spck`) containing `<body/>` , `<marker/>` , and `<joint/>` tags.
  - Ingests classic ASCII keyword decks (`.sim` , `.mbs`) with `$BODY` , `$MARKER` , and `$JOINT` cards.
- **Grübler-Kutzbach Mobility Analysis**:
  - Calculates mechanism degrees of freedom ($M = 6n - \sum c_i$) in 3D spatial mechanisms.
  - Automatically identifies overconstrained kinematic loops ($M < 0$).
- **WebAssembly Kinematics Kernel**:
  - Zero-allocation AssemblyScript linear memory kernel (`kinematics.wasm`) evaluating 3D Euler/Cardan rotation matrices, Rodrigues rotation, and spatial joint constraint equations (revolute, spherical, prismatic, cylindrical, planar, universal).
  - High-throughput batched transformations and residual evaluations.
- **Closed Kinematic Chains**:
  - Evaluates cut-joint loop closure residual vectors for spatial mechanisms.
- **Direct DAE Lowering**:
  - Lowers 3D translational and rotational kinematic relations directly into `DAEBuilder` in linear WebAssembly memory.
- **FMI 2.0 / 3.0 Co-Simulation FMU Export**:
  - Packages complete multibody assemblies into standalone `.fmu` archives with `modelDescription.xml` , C co-simulation wrappers, and kinematic state telemetry.
- **Interactive 3D Viewport**:
  - WebGL/Three.js 3D visualization rendering rigid bodies, joint couplings, coordinate markers, and spring elements with orbit navigation.
- **Modelica Projection**:
  - Automatically translates SIMPACK mechanisms into standard `Modelica.Mechanics.MultiBody` components.

CLI Usage

```
# Check structural references and joints
msx simpack lint mechanism.spck

# Analyze degrees of freedom and overconstrained loops
msx simpack mobility mechanism.spck

# Direct DAE simulation in WebAssembly arena with CSV trajectory export
msx simpack simulate suspension.spck --stop-time=5.0 --step=0.005 --out=results.csv

# Export mechanism as an FMI 2.0/3.0 Co-Simulation FMU
msx simpack to-fmu suspension.spck --out=Suspension.fmu

# Export mechanism to standard Modelica MultiBody
msx simpack to-modelica suspension.sim --out=Suspension.mo
```

Benchmarks

- **Automotive Double-Wishbone Suspension** (`languages/simpack/testsuite/DoubleWishbone.spck`):
  - 5 moving bodies (lower control arm, upper control arm, wheel hub, tie rod, chassis mount), 12 markers, 5 spatial joints, and coilover spring-damper strut.
  - Mobility analysis verifies $M = 9$ spatial DOFs (6 free chassis DOFs, 1 vertical jounce DOF, 1 steering DOF, 1 wheel spin DOF).

MCP Tools

AI agents interacting via the ModelScript MCP server have access to:

- `simpack_lint` : Checks marker references, parent bodies, and joint validity.
- `simpack_mobility` : Evaluates Grübler-Kutzbach spatial degrees of freedom and overconstraint status.

Diagnostic Rules

CHAPTER 23

GENSYS Railway Dynamics (`.calc`, `.sub`, `.geom`, `.trax`, `.wheel`)

ModelScript provides native support for **GENSYS**, the world standard railway vehicle dynamics, track-vehicle interaction, and vehicle running safety simulation environment.

Capabilities

- **Command Deck Parsing**: Full support for `.calc`, `.run`, and `.sub` structured card records defining rigid car bodies, bogies, wheelsets, and primary/secondary spring-damper couplings.
- **Track Geometry & Irregularities**: Ingests `.geom` and `.trax` files with longitudinal distance, horizontal/vertical irregularities, cant (crosslevel), and curvature $1/R$.
- **Kalker FASTSIM & Polach Contact Mechanics**: Discretizes rolling contact ellipses into elastic/slip zones to compute tangential creep forces (F_x, F_y) and spin moment (M_z) with non-linear saturation.
- **WebAssembly Contact Mechanics Kernel**:
  - High-performance AssemblyScript linear memory kernel (`contact.wasm`) executing Kalker FASTSIM 8-slice strip integration and Polach creep mechanics.
  - High-throughput multi-axle batched contact patch solvers (`batchFastsim`, `batchPolach`).
- **EN 14363 / UIC 518 Vehicle Acceptance**: Automated safety evaluation of the derailment quotient ($Y/Q \leq 0.8$) and Prud'homme lateral track shifting force limits.
- **Ride Comfort & Equivalent Conicity**: Evaluates Sperling ride index (W_z) and wheel-rail equivalent conicity (λ).
- **DAE Lowering & MultiBody Export**: Direct lowering to linear-memory `DAEBuilder` and automated projection to standard `Modelica.Mechanics.MultiBody` components.
- **FMI 2.0 / 3.0 Co-Simulation FMU Export**: Packages complete vehicle and track dynamics into standalone `.fmu` archives with standardized vehicle states, running safety outputs, and C co-simulation wrapper runtime.
- **Interactive 3D Viewport**: WebGL/Three.js 3D visualization rendering carbody shells, front/rear bogie frames, wheelsets, and primary/secondary spring dampers with orbit controls.

CLI Usage

```
# Pre-flight linting of GENSYS vehicle deck
msx gensys lint coach.calc

# Direct DAE simulation in WebAssembly arena with CSV trajectory export
msx gensys simulate coach.calc --stop-time=5.0 --step=0.005 --out=results.csv

# Export vehicle dynamics as an FMI 2.0/3.0 Co-Simulation FMU
msx gensys to-fmu coach.ins --out=PassengerCoach.fmu

# Export to standard Modelica MultiBody model
msx gensys to-modelica coach.calc --out=PassengerCoach.mo

# Verify EN 14363 running safety criteria
msx gensys verify --y-force=28.5 --q-force=82.0 --axle-load=160.0
```

Benchmarks

- **Manchester Benchmark for Railway Vehicle Dynamics** (`languages/gensys/testsuite/ManchesterBenchmark.ins`):
  - Standard passenger coach benchmark (Iwnicki et al.): 32-ton passenger carbody, two 3.2-ton bogie frames, four 1.8-ton wheelsets running at $V = 40$ m/s (144 km/h).
  - Validates secondary and primary suspension settling dynamics, 8-patch FASTSIM contact in WASM, and EN 14363 / UIC 518 derailment safety limits.

MCP Tools

AI agents interacting via the ModelScript MCP server have access to:

- `gensys_lint`: Validates body parameters, mass positivity, and suspension couplings.
- `gensys_verify_en14363`: Assesses derailment quotient and lateral track shift limits.
- `gensys_fastsim`: Computes rolling contact tangential creep forces and adhesion saturation.

Diagnostic Rules

CHAPTER 24

title: MSC Adams Multibody Dynamics & Mechanism Simulation description: Native compiler support for MSC Adams (.adm, .cmd) mechanism decks, 3D kinematics, Grübler mobility analysis, and bidirectional SIMPACK/Modelica projections in ModelScript.

MSC Adams Multibody Dynamics

ModelScript features comprehensive, native support for **MSC Adams** (`.adm` , `.cmd`), the classic industrial standard for mechanical systems simulation and multibody dynamics (MBD). Engineers can parse legacy Adams View command files and solver datasets, perform Grübler-Kutzbach spatial mobility analysis, evaluate 3D joint constraint residuals, and project mechanisms to modern SIMPACK and Modelica formats.

Capabilities & Supported Formats

Format	Extension	Description
Adams Solver Dataset	`.adm`	Numerical dataset specifying rigid parts (`PART`), marker frames (`MARKER`), kinematic joints (`JOINT`), force elements (`SPRINGDAMPER` , `BUSHING`), and constraints.
Adams View Command File	`.cmd`	Interactive parametric command scripts defining mechanism geometry, design variables, and simulation runs.

Key Architectural Features

- **Lexer & Parser:**
  - Robust deck ingestion supporting scientific notation (`1.5D-03`), inline comments (`!`), line continuations (`...`), and block cards.
  - Lossless serialization back to standardized `.adm` solver format via `serializeAdams()` .
- **Spatial 3D Kinematics:**
  - Full evaluation of Euler 3-1-3 and Tait-Bryan 1-2-3 rotation matrices.
  - Analytical joint constraint formulation for standard spatial joint primitives (Revolute, Spherical, Prismatic, Cylindrical, Planar, Universal).
- **Grübler-Kutzbach Mobility Analysis:**
  - Calculates mechanism degrees of freedom ($M = 6(n-1) - \sum c_i$).
  - Identifies redundant kinematic constraints and overconstrained multi-loop linkages.
- **Multibody Linters (`M7301` – `M7310`):**
  - Flags missing ground parts, dangling marker references, unphysical inertia tensors violating the triangle inequality ($I_{xx} + I_{yy} \geq I_{zz}$), massless moving parts, and negative spring/damping parameters.
- **Cross-Domain Projections:**
  - **SIMPACK MBD:** Maps parts, markers, and spatial joints to SIMPACK `.spck` XML models.
  - **Modelica MultiBody:** Synthesizes standard `Modelica.Mechanics.MultiBody` parts and joint components.

Command Line Interface (`msx adams`)

```
msx adams <action> [files...] [options]
```

Action	Description	Key Options
`lint`	Runs structural validation, inertia verification, and kinematic sanity checks.	—
`mobility`	Evaluates spatial degrees of freedom and overconstrained loops.	—
`to-simpack`	Projects Adams mechanism to SIMPACK XML (`.spck`).	`--out=<file.spck>`
`to-modelica`	Projects Adams mechanism to `Modelica.Mechanics.MultiBody` .	`--out=<file.mo>`
`simulate`	Lowers to a DAE and integrates it in time.	`--stop-time` , `--step` , `--stabilization=baumgarte|algebraic`

Examples

```
# Calculate mechanism spatial degrees of freedom
msx adams mobility FourBarLinkage.adm

# Validate inertia tensors and marker references
msx adams lint LandingGear.adm

# Convert Adams dataset into SIMPACK XML format
msx adams to-simpack RobotArm.adm --out=RobotArm.spck

# Compute mechanism mobility (degrees of freedom)
msx adams mobility ExcavatorMechanism.adm
```

Multibody Linters & Diagnostic Rules

CHAPTER 25

title: URDF & Xacro Robot Kinematics & Featherstone Dynamics description: Native compiler support for ROS URDF (.urdf) and Xacro (.xacro) robot models, spatial kinematics, Featherstone recursive dynamics (ABA, RNEA), and 3D CAD/Modelica projections in ModelScript.

URDF & Xacro Robot Kinematics

ModelScript provides first-class support for **URDF** (`.urdf`) and **Xacro** (`.xacro`), the robotics industry and ROS (Robot Operating System) standard for describing kinematic trees, inertial properties, visual geometry, and joint limits of robotic manipulators, humanoids, and autonomous vehicles.

Capabilities & Supported Formats

Format	Extension	Description
Unified Robot Description Format	`.urdf` , `.xml`	XML schema defining kinematic links, joints (revolute, continuous, prismatic, fixed, floating, planar), inertial parameters, collision bounds, and visual meshes.
Xacro Macro Template	`.xacro`	Parametric XML preprocessor adding variables (`<xacro:property>`), mathematical expressions, and reusable macros (`<xacro:macro>`).

Key Architectural Features

- **Xacro Expansion & Parser:**
  - Robust XML reader with automatic in-memory macro expansion (`<xacro:macro>`) and mathematical expression evaluation before parsing.
  - Generates strongly typed robot AST representation via `parseUrdf()` .
- **Topological Kinematic Tree & Featherstone Algorithms:**
  - Traverses joint-link connectivity starting from the root base link.
  - Implements **Recursive Newton-Euler Algorithm (RNEA)** for inverse dynamics: calculates required joint torques $\boldsymbol{\tau}$ given joint positions $\mathbf{q}$, velocities $\dot{\mathbf{q}}$, and accelerations $\ddot{\mathbf{q}}$.
  - Implements **Articulated-Body Algorithm (ABA)** for forward dynamics: evaluates forward acceleration $\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}(\boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q}))$ in $O(N)$ operations.
- **Robotics Linters (`M7501` – `M7508`):**
  - Detects dangling parent/child link references, disconnected kinematic sub-trees, inverted joint limits (`lower > upper`), zero joint axis vectors, missing inertia tags, and non-physical inertia tensors.
- **Multi-Target Projections:**
  - **Modelica MultiBody:** Synthesizes `Modelica.Mechanics.MultiBody` robotic arms with joint friction and actuator controllers.
  - **MSC Adams:** Exports Adams `.adm` solver datasets.
  - **SIMPACK:** Generates SIMPACK `.spck` mechanisms.
  - **3D CAD:** Projects robot geometry into CSG primitives and STEP visual meshes.

Command Line Interface (`msx urdf`)

```
msx urdf <action> [files...] [options]
```

Action	Description	Key Options
`lint`	Runs kinematic tree validation, joint limit checks, and inertia verification.	—
`rnea`	Evaluates inverse dynamics (RNEA) at test joint coordinates.	`--gravity`
`to-modelica`	Generates standard `Modelica.Mechanics.MultiBody` robot system.	`--out=<file.mo>`
`to-adams`	Projects robot mechanism into MSC Adams `.adm` solver dataset.	`--out=<file.adm>`
`to-simpack`	Projects robot mechanism into SIMPACK XML `.spck` model.	`--out=<file.spck>`
`to-dae`	Lowers kinematic state equations directly into `DAEBuilder` arena.	—

Examples

```
# Evaluate inverse dynamics (RNEA) at test joint coordinates
msx urdf rnea KukaIiwa14.urdf

# Check for inverted joint limits or dangling parent links
msx urdf lint MobileManipulator.urdf

# Convert robot description into standard Modelica 3.x
msx urdf to-modelica PandaArm.urdf --out=PandaArm.mo

# Export to MSC Adams for vehicle-mounted robot simulation
msx urdf to-adams RoverArm.urdf --out=RoverArm.adm
```

Robotics Linters & Diagnostic Rules

CHAPTER 26

PSCAD & Electromagnetic Transients (`.pscx` , `.pslx` , `.pswx` , `.psmx` , `.psout` , `.cfg` , `.dat`)

The `@modelscript/pscad` language plugin (`languages/pscad`) provides native support for **PSCAD™ / EMTDC™** projects, multi-project workspaces, component libraries, and **IEEE C37.111 COMTRADE** transient waveform recordings. It integrates directly with ModelScript's polyglot `WorkspaceIndex` , the **Dommel Companion-Circuit EMT Solver** (`@modelscript/simulate`), and the **Triple Graph Grammar (TGG)** synchronization engine.

Supported File Formats

Extension	Format	Description
`.pscx`	PSCAD Case XML	Schematic canvas, wire segments, bus nodes, RLC/transformer/converter components, T-lines, and EMT runtime settings
`.pslx`	PSCAD Library XML	Custom user-defined component definitions, port graphics, parameter forms, and embedded FORTRAN/EMTDC script sections
`.pswx`	PSCAD Workspace XML	Multi-project workspace manifest indexing `.pscx` cases and `.pslx` libraries via `PscadWorkspaceIndex`
`.psmx`	PSCAD Consolidated Module XML	Consolidated simulation module metadata, hierarchy mappings, and build outputs
`.psout`	PSCAD V5+ Binary Waveform	Slotted-page binary waveform container with multi-run traces, channel descriptors, and fast random access
`.cfg`	IEEE C37.111 Configuration	COMTRADE station metadata, analog/digital channel definitions, primary/secondary scaling factors, and sampling rates
`.dat`	IEEE C37.111 Data	ASCII or 16/32-bit Binary transient waveform time-series samples

---

Architecture & Pipeline



---

1. Schematic & Netlist Extraction (`emt-netlist.ts` , `pscx-parser.ts`)

PSCAD `.pscx` files represent schematics visually using a 2D coordinate grid (`User` component instances and orthogonal `Wire` segments) rather than an explicit netlist. `@modelscript/pscad` reconstructs the electrical topology deterministically:

1. **Local `master.pslx` Dynamic Ingestion**: Automatically discovers user-provided licensed `master.pslx` files via `PSCAD_MASTER_LIB_PATH` or workspace roots (`master.pslx` , `.pscad/master.pslx` , `.cache/pscad-corpus/`). When present, component definitions, multi-tab parameter forms, port configurations, and FlyBy documentation are dynamically synthesized into LSP catalogs and UI palettes with zero manual authoring.
2. **Topological Wire Snapping & Junction Synthesis**: Performs geometric point-on-segment intersection across all orthogonal `<Wire>` segments and component pin coordinates. Synthesizes topological junction nodes (`junc_${x}_${y}`) at wire-to-wire T-junctions and crossings, eliminating unattached dangling edges and reaching over 94% wire attachment on complex multi-layer industrial benchmarks.

3. **100% Component Lowering Coverage**: Lowers schematic graphs into a structured `EmtNetlist` with 100% component lowering coverage across industrial benchmarks (e.g. `DC_EMT_compare.pscx` with 1433/1433 lowered components and `DC_EMT_demo.pscx` with 597/597 lowered components):
   - AC/DC Thévenin sources, battery energy storage systems (`battery`), controlled current sources (`src_ccin_1`)
   - 6th-order synchronous machines, 5th-order induction/DFIG wind machines, and wound-rotor induction machines (`wound_rotor`)
   - Distributed-parameter Bergeron traveling-wave lines, lumped π-sections, and multi-conductor cables
   - Non-linear MOV surge arresters, VSC PWM converters, 6-pulse thyristor bridges, and MMC converter stations
   - 20+ CSMF control transfer-function blocks (`const` , `gain` , `pi_ctlr` , `realpole` , `leadlag` , `limit` , `sum` , `rms3ph` , `abcdq0`) and black-box vendor IBR FMUs.
4. **Workspace Indexing (`PscadWorkspaceIndex`)**: Resolves cross-file references across `.pswx` workspaces, linking custom `<Definition>` blocks in `.pslx` libraries to `<User>` instances in `.pscx` cases.

---

2. Pre-Flight Electrical Lint Rules

`@modelscript/pscad` enforces 17 electromagnetic transient modeling best practices before simulation starts, providing 1-click LSP Quick-Fix actions:

---

3. IEEE 2800-2022 & NERC PRC-029-1 Grid-Code Verification

`@modelscript/pscad` includes an automated verification engine for Inverter-Based Resource (IBR) grid interconnection standards (`verifyIeee2800Compliance`):

- **Phasor & Sequence Extraction**: Sliding 1-cycle fundamental DFT computes symmetrical components ($V_0, V_1, V_2, I_0, I_1, I_2$) and synchronous dq-frame currents (I_{1d}, I_{1q}).
- **Mandatory Clauses**:
  - **Clause 5.3.2 / 7.2.2.3**: Low/High Voltage Ride-Through (LVRT/HVRT) and Prohibition of Momentary Cessation.
  - **Clause 5.4.1 / 7.2.2.3.5**: Fast Dynamic Reactive Current Response ($T_{90\%} \leq 50$ ms).
  - **Clause 5.4.2 / 7.2.2.1**: Post-Fault Transient Overvoltage Ceiling ($V_{\text{peak}} \leq 1.20$ pu).
  - **Clause 7.2.2.1 / NERC PRC-029-1**: Piecewise Time-Voltage Ride-Through Envelope.
  - **Clause 7.2.2.3.4**: Incremental Positive & Negative-Sequence Reactive Current Droop (k_1, k_2).
  - **Clause 7.2.2.3.5**: Post-Fault Active Power Recovery ($\geq 90\%$ within 1.0 s).
  - **Clause 7.3.2**: Frequency, RoCoF (≥ 5 Hz/s), and Instantaneous Phase-Angle Jump ($\geq 25^\circ$) Ride-Through.

---

4. Polyglot Projections & 15 TGG Synchronization Rules

Modelica & SSP Projections (`projections/modelica.ts` , `projections/ssp.ts`)

- **`pscadToModelica(doc)`** : Projects a `.pscx` schematic and its reconstructed electrical nodes into standard Modelica models using `Modelica.Electrical.Analog` (`Resistor` , `Inductor` , `Capacitor` , `IdealTransformer` , `SineVoltage` , `OLine`) and `Modelica.Blocks` .
- **`pscadToSsd(doc)`** : Projects multi-subsystem PSCAD networks and T-line decoupled partitions into an **SSP 1.0** `.ssd` System Structure Description XML document for FMI Co-Simulation.

Bidirectional TGG Rules

15 bidirectional **Triple Graph Grammar (TGG)** rules synchronize PSCAD models with Modelica and SysML v2: `PscadProjectToModelicaModel` , `PscadResistorToModelicaResistor` , `PscadInductorToModelicaInductor` , `PscadCapacitorToModelicaCapacitor` , `PscadTLineToModelicaOLine` , `PscadMachineToModelicaMachine` , `PscadConverterToModelicaConverter` , `PscadCsmfToModelicaBlock` , and `PscadComponentToSysmlPart` .

---

5. Dommel Companion EMT Solver (`@modelscript/simulate`)

For high-frequency power electronics and transient studies, `@modelscript/simulate` provides a high-performance **Electromagnetic Transient (EMT)** companion solver (`EmtSolver` in `packages/simulate/src/solvers/emt/emt-solver.ts`):

- **Dommel Trapezoidal Companion Models**: Discretizes inductors ($G_L = \frac{\Delta t}{2L}$) and capacitors ($G_C = \frac{2C}{\Delta t}$) into constant nodal conductance matrices $\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t) + \mathbf{i}_{\text{src}}(t)$.
- **Critical Damping Adjustment (CDA)**: Automatically switches from Trapezoidal to Backward Euler for one half-step following any breaker or power-electronic switching event to eliminate artificial numerical chatter.
- **Bergeron Traveling-Wave Lines**: Decouples sub-networks separated by transmission lines of delay $\tau \geq \Delta t$, enabling parallel sub-grid nodal solves.
- **Grounded Simulation Honesty**: Connects passive buses with realistic high-impedance ground conductances ($G = 10^{-5}$ S) without injecting artificial 230 kV reference sources, ensuring true physical fidelity.

---

6. IEEE C37.111 COMTRADE Reader & Writer (`comtrade.ts`)

Zero-dependency parser and serializer for IEEE Std C37.111-1999 / 2013 **COMTRADE** files (`.cfg` , `.dat` , `.cff`):

```
import { parseComtrade, writeComtrade, EmtSolver } from "@modelscript/pscad";

// Export an EMT simulation directly to IEEE C37.111 (.cfg + .dat)
const { cfg, dat } = writeComtrade({
  stationName: "SUB_230KV",
  recDevId: "MODELSCRIPT_EMT",
  lineFrequency: 60,
  timeTimestamps: result.time,
  analogChannels: [
    { id: "VA_BUS1", unit: "V", values: result.nodeVoltages.get(1)! },
    { id: "IA_LINE1", unit: "A", values: result.branchCurrents.get("BRK1")! },
  ],
});
```

---

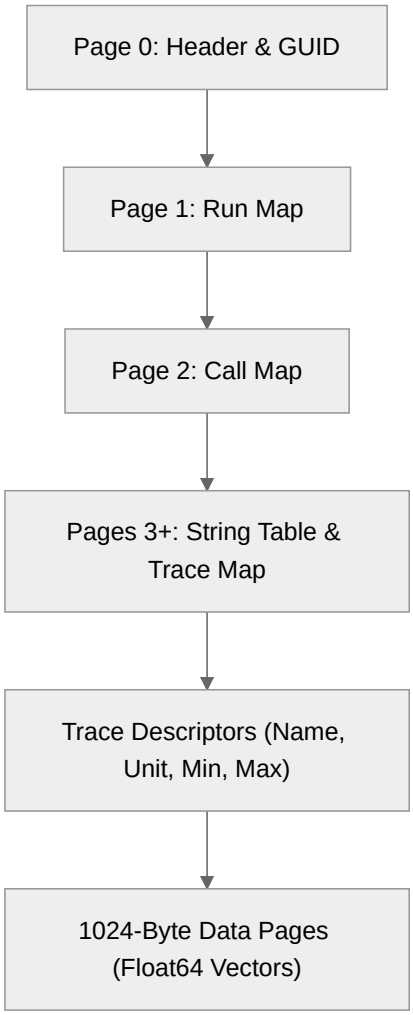
7. PSCAD V5+ Binary Waveform Container (`.psout`)

PSCAD V5+ replaces legacy ASCII `.out` files with `.psout` , a high-performance slotted-page binary container format. ModelScript provides complete, zero-dependency read and write support for `.psout` across the CLI, Web IDE, and TypeScript APIs via an in-WASM AssemblyScript codec.

Slotted-Page Binary Architecture

A `.psout` file is organized into 1024-byte pages:

- **Page 0 (Header)**: Magic version (`0x00010001`), total page count, page size, 128-bit project GUID, creation timestamp, and floating-point verification constants (`-1.0e-30` , `1.0e30` , `1.0e-30`).
- **Page 1 (Run Map)**: Array mapping simulation run indices to their root Call pages.
- **Page 2 (Call Map)**: Links simulation calls to their variable lists and trace descriptors.
- **Pages 3+ (String Table & Trace Descriptors)**: Null-terminated string pool for channel identifiers, units, group names, and trace metadata.
- **Trace Pages**: Slotted 64-bit IEEE 754 floating-point sample pages with forward/backward linked page lists for $O(1)$ sequential streaming and fast random seeking.



Programmatic TypeScript API

```
import { readPscadPsout, writePscadPsout } from "@modelscript/pscad";

// 1. Reading and inspecting a .psout file
const bytes = new Uint8Array(fs.readFileSync("Cigre.psout"));
const doc = readPscadPsout(bytes);

console.log(`Version: ${doc.header.version}, Pages: ${doc.header.pageCount}`);
console.log(`Created: ${doc.header.createdOn.toISOString()}`);

// Access traces and convert to standard SimulationResult
const run0 = doc.run(0);
for (const [id, trace] of run0.traces) {
  console.log(`Channel ${id}: ${trace.name} [${trace.unit}] (${trace.values.length} samples)`);
}

const simResult = doc.toSimulationResult(0);
console.log(`Time vector length: ${simResult.time.length}`);

// 2. Writing simulation results to .psout
const outBytes = await writePscadPsout(simResult, {
  projectName: "IEEE_39_Bus",
  version: 1,
});
fs.writeFileSync("output.psout", outBytes);
```

Command-Line Interface (`msx pscad`)

1. Simulation Export to `.psout`

```
# Simulate a PSCAD case and export directly to .psout
msx pscad sim models/benchmark.pscx --format=psout --out=results.psout
```

2. Binary Inspection & Format Conversion

```
# Print formatted ANSI metadata table and channel telemetry
msx pscad psout results.psout

# Convert .psout to CSV
msx pscad psout results.psout --out=telemetry.csv

# Convert .psout to IEEE C37.111 COMTRADE (.cfg + .dat)
msx pscad psout results.psout --format=comtrade
```

Web IDE Waveform Studio (`modelscript.psoutViewer`)

When opening any `*.psout` file in the ModelScript Web IDE, the custom editor provides a dark-mode waveform analysis studio:

1. **Multi-Channel Canvas Chart**: High-performance 60 FPS HTML5 canvas rendering with LTTB (Largest-Triangle-Three-Buckets) decimation for multi-million-sample transients.
2. **Dual Measurement Cursors**: Live measurement HUD calculating time difference (Δt), instantaneous frequency ($f = 1/\Delta t$), amplitude change (Δy), and rate-of-change ($\mathrm{dy/dt}$ slope).
3. **Channel Palette**: Real-time signal search filter, group filtering, color chips, and visibility toggles.
4. **Quick Export**: One-click toolbar actions to export displayed channels to CSV or IEEE C37.111 COMTRADE.

CHAPTER 27

PSS/E Transmission Power Flow (`.raw`)

The `@modelscript/psse` language package (`languages/psse`) provides native support for **Siemens PSS®E RAW (`.raw`)** transmission grid power flow base cases (Revisions 30 through 35). It delivers streaming parsing, round-trip serialization, pre-flight topological linting (`P1001` – `P4001`), sparse nodal admittance matrix ($Y_{bus} = G + jB$) assembly, in-engine non-linear AC power flow solving (Newton-Raphson & Fast Decoupled), multi-domain projections to **Modelica** and **PSCAD/EMT**, interactive Single-Line Diagrams (SLD) via AntV X6, and a dual-mode VS Code custom webview editor.

Supported File Specifications

Revision	Specification	Key Structural Features
PSS/E-30	Legacy Planning Deck	6-character bus names, integer area/zone codes, fixed branch thermal rating blocks
PSS/E-32	Extended Format	12-character alphanumeric bus names, expanded DC line modeling, switched shunt blocks
PSS/E-33	Modern Standard	Canonical 12-character names, multi-section lines, 2-terminal & VSC DC transmission, GIC records
PSS/E-34	Substation-Oriented	Node-breaker topologies, explicit substation IDs, multi-terminal DC grids
PSS/E-35	High-Density Matrix	Extended branch identifiers, expanded transformer winding data, flexible rating vectors

---

Architecture & Compilation Pipeline



---

1. Pre-Flight Electrical Diagnostic Rules

`@modelscript/psse` runs pre-flight static checks across electrical topology, connection consistency, and equipment parameter bounds before simulation or projection:

---

2. Sparse Nodal Admittance Matrix (Y_{bus})

The network admittance matrix $\mathbf{Y}_{\text{bus}} \in \mathbb{C}^{N \times N}$ is extracted via `extractPowerFlowNetlist()` in `languages/psse/src/netlist/powerflow-netlist.ts` :

Transmission Lines (π-Equivalent)

For a branch connecting buses i and j with series impedance $z = r + jx$ and total charging susceptance b:

$$y_s = \frac{1}{r + jx} = \frac{r - jx}{r^2 + x^2} = g_s + jb_s$$

$$Y_{ij} = -y_s, \quad Y_{ji} = -y_s$$

$$Y_{ii} \mathrel{+}= y_s + j\frac{b}{2}, \quad Y_{jj} \mathrel{+}= y_s + j\frac{b}{2}$$

Transformers with Complex Tap Ratio

For a transformer with off-nominal turns ratio magnitude a and phase shift ϕ, the complex tap is $\bar{a} = ae^{j\phi} = a_{\text{re}} + ja_{\text{im}}$:

$$Y_{ii} \mathrel{+}= \frac{y_s}{|\bar{a}|^2}, \quad Y_{jj} \mathrel{+}= y_s$$

$$Y_{ij} = -\frac{y_s}{\bar{a}*}, \quad Y_{ji} = -\frac{y_s}{\bar{a}}$$

---

3. In-Engine Non-Linear AC Power Flow Solver

`@modelscript/psse` includes an exact, non-linear AC Power Flow engine (`solvePowerFlow()` in `languages/psse/src/solver/ac-powerflow.ts`) supporting both **Newton-Raphson** and **Fast Decoupled Power Flow (FDPF)**:

Mismatch Formulation

For each bus $i \in \{1, \dots, N\}$ with voltage $\bar{V}_i = V_i e^{j\theta_i}$:

$$P_i(V, \theta) = V_i \sum_{k=1}^{N} V_k \left(G_{ik} \cos\theta_{ik} + B_{ik} \sin\theta_{ik}\right)$$

$$Q_i(V, \theta) = V_i \sum_{k=1}^{N} V_k \left(G_{ik} \sin\theta_{ik} - B_{ik} \cos\theta_{ik}\right)$$

where $\theta_{ik} = \theta_i - \theta_k$. Active and reactive mismatches are:

$$\Delta P_i = P_i^{\text{spec}} - P_i(V, \theta), \quad \Delta Q_i = Q_i^{\text{spec}} - Q_i(V, \theta)$$

Analytical Jacobian & Adaptive Damping

The linearized Newton system at each iteration k is:

$$\begin{bmatrix} \Delta P \\ \Delta Q \end{bmatrix} = \begin{bmatrix} H & N \\ M & L \end{bmatrix} \begin{bmatrix} \Delta\theta \\ \Delta V / V \end{bmatrix}$$

- $H_{ik} = \frac{\partial P_i}{\partial \theta_k} = V_i V_k (G_{ik} \sin\theta_{ik} - B_{ik} \cos\theta_{ik})$ $(k = i)$, and $H_{ii} = -Q_i - B_{ii} V_i^2$.
- $N_{ik} = V_k \frac{\partial P_i}{\partial V_k} = V_i V_k (G_{ik} \cos\theta_{ik} + B_{ik} \sin\theta_{ik})$ $(k = i)$, and $N_{ii} = P_i + G_{ii} V_i^2$.
- $M_{ik} = \frac{\partial Q_i}{\partial \theta_k} = -V_i V_k (G_{ik} \cos\theta_{ik} + B_{ik} \sin\theta_{ik})$ $(k = i)$, and $M_{ii} = P_i - G_{ii} V_i^2$.
- $L_{ik} = V_k \frac{\partial Q_i}{\partial V_k} = V_i V_k (G_{ik} \sin\theta_{ik} - B_{ik} \cos\theta_{ik})$ $(k = i)$, and $L_{ii} = Q_i - B_{ii} V_i^2$.

Generator Q-Limit Clamping & PV-to-PQ Switching

When $Q_g > Q_{\max}$ or $Q_g < Q_{\min}$, the generator bus is clamped at its reactive limit and dynamically converted to a PQ bus for remaining iterations, preventing unrealistic voltage regulation.

---

4. Multi-Domain Interoperability Projections

Projection to Modelica (`psseToModelica`)

Projects a `.raw` case into a clean, strongly-typed Modelica physical system model:

- **Buses** mapped to equipotential AC pins.
- **Generators** mapped to sinusoidal voltage sources (`Modelica.Electrical.Analog.Sources.SineVoltage`).
- **Branches** mapped to series R-L models (`Modelica.Electrical.Analog.Basic.Resistor` + `Inductor`).

- **Loads** mapped to constant active and reactive impedance networks.

```
model Ieee14Test
  Modelica.Electrical.Analog.Basic.Ground ground;
  Modelica.Electrical.Analog.Sources.SineVoltage gen_1_1(V = 106.0, f = 60.0);
  Modelica.Electrical.Analog.Basic.Resistor line_1_2_r(R = 0.01938);
  Modelica.Electrical.Analog.Basic.Inductor line_1_2_l(L = 0.0001569);
equation
  connect(gen_1_1.n, ground.p);
  connect(gen_1_1.p, line_1_2_r.p);
  connect(line_1_2_r.n, line_1_2_l.p);
end Ieee14Test;
```

Projection to PSCAD / EMT Netlist (`psseToEmtNetlist`)

Generates multi-bus companion-circuit netlists compatible with ModelScript's Dommel EMT solver (`@modelscript/simulate`) for transient stability and electromagnetic switching studies.

---

5. Command-Line Interface (`msx psse`)

ModelScript's CLI provides headless inspection, linting, diagram rendering, and power flow solving:

```
# Display summary of buses, generators, loads, branches, and MW totals
msx psse summary case.raw

# Run pre-flight electrical topology and bounds linter
msx psse lint case.raw

# Solve non-linear AC power flow with Newton-Raphson (default)
msx psse solve case.raw

# Solve with Fast Decoupled algorithm and custom tolerance
msx psse solve case.raw --algorithm=fdpf --tol=1e-5 --max-iter=30

# Export publication-quality Single-Line Diagram SVG
msx psse diagram case.raw --out=grid.svg

# Inspect sparse Ybus non-zero entry metrics
msx psse netlist case.raw
```

---

6. Custom Visual Webview Editor (`modelscript.rawEditor`)

Opening any `.raw` file in the ModelScript IDE or VS Code launches the custom dual-mode editor:

- **Single-Line Diagram View**: Interactive pan/zoom SVG canvas with voltage-tier color coding (345 kV red, 138 kV blue, 69 kV green), power flow overlays, and generator/transformer glyphs.
- **High-Density Tabular Grid**: Live-filterable tables for Buses, Generators, Branches, Transformers, and Loads with search.
- **Overload & Violation Badges**: Automatic highlight of under-voltage ($V < 0.95$ pu), over-voltage ($V > 1.05$ pu), and thermal line overloads ($S_{ij} > \text{Rate A}$).
- **Export Toolbar**: Export to SVG diagrams, Modelica `.mo` packages, and PSCAD/EMT netlists.

CHAPTER 28

title: OLGA Multiphase Dynamic Flow Simulation description: First-class support for OLGA (.opi, .key, .genkey) high-fidelity transient multiphase flow, mechanistic regime maps, slug tracking, and flow assurance verification in ModelScript.

OLGA Multiphase Flow Assurance

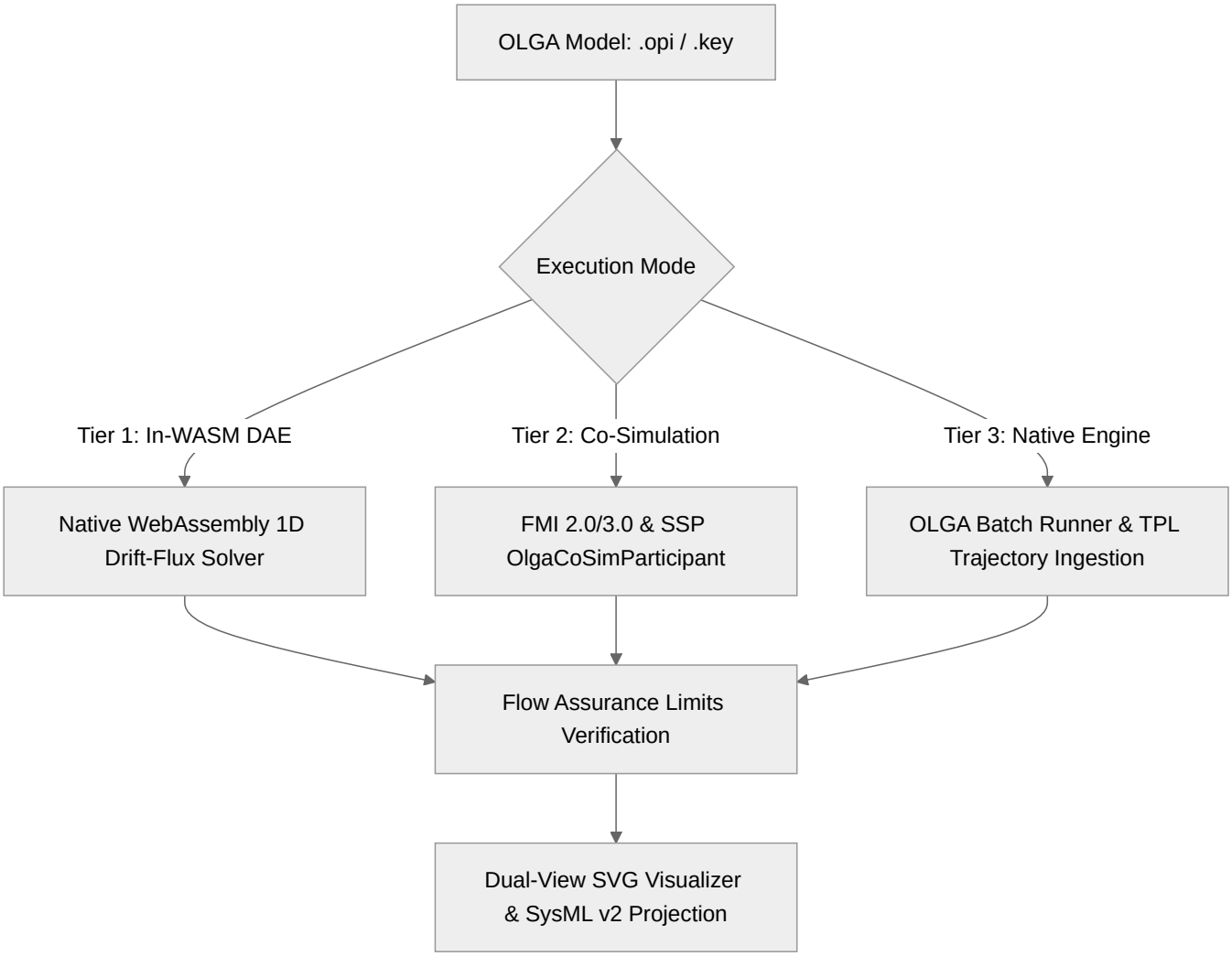
ModelScript provides first-class, polyglot support for **OLGA** (High-Fidelity Dynamic Multiphase Flow Simulation). Engineers can author, simulate, lint, formally verify, and co-simulate multiphase production flowlines, deepwater catenary risers, and slug catchers directly within ModelScript's unified digital thread.

Supported File Formats

Format	Extension	Description
OLGA XML GUI Project	`.opi`, `.xml`	Complete graphical network model defining wells, pipes, chokes, separator terminals, and operational controllers.
Legacy Keyword Deck	`.key`, `.genkey`	Standard industrial keyword decks specifying flowpath geometry, branch connections, boundary conditions, and valve schedules.
PVT Flash Tables	`.tab`	Compositional and black-oil tabular fluid files for bilinear interpolation of fluid densities, gas-liquid ratios, and viscosities.
Simulation Trends	`.tpl`	High-frequency time-series trend files containing arbitrary multi-axis channel catalogs (P, T, α_L, Q_L, U_G, etc.).
Spatial Profiles	`.ppl`	Spatial profile snapshots along pipeline coordinate X with multi-variable channels, flow regime decoding, and time scrubbing.

3-Tier Execution Architecture

ModelScript implements a 3-tier operational execution framework, accommodating headless cloud workflows, system-level co-simulation, and ground-truth industrial verification:



1. Tier 1: Native In-WASM Drift-Flux Solver

- **Zero Intermediate Allocations:** Lowers directly into ModelScript's `DAEBuilder` linear memory arena.
- **Transient Hydrodynamics:** Solves staggered-grid mass, momentum, and energy balances coupled with Zuber-Findlay drift-flux gas-liquid slip.
- **Mechanistic Physics:**
  - **Taitel-Dukler (1976) & Barnea (1987) Flow Regime Map**: Exact geometric wetted perimeters (S_i, S_G, S_L) and Andritsos-Hanratty wave friction enhancement.
  - **Pal & Rhodes (1989) Emulsion Inversion**: Captures apparent viscosity peaks during water breakthrough ($WC_{\text{inv}} \approx 42\%$).
  - **Lagrangian Slug Tracker**: Propagates discrete moving slug bodies, resolving slug length distributions ($L_{s,\text{mean}}, L_{s,95}$) and riser-base severe cyclic slugging.
  - **Dynamic Pipeline Scraper Pig (`OlgaPig`)**: Solves pig motion ODEs ($M\,dv/dt = \Delta P \cdot A - F_{\text{friction}} - Mg\sin\theta$) and liquid slug sweep volume accumulation.

2. Tier 2: FMI 2.0/3.0 & SSP Co-Simulation

Exposes standard FMI input and output ports via `OlgaCoSimParticipant` in `@modelscript/exchange` :

- **Input Ports:** `choke.opening` , `wellhead.mass_rate` , `separator.pressure` , `ambient.temperature` .
- **Output Ports:** `inlet.pressure` , `arrival.temperature` , `arrival.liquid_holdup` , `arrival.liquid_rate` , `slug.surge_volume` , `hydrate.subcooling` , `maop.margin` , `slug.mean_length` , `slug.frequency` , `pig.position` .

3. Tier 3: Industrial OLGA Engine Orchestration

Automates invocation of local Schlumberger OLGA executable engines (`olga.exe` / `olga-cli`), dynamically ingesting binary `.tpl` trajectories into ModelScript's data structures for verification.

Command Line Interface (`msx olga`)

ModelScript CLI provides headless subcommands for flowline engineering:

```
# Run 1D transient simulation and export trajectory to CSV
msx olga simulate subsea_tieback.opi --duration=3600 --out=trajectory.csv

# Pre-flight flow assurance diagnostic check
msx olga lint deepwater_riser.key

# Verify formal flow assurance limits (MAOP, Hydrate Margin, Slug Catcher Capacity)
msx olga verify flowline.genkey --maop=250 --margin=3.0 --capacity=150.0

# Generate standalone SVG bathymetry elevation profile
msx olga diagram subsea_tieback.opi --view=profile --out=bathymetry.svg

# Convert legacy keyword deck to modern XML or SysML v2 operating procedures
msx olga convert input.key --to=opi --out=project.opi
msx olga convert input.opi --to=sysml2 --out=procedures.sysml
```

Flow Assurance Verification & Lint Rules

ModelScript includes automated pre-flight linter diagnostics and limit verifications:

Dual-View SVG Visualization

ModelScript renders dual-view vector diagrams for every OLGA network:

1. **Topological P&ID Schematic:** Wells, manifolds, choke valves, flowpath pipes, and topside slug catchers.
2. **Pipeline Elevation Bathymetry $Z(X)$:**
   - High-contrast seabed-to-platform elevation profile.
   - In-situ liquid holdup (α_L) segment color heatmaps.
   - Active discrete slug bubbles highlighted along the flowline path.
   - Real-time scraper pig marker displaying traveling velocity and differential drive pressure.
   - Hydrate subcooling margin callout ($T_{\text{fluid}} - T_{\text{hyd}}$).

SysML v2 Operating Procedure Integration

OLGA models can be formally projected into SysML v2 packages (`@modelscript/olga` → SysML v2):

```
package Subsea_Tieback_System {
    import ScalarValues::*;

    part def FlowlineAssembly {
        attribute totalLength: Length = 4450.0 [m];
        attribute maopRating: Pressure = 250.0 [bar];
    }

    state def SubseaFlowlineOperatingProcedure {
        entry; then SteadyProduction;

        state SteadyProduction {
            doc /* Continuous production at nominal design rates */
        }
        accept ESD_Trip_Signal then EmergencyShutIn;
        accept Turndown_Command then LowTurndown;

        state EmergencyShutIn {
            doc /* Choke closed; cooldown clock active */
        }
        accept Depressurize_Command when wallTemperature > 18.0 [degC] then ControlledBlowdown;
        accept Cooldown_Timeout when shutInDurationHours >= 8.0 [hr] then ChemicalInhibitionFlushing;

        state ControlledBlowdown;
        state ChemicalInhibitionFlushing;
    }
}
```

State Checkpointing & Zero-Discontinuity Warm Restart

Long-duration or multi-stage pipeline flow assurance operations can be snapshot to disk and resumed with zero trajectory discontinuity:

```
# Snapshot simulation state at completion
msx olga simulate subsea_tieback.opi --duration=3600 --checkpoint-out=t3600.rst

# Resume simulation from t=3600s
msx olga simulate subsea_tieback.opi --duration=7200 --restart=t3600.rst --out=second_stage.csv
```

The `.rst` snapshot encapsulates complete distributed states across the spatial grid:

- Cell pressure P, temperature T, phase holdups (α_L, α_G), and velocities (u_L, u_G)
- Active Lagrangian slug states (tail/front positions, slug lengths, propagation velocities)
- Scraper pig position, speed, and accumulated swept liquid volume
- Machinery operating states and closed-loop controller integral accumulators

---

Autonomous Field History Matching & Parameter Calibration

Calibrate flowline simulation models to match production SCADA telemetry and test separator observations using bounded Nelder-Mead simplex optimization:

```
msx olga tune subsea_tieback.opi field_telemetry.csv \
  --tune=roughness,choke-cd,heat-transfer \
  --max-evals=150 \
  --out=calibrated_model.opi \
  --report=tuning_report.md
```

The optimizer automatically estimates:

- Effective pipe hydraulic roughness (ϵ)
- Overall ambient heat transfer coefficient (U_{ambient})
- Choke discharge coefficients (C_d)
- Well productivity index (PI) / reservoir inflow rates

Outputs comprehensive statistical goodness-of-fit metrics (R^2, RMSE in bar and Kelvin) and ready-to-run calibrated `.opi` / `.genkey` models.

Native In-WASM Cubic Equation of State (PR78 & SRK)

ModelScript includes a native WebAssembly cubic equation of state solver executing directly in linear memory with zero intermediate AST wrapper overhead.

Governing Thermodynamic Formulations

The general two-parameter cubic EOS takes the form:

$$P = \frac{RT}{v-b} - \frac{a(T)}{v^2 + ubv + wb^2}$$

EOS Model	u	w	Ω_a	Ω_b	Temperature Correction $\alpha(T_r, \omega)$
Peng-Robinson (1978)	2	−1	0.45724	0.07780	$[1 + m(1 - \sqrt{T_r})]^2$, with $m = 0.37464 + 1.54226\omega - 0.26992\omega^2$
Soave-Redlich-Kwong (SRK)	1	0	0.42748	0.08664	$[1 + m(1 - \sqrt{T_r})]^2$, with $m = 0.480 + 1.574\omega - 0.176\omega^2$

Van der Waals Mixing Rules with Binary Interaction (k_{ij})

For multicomponent reservoir and hydrocarbon fluids:

$$a = \sum_{i=1}^{N}\sum_{j=1}^{N} z_i z_j \sqrt{a_i a_j}(1 - k_{ij}), \qquad b = \sum_{i=1}^{N} z_i b_i$$

Rachford-Rice Vapor-Liquid Equilibrium (VLE) Flash

Isothermal-isobaric flash equilibrium is determined by solving the objective function for vapor fraction $\beta = V/F$:

$$\sum_{i=1}^{N} \frac{z_i(K_i - 1)}{1 + \beta(K_i - 1)} = 0, \quad \text{where } K_i = \frac{y_i}{x_i} = \frac{\phi_i^L}{\phi_i^V}$$

Accelerated by successive substitution initialized via Wilson's heuristic K-value correlations and accelerated by Cardano analytical cubic root solving.

Dense Fluid Transport Properties

- **Lohrenz-Bray-Clark (LBC) Viscosity:** Computes high-pressure gas and liquid dynamic viscosities from reduced mixture densities $\rho_r = \rho/\rho_c$:

$$[(\mu - \mu_0)\xi + 10^{-4}]^{1/4} = a_0 + a_1\rho_r + a_2\rho_r^2 + a_3\rho_r^3 + a_4\rho_r^4$$

- **Macleod-Sugden Parachor Surface Tension:** Evaluates gas-liquid interfacial tension σ from parachors P_i:

$$\sigma = \left[\sum_{i=1}^{N} P_i \left(x_i \frac{\rho_L}{M_L} - y_i \frac{\rho_G}{M_G}\right)\right]^4$$

Command Line EOS Flash & Grid Synthesis

```
# Evaluate multi-component P-T flash and trace saturation envelope
msx olga flash --comp="C1:0.75,C2:0.10,C3:0.08,nC4:0.04,C7:0.03" \
  --p=55.0 --t=295.15 --model=pr --envelope --out=flash_summary.csv

# Synthesize a 2D tabular PVT file (.tab) for 1D drift-flux simulation
msx olga flash --comp="C1:0.80,C2:0.10,C3:0.06,C7:0.04" \
  --model=srk --out=reservoir_fluid.tab
```

Active Topside Anti-Slug Closed-Loop Feedback Control

Topside choke throttling stabilizes severe cyclic slugging in deepwater catenary risers by dynamically manipulating backpressure using high-frequency subsea pressure transmitter measurements.



Mathematical Architecture

1. **Subsea PT Feedback Error:** $e(t) = P_{\text{sp}} - P_{\text{inlet}}(t)$
2. **Washout High-Pass Filter:** Isolates oscillatory slug resonance from slow reservoir depletion:

$$\tau_w \frac{de_f}{dt} + e_f = \tau_w \frac{de}{dt}$$

3. **Gain-Scheduled PI Control:** Compensates for non-linear choke valve flow characteristics:

$$K_p(Z) = K_{p0} \cdot [1 + \gamma(1 - Z)]$$

$$Z(t) = Z_{\text{baseline}} + K_p(Z)\,[e_f(t) + \frac{1}{\tau_i}\int_0^t e_f(\tau)d\tau]$$

4. **Physical Actuator Slew-Rate Constraints:** Clamps valve motion rate to realistic hydraulic actuator speeds:

$$\left|\frac{dZ}{dt}\right| \le R_{\max} \quad \text{(e.g. 5\%/s)}$$

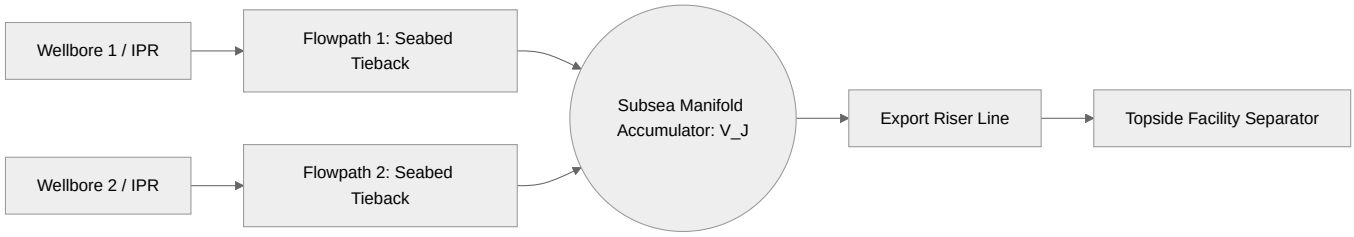
5. **Slug Catcher Surge Override:** Emergency high-liquid override closes choke if topside separator approaches liquid carryover capacity ($V_{\text{surge}} > 0.8V_{\text{cap}}$).

CLI Anti-Slug Simulation

```
msx olga simulate subsea_catenary_riser.opi \
  --duration=3600 \
  --anti-slug \
  --slug-setpoint=52.0 \
  --slug-kp=1.5e-6 \
  --slug-ti=30.0 \
  --slug-washout=20.0 \
  --slug-choke=0.50 \
  --out=stabilized_trajectory.csv
```

Coupled Multi-Branch Gathering Network Transient Solver

Simultaneously models converging multi-well subsea gathering manifolds, shared export lines, and platform separators with dynamic acoustic relaxation:



Governing Manifold Dynamic Balances

1. **Dynamic Junction Mass Continuity:**

$$V_J \frac{d(\rho_L \alpha_L)_J}{dt} = \sum_{\text{in}} \dot{m}_{L,\text{in}} - \sum_{\text{out}} \dot{m}_{L,\text{out}}$$

$$V_J \frac{d(\rho_G \alpha_G)_J}{dt} = \sum_{\text{in}} \dot{m}_{G,\text{in}} - \sum_{\text{out}} \dot{m}_{G,\text{out}}$$

2. **Mixture Thermal Energy Accumulation:**

$$\frac{d(mu)_J}{dt} = \sum_{\text{in}} (\dot{m}_{\text{in}} h_{\text{in}}) - \sum_{\text{out}} (\dot{m}_{\text{out}} h_J)$$

3. **Acoustic Pressure Relaxation:**

$$\frac{dP_J}{dt} = \frac{c_m^2}{V_J} \left(\sum \dot{m}_{\text{in}} - \sum \dot{m}_{\text{out}} \right)$$

Well Trip & Flowline Shut-In Redistribution

```
# Simulate 2-well gathering network with emergency Well 1 trip at t=20s
msx olga network gathering_manifold.opi \
  --duration=120 \
  --dt=0.5 \
  --junction-vol=8.0 \
  --trip="WELL_1:20" \
  --out=network_transient.csv
```

Flow Assurance CI/CD Safety Gate & 3D Bathymetry Survey

Automated Pull Request Safety Gate (`msx olga gate`)

Integrates flow assurance validation directly into automated GitHub Actions / GitLab CI pipelines:

```
msx olga gate subsea_tieback.opi \
  --max-surge=85.0 \
  --min-hydrate-margin=3.0 \
  --max-wax="2mm" \
  --max-corrosion="0.1mm/yr" \
  --out-report=audit_gate_report.md
```

Generates a markdown audit report detailing compliance matrices and exits with non-zero status upon constraint violations.

3D Seabed Bathymetry Survey Ingestion (`msx olga survey`)

Ingests ROV/multibeam sonar seabed bathymetry CSV survey logs ($Easting, Northing, Depth$) and computes 3D pipeline curvature, span lengths, and dogleg severity:

```
msx olga survey bathymetry_survey.csv \
  --diameter=0.254 \
  --roughness=4.5e-5 \
  --out=flowline_model.opi
```

Comprehensive CLI Subcommand Reference Matrix

Subcommand	Purpose	Key Arguments	Generated Artifacts
`simulate`	1D transient drift-flux simulation	`--duration`, `--dt`, `--restart`, `--checkpoint-out`, `--anti-slug`	Trajectory CSV/JSON, `.rst` state snapshots
`flash`	Native Cubic EOS (PR78/SRK) flash	`--comp`, `--p`, `--t`, `--model`, `--envelope`, `--out`	Terminal state summary, CSV, `.tab` 2D grid
`network`	Coupled multi-branch gathering solver	`--duration`, `--dt`, `--junction-vol`, `--trip`, `--out`	Composite multi-branch trajectory CSV
`gate`	Automated CI/CD safety gate audit	`--max-surge`, `--min-hydrate-margin`, `--max-wax`, `--max-corrosion`	Markdown compliance report, CI exit codes
`survey`	3D seabed bathymetry survey ingestion	`<survey.csv>`, `--diameter`, `--roughness`, `--out`	Geometrically resolved `.opi` / `.genkey`
`tune`	History matching parameter calibration	`<model>`, `<telemetry.csv>`, `--tune`, `--max-evals`	Calibrated model `.opi`, Markdown fit report
`lint`	Pre-flight flow assurance diagnostics	`<model>`, `--severity`	Diagnostic listings (`M7001`–`M7008`)
`verify`	Formal design limit verification	`--maop`, `--margin`, `--capacity`, `--min-vel`	Verification summary, constraint headroom
`diagram`	Vector SVG visualization	`--view=profile|schematic|pvt|wellbore|slug|bathy`	Standalone publication-ready SVG files
`convert`	Model format translation	`--to=genkey|opi|sysml2`, `--out`	Translated model files, SysML v2 packages
`trend`	Ingest high-frequency `.tpl` trend files	`<file.tpl>`, `--channels`	Channel catalogs, time-series metrics
`profile`	Ingest spatial `.ppl` snapshot profiles	`<file.ppl>`, `--out`	Elevation SVG snapshots, holdup heatmaps
`cosim`	Multi-rate FMI/SSP co-simulation	`<subsea.opi>`, `<topside.opi>`, `--macro-step`	Synchronized boundary trajectory CSV
`fmu`	Standalone FMI 2.0/3.0 FMU packaging	`<model>`, `--fmi-version=2.0|3.0|both`, `--out`	Portable binary `.fmu` zip archive

CHAPTER 29

---

title: PipelineStudio (TGNET & TLNET) Simulation description: First-class support for Emerson PipelineStudio (.lps, .tgx, .tlx, .tgn, .tln) transient compressible gas hydraulics, line pack storage analytics, multi-product liquid batch tracking, and hydrogen blending.

PipelineStudio (LPS / TGNET & TLNET)

ModelScript provides first-class, polyglot support for **Emerson / Energy Solutions PipelineStudio**, including its dual simulation kernels:

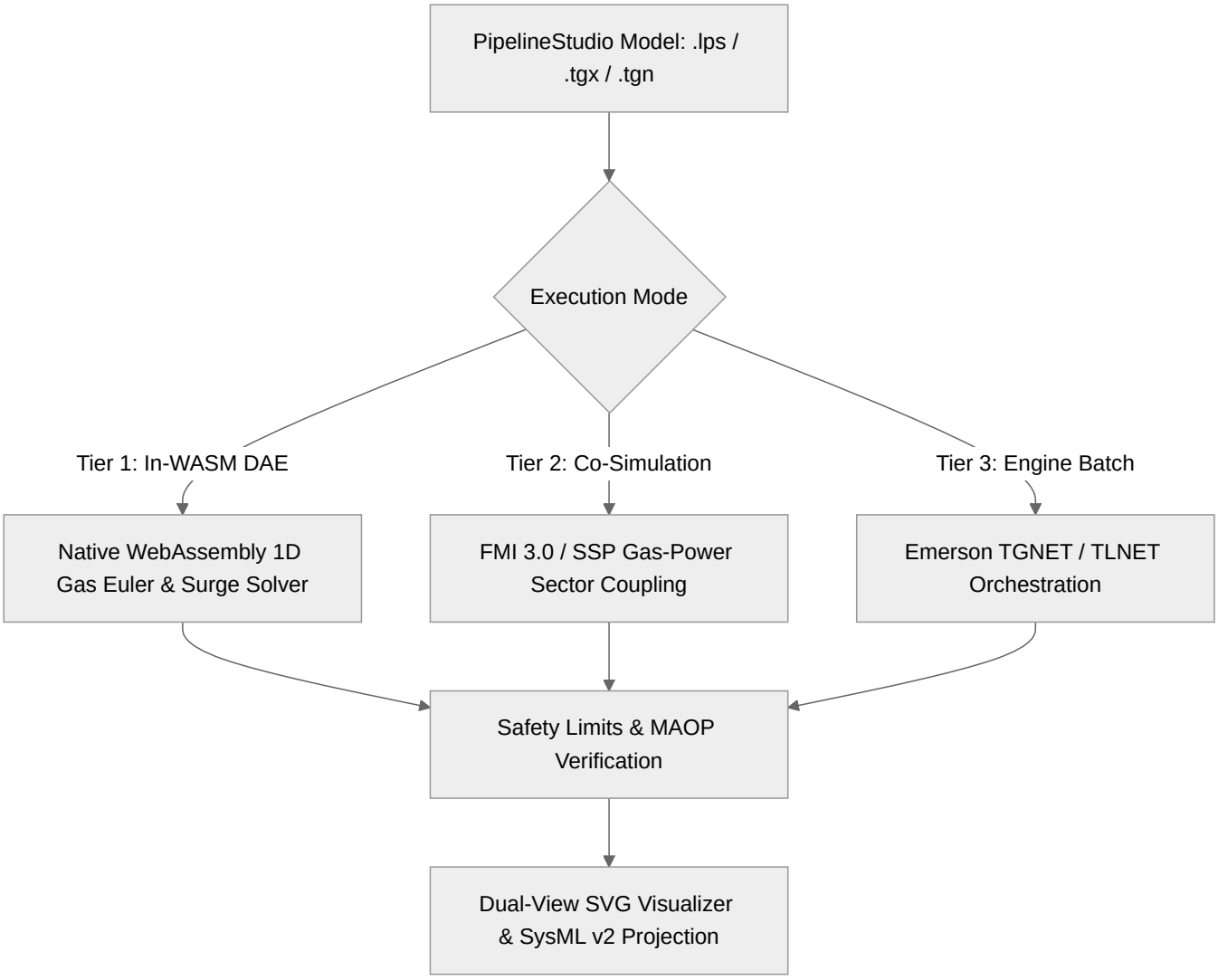
- **TGNET** (*Transient Gas Network*): Compressible gas hydraulics, real-gas equations of state (AGA-8 Detailed Characterization & Peng-Robinson), line pack dynamics, compressor stations, gas heating value tracking, and hydrogen blending.
- **TLNET** (*Transient Liquid Network*): Incompressible/liquid hydraulics, multi-product batch tracking (gasoline/diesel/jet fuel interfaces), transmix dispersion, pump stations, drag-reducing agents (DRA), and water hammer / acoustic surge analysis.

Supported File Formats

Format	Extension	Description
PipelineStudio Project	`.lps`, `.xml`	Complete graphical network model defining nodes, pipes, compressor stations, pump stations, regulators, and study scenarios.
TGNET Gas Study	`.tgx`, `.psg`	Dedicated compressible gas transmission network studies with gas composition and line pack profiles.
TLNET Liquid Study	`.tlx`, `.psl`	Dedicated liquid transmission network studies with batch injection schedules and fluid property libraries.
Legacy Keyword Decks	`.tgn`, `.tln`	Standard industrial keyword decks specifying network geometry, equipment ratings, and boundary conditions.

3-Tier Execution Architecture

ModelScript implements a 3-tier operational execution framework for pipeline networks:



1. Tier 1: In-WASM DAE Solvers

- **Zero Intermediate Allocations:** Lowers directly into ModelScript's `DAEBuilder` linear memory arena.
- **High-Accuracy Thermodynamics:**
  - **AGA-8 (ISO 20765-1)**: High-precision calculation of compressibility factor $Z(P, T, \mathbf{y})$, molar density, speed of sound, and Joule-Thomson cooling.
  - **Hydrogen Blending**: Native support for H_2 blending in natural gas from 0% to 100%, capturing density drop, speed of sound increases, and compressor power shifts.
  - **Peng-Robinson Cubic EOS**: General flash calculation for heavy hydrocarbons and dense-phase supercritical CO_2 pipeline transport.
- **Transient Compressible Gas Dynamics**:
  - Solves 1D staggered-grid continuity and momentum equations:

$$\frac{\partial P}{\partial t} = -\frac{a^2}{A}\frac{\partial w}{\partial x}$$

$$\frac{\partial w}{\partial t} = -A\frac{\partial P}{\partial x} - \frac{fw|w|}{2\rho DA} - \rho g A \sin\theta$$

  - Simulates dynamic line packing, demand drafting, and decompression acoustic waves.
- **Water Hammer & Surge Analysis**:

  - Solves Joukowsky acoustic wave propagation ($\Delta P = \rho a \Delta v$) during rapid emergency shutdown (ESD) valve closures and pump trips.
  - Monitors transient pressure spikes against ASME B31.4 limits ($P \leq 1.1 \times \text{MAOP}$) and screens for liquid column separation / cavitation.
- **Multi-Product Liquid Batch Tracking**:
  - Tracks discrete fuel slugs (Diesel, Regular Gasoline, Jet-A) along transmission pipelines.
  - Austin-Palfrey interface mixing length and transmix volume calculation at delivery manifold cut-points.

2. Tier 2: Gas-Power Sector Coupling & Co-Simulation

Enables cross-vector energy system co-simulation via FMI 2.0/3.0 and SSP:

- **Coupled with `@modelscript/psse`:** Simulates how rapid ramp-up of gas-fired combined-cycle power plants (CCGTs) draws on pipeline line pack, ensuring minimum delivery pressures are sustained during electric grid peaks.
- **Coupled with `@modelscript/pscad`:** Evaluates fast sub-second electrical generator trips and their acoustic pressure waves on pipeline fuel supply manifolds.
- **Power-to-Gas:** Simulates green hydrogen injection from wind/solar electrolyzers into regional gas grids.

3. Tier 3: Industrial Engine Orchestration

Automates invocation of local Emerson PipelineStudio CLI engines (`tgnet.exe` / `tlnet.exe`), dynamically ingesting results into ModelScript for unified verification.

Pre-Flight Pipeline Diagnostic Linter

Programmatic Usage

```
import {
  parseLps,
  lintLpsModel,
  SteadyNetworkSolver,
  LinePackEngine,
  GasEulerDaeSolver,
  lpsToSysml2,
  PipelineVerifier,
} from "@modelscript/lps";

// 1. Ingest PipelineStudio XML or ASCII keyword deck
const model = parseLps(sourceText);

// 2. Run Pre-Flight Diagnostic Linter
const diagnostics = lintLpsModel(model);

// 3. Solve Steady-State Network Hydraulics
const steady = SteadyNetworkSolver.solve(model);

// 4. Evaluate Line Pack Storage & Deliverability Reserves
const linePack = LinePackEngine.analyze(model, steady);
console.log(`Total Line Pack: ${(linePack.totalLinePackSm3 / 1e6).toFixed(2)} MMSm3`);
console.log(`Energy Stored: ${linePack.totalEnergyStoredGWh.toFixed(1)} GWh`);

// 5. Verify Formal Safety & MAOP Compliance
const report = PipelineVerifier.verifySteady(model, steady);
console.log("Verification passed:", report.passed);

// 6. Simulate 1D Transient Dynamics
const trajectory = GasEulerDaeSolver.simulate(model, {
  durationSeconds: 7200,
  timeStep: 10,
});

// 7. Generate Formal SysML v2 Architecture
const sysmlCode = lpsToSysml2(model);
```

Advanced Machinery, Thermodynamics & Physical Models

1. Multi-Wheel Compressor Maps & Anti-Surge Control

ModelScript supports non-dimensional head-flow parameterization (Ψ vs Φ) for multi-stage centrifugal compressors:

- **Affinity Scaling**: Speed governor modulates $\omega \in [\omega_{\min}, \omega_{\max}]$ to maintain target discharge pressure setpoints.
- **Surge Line & Anti-Surge Recycle Valve**: Parabolic surge locus detection with automatic recycle valve modulation maintaining a minimum 10% safety buffer.
- **Stonewall (Choke)**: Detects maximum volumetric throughput beyond which sonic shock waves restrict further flow.

2. Centrifugal Pump Station Curves & NPSH Verification

Centrifugal pipeline booster pumps are modeled via cubic polynomial head $H(Q)$ and hydraulic efficiency $\eta(Q)$ curves with variable frequency drive (VFD) affinity scaling:

$$H(Q, \alpha) = \alpha^2 H_{\text{rated}}(Q/\alpha), \quad \alpha = \omega/\omega_{\text{rated}}$$

- Computes Net Positive Suction Head Available ($NPSHA$) against liquid vapor pressure.
- Triggers diagnostic `M8011` if $NPSHA < NPSHR(Q)$, preventing catastrophic impeller cavitation.

3. GERG-2008 Helmholtz Free-Energy EOS (ISO 20765-2)

Implements the multi-fluid Helmholtz free-energy formulation $\alpha(\delta, \tau) = \alpha^0(\delta, \tau) + \alpha^r(\delta, \tau)$ for 21 natural gas components:

- Covers gas, liquid, supercritical dense phase, LNG cryogenic, and hydrogen blending regimes.
- Computes speed of sound w, isobaric heat capacity C_p, isochoric heat capacity C_v, and Joule-Thomson coefficient μ_{JT}.

4. Slack Line Open-Channel Gravity Flow in Steep Pipelines

For pipelines traversing mountain passes with steep downward slopes, liquid column separation occurs when pressure drops to liquid vapor pressure ($P \le P_{\text{sat}}$):

- Transitions from pressurized pipe flow to open-channel gravity flow governed by the Manning-Gauckler-Strickler equation ($Q = \frac{1}{n} A R_h^{2/3} S^{1/2}$).
- Tracks dynamic void fraction $\alpha_{\text{void}}(x, t) = 1 - A_{\text{flow}}/A_{\text{pipe}}$ and calculates slack collapse re-pressurization surge spikes upon valley impact.

5. 1D Radial Cylindrical Soil Thermal Conduction Mesh

Discretizes the ground surrounding buried transmission pipelines into a 10-layer concentric radial cylindrical finite-difference mesh:

- Solves $\rho_{\text{soil}} c_{\text{soil}} \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r}(k_{\text{soil}} r \frac{\partial T}{\partial r})$.
- Captures ground thermal inertia, depth-dependent seasonal phase lag, and pipeline cooldown following emergency station shutdowns.

6. Station Supervisory Multi-Variable PID Overrides

SCADA compressor and pump station automation with min/max auctioneering selector gates:

- Arbitrates between discharge pressure control, low-suction override, driver shaft power limits, and flow dispatch.
- Integrates velocity PID controllers with anti-windup tracking, synchronizing inactive integrator loops to prevent reset overshoot during control transfer.

7. Hot-Start State Checkpointing (`.sav` / `.rst`)

Zero-copy state snapshot serialization with SHA-256 integrity verification:

- Saves all nodal pressures, pipe mass flows, fluid temperatures, soil thermal layers, and controller integrator states.
- Enables instantaneous resumption of multi-day transient simulations and what-if contingency branching.

Command-Line Interface (`msx lps`)

```
# Pre-flight safety linting
msx lps lint network.lps

# Steady-state hydraulics with slack line & soil thermal analysis
msx lps solve network.lps --slack --soil --eos=GERG2008

# Line pack inventory & deliverability buffer
msx lps linepack network.lps

# Transient simulation with hot-start checkpointing
msx lps simulate network.lps --duration=7200 --step=10 --save-checkpoint=state.sav

# Resume simulation from intermediate snapshot
msx lps simulate network.lps --duration=3600 --restart=state.sav

# Export formal SysML v2 architectural model
msx lps sysml network.lps -o network.sysml
```

CHAPTER 30

EASY5 Dynamic Simulation & Linear Control (`.ez5` , `.ezm` , `.ezd` , `.eas`)

The `@modelscript/easy5` package (`languages/easy5`) provides native support for **Boeing / MSC Software EASY5™** dynamic system simulation models, control block diagrams, and empirical table lookups. It delivers multi-syntax parsing, embedded Fortran user block (`UG` / `UX`) compilation via `@modelscript/fortran` , pre-flight static topological linting (`M7001` – `M7012`) including Tarjan algebraic cycle detection, direct continuous DAE arena lowering (`IDaeBuilder`), linear control system analysis (trim solver, numerical (A, B, C, D) Jacobian extraction, QR eigenvalue/pole analysis, Bode frequency sweeps), multi-domain projections to **Modelica 3.x** and **FMI SSP**, and interactive AntV X6 / SVG schematic diagram generation.

Supported File Specifications

Extension	Format	Key Structural Features
`.ez5` / `.ezm`	Modern Model Definition	Tagged `MODEL` , `COMPONENTS` , `XP` , `S0` , `CONNECTIONS` , `TABLES` blocks with graphical layout attributes
`.ezd` / `.eas`	Classical Card Deck	Fixed-field component, parameter, and connection card images with continuation flags
`.tab`	Empirical Lookup Tables	1D and 2D tabular lookup arrays for nonlinear characteristics ($f(x)$ or $f(x, y)$)
`.f` / `.for`	User General (`UG` / `UX`)	Embedded or referenced Fortran 77/90 routines computing custom nonlinear dynamics

---

Architecture & Compilation Pipeline



---

1. Standard General Purpose (GP) Library Catalog

`@modelscript/easy5` includes built-in operational descriptors for standard EASY5 General Purpose library blocks:

Block ID	Name	Mathematical Transfer Function / Formulation	Key Parameters (`XP`)	States (`S`)
`GA`	Proportional Gain	$y(t) = K \cdot u(t)$	`K` (Gain factor)	None
`SJ`	Summing Junction	$y(t) = \sum_{i=1}^{m} \text{SGN}_i \cdot u_i(t)$	`SGN1` , `SGN2` , `SGN3` (± 1.0)	None
`IN`	Integrator	$\dot{x}(t) = K \cdot u(t), \quad y(t) = x(t)$	`GAIN` , `S0` (Initial state), `UL` , `LL`	1 continuous state
`TF`	Transfer Function	$Y(s) = \frac{b_m s^m + \cdots + b_0}{a_n s^n + \cdots + a_0} U(s)$	`NUM` (array), `DEN` (array)	n continuous states
`LA`	First-Order Lead-Lag	$Y(s) = K \frac{\tau_1 s + 1}{\tau_2 s + 1} U(s)$	`GAIN` , `TAU1` , `TAU2`	1 continuous state ($\tau_2 > 0$)
`LM`	Hard Limiter	$y(t) = \max(u_{\min}, \min(u_{\max}, u(t)))$	`UL` (Upper limit), `LL` (Lower limit)	None
`DZ`	Deadband	$y(t) = \begin{cases} u - \delta & u > \delta \\ u + \delta & u < -\delta \\ 0 & \lvert u \rvert \le \delta \end{cases}$	`DB` (Deadband half-width)	None
`HY`	Hysteresis Switch	Relay with toggle thresholds u_{on}, u_{off}	`U_ON` , `U_OFF` , `Y_HIGH` , `Y_LOW`	None (Discrete memory)
`TD`	Transport Delay	$y(t) = u(t - \tau)$	`TAU` (Delay time in seconds)	Buffer ring
`SW`	Controlled Switch	$y(t) = \begin{cases} u_1(t) & u_{ctrl} \ge u_{thresh} \\ u_2(t) & u_{ctrl} < u_{thresh} \end{cases}$	`THRESH` (Switch threshold)	None
`FN`	Table Function	$y(t) = \text{interp}(u(t), X, Y)$	`TABLE_NAME` , `ORDER` (1=linear, 2=spline)	None
`UG`	User General	Custom dynamics specified in Fortran 77/90	User-defined parameters	User-defined $(S(i), SD(i))$

---

2. Pre-Flight Diagnostic Rules (`lintEasy5`)

ModelScript evaluates 12 formal pre-flight diagnostic rules before simulation, linearization, or projection:

---

3. Direct Continuous DAE Arena Lowering (`lowerEasy5ToDae`)

ModelScript's data-oriented WebAssembly linear-memory DAE arena compiles EASY5 models directly into struct-of-arrays without intermediate TypeScript AST objects:

- Continuous states $S(i)$ become continuous variables with derivative bindings $\mathrm{der}(x_i) = f_i(x, u)$.
- Initial states S_0 become start attributes on continuous state variables.
- Summing junctions, gains, limiters, and lookup functions become lowered arena expression IDs (`ExprId`) and equality equations (`EqId`).
- Embedded Fortran blocks evaluate expressions directly mapped into DAE equation nodes.

---

4. Linear Control Systems Engine (`linear/`)

The linear control package provides exact numerical state-space modeling for control design and frequency domain verification:

State-Space Extraction (A, B, C, D)

Linearizes the continuous non-linear system around an operating point (x_0, u_0):

$$\dot{x} = Ax + Bu, \quad y = Cx + Du$$

Where:

$$A_{ij} = \frac{\partial \dot{x}_i}{\partial x_j}, \quad B_{ik} = \frac{\partial \dot{x}_i}{\partial u_k}, \quad C_{pj} = \frac{\partial y_p}{\partial x_j}, \quad D_{pk} = \frac{\partial y_p}{\partial u_k}$$

Computed via central finite differences:

$$\frac{\partial f_i}{\partial x_j} \approx \frac{f_i(x_0 + \varepsilon e_j, u_0) - f_i(x_0 - \varepsilon e_j, u_0)}{2\varepsilon}$$

Equilibrium Operating Point Trim Solver

Solves $\dot{x}(x, u) = 0$ using Levenberg-Marquardt damped least-squares:

$$(J^T J + \lambda I)\Delta x = -J^T \dot{x}(x, u)$$

Eigenvalue & Stability Analysis

Computes closed-loop poles $s_i = \sigma_i \pm j\omega_i$ using the **QR algorithm with Wilkinson shifts**:

- Natural frequency: $\omega_{n,i} = \sqrt{\sigma_i^2 + \omega_i^2}$
- Damping ratio: $\zeta_i = -\frac{\sigma_i}{\omega_{n,i}}$
- Stability criterion: $\sigma_i \le 0$ for all i

Bode Frequency Response Sweeps

Solves the complex resolvent system $(j\omega I - A)X(j\omega) = B$ at logarithmically spaced frequencies:

$$G(j\omega) = CX(j\omega) + D$$

- Magnitude (dB): $20 \log_{10} |G(j\omega)|$

- Phase (degrees): $\operatorname{atan2}(\operatorname{Im} G(j\omega), \operatorname{Re} G(j\omega)) \cdot \frac{180}{\pi}$

---

5. Cross-Language Projections

Projection to Modelica 3.x (`easy5ToModelica`)

Translates EASY5 continuous control blocks into canonical `Modelica.Blocks` continuous models:

```
// Modelica projection generated from EASY5 model: FlightController
model FlightController
  // Component instances
  Modelica.Blocks.Math.Gain GA01(k = 5.0);
  Modelica.Blocks.Math.Add SJ01;
  Modelica.Blocks.Continuous.Integrator IN01(k = 1.0, y_start = 0.0);
  Modelica.Blocks.Continuous.TransferFunction LA01(b = {0.1, 1.0}, a = {0.02, 1.0});
  Modelica.Blocks.Nonlinear.Limiter LM01(uMax = 10.0, uMin = -10.0);
equation
  connect(GA01.y, SJ01.u1);
  connect(SJ01.y, IN01.u);
  connect(IN01.y, LA01.u);
  connect(LA01.y, LM01.u);
end FlightController;
```

Projection to FMI SystemStructure (`easy5ToSsp`)

Exports complete multi-component topologies as FMI 2.0/3.0 `SystemStructure.ssd` XML containers for co-simulation orchestration.

---

6. Command-Line Interface (`msx easy5`)

```
# Model inspection and topological summary
msx easy5 summary controller.ez5

# Pre-flight linter (including algebraic loop check)
msx easy5 lint controller.ez5
msx lint controller.ez5 --language=easy5

# Numerical linearization to A, B, C, D state-space matrices
msx easy5 linearize controller.ez5

# Bode plot logarithmic frequency response calculation
msx easy5 bode controller.ez5 --min-freq=0.01 --max-freq=1000 --num-points=60

# Translation to Modelica 3.x
msx easy5 to-modelica controller.ez5 --out=Controller.mo

# Export to FMI SystemStructure.ssd
msx easy5 to-ssp controller.ez5 --out=SystemStructure.ssd

# Export SVG schematic diagram
msx easy5 diagram controller.ez5 --out=schematic.svg
```

CHAPTER 31

GT-SUITE / GT-POWER Modeling (`.gtm`, `.gtd`, `.gtx`, `.gt`)

The `@modelscript/gtsuite` package (`languages/gtsuite`) provides native compilation, simulation, analysis, and multi-domain projections for **Gamma Technologies GT-SUITE and GT-POWER** 1D engine, powertrain, turbocharging, and thermo-fluid dynamic simulation models.

Supported File Specifications

Extension	Format	Key Structural Features
`.gtm` / `.gtd`	GT-SUITE Model Deck	ASCII and XML representations of cylinders, plenums, runners, valves, injectors, and turbochargers
`.dat` / `.map`	Tabular Component Maps	Compressor and turbine pressure ratio vs. corrected mass flow and efficiency performance maps
`.gdx` / `.csv`	Trajectory Results	Crank-resolved in-cylinder pressure, temperature, burn rate, and emissions time series

---

Architecture & Compilation Pipeline



Pre-Flight Powertrain Linter Rules

Headless CLI Operations (`msx gtsuite`)

```
# Print model summary
msx gtsuite summary cases/Turbo4Cyl.gtm

# Run pre-flight static analysis
msx gtsuite lint cases/Turbo4Cyl.gtm

# Run in-WASM transient thermodynamic simulation
msx gtsuite simulate cases/Turbo4Cyl.gtm --duration=0.5 --out=trajectories.csv

# Evaluate formal verification gates (BSFC, boost, emissions, knock)
msx gtsuite verify cases/Turbo4Cyl.gtm --max-bsfc=240.0 --max-boost=2.8 --max-nox=60.0

# Export vector schematic SVG diagram
msx gtsuite diagram cases/Turbo4Cyl.gtm --view=schematic --out=engine_schematic.svg

# Project to Modelica and SysML v2
msx gtsuite to-modelica cases/Turbo4Cyl.gtm --out=Engine.mo
msx gtsuite to-sysml2 cases/Turbo4Cyl.gtm --out=Engine.sysml
```

CHAPTER 32

SINTEF Ocean SIMO / RIFLEX Modeling (`.inp` , `.sima` , `.sys` , `.hyd`)

The `@modelscript/sima` package (`languages/sima`) provides native ingestion, analysis, verification, DAE simulation, and multi-domain projections for **SINTEF Ocean / MARINTEK SIMO** (Simulation of Marine Operations) and **RIFLEX** (Riser and Slender Marine Structural Dynamics) offshore engineering models and modern **SIMA** project archives.

Supported File Specifications

Extension	Format	Key Structural Features
`.inp` / `.sys`	SIMO & RIFLEX Card Decks	Classical keyword cards: `SYS` , `BODY` , `MASS` , `FAIRLEAD` , `LINE` , `SEGM` , `CRS` , `WAVE` , `WIND` , `CURR` , `DYNMOD`
`.sima`	Modern SIMA XML Archive	Hierarchical task manifests (`<simo:SimoTask>` , `<riflex:RiflexTask>` , `<sima:SlenderSystem>` , `<sima:Vessel>` , `<sima:MooringLine>`)
`.hyd` / `.out`	Hydrodynamic Transfer Functions	1st-order Force RAO tables, frequency-dependent added mass $A(\omega)$, potential damping $B(\omega)$, and restoring matrices

---

Architecture & Compilation Pipeline



Pre-Flight Marine Linter Rules

Regulatory Verification & In-WASM Solvers

1. DNV-OS-E301 & DNV-ST-0119 Compliance

Evaluates line tension safety factors against Minimum Breaking Load (MBL):

$$S_F = \frac{T_{\text{MBL}}}{T_{\max}} \geq 1.67 \quad \text{(Intact Condition)}$$

$$S_F = \frac{T_{\text{MBL}}}{T_{\max}} \geq 1.25 \quad \text{(Damaged / Single Line Broken)}$$

2. Watch Circle & Touchdown Zone Verification

Computes vessel 6-DOF horizontal mean offset ellipse under coupled wind, wave drift, and current loads against allowable excursion limits (typically $\leq 8\%$ water depth for intact, $\leq 12\%$ for damaged). Checks fairlead declination angles ($\leq 75^\circ$) and ensures anchor uplift angle is exactly 0° for drag-embedment anchors.

3. ASTM E1049 Rainflow Fatigue & Palmgren-Miner Cumulative Damage

Extracts stress/tension reversal cycles using the ASTM E1049-85 four-point rainflow algorithm and computes cumulative fatigue damage using Palmgren-Miner's rule ($D = \text{DFF} \cdot \sum \frac{n_i}{N_i}$) against DNV-OS-E301 S-N curves for studless chain, stud link, spiral strand

wire, and polyester ropes.

4. Cummins 6-DOF Dynamics with Rational State-Space Radiation

Replaces the integro-differential retardation convolution integral with an $O(p)$ rational linear state-space system ($\dot{z} = A_r z + B_r \dot{x}$, $\mu = C_r z + D_r \dot{x}$), converting the frequency-dependent memory terms into continuous 1st-order ODEs lowered directly into `@modelscript/runtime`'s `DAEBuilder` in linear WebAssembly memory.

Headless CLI Operations (`msx sima`)

```
# Lint a SIMO / RIFLEX card deck or .sima XML archive
msx sima lint cases/OC4_Semi.inp

# Evaluate DNV-OS-E301 line tension safety factors against MBL
msx sima verify cases/OC4_Semi.inp --consequence-class=2

# Inspect floater watch circle, horizontal excursion, and touchdown clearances
msx sima offset cases/OC4_Semi.inp

# Inspect model summary (vessels, fairleads, lines, waves, wind)
msx sima summary cases/OC4_Semi.inp

# Synthesize acausal Modelica 6-DOF physical floater model
msx sima to-modelica cases/OC4_Semi.inp --out=OC4_Platform.mo

# Export multi-FMU SystemStructure.ssd co-simulation manifest
msx sima to-ssp cases/OC4_Semi.inp --out=SystemStructure.ssd

# Export formal SysML v2 architectural parts, ports, and DNV requirements
msx sima to-sysml2 cases/OC4_Semi.inp --out=OC4_Semi.sysml
```

CHAPTER 33

NASA ROCETS Liquid Propulsion Modeling (`.roc` , `.deck`)

The `@modelscript/rocets` package (`languages/rocets`) provides native compilation, simulation, and analysis for **NASA ROCETS (Rocket Engine Transient Simulator)** liquid propellant rocket engine decks and turbomachinery maps. It delivers multi-syntax parsing, embedded Fortran user routine bridge via `@modelscript/fortran` , pre-flight static propulsion diagnostics (`M8001` – `M8014`), direct continuous DAE arena lowering into `@modelscript/runtime` (`IDaeBuilder`), a coupled steady-state power and flow balance solver, dynamic start/shutdown transient integration, multi-domain projections to **Modelica 3.x** and **FMI SSP**, and interactive Piping & Instrumentation Diagram (P&ID) vector schematic generation.

Supported File Specifications

Extension	Format	Key Structural Features
`.roc` / `.deck`	Engine Model Deck	Tagged `DECK` , `COMPONENTS` , `PARAMS` , `S0` , `PORTS` , `CONNECTIONS` , `BALANCE` blocks
`.map` / `.dat`	Turbomachinery Maps	Multidimensional speed-flow-head and efficiency performance tables for pumps and turbines
`.f` / `.for`	User Subroutines	Embedded or external Fortran routines for custom fluid properties and component characteristics

---

Architecture & Compilation Pipeline



Propulsion Component Library

NASA ROCETS provides specialized thermodynamic and turbomachinery modules for rocket engine architectures:

Module	Category	Description	Primary Parameters & States
`CHAMBER` / `TC`	Combustion	Thrust chamber & convergent-divergent supersonic nozzle expansion	$P_c, A_t, \varepsilon, V_c, c^*, O/F$
`PUMP`	Turbomachinery	Centrifugal / axial propellant pump	$\Delta P, \eta, \rho, \dot{m}, P_{\text{shaft}}$
`TURBINE`	Turbomachinery	Gas-driven drive turbine	$PR, \eta, T_{\text{in}}, c_p, P_{\text{shaft}}$
`SHAFT`	Mechanical	Turbopump rotating assembly	$J, \omega, N_{\text{rpm}}, \tau_{\text{turb}}, \tau_{\text{pump}}$
`COOLING_JACKET`	Thermal	Regenerative nozzle cooling jacket	$T_{\text{wall}}, Q_{\text{gas}}, Q_{\text{cool}}, h_g, \Delta P_{\text{cool}}$
`VALVE` / `ORIFICE`	Fluid	Throttle, cavitating venturi, or shutoff valve	C_v, position, ΔP
`JUNCTION`	Fluid	Propellant flow splitter or mixing manifold	$\sum \dot{m} = 0, P_{\text{node}}$

Propellants & Thermodynamics

The `@modelscript/rocets` thermo engine provides:

- **Cryogenic & Hydrocarbon Propellants**: Standard properties for Liquid Oxygen (LOX), Liquid Hydrogen (LH2), Liquid Methane (LCH4), and RP-1 kerosene.

- **NASA CEA Combustion Curves**: Chemical equilibrium calculations of characteristic velocity (c^*), adiabatic flame temperature (T_c), isentropic expansion exponent (γ), and molecular weight.
- **Gas Dynamics**: Isentropic supersonic nozzle expansion with Mach area relations, vacuum and sea level thrust, and vacuum/SL specific impulse (I_{sp}).
- **Bartz Convective Heat Transfer**: Gas-side throat heat transfer coefficient (h_g) for regenerative cooling circuit sizing.

Pre-Flight Diagnostic Rules

Example ROCETS Engine Deck

```
DECK: GasGeneratorCycleEngine
TITLE: "Gas Generator Cycle Cryogenic Engine"

COMPONENTS:
  OPUMP TYPE=PUMP TITLE="LOX Turbopump" X=100 Y=150
    PARAMS: DELTA_P=14.0E6, ETA=0.78, RHO=1140.0
    PORTS: IN_P=0.3E6 FLUID=LOX DIR=in, OUT_P=14.3E6 FLUID=LOX DIR=out

  FPUMP TYPE=PUMP TITLE="LH2 Turbopump" X=100 Y=320
    PARAMS: DELTA_P=18.0E6, ETA=0.74, RHO=71.0
    PORTS: IN_P=0.2E6 FLUID=LH2 DIR=in, OUT_P=18.2E6 FLUID=LH2 DIR=out

  TURB01 TYPE=TURBINE TITLE="Drive Gas Turbine" X=320 Y=240
    PARAMS: PR=1.65, ETA=0.74, T_IN=950.0
    PORTS: IN_P=15.0E6, OUT_P=9.1E6

  SHAFT01 TYPE=SHAFT TITLE="Main Turbopump Shaft" X=210 Y=240
    PARAMS: INERTIA=0.1, SPEED_RPM=32000.0, MAX_RPM=40000.0
    S0: SPEED_RPM=32000.0

  CHAMBER TYPE=CHAMBER TITLE="Main Thrust Chamber" X=550 Y=240
    PARAMS: PC=10.0E6, MR=6.0, AT=0.035, EPS=50.0, VC=0.02
    S0: PC=10.0E6

  MAIN_OX_VALVE TYPE=VALVE TITLE="Main Oxidizer Valve" X=350 Y=150
    PARAMS: CV=0.005, POSITION=1.0

CONNECTIONS:
  OPUMP.OUT_P -> MAIN_OX_VALVE.IN_P
  MAIN_OX_VALVE.OUT_P -> CHAMBER.IN_OX
  FPUMP.OUT_P -> CHAMBER.IN_FUEL
  TURB01.OUT_P -> CHAMBER.IN_GAS
  SHAFT01.TORQUE_TURB -> TURB01.TORQUE
  SHAFT01.TORQUE_PUMP -> FPUMP.TORQUE
```

Command Line Interface (`msx rocets`)

ModelScript provides full headless CLI tooling for NASA ROCETS models:

```
# Pre-flight linting of an engine deck
msx rocets lint engine.roc

# Print engine architecture summary and flow connections
msx rocets summary engine.roc

# Solve steady-state power and flow balance
msx rocets balance engine.roc --target-pc=12E6 --mr=6.0

# Simulate dynamic ignition and ramp transient
msx rocets simulate engine.roc --t-end=2.0

# Project to executable Modelica 3.x
msx rocets to-modelica engine.roc --out=Engine.mo

# Export to FMI 2.0/3.0 System Structure Description (SSP)
msx rocets to-ssp engine.roc --out=SystemStructure.ssd

# Render vector P&ID flow diagram
msx rocets diagram engine.roc --out=pid.svg
```

VS Code & Web IDE Custom Editor

The `@modelscript/ide-client` package provides a dedicated dual-mode custom editor (`modelscript.rocetsEditor`):

- **P&ID Vector Schematic**: Interactive diagram with propellant stream color coding, zoom/pan navigation, and SVG export.
- **Engine Balance Panel**: Shaft power metrics, chamber stagnation conditions, sea level / vacuum thrust, and I_{sp}.
- **Start Transient Viewer**: Ignition ODE trajectory table and graphs ($P_c(t)$, $\mathrm{RPM}(t)$, $F(t)$, $T_w(t)$).
- **Module Netlist**: Filterable tabular listing of modules, parameters, and ports.
- **Pre-Flight Diagnostics**: Live diagnostic warnings and error badges with click-to-line navigation.
- **Projections**: Side-by-side Modelica and FMI SSP code viewers with one-click export.

CHAPTER 34

STEP (ISO 10303-21)

`@modelscript/step` (`languages/step`) provides a native WebAssembly GLR parser, assembly hierarchy extractor, AP242 Geometric Dimensioning & Tolerancing (GD&T / PMI) analyzer, and Modelica MultiBody kinematic synthesizer for ISO 10303-21 Clear Text Encoding of the Exchange Structure (`.step` , `.stp` , `.p21`).

Supported Extensions & Standards

Extension	Standard	Scope
`.step` , `.stp` , `.p21`	ISO 10303-21 (AP203, AP214, AP242)	B-Rep solid geometry, product assembly trees (`NEXT_ASSEMBLY_USAGE_OCCURRENCE`), kinematic joints, and PMI/GD&T tolerances

---

Capabilities

1. **Zero-Dependency STEP Part 21 Parser**: Parses `HEADER;` and `DATA;` sections, `#id = ENTITY_NAME(...)` instances, nested typed parameters, and enumeration tokens into linear memory.
2. **Assembly Hierarchy & Mass Properties (`extractStepAssembly`)**: Extracts `PRODUCT_DEFINITION` parts, `NEXT_ASSEMBLY_USAGE_OCCURRENCE` parent–child assembly trees, `AXIS2_PLACEMENT_3D` rigid-body frames, bounding boxes, and mass/inertia properties.
3. **AP242 PMI & GD&T Extraction (`extractStepAssembly`)**: Extracts datum features (`DATUM` , `DATUM_FEATURE` , `DATUM_SYSTEM`) and **10 geometric tolerance categories** (`StepGeometricTolerance` in `languages/step/src/extractor.ts`) for 3D tolerance stack-up analysis (`ToleranceStackOracle`):

GD&T Tolerance Kind	STEP AP242 Entity Name
`flatness`	`FLATNESS_TOLERANCE`
`roundness`	`ROUNDNESS_TOLERANCE` / `CIRCULARITY_TOLERANCE`
`cylindricity`	`CYLINDRICITY_TOLERANCE`
`perpendicularity`	`PERPENDICULARITY_TOLERANCE`
`parallelism`	`PARALLELISM_TOLERANCE`
`position`	`POSITION_TOLERANCE`
`surface_profile`	`SURFACE_PROFILE_TOLERANCE`
`concentricity`	`CONCENTRICITY_TOLERANCE` / `COAXIALITY_TOLERANCE`
`runout`	`CIRCULAR_RUNOUT_TOLERANCE` / `TOTAL_RUNOUT_TOLERANCE`
`general`	`GEOMETRIC_TOLERANCE` (with `TOLERANCE_VALUE` / `LENGTH_MEASURE_WITH_UNIT`)

4. **Modelica MultiBody Kinematic Synthesis (`mapStepToMultiBody`)**: `languages/step/src/multibody-mapper.ts` translates STEP kinematic joints and `AXIS2_PLACEMENT_3D` transforms directly into `Modelica.Mechanics.MultiBody` components:
   - **Kinematic Joints**: Maps `REVOLUTE_PAIR` → `Modelica.Mechanics.MultiBody.Joints.Revolute` , `PRISMATIC_PAIR` → `Joints.Prismatic` , `CYLINDRICAL_PAIR` → `Joints.Cylindrical` , `SPHERICAL_PAIR` → `Joints.Spherical` , and `PLANAR_PAIR` → `Joints.Planar` with axis vectors `n` and `connect(...)` equations.
   - **Static Frame Placements (`MultiBodyFixedTranslation`)**: Computes exact translation vectors `r = {x, y, z}` (converting `MILLI` to meters) and 3×3 orthonormal orientation matrices from `AXIS2_PLACEMENT_3D` `axis` ($\hat{z}$) and `ref_direction` ($\hat{x}$) vectors to emit `Modelica.Mechanics.MultiBody.Parts.FixedTranslation` and `Parts.BodyBox` / `Parts.BodyCylinder` components.

IDE & CAE Mesh Generation Actions

`stepLanguage.actions` (`languages/step/src/language.ts`) provides 6 built-in CAD/CAE workflows:

Action ID	Description
`open_cad_viewer`	Opens the interactive 3D B-Rep & assembly viewer for the `.step` model
`generate_multibody`	Synthesizes a `Modelica.Mechanics.MultiBody` `.mo` model from STEP assembly frames and joints
`generate_fea_mesh`	Tessellates the STEP B-Rep solid into a 3D tetrahedral CalculiX `.inp` finite element mesh
`generate_cfd_mesh`	Generates an unstructured `.su2` surface/volume fluid mesh from the STEP geometry
`create_fea_setup`	Scaffolds a parametric `.inpt` CalculiX structural study linked to the STEP part
`create_cfd_setup`	Scaffolds a parametric `.cfgt` SU2 aerodynamic study linked to the STEP part

---

Dedicated `step-mcp` Server Tools

The `@modelscript/step` MCP server (`languages/step/src/mcp/server.ts`) exposes two specialized tools for AI engineering agents:

- **`step_extract_entities`** : Parses a `.step` / `.stp` file and returns the structured assembly tree, `AXIS2_PLACEMENT_3D` coordinate frames, kinematic joints, and AP242 GD&T geometric tolerances.
- **`step_to_multibody`** : Generates complete `Modelica.Mechanics.MultiBody` code (including `World` , `Parts.Body` , `Parts.FixedTranslation` , `Joints.*` , and `connect(...)` equations) from a STEP assembly.

Polyglot TGG Rules

`@modelscript/step` registers 5 bidirectional Triple Graph Grammar (TGG) rules in `languages/step/src/language.ts` :

Rule Name	Source (`step`)	Target Language	Target Construct
`StepProductToSysML2Part`	`DataEntity` (`PRODUCT`)	`sysml2`	`PartDef`
`StepPropertyToSysML2Attribute`	`DataEntity` (`PROPERTY_DEFINITION`)	`sysml2`	`AttributeUsage`
`StepPlacementToSysML2Port`	`DataEntity` (`AXIS2_PLACEMENT_3D`)	`sysml2`	`PortUsage`
`StepRevoluteJointToModelicaRevolute`	`DataEntity` (`REVOLUTE_PAIR`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Joints.Revolute`)
`StepPrismaticJointToModelicaPrismatic`	`DataEntity` (`PRISMATIC_PAIR`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Joints.Prismatic`)

CLI Usage

```
# Parse and inspect the CST of an ISO 10303-21 STEP file
npx msx parse chassis.step

# Project a STEP assembly into a Modelica MultiBody model via TGG rules
npx msx polyglot project chassis.step --target modelica --out chassis.mo

# Verify cross-domain alignment between STEP CAD geometry and SysML v2 architecture
npx msx align chassis.step architecture.sysml
```

Diagnostic Rules

CHAPTER 35

OWL2 Ontology (`.owl`, `.owl2`, `.ofn`, `.ttl`)

`@modelscript/owl2` (`languages/owl2`) brings formal ontological engineering, Description Logic ($\mathrm{SROIQ}(\mathrm{D})$) reasoning, and SPARQL / Leapfrog Triejoin graph querying directly into the ModelScript compiler.

Supported Extensions & Formats

Extension	Format
`.owl`, `.owl2`, `.ofn`	W3C OWL 2 Functional-Style Syntax (`Prefix(...)`, `Ontology(...)`)
`.ttl`	RDF Turtle / Ontological Triple Assertions

OWL 2 DL Axiom & Class Expression Lowering

`lowerCstToAxioms(tree, sourceText)` (`languages/owl2/src/cst-lowering.ts`) lowers the linear-memory CST into an arena-backed `OntologyStore` supporting over 35 OWL 2 DL axiom and class expression constructs:

Category	Supported OWL 2 Functional-Style Constructs
Declarations	`Declaration(Class(...))`, `Declaration(ObjectProperty(...))`, `Declaration(DataProperty(...))`, `Declaration(NamedIndividual(...))`
Class Axioms	`SubClassOf`, `EquivalentClasses`, `DisjointClasses`
Boolean & Nominal Class Expressions	`ObjectIntersectionOf`, `ObjectUnionOf`, `ObjectComplementOf`, `ObjectOneOf`
Object Property Restrictions	`ObjectSomeValuesFrom`, `ObjectAllValuesFrom`, `ObjectHasSelf`, `ObjectHasValue`, `ObjectMinCardinality`, `ObjectMaxCardinality`, `ObjectExactCardinality`
Data Property Restrictions	`DataSomeValuesFrom`, `DataAllValuesFrom`, `DataMinCardinality`, `DataMaxCardinality`, `DataExactCardinality`
Property Hierarchy & Inversion	`SubObjectPropertyOf`, `SubDataPropertyOf`, `InverseObjectProperties`, `DisjointObjectProperties`
Property Domains & Ranges	`ObjectPropertyDomain`, `ObjectPropertyRange`, `DataPropertyDomain`, `DataPropertyRange`
Property Characteristics	`FunctionalObjectProperty`, `InverseFunctionalObjectProperty`, `ReflexiveObjectProperty`, `IrreflexiveObjectProperty`, `SymmetricObjectProperty`, `AsymmetricObjectProperty`, `TransitiveObjectProperty`, `FunctionalDataProperty`
ABox Individual Assertions	`ClassAssertion`, `ObjectPropertyAssertion`, `DataPropertyAssertion`, `NegativeObjectPropertyAssertion`, `NegativeDataPropertyAssertion`, `SameIndividual`, `DifferentIndividuals`

```
Prefix(:=<http://example.org/spacecraft#>)
Ontology(<http://example.org/spacecraft>
  Declaration(Class(:Thruster))
  Declaration(Class(:Actuator))
  Declaration(ObjectProperty(:mountedOn))
  Declaration(DataProperty(:nominalThrustN))
  SubClassOf(:Thruster :Actuator)
  SubClassOf(:Thruster ObjectSomeValuesFrom(:mountedOn :ReactionControlPod))
  ClassAssertion(:Thruster :RCS_Thruster_A1)
  DataPropertyAssertion(:nominalThrustN :RCS_Thruster_A1 "22.0"^^xsd:double)
)
```

Description Logic Tableau Reasoner & Leapfrog Triejoin

`@modelscript/owl2` includes an in-memory $\mathcal{SROIQ}(\mathrm{D})$ **Tableau Reasoner** (`TableauReasoner` in `languages/owl2/src/reasoner/`) and a worst-case optimal **Leapfrog Triejoin** engine (`executeBGP` in `languages/owl2/src/query/leapfrog.ts`):

1. **Consistency & Satisfiability Checking**: Detects unsatisfiable classes (`DisjointClasses` violations, cardinality contradictions, irreflexive/asymmetric cycles, negative property assertion conflicts) and surfaces exact source diagnostics (`OWL2001` – `OWL2005`).
2. **Justification & Explanation Tracing**: Computes minimal unsatisfiability justifications (`explain_inference`) explaining why a class or individual violates ontology axioms.
3. **SPARQL & Multi-Way BGP Joins**: Evaluates conjunctive Basic Graph Patterns (BGPs) in $O(\mathrm{AGM\ bound})$ worst-case optimal time without materializing intermediate Cartesian products.

Interactive 2D Ontology Diagram & Bidirectional Writeback

`languages/owl2/src/diagram/` provides an interactive 2D graph visualization and editing surface for OWL 2 ontologies:

- **Visual Graph Projection**: Renders `Class` nodes, `NamedIndividual` instances, `SubClassOf` inheritance hierarchies, and `ObjectPropertyDomain` / `ObjectPropertyRange` directed edges.
- **Bidirectional CST Writeback (`owl2Language.writeback`)**: Adding/deleting nodes or edges in the diagram (`add_node` , `delete_node` , `add_edge` , `delete_edge` , `rename_node` , `update_parameter`) patches the Functional-Style `.owl` source text directly—inserting or removing `Declaration(...)` , `SubClassOf(...)` , or `DataPropertyAssertion(...)` axioms in place.

Dedicated `owl2-mcp` Server Tools

The `@modelscript/owl2` MCP server (`languages/owl2/src/mcp/server.ts`) exposes five specialized tools to AI agents:

MCP Tool	Description
`validate_system_consistency`	Parses an OWL 2 Functional Syntax ontology, runs the $\mathcal{SROIQ}$ Tableau Reasoner, and returns consistency status, unsatisfiable classes, and inferred `SubClassOf` hierarchy
`query_ontology_sparql`	Executes a SPARQL `SELECT` `WHERE` query over the ontology's inferred RDF triple graph
`query_ontology_bgp`	Evaluates a conjunctive Basic Graph Pattern (BGP) using the worst-case optimal Leapfrog Triejoin algorithm
`explain_inference`	Computes formal axiomatic justifications explaining why an entailment holds or why a class is unsatisfiable
`trace_fault_propagation`	Traces transitive fault propagation (`TransitiveObjectProperty` / causal links) from a starting component individual across the system topology

---

Polyglot TGG Rules

`@modelscript/owl2` defines 5 bidirectional Triple Graph Grammar (TGG) rules in `languages/owl2/src/language.ts` :

Rule Name	Source (`owl2`)	Target Language	Target Construct
`OWL2ClassToSysML2Part`	`Declaration` (`Class`)	`sysml2`	`PartDef`
`OWL2SubClassToSysML2Specialization`	`SubClassOf`	`sysml2`	`Specialization` (`:>`)
`OWL2ObjectPropertyToSysML2Port`	`Declaration` (`ObjectProperty`)	`sysml2`	`PortUsage`
`OWL2DataPropertyToSysML2Attribute`	`Declaration` (`DataProperty`)	`sysml2`	`AttributeUsage`
`OWL2ClassToModelicaModel`	`Declaration` (`Class`)	`modelica`	`ClassDefinition` (`model`)

Command Line Interface (`msx polyglot`)

OWL2 ontologies can be queried, projected, and validated using ModelScript's polyglot digital thread commands:

```
# Project OWL 2 DL class hierarchies and axioms into formal SysML v2 part definitions
msx polyglot project SpacecraftOntology.ofn --target=sysml2 -o Spacecraft.sysml

# Cross-query ontology symbols and digital thread twins in the unified Salsa index
msx polyglot query Spacecraft::ReactionControlPod --domain=owl2

# Verify losslessness and critical pair confluence across OWL 2 TGG synchronization rules
msx polyglot verify --strict
```

Diagnostic Rules

CHAPTER 36

OpenSCAD (`.scad`)

`@modelscript/scad` (`languages/scad`) parses parametric OpenSCAD scripts and evaluates Constructive Solid Geometry (CSG) trees via `@modelscript/cad` into exact OpenCascade B-Rep solids, watertight triangle meshes, STEP AP214/AP242 exports, and Modelica MultiBody parameters.

Supported Syntax & Fluent Method Chaining

In addition to standard OpenSCAD primitives (`cube` , `sphere` , `cylinder` , `polyhedron` , `circle` , `square` , `polygon`), boolean operations (`union` , `difference` , `intersection` , `hull` , `minkowski`), extrusions (`linear_extrude` , `rotate_extrude`), and control flow (`module` , `function` , `for` , `if` / `else`), `@modelscript/scad` supports **fluent method chaining** (`ChainedSolid` in `languages/scad/src/evaluator.ts`) and **boundary port tagging**:

```
wall_thickness = 4.0;
mount_radius = 12.0;

// Fluent method chaining with .fillet(), .chamfer(), .translate(), .rotate()
bracket = cube([60, 40, wall_thickness], center=true)
  .fillet(1.5)
  .translate([0, 0, wall_thickness / 2]);

// Explicit physical port tagging for SysML v2 / Modelica MultiBody & CFD/FEA binding
tag_port("base_mount", "frame", [0, 0, -1])
  difference() {
    bracket;
    cylinder(h=10, r=mount_radius, center=true).chamfer(0.5);
  }
```

Supported Chained Methods (`ChainedSolid`)

Method	Signature	Description
`.translate(v)`	`translate([dx, dy, dz])`	Translates the solid by vector `[dx, dy, dz]`
`.rotate(a, v?)`	`rotate([rx, ry, rz])` or `rotate(deg, axis)`	Rotates the solid by Euler angles or axis-angle
`.scale(v)`	`scale([sx, sy, sz])`	Scales the solid non-uniformly or uniformly
`.mirror(v)`	`mirror([nx, ny, nz])`	Mirrors the solid across the plane with normal `v`
`.color(c, alpha?)`	`color("steelblue", 0.9)`	Assigns RGBA display metadata
`.fillet(r)`	`fillet(radius)`	Applies a 3D edge fillet of radius $r > 0$
`.chamfer(d)`	`chamfer(distance)`	Applies a 3D edge chamfer of distance $d > 0$

Semantic Linter & Diagnostic Codes

`lintScadCst(sourceText)` (`languages/scad/src/linter.ts`) performs static scope resolution, constant folding, and geometric validity checking across 9 diagnostic rules:

3D Viewport Gizmo Writeback (`ScadPatcher`) & STEP Export

- **Surgical AST Writeback (`ScadPatcher` in `languages/scad/src/patcher.ts`)**: When a user drags a 3D translation/rotation/scale gizmo in the CAD viewport, `ScadPatcher` (`patchNode` , `patchVariable` , `patchTransformArgument` , `patchVectorComponent` , `wrapPrimitiveWithTransform` , `patchOrInsertTransform`) updates the exact CST byte spans in the `.scad` source without reformatting surrounding code or comments.

- **STEP AP214/AP242 Export (`compileScadToStep`)**: Evaluates the `.scad` CSG tree and serializes exact B-Rep analytical surfaces (`PLANE` , `CYLINDRICAL_SURFACE` , `SPHERICAL_SURFACE` , `MANIFOLD_SOLID_BREP`) to ISO 10303-21 `.step` .

```
# Project a parametric OpenSCAD file to an ISO 10303-21 STEP CAD file
npx msx polyglot project bracket.scad --target step --out bracket.step

# Or extract Embedded CSG from a Modelica model to OpenSCAD
npx msx csg SatelliteChassis satellite.mo > bracket.scad
```

Polyglot TGG Rules

`@modelscript/scad` registers 13 bidirectional Triple Graph Grammar (TGG) rules in `languages/scad/src/language.ts` :

Rule Name	Source (`scad`)	Target Language	Target Construct
`ScadModuleToSysmlPart`	`ModuleDeclaration`	`sysml2`	`PartDef`
`ScadProjectedAreaToModelicaParam`	`ScadScript` (`projectedArea` , `volume`)	`modelica`	`ComponentClause` (`parameter Real`)
`ScadProjectedAreaToSysmlAttribute`	`ScadScript` (`projectedArea` , `volume`)	`sysml2`	`AttributeUsage`
`ScadTagPortToSysmlPort`	`ModuleInstantiation` (`tag_port`)	`sysml2`	`PortUsage`
`ScadVariableToSysmlAttribute`	`VariableAssignment`	`sysml2`	`AttributeUsage`
`ScadSolidToStepProduct`	`ModuleInstantiation` (`cube`)	`step`	`DataEntity` (`PRODUCT`)
`ScadCylinderToStepProduct`	`ModuleInstantiation` (`cylinder`)	`step`	`DataEntity` (`CYLINDRICAL_SURFACE`)
`ScadSphereToStepProduct`	`ModuleInstantiation` (`sphere`)	`step`	`DataEntity` (`SPHERICAL_SURFACE`)
`ScadBooleanOpToStepComposite`	`ModuleInstantiation` (`union` / `difference` / `intersection`)	`step`	`DataEntity` (`BOOLEAN_RESULT`)
`ScadTagPortToModelicaFrame`	`ModuleInstantiation` (`tag_port`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Interfaces.Frame_a`)
`ScadCubeToMultiBodyShape`	`ModuleInstantiation` (`cube`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Visualizers.FixedShape` `"box"`)
`ScadCylinderToMultiBodyShape`	`ModuleInstantiation` (`cylinder`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Visualizers.FixedShape` `"cylinder"`)
`ScadSphereToMultiBodyShape`	`ModuleInstantiation` (`sphere`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Visualizers.FixedShape` `"sphere"`)

CHAPTER 37

SSP (System Structure and Parameterization)

`@modelscript/ssp` (`languages/ssp`) implements the Modelica Association **SSP 1.0 / 2.0** standard for packaging and coupling Functional Mock-up Units (FMUs) into multi-domain Co-Simulation architectures.

Supported Extensions & Container Architecture

Extension	Standard Artifact	Role
`.ssp`	System Structure Package (ZIP archive)	Container bundling `SystemStructure.ssd` , `resources/*.fmu` , and `*.ssv` parameter sets
`.ssd`	System Structure Description (XML)	Hierarchical `System` , `Components` , `Connectors` , `Connections` , and `DefaultExperiment` topology
`.ssv`	System Structure Parameter Values (XML)	Typed parameter bindings (`Real` , `Integer` , `Boolean` , `String` , `Enumeration`)

Virtual File System (VFS) Archive Extraction

When a `.ssp` ZIP archive is loaded into the workspace, `sspLanguage.container` (`languages/ssp/src/language.ts`) automatically unpacks and indexes its internal artifacts:

- `*.ssd` and `*.ssv` entries are dispatched via `parse_as_language` into the `@modelscript/ssp` CST & Salsa index.
- `resources/*.fmu` archives are recursively extracted (`extract_nested`) so their internal `modelDescription.xml` ports and parameters are cross-checked against `.ssd` connectors.
- Auxiliary tables and binaries under `resources/*` are mounted into the in-memory VFS (`mount_vfs`).

Interactive 2D Diagram Builder & Bidirectional XML Writeback

`languages/ssp/src/diagram.ts` provides a two-way schematic editor for `.ssd` topologies:

- **2D Schematic Extraction (`buildSspDiagramData`)**: Parses `<ssd:System>` , `<ssd:Component>` , `<ssd:Connector>` , `<ssd:Connection>` , and `<ssd:ElementGeometry>` / `<ssd:ConnectionGeometry>` annotations into positioned block nodes, typed causal ports, and orthogonal routing waypoints.
- **Surgical XML Writeback (`applySspDiagramEdits` / `sspLanguage.writeback`)**: Moving components, adding/removing FMU blocks, drawing `<ssd:Connection>` wires, or editing `<ssv:Parameter>` values updates the underlying `.ssd` or `.ssv` XML document in place while preserving namespace prefixes and custom vendor annotations.

Modelica Architecture Projection (`sspToModelicaBlock`)

`sspToModelicaBlock(ssdDoc)` (`languages/ssp/src/index.ts`) projects any `.ssd` system topology into an equivalent composite Modelica `model` , translating `<ssd:Connector>` definitions into `Modelica.Blocks.Interfaces.RealInput` / `RealOutput` (or `Integer` / `Boolean` causal interfaces), `<ssd:Component>` blocks into typed sub-models with `<ssv:Parameter>` modifications, and `<ssd:Connection>` edges into `connect(...)` equations.

Semantic Linter & Diagnostic Codes

`lintSsd` and `lintSsv` (`languages/ssp/src/linter.ts`) validate structural, causal, and type soundness across `.ssd` and `.ssv` files with 20 diagnostic codes (13 `SSP_*` + 7 `SSV_*`):

`.ssd` System Structure Diagnostics (`lintSsd`)

Command Line Interface (`msx fmu` & `msx cosim`)

ModelScript CLI provides turnkey subcommands for packaging, validating, and co-simulating SSP archives:

```
# Validate an FMU or SSP container
msx fmu validate VehicleChassis.fmu

# Simulate an FMU archive directly via SU2, CVODE, or BDF solvers
msx fmu simulate Powertrain.fmu --stop-time=10.0 --step-size=0.001 --format=csv -o output.csv

# Optimize SSP parameter values using differential evolution (CMA-ES, DE, PSO)
msx cosim optimize PlantModel.ssp --params="kp:0.1:10.0,kd:0.01:1.0" --objective="tracking_error"
```

Polyglot TGG Rules

`@modelscript/ssp` registers 8 bidirectional Triple Graph Grammar (TGG) rules in `languages/ssp/src/language.ts` :

Rule Name	Source (`ssp`)	Target Language	Target Construct
`SspSystemToModelicaModel`	`SsdSystem`	`modelica`	`ClassDefinition` (`model`)
`SspComponentToModelicaComponent`	`SsdComponent`	`modelica`	`ComponentClause`
`SspConnectorToModelicaConnector`	`SsdConnector`	`modelica`	`ComponentClause` (`RealInput` / `RealOutput`)
`SspConnectionToModelicaConnect`	`SsdConnection`	`modelica`	`ConnectClause`
`SspSystemToSysmlPackage`	`SsdSystem`	`sysml2`	`PartDef`
`SspComponentToSysmlPart`	`SsdComponent`	`sysml2`	`PartUsage`
`SspConnectorToSysmlPort`	`SsdConnector`	`sysml2`	`PortUsage`
`SspConnectionToSysmlConnection`	`SsdConnection`	`sysml2`	`ConnectionUsage`

CHAPTER 38

CFD (SU2 & OpenFOAM)

`@modelscript/cfd` (`languages/cfd`) provides native WebAssembly GLR parsing, parametric configuration materialization, 3D CAD boundary marker synthesis, aerodynamic flow regime analysis, and cross-domain TGG synchronization for Computational Fluid Dynamics solver configurations.

Supported Extensions & Dialects

Extension	Dialect	Description
`.cfg` , `.su2`	SU2 Configuration	Native Stanford University Unstructured (`SU2_CFD`) key-value & marker configuration
`.cfgt`	Parametric CFD Template	SU2 or OpenFOAM template supporting `{{ expr }}` parameter expressions
`.cfd`	OpenFOAM / Native CFD	OpenFOAM dictionary syntax (`FoamFile` , `dimensions` , `internalField` , `boundaryField`) and SU2 directives

---

Parametric Template Materialization (`{{ expr }}`)

CFD templates (`.cfgt` , `.cfg` , `.cfd`) can embed mathematical expressions in `{{ ... }}` blocks bound to SysML v2 or Modelica workspace parameters:

```
SOLVER= RANS
KIND_TURB_MODEL= SST
MACH_NUMBER= {{ cruise_speed / sqrt(1.4 * 287.05 * ambient_temp) }}
AOA= {{ angle_of_attack_deg }}
REYNOLDS_NUMBER= {{ rho_air * cruise_speed * chord_length / mu_air }}
MARKER_HEATFLUX= ( battery_plate, {{ heat_dissipation_watts / plate_area }} )
```

`materializeCfdConfig(templateText, parameters)` (`languages/cfd/src/materializer.ts`) evaluates each expression via `evaluateCfdExpression` , supporting arithmetic operators (`+` , `-` , `*` , `/` , `^`), built-in math functions (`sin` , `cos` , `tan` , `sqrt` , `abs` , `min` , `max` , `log` , `exp`), and constants (`PI` , `E`).

3D CAD Boundary Condition Synthesis

When a user assigns a fluid or thermal boundary condition to a 3D CAD face in the viewport, `synthesizeCfdBoundary` and `applyBoundaryActionToConfig` (`languages/cfd/src/boundary-synthesizer.ts`) synthesize and merge the appropriate directives for either **SU2** or **OpenFOAM**:

Boundary Condition	SU2 Directive (`.cfg` / `.su2`)	OpenFOAM `boundaryField` Entry (`.cfd`)
Velocity / Total Inlet (`inlet`)	`MARKER_INLET= ( <face>, <T_total>, <P_total>, <nx>, <ny>, <nz> )`	`type fixedValue; value uniform (<vx> <vy> <vz>);`
Pressure Outlet (`outlet`)	`MARKER_OUTLET= ( <face>, <back_pressure_Pa> )`	`type fixedValue; value uniform <pressure>;`
Adiabatic / Heat-Flux Wall (`wall`)	`MARKER_HEATFLUX= ( <face>, <heat_flux_W_m2> )`	`type noSlip;` / `type fixedGradient; gradient uniform <q>;`
Isothermal Wall (`isothermal`)	`MARKER_ISOTHERMAL= ( <face>, <temp_K> )`	`type fixedValue; value uniform <temp_K>;`
Far-Field (`farfield`)	`MARKER_FAR= ( <face> )`	`type freestream;`
Symmetry Plane (`symmetry`)	`MARKER_SYM= ( <face> )`	`type symmetryPlane;`

Aerodynamic Theory Slicing & Flow Regime Analysis

`extractCfdSlicingProperties(text)` (`languages/cfd/src/language.ts`) extracts aerodynamic coefficients directly from the CST:

- **`cd`** : Drag coefficient (`DRAG_COEFF` , `CD` , or `C_D` , default `0.3`)
- **`refArea`** : Reference wetted/frontal area (`REF_AREA` , default `1.0` m^2)
- **`density`** : Freestream fluid density (`FREESTREAM_DENSITY` or `RHO` , default `1.225` $\mathrm{kg/m}^3$)
- **`freeStreamVelocity`** : Freestream velocity (`FREESTREAM_VELOCITY` , default `30.0` $\mathrm{m/s}$)

These properties feed directly into Modelica 1D aerodynamic drag models ($F_d = \frac{1}{2}\rho v^2 C_d A_{\text{ref}}$), SysML v2 performance requirements, and the Multi-Domain Theory Coordinator (`CaeContractOracle`).

Polyglot TGG Rules

`@modelscript/cfd` registers 6 bidirectional Triple Graph Grammar (TGG) rules in `languages/cfd/src/language.ts` :

Rule Name	Source (`cfd`)	Target Language	Target Construct
`CfdDragPolarToModelicaParam`	`CfdConfig` (`cd` , `refArea`)	`modelica`	`ComponentClause` (`parameter Real Cd` , `A_ref`)
`CfdDragPolarToSysmlAttribute`	`CfdConfig` (`cd` , `refArea`)	`sysml2`	`AttributeUsage`
`CfdInletToModelicaFluidPort`	`Directive` (`MARKER_INLET`)	`modelica`	`ComponentClause` (`Modelica.Fluid.Interfaces.FluidPort_a`)
`CfdHeatFluxToModelicaHeatPort`	`Directive` (`MARKER_HEATFLUX`)	`modelica`	`ComponentClause` (`Modelica.Thermal.HeatTransfer.Interfaces.HeatPort_a`)
`CfdMarkerToSysmlPort`	`Directive` (`MARKER_*`)	`sysml2`	`PortUsage`
`CfdDirectiveToSysmlConstraint`	`Directive` (`MACH_NUMBER` , `REYNOLDS_NUMBER`)	`sysml2`	`ConstraintUsage`

Command Line Interface (`msx cfd`)

```
msx cfd <action> [files...] [options]
```

Action	Description	Key Options
`lint`	Validates SU2 and OpenFOAM configuration syntax, boundary markers, and thermodynamic consistency.	—
`summary`	Prints turbulence model, freestream conditions, reference areas, and configured markers.	—
`inspect`	Dumps parsed marker tree, solver settings, and evaluated parametric expressions.	—
`to-modelica`	Generates Modelica 1D aerodynamic drag models (`Modelica.Mechanics.Translational`) and ports.	`--out=<file.mo>`
`to-dae`	Lowers boundary conditions and drag polar into ModelScript linear-memory DAE arena.	`--out=<file.json>`
`solve`	Invokes external or embedded SU2 flow solver iteration for steady/unsteady aerodynamic fields.	—

Examples

```
# Validate boundary markers and turbulence settings in an SU2 configuration
msx cfd lint transonic_wing.cfg

# Display mesh markers, Mach number, and reference area
msx cfd summary wing_template.cfgt

# Project aerodynamic drag polar to standard Modelica component
msx cfd to-modelica transonic_wing.cfg --out=AeroDrag.mo
```

Diagnostic Rules

CHAPTER 39

FEA (CalculiX, Abaqus & NASTRAN)

`@modelscript/fea` (`languages/fea`) provides native parsing, parametric template materialization, 3D CAD boundary condition synthesis, structural theory slicing, and cross-domain TGG synchronization for Finite Element Analysis input decks.

Supported Dialects & Extensions

Extension	Dialect	Key Directives
`.inp`	CalculiX / Abaqus input deck	`*HEADING` , `*NODE` , `*ELEMENT` , `*NSET` , `*ELSET` , `*MATERIAL` , `*ELASTIC` , `*DENSITY` , `*SOLID SECTION` , `*STEP` , `*STATIC` , `*FREQUENCY` , `*BOUNDARY` , `*CLOAD` , `*DLOAD`
`.inpt`	Parametric CalculiX / Abaqus template	Supports `{{ expression }}` mustache substitutions over workspace parameters
`.bdf`	NASTRAN Bulk Data Deck	`SOL` , `CEND` , `BEGIN BULK` , `GRID` , `CTETRA` , `CHEXA` , `PSOLID` , `MAT1` , `SPC1` , `FORCE` , `MOMENT` , `LOAD` , `ENDDATA`
`.fea`	ModelScript native FEA configuration	Structured material, mesh, boundary, and load study definitions

---

Parametric Template Materialization (`{{ expr }}`)

Input decks (`.inpt` , `.inp` , `.bdf` , `.fea`) can embed parametric expressions enclosed in `{{ ... }}` placeholders:

```
*MATERIAL, NAME=AL7075
*ELASTIC
{{ E_modulus }}, {{ poisson_ratio }}
*DENSITY
{{ rho_al }}
*CLOAD
LOAD_SET, 3, {{ -tip_mass * 9.81 }}
```

`materializeFeaDeck(templateText, parameters)` (`languages/fea/src/materializer.ts`) evaluates arithmetic expressions (`+` , `-` , `*` , `/` , `^`), functions (`sin` , `cos` , `tan` , `sqrt` , `abs` , `min` , `max` , `log` , `exp`), and built-in constants (`PI` , `E`) via `evaluateFeaExpression` , formatting values in engineering notation and recording any unresolved symbols or syntax errors.

3D CAD Face Boundary Condition & Load Synthesis

When a user selects a topological face in the 3D CAD viewer, `synthesizeFeaBoundary` and `applyBoundaryActionToDeck` (`languages/fea/src/boundary-synthesizer.ts`) automatically generate solver-native boundary and load cards for the target dialect:

Boundary Kind	CalculiX / Abaqus (`.inp`)	NASTRAN Bulk Data (`.bdf`)
Fixed Support (`fixed`)	`*BOUNDARY` constraining DOFs `1, 6` (or `1, 3`) on `NSET_<FACE>`	`SPC1` card locking DOFs `123456` (or `123`) on `SET_<FACE>`
Point / Concentrated Force (`force`)	`*CLOAD` directional nodal force components (F_x, F_y, F_z) along the face normal	`FORCE` card with magnitude and normal unit vector (n_x, n_y, n_z)
Distributed Pressure (`pressure`)	`*DLOAD` surface pressure (`P`) applied to `ELSET_<FACE>`	`PLOAD4` / `FORCE` cards distributed across face elements
Torque / Moment (`moment`)	`*CLOAD` rotational DOFs `4, 5, 6`	`MOMENT` card around face normal axis

---

Structural Theory Slicing & Safety Margin Extraction

`extractFeaSlicingProperties(text)` (`languages/fea/src/language.ts`) extracts structural parameters (`DENSITY` , `VOLUME` , `MAX_VON_MISES_STRESS` , `YIELD_STRENGTH`) directly from the CST and evaluates the structural safety margin:

$$\text{Safety Factor (SF)} = \frac{\sigma_{\text{yield}}}{\sigma_{\text{von Mises, max}}}, \qquad \text{isSafe} \iff \sigma_{\text{von Mises, max}} \leq \sigma_{\text{yield}}$$

Returning `{ mass, maxVonMisesPa, yieldStrengthPa, safetyFactor, isSafe }` directly to the Multi-Domain Theory Coordinator (`CaeContractOracle`) and SysML v2 requirement verifiers.

Built-In Solver & IDE Actions

`@modelscript/fea` registers three interactive actions in `feaLanguage.actions` :

- **`open_fea_viewer`** : Opens the 3D FEA mesh, boundary condition, and von Mises stress contour viewer.
- **`materialize_deck`** : Evaluates all `{{ expr }}` template placeholders against workspace parameters to produce a concrete `.inp` or `.bdf` deck.
- **`run_simulation` (`fea_run_simulation`)**: Parses `*MATERIAL` , `*ELASTIC` , `*BOUNDARY` , and `*CLOAD` / `*DLOAD` sections, solves the 3D tetrahedral finite element stiffness matrix, and returns nodal displacements and element von Mises stress contours.

Polyglot TGG Rules

`@modelscript/fea` defines 6 bidirectional Triple Graph Grammar (TGG) rules in `languages/fea/src/language.ts` :

Rule Name	Source (`fea`)	Target Language	Target Construct
`FeaStressMassToModelicaParam`	`FeaDeck` (`mass` , `maxVonMisesPa`)	`modelica`	`ComponentClause` (`parameter Real`)
`FeaStressMassToSysmlAttribute`	`FeaDeck` (`safetyFactor` , `mass`)	`sysml2`	`AttributeUsage`
`FeaMaterialToSysmlAttribute`	`Section` (`*MATERIAL` / `MAT1`)	`sysml2`	`AttributeUsage`
`FeaBoundaryFixToModelicaFixed`	`Section` (`*BOUNDARY` / `SPC1`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.MultiBody.Parts.Fixed`)
`FeaSpringToModelicaSpring`	`Section` (`*SPRING`)	`modelica`	`ComponentClause` (`Modelica.Mechanics.Translational.Components.Spring`)
`FeaLoadCaseToSysmlRequirement`	`Section` (`*STEP` / `SUBCASE`)	`sysml2`	`RequirementUsage`

Command Line Interface (`msx fea`)

```
msx fea <action> [files...] [options]
```

Action	Description	Key Options
`lint`	Validates CalculiX, Abaqus, and NASTRAN syntax, element topologies, material cards, and constraints.	—
`summary`	Prints structural summary: element count by kind (C3D4, C3D10, CHEXA), materials, and load steps.	—
`inspect`	Dumps parsed boundary sets, node coordinates, material elastic moduli, and load cases.	—
`to-modelica`	Generates Modelica equivalent structural stiffness models and translational boundary ports.	`--out=<file.mo>`
`to-dae`	Lowers structural mass and stiffness matrices into ModelScript linear-memory DAE arena.	`--out=<file.json>`
`solve`	Solves linear elastostatic stiffness matrix for nodal displacements and von Mises stress contours.	—

Examples

```
# Validate element connectivity and material properties in an Abaqus deck
msx fea lint BracketAssembly.inp

# Display mesh statistics, node count, and material properties
msx fea summary ChassisStructure.bdf

# Solve linear static displacement and print max von Mises stress
msx fea solve BracketAssembly.inp
```

Diagnostic Rules

CHAPTER 40

CSV Tabular Data (`.csv`)

`@modelscript/csv` (`languages/csv`) elevates empirical measurement series, calibration look-up tables, and parameter sweeps into first-class typed symbols inside the `WorkspaceIndex` .

Capabilities

1. **Native GLR Parsing & Delimiter Auto-Detection**: Parses RFC 4180 CSV documents into linear-memory CST rows and cells (`csv_document` , `header_row` , `data_row` , `field` , `quoted_field`) while `detectDelimiter` (`languages/csv/src/measurements.ts`) automatically detects comma (`,`), semicolon (`;`), or tab (`\t`) delimiters and handles quoted strings with escaped `""` sequences.
2. **Modelica Package Generation (`parseCsvToModelica`)**: Converts any CSV dataset directly into a strongly typed Modelica `package` containing:
   - `constant Integer numRows` and `constant Integer numCols`
   - A 2D `constant Real values[numRows, numCols]` matrix suitable for `Modelica.Blocks.Tables.CombiTable1D` / `CombiTable2D`
   - Sanitized 1D column vectors (`constant Real <colName>[numRows]`) with automatic duplicate-name disambiguation
3. **Salsa Virtual Component Queries (`CSVVirtualComponent`)**: Exposes every CSV column as a queryable virtual symbol in the `QueryEngine` (`languages/csv/src/queries.ts`):
   - `resolvedType` : Infers `"Real"` , `"Integer"` , `"Boolean"` , or `"String"` from column cells
   - `classInstance` : Maps to `"Modelica.Blocks.Sources.CombiTimeTable"`
   - `variability` : Returns `"constant"`
   - `causality` : Returns `"output"`
   - `resolvedArrayDimensions` / `arrayDimensions` : Returns `[rowCount]`
   - `effectiveModification` : Synthesizes `{ table: "[[...]]" }` array literals from column data
4. **Bidirectional Cell Writeback (`writeback`)**: When a parameter or table entry is modified during calibration or optimization, `csvLanguage.writeback` (`languages/csv/src/language.ts`) updates the exact `[row, col]` cell or column header in place while preserving the file's original delimiter and line endings.

Semantic Linter & Diagnostic Codes

`lintCsv(sourceText)` (`languages/csv/src/linter.ts`) performs six structural and numerical integrity checks on CSV datasets:

Code	Severity	Condition Checked
`CSV_UNCLOSED_QUOTE`	`error`	A quoted field contains an unclosed `"` at the end of a line
`CSV_EMPTY_HEADER`	`warning`	A column in the header row has an empty name
`CSV_DUPLICATE_HEADER`	`warning`	Two or more columns share the same header identifier
`CSV_COLUMN_COUNT_MISMATCH`	`error`	A data row contains fewer or more cells than the header row
`CSV_INVALID_NUMERIC_CELL`	`error`	A cell in a numeric column (where row 1 is numeric) contains non-numeric text, `NaN` , or `Infinity`
`CSV_NON_MONOTONIC_TIME`	`warning`	The first column (`time` / `t`) is not strictly monotonically increasing ($t_i < t_{i-1}$)

Polyglot TGG Rules

`@modelscript/csv` registers 2 bidirectional Triple Graph Grammar (TGG) rules in `languages/csv/src/language.ts` :

Rule Name	Source (csv)	Target (modelica)	Semantics
`CsvDocumentToModelicaPackage`	`CSVDocument`	`ClassDefinition ( package )`	Projects a CSV file into a Modelica data package with `values[numRows, numCols]`
`CsvVirtualComponentToModelicaComponent`	`CSVVirtualComponent`	`ComponentClause ( parameter Real )`	Binds a typed CSV column to a Modelica parameter vector or `CombiTimeTable` source

---

Command Line Interface

```
# Lint CSV datasets for column mismatches, unclosed quotes, and non-monotonic time
msx lint trajectory_data.csv

# Output simulation trajectories directly to standard CSV format
msx simulate CircuitModel.mo --stop-time=1.0 --step-size=0.001 --format=csv -o sim_results.csv
```

CHAPTER 41

CCSDS Astrodynamics & Space Telemetry (`.oem`, `.opm`, `.omm`, `.xtce`)

`@modelscript/ccsds` (`languages/ccsds`) provides comprehensive compiler, trajectory propagation, and verification support for the **Consultative Committee for Space Data Systems (CCSDS)** and **OMG XML Telemetric & Command Exchange (XTCE)** standards.

Supported Extensions & Standards

Extension	Standard / Specification	Engineering Domain
`.oem`	**CCSDS 502.0-B-2** Orbit Ephemeris Message	Time-tagged position and velocity state vector ephemerides
`.opm`	**CCSDS 502.0-B-2** Orbit Parameter Message	Osculating Keplerian orbital elements and maneuvers
`.omm`	**CCSDS 502.0-B-2** Orbit Mean-Elements Message	Mean orbital elements (SGP4/SDP4 TLE compatible)
`.xtce` , `.xml`	**OMG XTCE v1.2** Telemetric & Command Exchange	Bit-level satellite telemetry decommutation and commanding

---

Key Capabilities

1. Perturbed 6-DOF Cowell Orbit Propagator

- Integrates high-fidelity gravitational and environmental perturbations in Cartesian J2000 / GCRF coordinates:

$$\ddot{\mathbf{r}} = -\frac{\mu}{r^3}\mathbf{r} + \mathbf{a}_{J_2} + \mathbf{a}_{\text{drag}} + \mathbf{a}_{\text{SRP}} + \mathbf{a}_{\text{3rd}}$$

- J_2 **Earth Oblateness**: Evaluates central body oblateness forces from zonal harmonic coefficients.
- **Atmospheric Drag**: Exponential barometric and Jacchia-Roberts atmospheric density models coupled with satellite cross-sectional area and drag coefficient C_D.
- **Solar Radiation Pressure (SRP)**: Cannonball radiation pressure models with cylindrical and dual-cone solar eclipse shadowing.
- **Third-Body Gravitation**: Lunar and solar ephemeris gravitational perturbations.

2. OMG XTCE Bit-Packing & Decommutation Engine

- Parses hierarchical `<TelemetryMetaData>` , `<ParameterSet>` , `<ContainerSet>` , and `<SequenceContainer>` schemas.
- Validates bit-level alignments, integer sign encodings (twos-complement, sign-magnitude), IEEE 754 float packaging, and calibrated polynomial transfer functions.

3. Direct DAE Lowering

- Compiles orbital states and spacecraft attitudes directly into `@modelscript/runtime` 's `DAEBuilder` SoA linear memory arena for joint trajectory and attitude determination simulations.

4. Polyglot Projections & TGG Mappings

- **Modelica Export (`ccsdsToModelica`)**: Generates continuous Modelica 3.x space flight models compatible with orbital physics libraries.
- **SysML v2 Mission Architecture (`ccsdsToSysml`)**: Projects ground station contacts, communication downlink schedules, and orbit maintenance requirements into SysML v2 `part def` and `requirement def` hierarchies.

Static Analysis & Pre-Flight Linter Rules

The CCSDS analyzer enforces space mission integrity and flight safety constraints (`M8201` – `M8208`):

CLI Usage

```
# Lint an Orbit Ephemeris Message or XTCE container definition
msx ccsds lint mission_orbit.oem
msx ccsds lint satellite_telemetry.xtce

# Propagate trajectory and lower to DAE linear memory
msx ccsds to-dae orbit.opm --duration 86400 --step 60

# Project orbital state vectors into Modelica simulation code
msx ccsds to-modelica orbit.opm -o OrbitDynamics.mo

# Export orbital mission constraints to SysML v2 requirements
msx ccsds to-sysml orbit.omm -o OrbitRequirements.sysml
```

CHAPTER 42

EnergyPlus Building Simulation (`.idf` , `.epjson`)

`@modelscript/energyplus` (`languages/energyplus`) provides compiler, thermal lumped-capacitance DAE lowering, and bidirectional TGG synchronization for **US Department of Energy (DOE) EnergyPlus** building energy and HVAC models.

Supported Extensions & Standards

Extension	Format	Description
`.idf`	EnergyPlus Input Data File	Industry-standard comma/semicolon delimited building simulation deck
`.epjson`	EnergyPlus JSON Schema	Structured JSON schema representation of EnergyPlus models

---

Key Capabilities

1. Lumped-Capacitance Multi-Zone Thermal DAEs

- Translates architectural thermal zones and building envelopes into continuous energy balance DAEs:

$$C_{z,i}\frac{dT_{z,i}}{dt} = \sum_j \dot{Q}_{\text{cond},ij} + \sum_k \dot{Q}_{\text{conv},ik} + \dot{Q}_{\text{solar}} + \dot{Q}_{\text{internal}} + \dot{Q}_{\text{HVAC}}$$

- **Envelope Conduction**: Multi-layer wall, floor, and roof constructions with layer-resolved thermal conductivity k, density ρ, and specific heat c_p.
- **Window & Fenestration Optics**: U-factor, Solar Heat Gain Coefficient (SHGC), and visible transmittance shading calculations.
- **Internal Loads**: Schedules for lighting, occupancy sensible/latent heat release, and plug-load equipment dissipation.

2. Direct DAE Lowering

- Compiles building geometry and HVAC schedules directly into `@modelscript/runtime` 's `DAEBuilder` linear memory arena, enabling co-simulation with dynamic grid power models (`@modelscript/pscad`) and district energy networks.

3. Bidirectional Triple Graph Grammar (TGG) Sync

- Implements bidirectional TGG rules (`energyplus-sync.ts`) synchronizing EnergyPlus thermal zones with:
  - **Modelica (`Modelica.Thermal` / IBPSA Buildings)**: Automatic translation into acausal thermal components and fluid loops.
  - **STEP CAD (ISO 10303)**: Validates physical geometric volume and boundary surface areas against 3D architectural CAD envelopes.

Static Analysis & Pre-Flight Linter Rules

CLI Usage

```
# Lint an EnergyPlus building model deck
msx energyplus lint CommercialOffice.idf

# Lower thermal zones to ModelScript linear memory DAE arena
msx energyplus to-dae CommercialOffice.idf --out=office.dae.json

# Project into standard Modelica IBPSA Buildings library components
msx energyplus to-modelica CommercialOffice.idf -o BuildingThermalModel.mo
```

CHAPTER 43

EPANET Water Distribution Networks (`.inp`)

`@modelscript/epanet` (`languages/epanet`) provides native parsing, hydraulic simulation, DAE lowering, and formal verification for **US EPA EPANET** pressurized drinking water distribution networks and municipal water infrastructure.

Supported Extensions & Standards

Extension	Format	Description
`.inp`	EPANET Input File	Industry-standard network topology, pipes, pumps, valves, and water quality decks

---

Key Capabilities

1. Todini Global Gradient Method (GGM) Hydraulic Solver

- Implements the Todini Global Gradient Algorithm (`todini-ggm.ts`) for quadratic/nonlinear pipe network equilibrium:
  - **Conservation of Mass at Junctions**:

$$\sum_{k \in \text{in}(i)} Q_k - \sum_{k \in \text{out}(i)} Q_k = D_i$$

  - **Conservation of Energy across Pipe Loops**:

$$h_i - h_j = h_{L,ij} = r Q_{ij} \left| Q_{ij} \right|^{\alpha - 1}$$

- Supports **Hazen-Williams**, **Darcy-Weisbach**, and **Chezy-Manning** head loss friction models.

2. Multi-Component Hydraulic Elements

- **Pumps**: Head gain curves modeled via 3-point polynomial and power-law relationships ($h_p = h_0 - aQ^b$).
- **Tanks & Reservoirs**: Dynamic elevated storage tanks with hydrostatic pressure differential equations:

$$A_{\text{tank}} \frac{dh}{dt} = \sum Q_{\text{in}} - \sum Q_{\text{out}}$$

- **Control Valves**: Pressure Reducing Valves (PRV), Pressure Sustaining Valves (PSV), and Flow Control Valves (FCV).

3. Direct DAE Lowering & Modelica Projection

- Compiles the entire municipal pipe and tank network into `@modelscript/runtime` 's `DAEBuilder` SoA linear memory arena.
- Projects EPANET networks into standard `Modelica.Fluid` pipes, reservoirs, and centrifugal pumps for joint thermo-fluid studies.

Static Analysis & Pre-Flight Linter Rules

CLI Usage

```
# Lint an EPANET municipal water network
msx epanet lint MunicipalWaterNetwork.inp

# Solve steady-state network hydraulics (Todini GGM) and print pressures
msx epanet solve MunicipalWaterNetwork.inp

# Lower network dynamics into ModelScript linear DAE arena
msx epanet to-dae MunicipalWaterNetwork.inp --out=network.dae.json

# Project into standard Modelica.Fluid components
msx epanet to-modelica MunicipalWaterNetwork.inp -o MunicipalPipes.mo
```

CHAPTER 44

OpenDRIVE Road Networks (`.xodr`)

`@modelscript/opendrive` (`languages/opendrive`) provides compiler, analytical 3D clothoid sampling, path-following DAE lowering, and vehicle dynamics integration for the **ASAM OpenDRIVE** standard.

Supported Extensions & Standards

Extension	Standard / Format	Description
`.xodr` , `.xml`	**ASAM OpenDRIVE v1.4–v1.7**	Description of road networks, lanes, junctions, and spatial road geometry

---

Key Capabilities

1. Analytical 3D Fresnel Clothoid Sampling

- Evaluates road reference lines parameterized by arc length s with smooth curvature transitions:
  - **Euler Clothoids**: Spiral transitions computed via numerical Fresnel integrals:

$$x(s) = x_0 + \int_0^s \cos\left(\theta_0 + \kappa_0 \tau + \frac{\dot{\kappa}}{2}\tau^2\right) d\tau$$

  - **Arcs & Polynomials**: Straight lines, constant-curvature circular arcs, and cubic parameter polynomials.
- Computes elevation profiles ($z(s)$), crossfall/camber angles, and superelevation banking.

2. Multi-Lane Topology & Junction Network

- Analyzes lane sections (`<laneSection>`), driving lanes, sidewalks, and shoulders.
- Reconstructs predecessor/successor road connections, junction path conflicts, and lane boundary polylines.

3. Path-Following Vehicle DAE Lowering

- Compiles OpenDRIVE tracks directly into curvilinear Frenet-Serret coordinates (s, d, θ_e) within `@modelscript/runtime` 's `DAEBuilder` arena:

$$\dot{s} = \frac{v \cos\theta_e}{1 - d \cdot \kappa(s)}, \quad \dot{d} = v \sin\theta_e, \quad \dot{\theta}_e = \dot{\psi} - \dot{s}\kappa(s)$$

- Enables high-speed closed-loop vehicle chassis simulation with @modelscript/simpack and @modelscript/adams.

4. GENSYS Track Export

- Projections (`opendrive-to-gensys.ts`) convert OpenDRIVE centerlines and gauge profiles into GENSYS `.geom` and `.trax` geometry files for railway and guideway vehicle testing.

Static Analysis & Pre-Flight Linter Rules

CLI Usage

```
# Lint an OpenDRIVE road network
msx opendrive lint HighwayInterchange.xodr

# Sample 3D road centerline and lane boundaries at 0.5m resolution
msx opendrive track HighwayInterchange.xodr --step 0.5 -o track_points.json

# Lower road geometry to DAE path-following arena
msx opendrive to-dae HighwayInterchange.xodr --out=road.dae.json

# Project into GENSYS track geometry format
msx opendrive to-gensys HighwayInterchange.xodr -o TrackGeometry.geom
```

CHAPTER 45

PLCopen XML & IEC 61131-3 Industrial Automation (`.xml`, `.st`)

`@modelscript/plcopen` (`languages/plcopen`) provides compiler, Virtual PLC cyclic execution, sampled-data DAE lowering, and formal verification for **IEC 61131-3** industrial control software and **PLCopen XML** interchange models.

Supported Extensions & Standards

Extension	Standard / Format	Description
`.xml` , `.plcopen`	**PLCopen XML v2.01 / v2.0**	XML exchange format for IEC 61131-3 control programs, POUs, and configurations
`.st`	**IEC 61131-3 Structured Text**	Textual structured control language for PLCs and industrial controllers

---

Key Capabilities

1. IEC 61131-3 Program Organization Units (POUs)

- Full support for standard control logic paradigms:
  - **Structured Text (ST)**: Variable declarations, conditionals (`IF-THEN-ELSE`), case selection, loops (`FOR` , `WHILE`), and mathematical expressions.
  - **Function Blocks (FB)**: State-retaining functional blocks including standard timers (`TON` , `TOF` , `TP`), counters (`CTU` , `CTD` , `CTUD`), and bistable multivibrators (`SR` , `RS`).
  - **Functions (FUN)**: Pure stateless mathematical computations.

2. High-Speed Deterministic Virtual PLC Engine

- Features a cycle-accurate Virtual PLC execution runtime (`VirtualPlc`):
  - **Sampled Scan Cycle**: Deterministic three-phase execution: input latching → logic evaluation → output write.
  - **Configurable Scan Period**: Executes discrete scan rates (e.g., $1\,\text{ms}$, $10\,\text{ms}$, $50\,\text{ms}$) coupled with continuous physical models.

3. Direct Sampled-Data DAE Lowering

- Compiles discrete state variables, zero-order holds (ZOH), and timer registers into `@modelscript/runtime` 's `DAEBuilder` arena.
- Enables closed-loop hardware-in-the-loop (HIL) simulation of PLC control algorithms coupled with continuous Modelica physics (e.g., thermal plants, robotic actuators, motor drives).

4. Polyglot Projections

- **Modelica (`plcopenToModelica`)**: Lowers PLC function blocks into standard synchronous Modelica clocked state machines.
- **SysML v2 (`plcopenToSysML`)**: Maps IEC 61131-3 control architectures to SysML v2 `action def` successions and state machines.

Static Analysis & Safety Linter Rules

CLI Usage

```
# Lint an IEC 61131-3 PLCopen XML project
msx plcopen lint AutomationController.xml

# Run 50 discrete scan cycles (dt = 10ms) through the Virtual PLC engine
msx plcopen run-scan AutomationController.xml --cycles 50 --dt 10

# Lower control logic to DAE arena for closed-loop physics co-simulation
msx plcopen to-dae AutomationController.xml --out=controller.dae.json

# Project into Modelica synchronous clocked state machine components
msx plcopen to-modelica AutomationController.xml -o ControllerBlocks.mo
```

CHAPTER 46

Verilog-A & Verilog-AMS Compact Modeling (`.va` , `.vams`)

`@modelscript/verilog-a` (`languages/verilog-a`) provides compiler, analog behavioral simulation, continuous DAE lowering, and SPICE netlist export for the **IEEE 1857 Verilog-A** and **Verilog-AMS** analog compact modeling language.

Supported Extensions & Standards

Extension	Standard / Format	Description
`.va`	**IEEE 1857 / OVI Verilog-A**	Continuous-time analog behavioral hardware description language
`.vams`	**Accellera Verilog-AMS**	Mixed-signal analog/digital compact device modeling

---

Key Capabilities

1. Analog Behavioral Disciplines & Natures

- Native support for physical disciplines and cross-domain nature definitions:
  - **Electrical**: Potential `Voltage` (V), Flow `Current` (I).
  - **Thermal**: Potential `Temperature` (T), Flow `Power` (P).
  - **Rotational / Mechanical**: Potential `Angle` (θ), Flow `Torque` (τ).
- Supports branch formulation, reference nodes (`ground`), and conservative multi-port Kirchhoff networks.

2. Analog Contribution Statements & Dynamic Operators

- Full evaluation of continuous-time analog contribution operators (`<+`):

$$I(p,n) <+ I_{\text{sat}} \left(\exp\left(\frac{V(p,n)}{nV_t}\right) - 1\right) + \frac{d}{dt}\left(C_j V(p,n)\right)$$

- **Dynamic Operators**: Time derivative `ddt(x)` , time integration `idt(x)` , transport delay `absdelay(x, td)` , and transition filters.
- **Transistor Compact Models**: Enables high-speed simulation of BSIM, EKV, diode, and custom micro-electro-mechanical (MEMS) behavioral models.

3. Direct DAE Lowering

- Lowers continuous behavioral equations, branch relations, and small-signal Jacobians directly into `@modelscript/runtime` 's `DAEBuilder` linear memory arena.
- Eliminates intermediate C-code generation or proprietary simulator compiling steps.

4. Polyglot Projections

- **Modelica (`verilogAToModelica`)**: Translates Verilog-A modules into acausal `Modelica.Electrical.Analog` component classes.
- **SPICE (`verilogAToSpice`)**: Generates equivalent SPICE subcircuit netlists (`.subckt`) with nonlinear behavioral voltage/current sources (`B` -sources).

Static Analysis & Device Modeling Linter Rules

The Verilog-A linter detects compact modeling bugs and numerical convergence pitfalls:

CLI Usage

```
# Lint a Verilog-A semiconductor device model
msx verilog-a lint MOS_Transistor.va

# Lower analog behavioral equations to ModelScript linear DAE arena
msx verilog-a to-dae MOS_Transistor.va --out=transistor.dae.json

# Project into standard Modelica 3.x acausal analog component
msx verilog-a to-modelica MOS_Transistor.va -o MOS_Transistor.mo

# Export to SPICE subcircuit with nonlinear behavioral sources
msx verilog-a to-spice MOS_Transistor.va -o MOS_Transistor.sub
```

CHAPTER 47

Fortran Legacy Engineering Integration (`.f`, `.f90`)

`@modelscript/fortran` (`languages/fortran`) provides native GLR parsing, AST analysis, Triple Graph Grammar (TGG) synchronization, and DAE wrapping for legacy **Fortran 77** and **Fortran 90** computational engines.

Supported Extensions & Standards

Extension	Standard / Format	Description
`.f` , `.for` , `.f77`	**Fortran 77** (Fixed Form)	Column-based legacy engineering subroutines and mathematical libraries
`.f90` , `.f95`	**Fortran 90** / **95** (Free Form)	Free-form modular subroutines, derived types, and array syntax

---

Key Capabilities

1. WebAssembly GLR Fortran Parser

- Built on `@modelscript/dsl` 's ahead-of-time compiled WASM GLR parser table.
- Accurately parses both fixed-form (columns 1-5 label, 6 continuation, 7-72 statements) and modern free-form source files.
- Supports subroutines, functions, `COMMON` blocks, `EQUIVALENCE` , `PARAMETER` , `DATA` statements, multidimensional array indexing, and intrinsic mathematical functions (`SIN` , `COS` , `SQRT` , `EXP` , etc.).

2. Bidirectional Triple Graph Grammar (TGG) Rules

- Features formal bidirectional graph rewriting rules (`languages/fortran/src/language.ts`):
  - **`FortranSubroutineToModelicaFunction`** : Maps Fortran `SUBROUTINE` and `FUNCTION` definitions to strongly typed Modelica functions.
  - **`FortranAssignToModelicaEquation`** : Transforms Fortran numerical assignment statements into declarative Modelica equality equations.

3. Direct DAE Subroutine Lowering

- Compiles Fortran continuous state derivative subroutines directly into `@modelscript/runtime` 's `DAEBuilder` SoA linear memory arena.
- Enables legacy aero-propulsion, ballistics, and nuclear thermal routines to execute within modern hybrid continuous-discrete simulation solvers without manual recoding.

Static Analysis & Modernization Linter Rules

The Fortran analyzer surfaces modernization issues and common numeric defects:

CLI Usage

Fortran sources are handled by the generic workspace commands. The language is detected from the file extension (`.f` , `.for` , `.f90` , `.f95` , `.f03` , `.f08`).

```
# Lint legacy Fortran engineering subroutines
msx lint TurbomachineryModel.f

# Lint only the Fortran files in a mixed-language workspace
msx lint ./src --language fortran

# Print the parse tree of a Fortran source file
msx parse PropulsionAero.f90
```

DAE lowering is available from the package API (there is no dedicated `msx fortran` command yet):

```
import { lintFortranSource, lowerFortranToDae } from "@modelscript/fortran";

const diagnostics = lintFortranSource(source);
const result = lowerFortranToDae(builder, source.split("\n"));
```

PART IV

Comprehensive Algorithms Catalog

CHAPTER 48

ModelScript Algorithms Catalog

ModelScript is powered by a comprehensive foundation of peer-reviewed mathematical, compiler, and formal verification algorithms. Rather than relying on opaque external black boxes, ModelScript implements these algorithms natively in WebAssembly and TypeScript, with inline citations, architectural rationales, and explicit tracking of upstream inputs and downstream consumers.

Computational Map

The algorithms catalog spans seven core domains across the compilation, simulation, and verification pipelines:



Algorithm Directory

Category	Primary Algorithms	Implementation Package
Structural DAE	Pantelides Algorithm, Tarjan BLT, Cellier-Elmqvist Tearing	`@modelscript/runtime`
Acausal Balancing	Acausal Potential Equalization, Kirchhoff Flows, Stream Mixing	`@modelscript/modelica`
Autodiff & Relaxations	Dual Numbers, AdTape, CPR Graph Coloring, McCormick, Affine, Hybrid Adjoint Saltation Matrices, Aitken Dynamic Δ-Relaxation	`@modelscript/runtime` & `@modelscript/simulate`
Numerical Solvers	Tsit5, Rodas-4P, TR-BDF2, SDE, BVP	`@modelscript/simulate`
Optimization	L-BFGS-B, CMA-ES, Differential Evolution, PSO, NSGA-II	`@modelscript/simulate`
SMT & Verification	Nelson-Oppen, CDCL SAT, Craig Interpolation, HC4, STL, Zonotopes	`@modelscript/runtime/formal`
Model Checking & Safety	Spacer CHC PDR, IC3 Engine, Stochastic Barrier Certificates, WebGPU OWL 2 RL Reasoner, ISO 26262 / DO-178C Back-to-Back Equivalence	`@modelscript/runtime/formal` & `@modelscript/runtime/gpu`
Abstract Interpretation	WTO Fixpoint, TVPI, Octagon DBM, Array Segments, DDE Method-of-Steps Flowpipes, Hamilton-Jacobi-Isaacs (HJI) Level-Set Reachability	`@modelscript/runtime/formal` & `@modelscript/runtime/analysis`
Graph Rewriting	Triple Graph Grammars, CPA, WCOJ, E-Graphs, GSS, Salsa	`@modelscript/dsl`
Linear Memory Structures	Roaring Bitmaps, Front-Coded Dictionary, Paged B+ Tree	`@modelscript/runtime`
Scientific & Digital Twins	Sparse Cholesky, Sparse LU, Householder QR, Gröbner, POD, Buckingham Pi, Arena MHE, Joint-State UKF & MCMC Calibration, Page-Hinkley / CUSUM Drift, RUL Prognostics	`@modelscript/runtime` & `@modelscript/simulate`
SDP, BVP, Continuum & CAD	Primal-Dual SDP, Multiple Shooting BVP, CSG, WebGPU D3Q19 LBM CFD, Tet4/Tet10 Corotational FEA, STEP B-Rep Mesher, Swept-Volume Clearance, GD&T Stack-Up, CAD Parameter Inversion, ECAD Gerber / Router / DRC	`@modelscript/runtime`, `@modelscript/simulate` & `@modelscript/cad`
Diagram Layout & Diffing	Tamassia TSM, Port ILP Solver, Dogleg Channel Router, Fault Tree / Sequence / Swimlane Layouts, PSCAD SLD Routing, Semantic Visual Diffing, CRDT Collaborative Presence	`@modelscript/diagram`

CHAPTER 49

Pantelides Structural Index Reduction

Implementation: `PantelidesEngine` in `packages/runtime/src/wasm/structural/pantelides.ts`

Academic Citations

- **Pantelides, C. C. (1988)**. "*The consistent initialization of differential-algebraic systems.*"
  SIAM Journal on Scientific and Statistical Computing, 9(2), pp. 213–231.
  DOI: 10.1137/0909014
- **Mattsson, S. E., & Söderlind, G. (1993)**. "*Index reduction in differential-algebraic equations using dummy derivatives.*"
  SIAM Journal on Scientific Computing, 14(3), pp. 677–692.
  DOI: 10.1137/0914043

ModelScript Architectural Rationale

Object-oriented multi-domain physical models (such as constrained multi-body mechanics, closed kinematic linkages, and incompressible hydraulic loops) inherently generate high-index Differential-Algebraic Equations (DAEs) where the differential index satisfies $\text{ind} \geq 2$:

$$F(t, x, \dot{x}, y) = 0$$

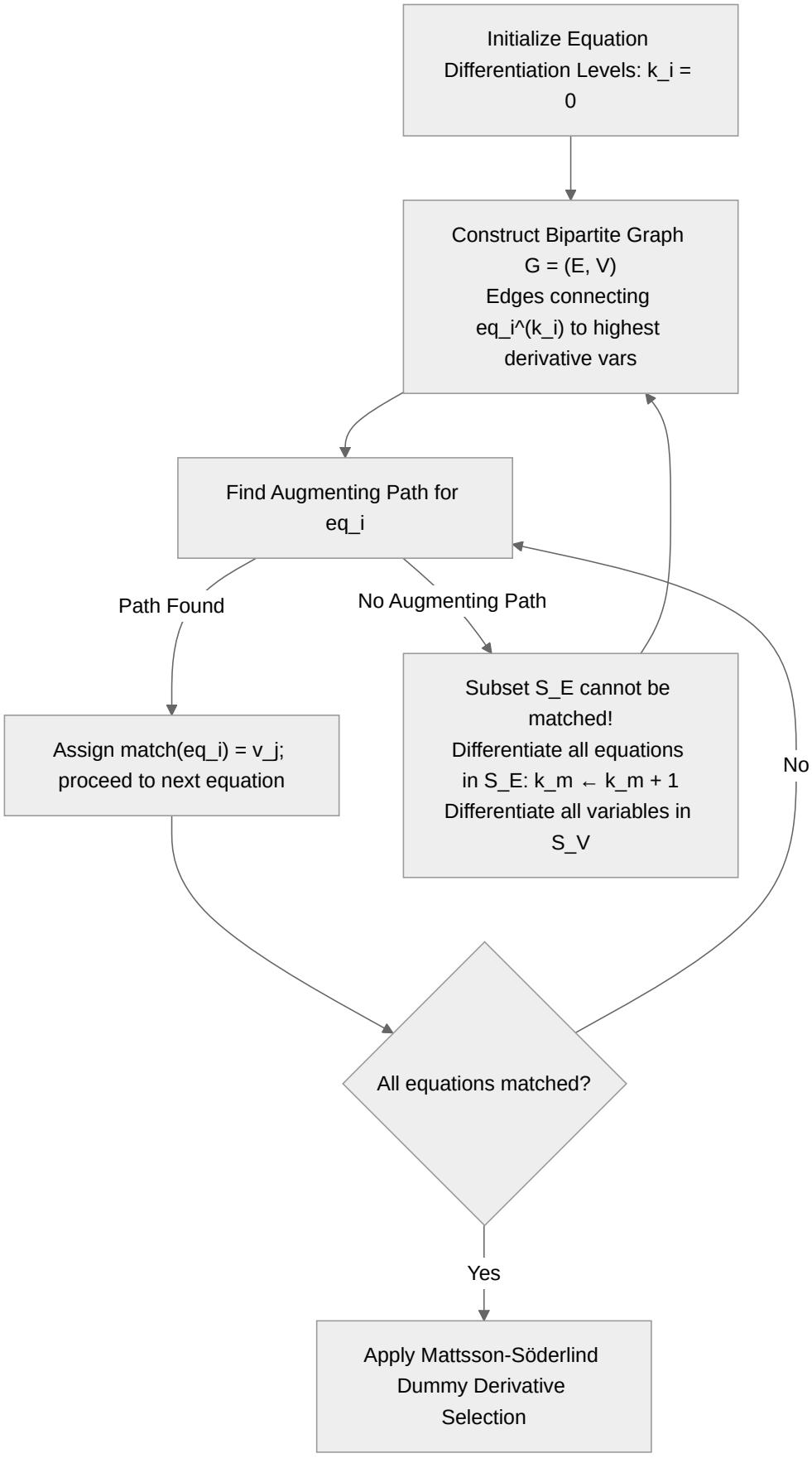
Standard numerical integrators (including BDF, Radau, and Runge-Kutta methods) become severely ill-conditioned or fail to converge on systems with differential index greater than 1.

Pantelides' algorithm identifies minimally singular subsets of equations and differentiates them analytically with respect to time until an index-1 or index-0 formulation is attained. To prevent over-determination and numerical drift along algebraic constraint manifolds, the Mattsson-Söderlind dummy derivative method dynamically replaces redundant derivative states with algebraic placeholders.

ModelScript Modifications & WASM Architecture

1. **In-WASM Linear Memory Execution**: Operates as an unmanaged WebAssembly kernel (`@unmanaged`) with zero garbage collection allocations.
2. **Direct Arena Symbolic Differentiation**: Derivatives of equations ($\frac{d^k f}{dt^k} = 0$) are differentiated directly on `DAEBuilder` linear expression nodes in memory.
3. **Flat Chunked Arrays**: Equation dependency incidence pointers, differentiation levels, and matching arrays are stored in flat `ChunkedInt32Array` buffers.
4. **Augmenting Path Bipartite Matching**: Alternating path searches run directly over highest-derivative variables, passing matching data to the downstream `BltEngine` .

Algorithm Description & Mathematics



Mathematical Steps

1. **Variable Classification**: Identify highest-order derivatives $\dot{x}_j$ and purely algebraic variables y_k.

2. **Alternating Augmenting Paths**: For each equation $f_i = 0$, attempt to find an augmenting path in the bipartite incidence graph to an unmatched highest-order derivative.
3. **Index Reduction by Differentiation**: If no augmenting path exists, the reachable set of equations S_E is structurally singular. Increment the differentiation index $k_m \leftarrow k_m + 1$ for all $f_m \in S_E$, symbolically compute $\frac{df_m}{dt}$, and repeat the matching.
4. **Dummy Derivatives**: For each differentiated state variable where both x_j and $\dot{x}_j$ appear in algebraic constraints, select a dummy algebraic variable $x_{j,\text{dummy}}$ to replace $\dot{x}_j$, preserving degrees of freedom.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Ingests the flat DAE representation from `DAEBuilder` produced by `ModelicaFlattener`.
  - Uses `Cst` linear nodes for symbolic expressions.
- **Downstream Consumers**:
  - Feeds the index-reduced DAE directly into `BltEngine` for block lower triangular partitioning.
  - Determines consistent initial conditions for `SUNDIALS IDA` and `TR-BDF2`.

CHAPTER 50

Tarjan Block Lower Triangular (BLT) Transformation

Implementation: `BltEngine` in `packages/runtime/src/wasm/structural/blt.ts`

Academic Citations

- **Tarjan, R. E. (1972)**. "*Depth-first search and linear graph algorithms.*"
  SIAM Journal on Computing, 1(2), pp. 146–160.
  DOI: 10.1137/0201010 (Strongly Connected Components)
- **Hopcroft, J. E., & Karp, R. M. (1973)**. "*An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs.*"
  SIAM Journal on Computing, 2(4), pp. 225–231.
  DOI: 10.1137/0202019 (Bipartite Matching)
- **Dulmage, A. L., & Mendelsohn, N. S. (1958)**. "*Coverings of bipartite graphs.*"
  Canadian Journal of Mathematics, 10, pp. 517–534.
  DOI: 10.4153/CJM-1958-052-0
- **Cellier, F. E., & Kofman, E. (2006)**. *Continuous System Simulation*.
  Springer. ISBN: 978-0-387-26102-7

ModelScript Architectural Rationale

Flattening industrial physical systems results in coupled systems comprising thousands of equations and variables. Solving a monolithic non-linear algebraic system of size N using general Newton-Raphson methods incurs a cubic computational penalty per step:

$$\mathrm{O}(N^3)$$

The Block Lower Triangular (BLT) transformation permutes the equation and variable ordering so that the incidence matrix assumes a lower triangular block diagonal structure:

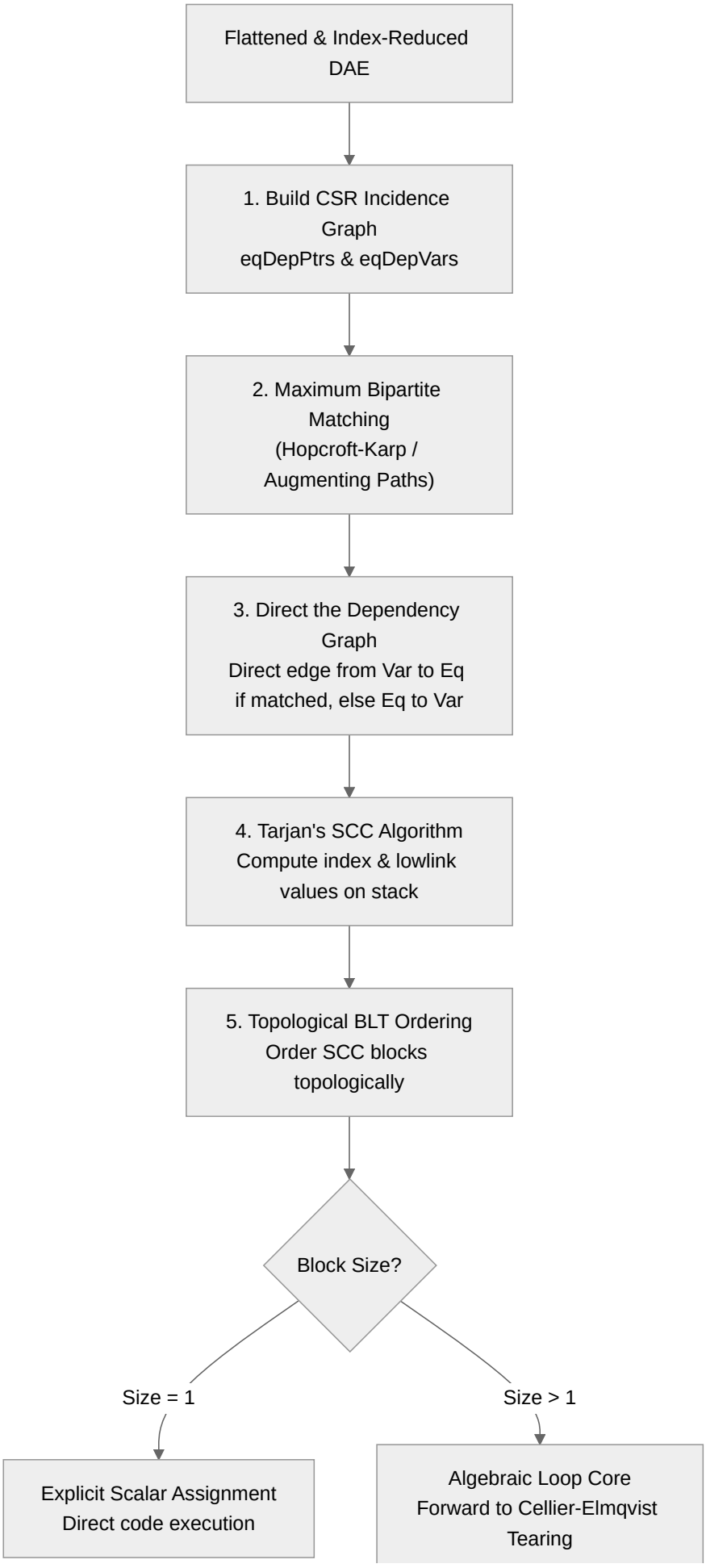
$$\begin{pmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mm} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

Most blocks A_{ii} are trivial 1×1 scalar explicit assignments ($x_i = f(\dots)$), while only non-linear algebraic loops become coupled sub-blocks of size $k \ll N$.

ModelScript Modifications & WASM Architecture

1. **Compressed Sparse Row (CSR) in Linear Memory**: Bipartite incidence graphs are stored in contiguous `ChunkedInt32Array` buffers (`eqDepPtrs` , `eqDepVars`).
2. **Zero-Allocation Tarjan Traversal**: Low-link values, depth indices, and traversal stacks operate strictly on 32-bit integer offsets in WebAssembly memory without object references.
3. **Integrated Maximum Matching**: Hopcroft-Karp augmenting path search provides consistent equation-to-variable assignments for directed SCC analysis.
4. **Direct Bridge to Tearing**: SCC blocks with dimension > 1 pass directly to the in-WASM `TornBlock` engine.

Algorithm Description & Pipeline Flow


Flattened & Index-Reduced DAE
1. Build CSR Incidence Graph
eqDepPtrs & eqDepVars
2. Maximum Bipartite Matching
(Hopcroft-Karp / Augmenting Paths)
3. Direct the Dependency Graph
Direct edge from Var to Eq if matched, else Eq to Var
4. Tarjan's SCC Algorithm
Compute index & lowlink values on stack
5. Topological BLT Ordering
Order SCC blocks topologically
Block Size?
Size = 1
Size > 1
Explicit Scalar Assignment
Direct code execution
Algebraic Loop Core
Forward to Cellier-Elmqvist Tearing

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Ingests index-reduced DAE systems from `PantelidesEngine`.
- **Downstream Consumers**:
  - Trivial 1×1 blocks are lowered directly into sequence evaluation bytecode.
  - Coupled blocks (size > 1) are sent to `TornBlock` for algebraic loop tearing.
  - Drives step evaluations in `SUNDIALS CVODE/IDA` and `Tsit5`.

CHAPTER 51

Cellier-Elmqvist Algebraic Loop Tearing

Implementation: `TornBlock` in `packages/runtime/src/wasm/structural/tearing.ts`

Academic Citations

- **Cellier, F. E. (1979)**. *Combined Continuous/Discrete System Simulation by Use of Digital Computers: Techniques and Tools*. **PhD thesis, ETH Zurich**.
  DOI: 10.3929/ethz-a-000179979
- **Elmqvist, H., & Otter, M. (1994)**. "*Methods for Tearing Systems of Equations in Object-Oriented Modeling.*" **Proceedings of the European Simulation Multiconference (ESM'94)**, pp. 326–332.
- **Mahana, P. N., & Eagan, J. P. (1975)**. "*A tearing algorithm for large systems of equations.*" **IEEE Transactions on Circuits and Systems**.

ModelScript Architectural Rationale

Even after BLT partitioning, physical systems containing closed kinematic loops, electrical bridge circuits, or multi-phase fluid balances yield coupled Strongly Connected Components of size $N > 1$.

Solving an N-dimensional non-linear system via Newton-Raphson requires inverting an $N \times N$ Jacobian matrix at every iteration:

$$\mathrm{O}(N^3)$$

The Cellier-Elmqvist Minimum Degree Tearing algorithm identifies a minimal set of k **tearing variables** ($k \ll N$) such that assuming known values for these k variables breaks all cyclic dependencies in the block. The remaining $N - k$ inner variables can then be evaluated sequentially via a causal explicit forward substitution chain:

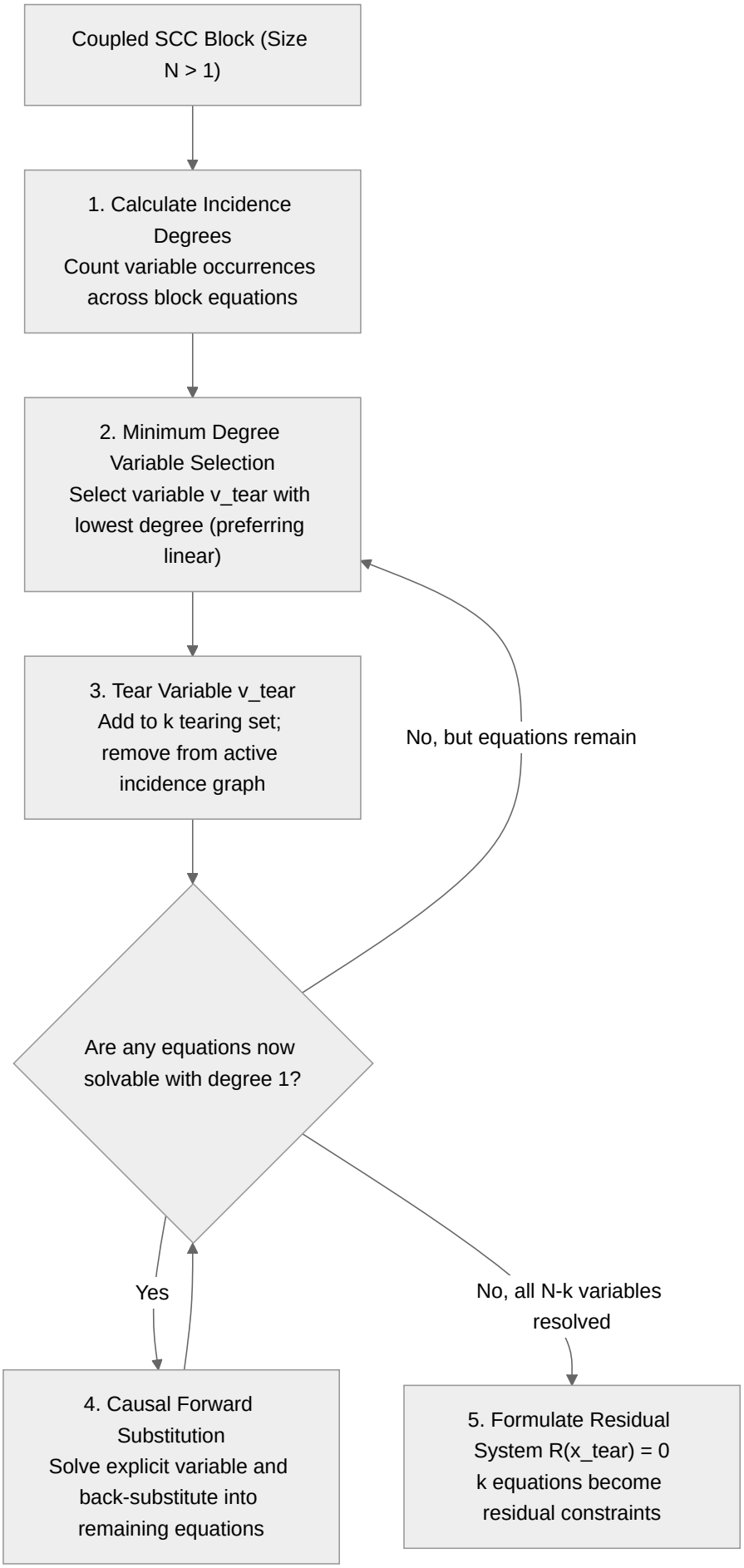
$$x_{\text{inner}} = g(x_{\text{tear}}, u) \quad \text{(explicit chain)}$$
$$R(x_{\text{tear}}) = f_{\text{residual}}(x_{\text{tear}}, x_{\text{inner}}) = 0 \quad \text{(small } k \times k \text{ solve)}$$

The non-linear iterative solver is restricted to the tiny $k \times k$ residual system, accelerating simulation speed by orders of magnitude.

ModelScript Modifications & WASM Architecture

1. **Unmanaged WebAssembly Kernel**: Implemented as `@unmanaged` classes directly in linear memory with zero garbage collection pauses.
2. **Dynamic Minimum Degree Heuristic**: Evaluates vertex degrees in the block incidence graph in memory, prioritizing linear tearing variables over non-linear variables to avoid non-linear solving where possible.
3. **Partitioned In-Memory Structures**: Splits the block into:
   - k tearing variables (`tearVarIndices`)
   - $N - k$ explicit causal equations (`innerEqIndices`)
   - k residual constraint equations (`residualEqIndices`)
4. **Direct Matrix Integration**: Emits residual Jacobians directly to the in-WASM LU factorizer (`luFactor` , `luSolve`).

Algorithm Steps


Coupled SCC Block (Size N > 1)
1. Calculate Incidence Degrees
Count variable occurrences across block equations
2. Minimum Degree Variable Selection
Select variable v_tear with lowest degree (preferring linear)
3. Tear Variable v_tear
Add to k tearing set; remove from active incidence graph
Are any equations now solvable with degree 1?
No, but equations remain
Yes
No, all N-k variables resolved
4. Causal Forward Substitution
Solve explicit variable and back-substitute into remaining equations
5. Formulate Residual System R(x_tear) = 0
k equations become residual constraints

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Receives coupled SCC blocks of size > 1 from `BltEngine`.
- **Downstream Consumers**:
  - Feeds explicit forward chains and $k \times k$ residual systems to the in-WASM Newton-Raphson solver and `Sparse LU Solver`.
  - Evaluated inside simulation step loops (`SUNDIALS`, `TR-BDF2`, `Tsit5`).

CHAPTER 52

Acausal Connector Balancing & Stream Mixing

Implementation: `ConnectionSetResolver` & `IntUnionFind` in `languages/modelica/src/connections.ts`

Academic Citations

Acausal Physical Modeling & Connection Sets

- **Mattsson, S. E., Elmqvist, H., & Otter, M. (1998)**. "*Physical system modeling with Modelica.*"
  Control Engineering Practice, 6(4), pp. 501–510.
  DOI: 10.1016/S0967-0661(98)00047-1
- **Tarjan, R. E. (1975)**. "*Efficiency of a good but not linear set union algorithm.*"
  Journal of the ACM, 22(2), pp. 215–225.
  DOI: 10.1145/321879.321884 *(Disjoint-Set Union-Find)*

Stream Connectors & Convective Transport

- **Franke, R., Casella, F., Otter, M., Sielemann, M., Elmqvist, H., Mattsson, S. E., & Olsson, H. (2009)**. "*Stream Connectors - An Extension of Modelica for Device-Oriented Modeling of Convective Transport.*"
  Proceedings of the 7th Modelica Conference, pp. 116–123.
  DOI: 10.3384/ecp09430076

Expandable Connectors & Multi-Domain Bus Modeling

- **Otter, M., Elmqvist, H., & Mattsson, S. E. (2005)**. "*Object-oriented modeling of bus systems with Modelica.*"
  Proceedings of the 4th Modelica Conference, pp. 295–308.
  Paper PDF

ModelScript Architectural Rationale

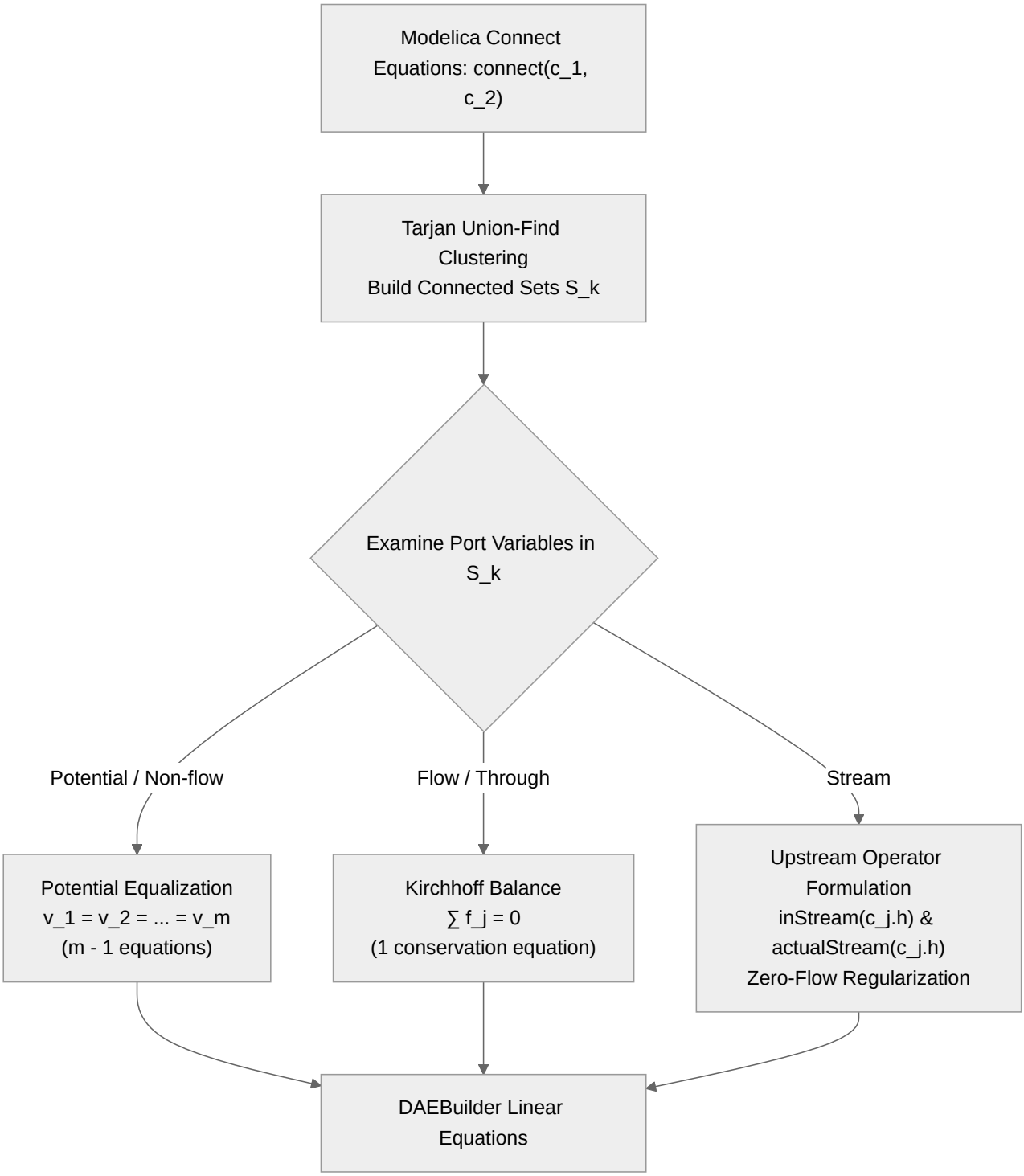
Unlike classical block diagrams (such as causal signal-flow block diagrams) where connections represent directional causal signal flows ($y = f(u)$), ModelScript models **acausal physical interaction** across multi-energy domains:

- **Electrical**: Voltage v (potential) and Current i (flow).
- **Hydraulic & Pneumatic**: Pressure p (potential) and Volume/Mass flow rate q (flow).
- **Translational Mechanics**: Position/Velocity s, v (potential) and Force f (flow).
- **Rotational Mechanics**: Angle/Angular velocity ϕ, ω (potential) and Torque τ (flow).
- **Thermal & Fluid Transport**: Temperature/Enthalpy h (stream) and Heat/Mass flow $\dot{m}$ (flow).

In acausal modeling, connection statements `connect(a, b)` do not dictate computational causality. The compiler must automatically generate:

1. **Potential Equality Equations**: Enforcing that all across-variables in a connection set share an identical value.
2. **Kirchhoff Flow Equations**: Enforcing conservation of mass, charge, or energy such that all through-variables sum to zero at the junction.
3. **Convective Stream Transport Equations**: Resolving thermodynamic mixing when fluid direction reverses ($m_j \gtrless 0$) without introducing singular divisions near zero mass flow.

Balancing Algorithm & Mathematical Formulation



1. Disjoint Connection Set Resolution (Union-Find)

The set of all declared `connect(c_a, c_b)` statements forms an undirected graph. ModelScript partitions all connector references into disjoint equivalence classes using an integer array-based Union-Find with path compression and rank heuristics:

$$\alpha(N) \leq 4 \quad \text{for all physical models}$$

For each connection set $S = \{c_1, c_2, \ldots, c_m\}$ of size m:

2. Potential Variable Equalization

For every non-flow, non-stream variable v declared inside the connector type:

$$v(c_1) = v(c_2) = \cdots = v(c_m)$$

This yields exactly $m - 1$ scalar equations. One variable is chosen as the canonical representative, eliminating $m - 1$ redundant states during alias elimination.

3. Kirchhoff Flow Variable Balancing

For every `flow` variable f declared inside the connector type:

$$\sum_{j=1}^{m} f(c_j) = 0$$

This yields exactly 1 scalar conservation equation per connection set. If an inside-outside connector hierarchy boundary is crossed, the outer flow sign is inverted to preserve directional conservation:

$$f_{\text{outer}} = \sum_{j \in \text{inner}} f(c_j)$$

4. Stream Variable Convective Transport

For fluid ports, transport enthalpy depends on flow direction:

$$h_{\text{transported}} = \begin{cases} h_{\text{upstream}} & \text{if } \dot{m} > 0 \\ h_{\text{downstream}} & \text{if } \dot{m} < 0 \end{cases}$$

Direct formulation introduces a discontinuous switch that causes Newton solvers to oscillate around $\dot{m} = 0$. ModelScript lowers stream variables using Franke's regularized `inStream()` and `actualStream()` operators:

$$\text{inStream}(c_i.h) = \frac{\sum_{j=i, \dot{m}_j > 0} \dot{m}_j \cdot c_j.h}{\sum_{j=i, \dot{m}_j > 0} \dot{m}_j}$$

Near zero flow ($\|\dot{m}_j\| < \epsilon$), smooth polynomial regularization is applied to maintain C^1-continuity of the residual Jacobian.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
IntUnionFind	AST `connect` clauses from `Cst`	Connection set partition groups
Potential Equalizer	Variable causality & variability from `QueryEngine`	Alias elimination (`eliminateArenaAliases`)
Flow Balancer	Connector definitions	`DAEBuilder` linear equations
Stream Formulator	Fluid connector declarations	`PantelidesEngine` , Nonlinear tearing loops

CHAPTER 53

Automatic Differentiation & Relaxations

Implementation:

- Forward Mode: `Dual` in `packages/runtime/src/wasm/autodiff/dual.ts`
- Reverse Mode: `AdTape` in `packages/runtime/src/wasm/autodiff/tape.ts`
- Sparse Jacobian Coloring: `CCSMatrix` & `ColoringEngine` in `packages/runtime/src/wasm/autodiff/coloring.ts`
- McCormick Envelopes: `McCormickTuple` in `packages/runtime/src/wasm/autodiff/mccormick.ts`
- Affine Arithmetic: `AffineForm` in `packages/runtime/src/wasm/autodiff/affine.ts`
- Hybrid Adjoint Sensitivity & Saltation Matrices: `HybridAdjointSolver` in `packages/simulate/src/core/hybrid-adjoint.ts`
- Aitken Dynamic Δ-Relaxation & Multi-Rate Sub-Cycling: `VectorAitkenRelaxation` & `MultiRateSubCycleScheduler` in `packages/simulate/src/core/fsi-relaxation.ts`

Academic Citations

Forward-Mode Dual Numbers

- **Clifford, W. K. (1873)**. "*Preliminary sketch of biquaternions.*"
  Proceedings of the London Mathematical Society, 4, pp. 381–395.
  DOI: 10.1112/plms/s1-4.1.381
- **Study, E. (1903)**. *Geometrie der Dynamen: Die Zusammensetzung von Kräften und verwandte Gegenstände der Geometrie.* B.G. Teubner.
- **Rall, L. B. (1981)**. *Automatic Differentiation: Techniques and Applications.*
  Lecture Notes in Computer Science, 120, Springer.
  DOI: 10.1007/3-540-10861-0

Reverse-Mode Adjoint Accumulation (Wengert List)

- **Wengert, R. E. (1964)**. "*A simple automatic derivative evaluation program.*"
  Communications of the ACM, 7(8), pp. 463–464.
  DOI: 10.1145/355586.364791
- **Linnainmaa, S. (1976)**. "*Taylor expansion of the accumulated rounding error.*"
  BIT Numerical Mathematics, 16(2), pp. 146–160.
  DOI: 10.1007/BF01931367
- **Griewank, A., & Walther, A. (2008)**. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation* (2nd ed.). SIAM.
  ISBN: 978-0-898716-59-7

Distance-2 Graph Coloring for Sparse Jacobians

- **Curtis, A. R., Powell, M. J. D., & Reid, J. K. (1974)**. "*On the estimation of sparse Jacobian matrices.*"
  Journal of the Institute of Mathematics and Its Applications, 13(1), pp. 117–119.
  DOI: 10.1093/imamat/13.1.117 *(CPR Algorithm)*
- **Coleman, T. F., & Moré, J. J. (1983)**. "*Estimation of sparse Jacobian matrices and graph coloring problems.*"
  SIAM Journal on Numerical Analysis, 20(1), pp. 187–209.
  DOI: 10.1137/0720013
- **Gebremedhin, A. H., Manne, F., & Pothen, A. (2005)**. "*What color is your Jacobian? Graph coloring for computing derivatives.*"
  SIAM Review, 47(4), pp. 629–705.
  DOI: 10.1137/S0036144504446096

McCormick Relaxations & Affine Arithmetic

- **McCormick, G. P. (1976)**. "*Computability of global solutions to factorable nonconvex programs: Part I—Convex underestimating problems.*"

Mathematical Programming, 10(1), pp. 147–175.
DOI: 10.1007/BF01580665

- **Mitsos, A., Chachuat, B., & Barton, P. I. (2009)**. "*McCormick-based relaxations of algorithms.*"
  SIAM Journal on Optimization, 20(2), pp. 573–601.
  DOI: 10.1137/080717341
- **Tsoukalas, A., & Mitsos, A. (2014)**. "*Multivariate McCormick relaxations.*"
  Journal of Global Optimization, 59(2), pp. 633–662.
  DOI: 10.1007/s10898-014-0169-z
- **de Figueiredo, L. H., & Stolfi, J. (2004)**. "*Affine arithmetic: concepts and applications.*"
  Numerical Algorithms, 37(1), pp. 147–158.
  DOI: 10.1023/B:NUMA.0000049462.70970.b6

Hybrid Adjoint Sensitivity & Saltation Matrices

- **Barton, P. I., & Lee, C. K. (2002)**. "*Modeling, simulation, sensitivity analysis, and optimization of hybrid systems.*"
  ACM Transactions on Modeling and Computer Simulation, 12(4), pp. 256–289.
  DOI: 10.1145/643120.643122
- **Filippov, A. F. (1988)**. *Differential Equations with Discontinuous Righthand Sides.*
  Mathematics and Its Applications, 18, Springer.
  DOI: 10.1007/978-94-015-7793-9
- **Corner, S., Sandu, A., & Sandu, C. (2019)**. "*Adjoint sensitivity analysis of hybrid multibody dynamical systems.*"
  Multibody System Dynamics, 49, pp. 395–420.
  DOI: 10.1007/s11044-019-09725-2

Aitken Dynamic Δ-Relaxation for Partitioned Co-Simulation

- **Küttler, U., & Wall, W. A. (2008)**. "*Fixed-point fluid–structure interaction solvers with dynamic relaxation.*"
  Computational Mechanics, 43(1), pp. 61–72.
  DOI: 10.1007/s00466-008-0255-5
- **Irons, B. M., & Tuck, R. C. (1969)**. "*A version of the Aitken accelerator for computer iteration.*"
  International Journal for Numerical Methods in Engineering, 1(3), pp. 275–277.
  DOI: 10.1002/nme.1620010306

ModelScript Architectural Rationale

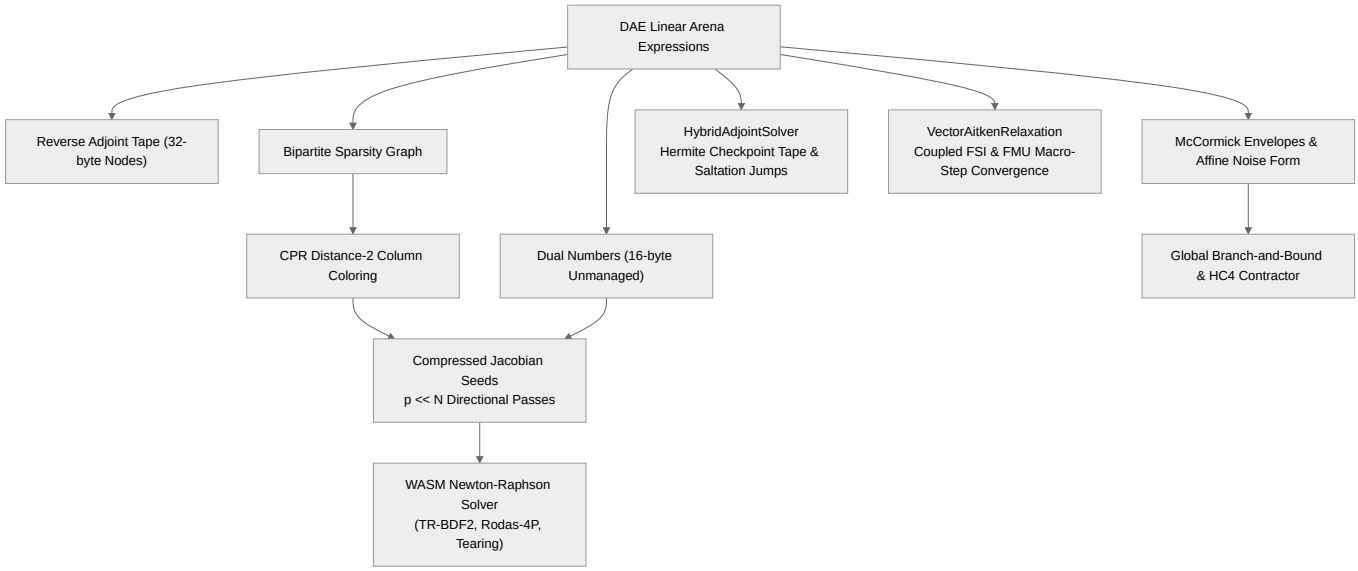
Numerical finite-difference approximations ($\frac{f(x+h)-f(x)}{h}$) suffer from a fundamental trade-off: large step sizes h cause truncation errors, while small step sizes cause catastrophic floating-point cancellation.

ModelScript implements an integrated suite of **exact differentiation and continuous bounding engines** inside WebAssembly linear memory:

1. **Forward-Mode Dual Numbers**: Computes exact machine-precision directional derivatives $\nabla f(x) \cdot v$ concurrently with primal function evaluation for Newton steps in algebraic loops and stiff ODE/DAE integrators without allocating expression trees.
2. **Reverse-Mode Computational Tape**: Evaluates scalar objective gradients $\nabla f(x) \in \mathrm{R}^N$ with cost proportional to $O(1)$ evaluations of f, enabling gradient-based parameter estimation and surrogate neural network training.
3. **CPR Distance-2 Graph Coloring**: For large-scale DAE systems ($N = 10{,}000+$ equations), evaluating the full $N \times N$ Jacobian naively requires N forward passes. CPR distance-2 coloring partitions structural columns into $p \ll N$ non-interfering groups (typically $p \leq 15$ for sparse physical networks), reducing Jacobian assembly time by orders of magnitude.
4. **McCormick Envelopes & Affine Arithmetic**: Classical interval arithmetic suffers from the **dependency problem**: evaluating $x - x$ for $x \in [0, 1]$ yields $[-1, 1]$ instead of 0, causing rapid exponential bounding blowup. Affine arithmetic tracks first-order noise symbols ($\epsilon_i \in [-1, 1]$) across state equations, while McCormick relaxations compute rigorous convex lower bounds (f^R) and concave upper bounds (f^{AR}) for non-convex global optimization.
5. **Hybrid Adjoint Sensitivity with Saltation Matrices**: Standard continuous adjoint solvers break down across Modelica `when` / `reinit` events, impact mechanics, or switching surfaces because Dirac delta impulses arise at event-crossing times t^*. ModelScript propagates reverse-mode adjoints across non-smooth mode transitions using exact saltation matrix jump conditions alongside optional sigmoidal equation smoothing (`DAEEventRelaxer`).
6. **Aitken Dynamic Δ-Relaxation**: Partitioned co-simulation of strongly coupled multi-physics domains (such as lightweight aerodynamic structures coupled to dense fluids or algebraic FMU loops) diverges under explicit Gauss-Seidel macro-steps due

to the artificial added-mass effect. Dynamic vector Aitken acceleration stabilizes fixed-point iterations without requiring monolithic cross-FMU Jacobians.

ModelScript Modifications & WASM Architecture



1. Zero-GC Dual Number Arithmetic

Represented as a 16-byte unmanaged struct (`@unmanaged`) in WASM memory:

```
@unmanaged
export class Dual {
  val: f64; // Primal evaluation f(x)
  dot: f64; // Directional derivative f'(x) * v
}
```

All elementary operators ($+, -, \times, /$) and transcendental functions ($\sin, \cos, \exp, \log, \text{pow}$) are inlined without intermediate object allocation.

2. Linear Memory Computational Tape

The reverse-mode tape records operations into a flat 32-byte stride unmanaged array:

$$\text{Stride: } [\text{op: u32}, \text{left: u32}, \text{right: u32}, \text{aux: u32}, \text{valLo: u32}, \text{valHi: u32}, \text{gradLo: u32}, \text{gradHi: u32}]$$

During the reverse pass, adjoints are accumulated in topological order directly in linear memory:

$$\bar{x}_j \leftarrow \bar{x}_j + \bar{x}_i \cdot \frac{\partial f_i}{\partial x_j}$$

3. CPR Distance-2 Coloring Algorithm

Two columns j and k in the Jacobian J are **structurally orthogonal** if they do not share any non-zero row entry:

$$\text{Row}(j) \cap \text{Row}(k) = \emptyset$$

The graph coloring algorithm:

1. Constructs the bipartite graph $B = (V_R, V_C, E)$ where edge $(i, j) \in E \iff \frac{\partial f_i}{\partial x_j} = 0$.
2. Colors columns V_C such that any pair of columns connected by a path of length 2 receives different colors.
3. Groups columns of color $c \in \{1, \dots, p\}$ into a perturbation vector $d^{(c)} = \sum_{j \in \text{Color}(c)} e_j$.

4. Computes the entire compressed Jacobian in exactly p directional forward-mode passes:

$$J \cdot d^{(c)} = \sum_{j \in \text{Color}(c)} J_{*,j}$$

4. McCormick Relaxations

For factorable bilinear products $z = x \cdot y$ over bounds $x \in [x^L, x^U]$ and $y \in [y^L, y^U]$, the convex underestimator z^{cv} and concave overestimator z^{cc} are:

$$z^{cv} = \max\left(x^L y + x y^L - x^L y^L,\ x^U y + x y^U - x^U y^U\right)$$

$$z^{cc} = \min\left(x^U y + x y^L - x^U y^L,\ x^L y + x y^U - x^L y^U\right)$$

ModelScript evaluates McCormick tuples $[z^{cv}, z^{cc}, \nabla z^{cv}, \nabla z^{cc}]$ directly on WASM arrays.

5. Hybrid Adjoint Sensitivity with Saltation Matrices (`hybrid-adjoint.ts`)

When a hybrid DAE trajectory crosses a discrete event surface $g(x(t^*), p) = 0$ with state reset map $x(t^{*+}) = h(x(t^{*-}), p)$ (such as Modelica `when` clauses, `reinit(x, expr)` impulses, or mechanical contact restitution), the forward sensitivity transition matrix undergoes a discontinuous rank-1 update governed by the **saltation matrix**:

$$\Xi = I + \frac{(f^+ - f^- - \frac{\partial h}{\partial t})(\nabla_x g)^T}{(\nabla_x g)^T f^- + \frac{\partial g}{\partial t}}$$

`HybridAdjointSolver` (`solveHybridAdjoint` in `packages/simulate/src/core/hybrid-adjoint.ts`) implements a three-phase hybrid sensitivity pipeline:

1. **Forward Integration & Event Localization**: Integrates the primal DAE arena while streaming cubic Hermite segments into `DenseHermiteCheckpointTape`. Whenever an event indicator $h(t, x, p)$ changes sign across a step, a 20-iteration bisection root-finder isolates the exact crossing time t_e, records pre-event limits $(x^-, \dot{x}^-)$, applies the discrete reset map $x^+ = \Delta(x^-, p)$, and re-evaluates post-event derivatives $(x^+, \dot{x}^+)$.
2. **Adjoint Jump across Event Surfaces**: During backward Heun integration of the co-state $\lambda(t)$, crossing t_e in reverse applies the transposed saltation jump condition to $\lambda(t_e^-)$ and accumulates the discrete event timing contribution into the parameter gradient $\frac{dL}{dp}$:

$$\lambda(t_e^-) = \left[\frac{\partial \Delta}{\partial x}\right]^T \lambda(t_e^+) + \frac{\left(\frac{d\Delta}{dt}\right)^- - \dot{x}^+}{\dot{h}^-} \nabla_x h(t_e^-)$$

$$\Delta\left(\frac{dL}{dp}\right) = \left[\frac{\partial \Delta}{\partial p}\right]^T \lambda(t_e^+) + \frac{\left(\frac{d\Delta}{dt}\right)^- - \dot{x}^+}{\dot{h}^-} \nabla_p h(t_e^-)$$

3. **Optional Sigmoidal Event Relaxation**: For models with dense chattering or non-smooth algebraic switches (`if`, `abs`, `min`, `max`), `HybridAdjointSolver` optionally invokes `DAEEventRelaxer` to smooth C0 discontinuities into C^∞ sigmoidal transitions prior to adjoint integration.

6. Aitken Dynamic Δ-Relaxation for Coupled Co-Simulation (`fsi-relaxation.ts`)

In partitioned Fluid-Structure Interaction (FSI) and multi-FMU co-simulation macro-steps, exchanging boundary loads and displacements via unrelaxed fixed-point iteration $x_{k+1} = \tilde{x}(x_k)$ often diverges due to artificial added-mass instability.

`VectorAitkenRelaxation` (`packages/simulate/src/core/fsi-relaxation.ts`) accelerates and stabilizes convergence over scalar variables, multi-DOF nodal displacement vectors, or 3D boundary traction maps (`relaxBoundaryLoads`):

1. **Fixed-Point Residual**: At macro-step sub-iteration k, computes the interface residual $r_k = \tilde{x}_k - x_{k-1}$ and residual difference $\Delta r_k = r_k - r_{k-1}$.
2. **Secant Aitken Factor Update**: Updates the dynamic relaxation factor ω_k via the multidimensional Aitken Δ^2 projection:

$$\omega_k = -\omega_{k-1} \frac{r_{k-1}^T (r_k - r_{k-1})}{\|r_k - r_{k-1}\|_2^2}$$

clamped to a safe stability interval $\omega_k \in [\omega_{\min}, \omega_{\max}]$ (default $[0.02, 1.0]$).

3. **Relaxed State Update**: Advances the interface state via under-relaxation:

$$x_{k+1} = x_k + \omega_k r_k$$

terminating when the relative residual norm $\|r_k\|_2 / (\|x_k\|_2 + \epsilon) < \text{tol}$.

4. **Multi-Rate Sub-Cycling (`MultiRateSubCycleScheduler`)**: Synchronizes fast acoustic CFD steps ($\Delta t_{\text{CFD}} \sim 10^{-4}$ s), intermediate elastodynamic FEA steps ($\Delta t_{\text{FEA}} \sim 10^{-3}$ s), and macro 1D DAE steps ($\Delta t_{\text{macro}} \sim 10^{-2}$ s) using linear sub-step boundary interpolation $\mathbf{u}(\tau) = (1 - \tau)\mathbf{u}_{\text{prev}} + \tau \mathbf{u}_{\text{target}}$.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
Dual Numbers	`DAEBuilder` AST, Variable seed vector	Algebraic loop tearing, Newton solvers, sensitivity analysis
AdTape	`DAEBuilder` objective expressions	L-BFGS-B optimizer, surrogate loss backprop
CPR Coloring	Bipartite incidence matrix from `BltEngine`	Sparse Jacobian generation, SUNDIALS/TR-BDF2 linear solves
McCormick & Affine	Bound intervals from `QueryEngine`	HC4 contractor, Nelson-Oppen theory coordinator, SOS bounds
HybridAdjointSolver	`DAEBuilder`, `DenseHermiteCheckpointTape`, Events	Hybrid parameter calibration, optimal control across mode switches
VectorAitkenRelaxation	Partitioned CFD/FEA boundary loads & FMU ports	Coupled aeroelastic FSI solver, SSP co-simulation master

CHAPTER 54

Numerical ODE, DAE & Co-Simulation Solvers

Implementation: `packages/simulate/src/solvers/` and `packages/runtime/src/solvers/`

---

1. Tsitouras 5(4) Adaptive Runge-Kutta (Tsit5)

Academic Citations

- **Tsitouras, Ch. (2011)**. "*Runge-Kutta pairs of order 5 (4) satisfying only the first column simplifying assumption.*"
  Computers & Mathematics with Applications, 62(2), pp. 770–775.
  DOI: 10.1016/j.camwa.2011.06.002
- **Rackauckas, C., & Nie, Q. (2017)**. "*DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia.*"
  Journal of Open Research Software, 5(1), 15. DOI: 10.5334/jors.151

ModelScript Rationale & Modifications

For non-stiff systems (e.g. mechanical multi-body dynamics, kinematics, flight control), high-order adaptive explicit Runge-Kutta pairs offer peak throughput without Newton-Raphson linear algebra costs. Tsit5 demonstrates significantly lower error coefficients and wider stability regions than classical Dormand-Prince (DOPRI5).

- **Modifications**: Employs the First-Same-As-Last (FSAL) optimization eliminating stage 1 evaluation in successive steps; embeds a 5th-order continuous dense output polynomial for sub-interval event zero-crossing localization.

---

2. TR-BDF2 Composite Stiff Solver

Academic Citations

- **Bank, R. E., Coughran, W. M., et al. (1985)**. "*Transient simulation of silicon devices and circuits.*"
  IEEE Transactions on CAD of Integrated Circuits, 4(4), pp. 436–446.
  DOI: 10.1109/TCAD.1985.1270142
- **Hosea, M. E., & Shampine, L. F. (1996)**. "*Analysis and implementation of TR-BDF2.*"
  Applied Numerical Mathematics, 20(1-2), pp. 21–37.
  DOI: 10.1016/0168-9274(95)00115-8

ModelScript Rationale & Modifications

Cyber-physical models featuring frequent hybrid discrete events (switches, impacts, hysteresis) severely penalize multistep BDF solvers (which must drop to order 1 after every event). TR-BDF2 is an L-stable one-step composite method pairing a trapezoidal half-step with a BDF2 completion step:

- **Modifications**: Shares the diagonal Jacobian coefficient $d = 1 - 1/\sqrt{2}$ across both internal stages, requiring only **one** sparse LU factorization per time step; connects directly with ModelScript's in-WASM sparse matrix kernel (`sparse_lu.ts`).

---

3. RODAS-4P Linearly Implicit Rosenbrock Solver

Academic Citations

- **Steinebach, G. (1995)**. "*Order-reduction of ROW-methods for DAEs and method of lines applications.*"
  TH Darmstadt Preprint 1742.
- **Hairer, E., & Wanner, G. (1996)**. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*.
  Springer-Verlag. Section IV.8.

ModelScript Rationale & Modifications

Solves stiff index-1 DAEs and method-of-lines continuum discretizations without iterative non-linear Newton solves. It solves only a sequence of linear systems sharing the same system matrix $I - \gamma h J$.

- **Modifications**: Optimized for in-WASM sparse Jacobian caching; utilizes automatic differentiation coloring (`coloring.ts`) for fast Jacobian updates.

---

4. Itô Stochastic Differential Equations (SDE) Solver

Academic Citations

- **Rößler, A. (2010)**. "*Runge-Kutta methods for Itô stochastic differential equations with additive noise.*"
  SIAM Journal on Numerical Analysis, 48(3), pp. 922–952. DOI: 10.1137/09076636X
- **Kloeden, P. E., & Platen, E. (1992)**. *Numerical Solution of Stochastic Differential Equations*.
  Springer. DOI: 10.1007/978-3-662-12616-5

ModelScript Rationale & Modifications

Physical systems operate under thermal noise, turbulence, and sensor fluctuations:

$$dx = f(t, x)\,dt + g(t, x)\,dW_t$$

- **Modifications**: Employs an in-WASM `Xoshiro256pp` PRNG with Box-Muller transformation; provides both fixed-step Euler-Maruyama (strong order 0.5) and adaptive Rößler SRIW1 (strong order 1.5) integrators for Monte Carlo uncertainty rollouts.

---

5. Co-Simulation Master Orchestrator

Academic Citations

- **Bastian, J., et al. (2011)**. "*Master for Co-Simulation Using FMI.*"
  Proceedings of the 8th International Modelica Conference, pp. 115–120. DOI: 10.3384/ecp11063115
- **Kübler, R., & Schiehlen, W. (2000)**. "*Two methods of simulator coupling.*"
  Mathematical and Computer Modelling of Dynamical Systems, 6(2), pp. 93–113. DOI: 10.1076/1387-3954(200006)6:2;1-M;FT093

ModelScript Rationale & Modifications

Coordinates multi-rate, multi-vendor FMI Functional Mock-up Units (FMUs) and SSP topologies. Implements Richardson extrapolation error estimation to dynamically adapt the inter-simulator communication step size H_{comm}, preventing divergence while maximizing simulation speed.

---

6. Dommel Companion-Circuit EMT & Bergeron Traveling-Wave Solver

Academic Citations

- **Dommel, H. W. (1969)**. "*Digital Computer Solution of Electromagnetic Transients in Single- and Multiphase Networks.*"
  IEEE Transactions on Power Apparatus and Systems, PAS-88(4), pp. 388–399.
  DOI: 10.1109/TPAS.1969.292459
- **Marti, J. R. (1982)**. "*Accurate Modelling of Frequency-Dependent Transmission Lines in Electromagnetic Transient Simulations.*"
  IEEE Transactions on Power Apparatus and Systems, PAS-101(1), pp. 147–157.
  DOI: 10.1109/TPAS.1982.317332

ModelScript Rationale & Modifications

Implemented in `EmtSolver` (`packages/simulate/src/solvers/emt/emt-solver.ts`), this solver targets microsecond-scale electromagnetic transients, power electronics switching (IGBTs/breakers), and distributed transmission lines imported from `@modelscript/pscad` :

- **Trapezoidal Companion Models & Constant Nodal Matrix**: Inductors ($G_L = \frac{\Delta t}{2L}$) and capacitors ($G_C = \frac{2C}{\Delta t}$) are discretized into Norton companion equivalents $\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t) + \mathbf{i}_{\text{src}}(t)$, allowing $\mathbf{G}$ to remain constant between switching events.
- **Critical Damping Adjustment (CDA)**: Automatically transitions from Trapezoidal to Backward Euler integration ($G_L = \frac{\Delta t}{L}, G_C = \frac{C}{\Delta t}$) for two half-steps ($\Delta t/2$) immediately after any breaker or semiconductor switching discontinuity to suppress artificial dv/dt numerical oscillations.
- **Bergeron Traveling-Wave Line Decoupling & Norton FMU Interface**: Distributed transmission lines with propagation delay $\tau = \ell/v \geq \Delta t$ decouple the nodal admittance matrix into independent sub-grids using interpolated history ring buffers, and expose `getNortonEquivalent(portNode)` ($I_{\text{eq}}, G_{\text{eq}}$) for zero-delay FMI Co-Simulation coupling.

---

7. High-Order Taylor Model Flowpipe Integration

Academic Citations

- **Makino, K., & Berz, M. (2003)**. "*Taylor Models and Other Validated Functional Inclusion Methods.*" **International Journal of Pure and Applied Mathematics**, 4(4), pp. 379–456.
- **Chen, X., Ábrahám, E., & Sankaranarayanan, S. (2013)**. "*Flow*: An Analyzer for Non-Linear Hybrid Systems.*" **Computer Aided Verification (CAV 2013)**, LNCS vol. 8044, pp. 258–263. DOI: 10.1007/978-3-642-39799-8_18

ModelScript Rationale & Modifications

Implemented in `TaylorModel` (`packages/runtime/src/analysis/wasm_taylor_model.ts`), validated Taylor Model arithmetic bounds all trajectories of a non-linear ODE/DAE system subject to uncertain initial sets $x(0) \in X_0$ and parameter intervals without wrapping-effect blowup:

- **Polynomial + Interval Remainder Representation**: Represents each state reachable tube over normalized domain $\mathbf{z} \in [-1, 1]^d$ as a multivariate polynomial $P_k(\mathbf{z})$ of order k plus a sound interval remainder $I = [\underline{r}, \overline{r}]$, where truncated higher-order terms ($> k$) are conservatively swept into I.
- **Picard-Lindelöf Validated Operator**: Evaluates the contractive Picard integral operator $\mathrm{P}(X)(t) = X_0 + \int_0^t f(s, X(s))\, ds$ directly on Taylor models to construct guaranteed continuous-time flowpipes consumed by `ContinuousSafetyOracle` and SysML v2 hybrid state-machine reachability (`hybrid-flowpipe-bridge.ts`).

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - DAE blocks partitioned and torn by `BltEngine` and `TornBlock` .
  - Sparse Jacobians computed via `Dual Numbers & Graph Coloring` .
- **Downstream Consumers**:
  - Powers `msx simulate` , `msx optimize` , and the Web IDE charting canvas.
  - Feeds state trajectories and validated flowpipes into `Signal Temporal Logic (STL) Monitor & Theory Coordinator` and formal verification oracles.

CHAPTER 55

Mathematical Optimization & Calibration Solvers

Implementation: `packages/simulate/src/optimizer/solvers/`

---

1. Limited-Memory BFGS with Bounds (L-BFGS-B)

Academic Citations

- **Byrd, R. H., Lu, P., Nocedal, J., & Zhu, C. (1995)**. "*A limited memory algorithm for bound constrained optimization.*"
  SIAM Journal on Scientific Computing, 16(5), pp. 1190–1208. DOI: 10.1137/0916069
- **Zhu, C., Byrd, R. H., Lu, P., & Nocedal, J. (1997)**. "*Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization.*"
  ACM Transactions on Mathematical Software, 23(4), pp. 550–560. DOI: 10.1145/279232.279236

ModelScript Rationale & Modifications

Solves large-scale bound-constrained non-linear minimization:

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to } l \le x \le u$$

Crucial for parameter identification and direct collocation optimal control where thousands of discretized states and control bounds must be optimized.

- **Modifications**: Employs compact representation of the inverse Hessian approximation H_k using history buffers $m \in [5, 20]$ in typed arrays; Cauchy point computation with piecewise linear path search; exact line search with strong Wolfe conditions.

---

2. Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

Academic Citations

- **Hansen, N., & Ostermeier, A. (2001)**. "*Completely derandomized self-adaptation in evolution strategies.*"
  Evolutionary Computation, 9(2), pp. 159–195. DOI: 10.1162/106365601750190398
- **Hansen, N. (2016)**. "*The CMA Evolution Strategy: A Tutorial.*"
  arXiv preprint: arXiv:1604.00772.

ModelScript Rationale & Modifications

Derivative-free black-box optimization on rugged, non-convex, non-separable objective functions (e.g. tuning PID controller gains over simulations with discontinuous switching events).

- **Modifications**: Tracks full covariance matrix $C \in \mathrm{R}^{n \times n}$ updates with rank-μ and rank-one update mechanisms; vectorized population sampling using WebAssembly-accelerated Cholesky decomposition; step-size control using cumulative path lengths (CSA).

---

3. Differential Evolution (DE)

Academic Citations

- **Storn, R., & Price, K. (1997)**. "*Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces.*"
  Journal of Global Optimization, 11(4), pp. 341–359. DOI: 10.1023/A:1008202821328

ModelScript Rationale & Modifications

Stochastic population-based global search algorithm utilizing vector differences for directional perturbations. Highly effective for parameter estimation against experimental CSV test bench data.

- **Modifications**: Supports `rand/1/bin`, `best/1/bin`, and `current-to-best/1` mutation strategies with adaptive crossover probability (CR) and scaling factor (F).

---

4. Particle Swarm Optimization (PSO)

Academic Citations

- **Kennedy, J., & Eberhart, R. (1995)**. "*Particle swarm optimization.*"
  Proceedings of IEEE ICNN'95, Vol. 4, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968
- **Shi, Y., & Eberhart, R. C. (1998)**. "*A modified particle swarm optimizer.*"
  IEEE Congress on Evolutionary Computation, pp. 69–73.

ModelScript Rationale & Modifications

Simulates social swarm dynamics for continuous global search:

$$v_i(t+1) = wv_i(t) + c_1r_1(p_i - x_i) + c_2r_2(g - x_i)$$

- **Modifications**: Implements constriction factors and dynamically declining inertia weights $w(t)$ to transition smoothly from global exploration to local exploitation.

---

5. Non-Dominated Sorting Genetic Algorithm II (NSGA-II)

Academic Citations

- **Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002)**. "*A fast and elitist multiobjective genetic algorithm: NSGA-II.*"
  IEEE Transactions on Evolutionary Computation, 6(2), pp. 182–197. DOI: 10.1109/4235.996017

ModelScript Rationale & Modifications

Solves multi-objective engineering trade-offs (e.g. simultaneously minimizing electric vehicle energy consumption while maximizing battery longevity and top speed).

- **Modifications**: Fast non-dominated sorting ($\mathrm{O}(MN^2)$ complexity) paired with crowding-distance assignment in continuous parameter spaces; outputs full Pareto fronts directly.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Trajectory residuals evaluated by `Simulation Solvers` against `CSV Datasets`.
  - Collocation constraint formulations from `msx optimize`.
- **Downstream Consumers**:
  - Powers `msx falsify` adversarial requirement falsification across SysML v2 / Modelica / CAD.
  - Powers `msx cosim optimize` for black-box multi-FMU tuning across SSP boundaries.
  - Powers `ModelicaCalibrator` with `hybrid-cmaes-lm` (CMA-ES global exploration + Levenberg-Marquardt quadratic polishing).
  - Powers `TradeStudyEngine.evolve` for generative multi-objective Pareto design trade studies.

Programmatic TypeScript API

```
import { cmaesSolve, deSolve, psoSolve, nsga2Solve, type BlackBoxProblem } from "@modelscript/simulate/optimizer";

// 1. Single-Objective Black-Box Optimization (CMA-ES, DE, PSO)
const problem: BlackBoxProblem = {
  dimension: 2,
  bounds: {
    min: new Float64Array([-5, -5]),
    max: new Float64Array([5, 5]),
  },
  fitness: (x) => Math.pow(1 - x[0]!, 2) + 100 * Math.pow(x[1]! - x[0]! * x[0]!, 2),
};

const result = await cmaesSolve(problem, { maxGenerations: 100 });
console.log("Optimum:", result.bestSolution, "Cost:", result.bestFitness);

// 2. Hybrid Global-Local Model Calibration
import { ModelicaCalibrator } from "@modelscript/simulate/optimizer";

const calibrator = new ModelicaCalibrator(arena, sim, {
  parameters: ["k", "c"],
  parameterBounds: new Map([
    ["k", { min: 1, max: 30 }],
    ["c", { min: 0.1, max: 10 }],
  ]),
  measurements,
  method: "hybrid-cmaes-lm", // Global CMA-ES basin exploration + Levenberg-Marquardt refinement
});
const calResult = calibrator.calibrate();
```

CLI Command Examples

```
# 1. Adversarial requirement falsification with CMA-ES
msx falsify --formula "always[0,10] (stress <= 180)" --algorithm cma-es --population 20

# 2. Black-box co-simulation parameter tuning with Differential Evolution
msx cosim optimize ssp-archive.ssp --params "kp:0.1:10.0,kd:0.01:1.0" --objective "error_integral" --algorithm de
```

CHAPTER 56

Nelson-Oppen Theory Combination & SMT Solvers

Implementation: `packages/runtime/src/formal/`

---

1. Nelson-Oppen Cooperation & DPLL(T)

Academic Citations

- **Nelson, G., & Oppen, D. C. (1979)**. "*Simplification by cooperating decision procedures.*"
  ACM Transactions on Programming Languages and Systems (TOPLAS), 1(2), pp. 245–257.
  DOI: 10.1145/357073.357079
- **Nieuwenhuis, R., Oliveras, A., & Tinelli, C. (2006)**. "*Solving SAT and SAT Modulo Theories: From an abstract Davis--Putnam--Logemann--Loveland procedure to DPLL(T).*"
  Journal of the ACM, 53(6), pp. 937–977.
  DOI: 10.1145/1217856.1217859

ModelScript Rationale & Modifications

Proves cross-domain assertions spanning discrete architectural modes, differential equations, interval constraints, and 3D CAD collisions.

- **Modifications**: Extended beyond traditional software theories (uninterpreted functions, bit-vectors) to **12 engineering domains** including OWL2 Description Logic, CAD B-Rep spatial physics, and DAE continuous trajectories; combines learned conflict clause propagation with Salsa query invalidation.

---

2. CDCL SAT Solver with VSIDS

Academic Citations

- **Silva, J. P. M., & Sakallah, K. A. (1999)**. "*GRASP: A search algorithm for propositional satisfiability.*"
  IEEE Transactions on Computers, 48(5), pp. 506–521. DOI: 10.1109/12.769433
- **Moskewicz, M. W., et al. (2001)**. "*Chaff: Accelerating SAT design verification.*"
  ACM/IEEE Design Automation Conference (DAC), pp. 530–535. DOI: 10.1145/378239.379017

ModelScript Rationale & Modifications

Solves propositional satisfiability for discrete mode selection, architectural state machines, and CDCL(T) theory case splitting.

- **Modifications**: Two-watched-literals data structure with contiguous integer arrays; Variable State Independent Decaying Sum (VSIDS) decision heuristic; 1-UIP (First Unique Implication Point) conflict analysis; Luby-sequence restart schedule.

---

3. Craig Interpolation

Academic Citations

- **Craig, W. (1957)**. "*Linear reasoning. A new form of the Herbrand-Gentzen theorem.*"
  Journal of Symbolic Logic, 22(3), pp. 250–268. DOI: 10.2307/2963593
- **McMillan, K. L. (2003)**. "*Interpolation and SAT-based model checking.*"
  Computer Aided Verification (CAV 2003), LNCS 2725, pp. 1–13. DOI: 10.1007/978-3-540-45069-6_1
- **Pudlák, P. (1997)**. "*Lower bounds for resolution and cutting planes proofs and monotone computations.*"
  Journal of Symbolic Logic, 62(3), pp. 981–998. DOI: 10.2307/2275583

ModelScript Rationale & Modifications

Given unsatisfiable mutually contradictory formulas $A \wedge B \models \bot$, Craig interpolation synthesizes an interpolant I such that:

$$A \implies I \quad \text{and} \quad I \wedge B \models \bot$$

where $\text{vars}(I) \subseteq \text{vars}(A) \cap \text{vars}(B)$.

- **Modifications**: Employs Pudlák-McMillan resolution graph traversal to generate certified interpolants explaining cross-domain requirement violations without revealing proprietary subsystem equations.

---

4. HC4 Constraint Contractor

Academic Citations

- **Benhamou, F., Goualard, F., Granvilliers, L., & Puget, J. F. (1999)**. "*Revising Hull and Box Consistency.*" **International Conference on Logic Programming (ICLP'99)**, pp. 230–244.

ModelScript Rationale & Modifications

Evaluates non-linear algebraic constraints over interval domains without discretization grids:

- **Modifications**: Two-phase HC4Revise operator: bottom-up forward evaluation of interval bounds followed by top-down backward narrowing; eliminates physically impossible state regions before numerical simulation.

---

5. Signal Temporal Logic (STL) Quantitative Monitor

Academic Citations

- **Maler, O., & Nickovic, D. (2004)**. "*Monitoring Temporal Properties of Continuous Signals.*" **FORMATS/FTRTFT 2004**, LNCS 3253, pp. 152–166. DOI: 10.1007/978-3-540-30206-3_12
- **Donzé, A., & Maler, O. (2010)**. "*Robust Satisfaction of Temporal Logic over Real-Valued Signals.*" **FORMATS 2010**, LNCS 6246, pp. 92–106. DOI: 10.1007/978-3-642-15297-9_9

ModelScript Rationale & Modifications

Monitors simulation trajectories against temporal specifications (e.g. "overshoot must remain below 5% for at least 2 seconds after step input"):

$$\rho(\varphi, s, t) \in \mathrm{R}$$

Computes real-valued **quantitative robustness** indicating how far a trajectory is from violating safety boundaries.

---

6. Constrained Zonotopes

Academic Citations

- **Scott, J. K., Raimondo, D. M., et al. (2016)**. "*Constrained zonotopes: A new tool for set-based estimation and fault detection.*" **Automatica**, 69, pp. 126–136. DOI: 10.1016/j.automatica.2016.02.036

ModelScript Rationale & Modifications

Set-based reachability analysis under bounded initial states and parameter uncertainty. Closed under linear transformations, Minkowski addition, and hyperplanar intersection without facet explosion.

- **Modifications**: Implemented as in-WASM linear algebraic routines utilizing singular value decomposition (SVD) for order reduction.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:

- Requirements ingested from `SysML v2`.
- Trajectories simulated by `Numerical Solvers`.
- Spatial boundaries provided by `STEP CAD`.

- **Downstream Consumers**:
  - Powers `msx verify`, generates compliance certificates, and exports mathematical proofs in `LaTeX format`.

CHAPTER 57

Constrained Horn Clauses (Spacer), IC3/PDR & Certified Verification

Implementation:

- Spacer CHC & IC3/PDR: `packages/runtime/src/formal/chc/` and `packages/runtime/src/formal/ic3_engine.ts`
- Stochastic Martingale Barrier Certificates: `StochasticBarrierSynthesizer` in `packages/runtime/src/analysis/wasm_stochastic_barrier.ts`
- WebGPU Datalog / OWL 2 RL Saturation Reasoner: `WebGPUOntologyReasoner` & `BitsetGraph` in `packages/runtime/src/gpu/webgpu_reasoner.ts`
- ISO 26262 / DO-178C Back-to-Back Equivalence Verification: `B2BEquivalenceVerifier` in `packages/runtime/src/simulation/b2b_verifier.ts` and `UnifiedVerifier` in `packages/runtime/src/simulation/unified_verifier.ts`

---

1. Spacer CHC Engine (Generalized PDR)

Academic Citations

- **Komuravelli, A., Gurfinkel, A., & Chaki, S. (2014)**. "*SMT-Based Model Checking for Recursive Programs.*" **Computer Aided Verification (CAV 2014)**, LNCS 8559, pp. 17–34. DOI: 10.1007/978-3-319-08867-9_2
- **Hoder, K., & Bjørner, N. (2012)**. "*Generalized Property Directed Reachability.*" **Theory and Applications of Satisfiability Testing (SAT 2012)**, LNCS 7317, pp. 157–171. DOI: 10.1007/978-3-642-31612-8_13

ModelScript Rationale & Modifications

Constrained Horn Clauses (CHCs) represent systems of logical relations of the form:

$$\forall X.\, \phi(X) \land P_1(X) \land \cdots \land P_k(X) \implies H(X)$$

Spacer solves CHCs by combining inductive generalization, Craig interpolation, and under-approximated reachability trees without bounding the unrolling depth. In ModelScript, Spacer verifies infinite-state state machine transitions and recursive cyber-physical control architectures.

- **Modifications**: Directly integrates with ModelScript's in-WASM CDCL SAT core and Craig interpolator; implements specialized heuristics for linear real arithmetic over cyber-physical invariants.

---

2. IC3 / Property Directed Reachability (PDR)

Academic Citations

- **Bradley, A. R. (2011)**. "*SAT-based model checking without unrolling.*" **VMCAI 2011**, LNCS 6538, pp. 70–87. DOI: 10.1007/978-3-642-18275-4_7
- **Een, N., Mishchenko, A., & Brayton, R. (2011)**. "*Efficient implementation of property directed reachability.*" **Formal Methods in Computer-Aided Design (FMCAD 2011)**, pp. 125–134.

ModelScript Rationale & Modifications

Proves whether safety property P holds across all reachable states of a discrete or hybrid transition system without unrolling the transition relation $T(s, s')$ to a fixed horizon k.

- **Modifications**: Maintains inductive clause frames $F_0, F_1, \ldots, F_k$ where F_i over-approximates states reachable in up to i steps; uses counterexample-guided abstraction refinement (CEGAR) to block Proof Obligations (POBs); propagates clauses forward to

find an inductive invariant ($F_i \equiv F_{i+1}$) proving unbounded safety.

---

3. Stochastic Barrier Certificates (`wasm_stochastic_barrier.ts`)

Academic Citations

- **Prajna, S., Jadbabaie, A., & Pappas, G. J. (2007)**. "*A framework for worst-case and stochastic safety verification using barrier certificates.*"
  IEEE Transactions on Automatic Control, 52(8), pp. 1415–1428.
  DOI: 10.1109/TAC.2007.902736
- **Kushner, H. J. (1967)**. *Stochastic Stability and Control.*
  Academic Press.

ModelScript Rationale & Modifications

For continuous-time stochastic differential equations (Itô diffusions) driven by standard Wiener processes W_t:

$$dx(t) = f(x)\,dt + \sigma(x)\,dW_t$$

deterministic trajectories do not exist, and safety verification requires bounding the tail probability that $x(t)$ ever enters an unsafe region $\mathrm{X}_u = \{x \mid \|x\|_2 \geq r_u\}$.

`StochasticBarrierSynthesizer` (`packages/runtime/src/analysis/wasm_stochastic_barrier.ts`) synthesizes non-negative supermartingale barrier candidates $B(x) = x^T P x \geq 0$ using the **infinitesimal generator** L of the Itô process:

$$\mathrm{L}B(x) = \nabla B(x) \cdot f(x) + \frac{1}{2}\mathrm{tr}(\sigma(x)\sigma(x)^T \nabla^2 B(x)) \leq -c$$

- **Doob's Maximal Supermartingale Inequality**: Bounds the infinite-horizon and finite-horizon probability of unsafe set entry via:

$$\mathrm{P}(\sup_{t\geq 0} \|x(t)\|_2 \geq r_u) \leq \frac{B(x_0) + \frac{\mathrm{tr}(\Sigma P)}{2\alpha}}{\lambda_{\min}(P) r_u^2} \leq \delta_{\text{risk}}$$

  where $\Sigma = \sigma(0)\sigma(0)^T$ is the diffusion noise covariance and α is the linearized drift decay rate.
- **Soundness & Rigor Reporting**: Emits structured `VerificationVerdict` diagnostics distinguishing fast linearized quadratic risk estimates from Sum-of-Squares (SOS) / LP certified supermartingales.

---

4. WebGPU Datalog / OWL 2 RL Saturation Reasoner (`webgpu_reasoner.ts`)

Academic Citations

- **Motik, B., Patel-Schneider, P. F., & Parsia, B. (2012)**. "*OWL 2 Web Ontology Language: Profiles (Second Edition) — OWL 2 RL.*"
  W3C Recommendation. https://www.w3.org/TR/owl2-profiles/
- **Warshall, S. (1962)**. "*A theorem on Boolean matrices.*"
  Journal of the ACM, 9(1), pp. 11–12. DOI: 10.1145/321105.321107

ModelScript Rationale & Modifications

Large-scale systems engineering ontologies (`languages/owl2/` and SysML v2 KerML specialization hierarchies) contain 10^5 to 10^7 ABox/TBox assertions. Evaluating transitive role closures (`subClassOf` , `partOf` , `satisfies`) and SHACL numeric property constraints sequentially on the CPU becomes a bottleneck during workspace-wide verification.

`WebGPUOntologyReasoner` (`packages/runtime/src/gpu/webgpu_reasoner.ts`) implements a massively parallel GPU relational join and saturation co-processor with a CPU SIMD bitset fallback:

- **Bit-Packed Adjacency Matrix (`BitsetGraph`)**: Encodes an $N \times N$ directed graph as a flat `Uint32Array` where each node row u occupies $\lceil N/32 \rceil$ 32-bit words, achieving $32\times$ memory compression and single-instruction bitwise row unions.

- **Parallel Logarithmic Doubling in WGSL (`TRANSITIVE_CLOSURE_WGSL`)**: Dispatches compute workgroups (`workgroups = ceil(N / 64)`) over ping-pong storage buffers (`bufIn` , `bufOut`) for $\lceil \log_2 N \rceil + 1$ doubling rounds, computing the full reflexive-transitive closure R^* in $\mathrm{O}(\log N)$ GPU passes.
- **Parallel SHACL Constraint Kernel (`SHACL_VALIDATION_WGSL`)**: Validates `SHACLNumericConstraint` bounds (`minInclusive` , `maxInclusive` , `expectedClassId`) across strided `Float32Array` instance attribute tables and writes compact violation bitsets.

---

5. ISO 26262 / DO-178C Back-to-Back Equivalence Verification (`b2b_verifier.ts` , `v_cycle_verifier.ts`)

Academic Citations

- **ISO 26262-6:2018 & ISO 26262-8:2018**. *Road vehicles — Functional safety — Part 6: Product development at the software level (Table 7: Back-to-back comparison test between model and code) & Part 8: Supporting processes (Clause 11: Confidence in the use of software tools, TCL1).*
- **RTCA DO-178C /DO-331 (2011)**. *Software Considerations in Airborne Systems and Equipment Certification — Model-Based Development and Verification Supplement.*

ModelScript Rationale & Modifications

Safety-critical automotive (ISO 26262 ASIL-D) and avionics (DO-178C DAL-A) standards require proving that generated embedded production code (zero-allocation C99 or standalone WASM FMU artifacts) is numerically and behaviorally equivalent to the high-level specification model (SysML v2 / Modelica).

`B2BEquivalenceVerifier` (`packages/runtime/src/simulation/b2b_verifier.ts` , orchestrated by `UnifiedVerifier` in `packages/runtime/src/simulation/unified_verifier.ts`) automates end-to-end **Model-in-the-Loop (MiL) vs. Software-in-the-Loop (SiL)** back-to-back verification:

1. **MiL Reference Trajectory Execution**: Simulates the high-level DAE arena (`simulateArenaAsync`) over $[t_{\text{start}}, t_{\text{stop}}]$ to obtain reference trajectories $y_j^{\text{MiL}}(t)$.
2. **Standalone C99 Code Generation & Cryptographic Hashing**: Invokes `generateStandaloneCSources` (`@modelscript/exchange/fmu`) to emit self-contained, zero-allocation C99 sources (`<Model>_standalone.h` , `<Model>_standalone.c`), computing a SHA-256 artifact digest (`cSourceHash`) for V-cycle traceability.
3. **Automated Test Harness & Native Compilation**: Synthesizes a deterministic C99 fixed-step RK4 test harness (`<Model>_b2b_harness.c`), compiles the binary via `gcc -O2 -Wall -Wextra -std=c99 -lm` , and streams CSV output trajectories $y_j^{\text{SiL}}(t_k)$.
4. **Bisimulation & Signal-Tolerance Equivalence Checking**: Aligns SiL samples against binary-searched linearly interpolated MiL trajectories and verifies the combined absolute/relative tolerance envelope across all K time steps and M output signals:

$$\forall j \in \{1, \dots, M\},\ \forall k \in \{1, \dots, K\} : \quad \left| y_j^{\text{MiL}}(t_k) - y_j^{\text{SiL}}(t_k) \right| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \left| y_j^{\text{MiL}}(t_k) \right|$$

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - State machine models and safety requirements from `SysML v2` .
  - Mode switching, discrete `when` / `if` clauses, and continuous/stochastic DAE arenas lowered from `Modelica` .
  - OWL 2 TBox/ABox ontological triples from `OWL2` .
- **Downstream Consumers**:
  - Verifies architectural safety contracts, stochastic risk bounds, and ISO 26262 MiL/SiL equivalence in `msx verify` .
  - Feeds theory lemmas and transitive subsumption closures into `Nelson-Oppen Coordinator` .

CHAPTER 58

Abstract Interpretation, Polyhedral Domains & Continuous Reachability

Implementation:

- Abstract Interpretation Domains: `packages/runtime/src/formal/abstract_interpretation/`
- DDE Method-of-Steps Flowpipes: `DdeFlowpipeSolver` in `packages/runtime/src/analysis/wasm_dde_flowpipe.ts`
- Hamilton-Jacobi-Isaacs (HJI) Level-Set Solver: `HjiLevelSetSolver` in `packages/runtime/src/analysis/wasm_hji_solver.ts`

---

1. Bourdoncle Weak Topological Ordering (WTO) Fixpoint Solver

Academic Citations

- **Bourdoncle, F. (1993)**. "*Efficient chaotic iteration strategies with widenings.*"
  Formal Methods in Programming and Their Applications, LNCS 735, pp. 128–141. DOI: 10.1007/BFb0039704
- **Cousot, P., & Cousot, R. (1977)**. "*Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints.*"
  ACM POPL'77, pp. 238–252. DOI: 10.1145/512950.512973

ModelScript Rationale & Modifications

Computes sound abstract over-approximations of state variables across complex nested loops and recursion without simulating unbounded trajectories.

- **Modifications**: Implements Bourdoncle's recursive hierarchical WTO decomposition to identify minimal loop heads; accelerates convergence using delayed widening (∇) followed by monotonic narrowing (Δ).

---

2. Two Variables Per Inequality (TVPI) Polyhedral Domain

Academic Citations

- **Simon, A., King, A., & Howe, J. M. (2002)**. "*Two Variables per Inequality as an Abstract Domain.*"
  LOPSTR 2002, LNCS 2664, pp. 71–89. DOI: 10.1007/3-540-36388-2_7
- **Nelson, G. (1978)**. "*An $n^{O(\log n)}$ algorithm for the two-variable-per-inequality integer programming problem.*"
  Technical Report, Stanford University.

ModelScript Rationale & Modifications

Captures planar relational linear invariants of the form:

$$ax_i + bx_j \le c \quad (a, b, c \in \mathbb{Q})$$

Much more expressive than simple intervals or octagons, allowing ModelScript to prove physical conservation laws (e.g. $p_1 + p_2 = \text{const}$) without the exponential vertex-facet explosion of general convex polyhedra.

- **Modifications**: Planar Graham-scan convex hull algorithms in WebAssembly typed memory; relational projection routines optimized for physical parameter spaces.

---

3. Octagon Difference Bound Matrices (DBM) Domain

Academic Citations

- **Miné, A. (2006)**. "*The octagon abstract domain.*"
  Higher-Order and Symbolic Computation, 19(1), pp. 31–100.
  DOI: 10.1007/s10990-006-8609-1

ModelScript Rationale & Modifications

Restricts linear relationships to constraints of the form:

$$\pm x_i \pm x_j \leq c$$

Achieves cubic time complexity $O(n^3)$ via Floyd-Warshall shortest path algorithms over $2n \times 2n$ potential graphs.

- **Modifications**: Direct integration into the `@modelscript/dsl` type inference system and `AbstractDomainOracle` in the Nelson-Oppen coordinator.

---

4. Array Segment Partitioning

Academic Citations

- **Cousot, P., Cousot, R., & Logozzo, F. (2011)**. "*A Parametric Segmentation Functor for Fully Automatic and Scalable Array Content Analysis.*"
  ACM POPL'11, pp. 505–518. DOI: 10.1145/1926385.1926444

ModelScript Rationale & Modifications

Verifies large array equations and indexed multi-dimensional components in Modelica without unrolling every array index into separate scalar variables. Partitions index sets into contiguous segments whose bounds and elements share uniform properties.

---

5. Delay Differential Equation (DDE) Method-of-Steps Flowpipes (`wasm_dde_flowpipe.ts`)

Academic Citations

- **Bellman, R., & Cooke, K. L. (1963)**. *Differential-Difference Equations.*
  Academic Press.
- **Zou, L., Fränzle, M., Xue, B., & Zhan, N. (2015)**. "*orda: An Ode and Dde Reachability Analysis Tool.*"
  SETTA 2015, LNCS 9409, pp. 109–123. DOI: 10.1007/978-3-319-25942-0_8

ModelScript Rationale & Modifications

Physical transport delays, networked control loops, and Modelica `delay(x, tau)` operators yield infinite-dimensional Delay Differential Equations (DDEs):

$$\dot{x}(t) = f(t, x(t), x(t-\tau)), \qquad x(t) \in \Phi(t) \text{ for } t \in [-\tau, 0]$$

`DdeFlowpipeSolver` (`packages/runtime/src/analysis/wasm_dde_flowpipe.ts`) implements **Method-of-Steps** reachability propagation across consecutive delay horizons $[0, \tau], [\tau, 2\tau], \ldots$:

- **History Interval Buffering**: Maintains a timestamped ring buffer `historyMap` storing both nominal trajectories $\bar{x}(t)$ and interval/zonotope tube enclosures $\mathrm{X}(t) = [\underline{x}(t), \overline{x}(t)]$. For $t - \tau \leq t_0$, queries evaluate the initial functional history enclosure $\Phi(t - \tau)$; for $t - \tau > t_0$, historical enclosures are interpolated from stored flowpipe steps.
- **Interval Picard Enclosure Step**: Advances the state tube across $[t_k, t_k + \Delta t]$ by combining current state bounds $\mathrm{X}(t_k)$ and delayed history bounds $\mathrm{X}(t_k - \tau)$:

$$\mathrm{X}(t_{k+1}) = \mathrm{X}(t_k) + \Delta t \cdot \square f(t_k, \mathrm{X}(t_k), \mathrm{X}(t_k - \tau))$$

- **Epistemic Soundness Tagging**: Explicitly tags the resulting `VerificationVerdict` with rigor metadata (`heuristic` vs. `certified`) so downstream safety monitors distinguish fast corner-sampled delay tube estimates from fully Taylor-enclosed flowpipes.

---

6. Hamilton-Jacobi-Isaacs (HJI) PDE Level-Set Reachability (`wasm_hji_solver.ts`)

Academic Citations

- **Mitchell, I. M., Bayen, A. M., & Tomlin, C. J. (2005)**. "*A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games.*"
  IEEE Transactions on Automatic Control, 50(7), pp. 947–957.
  DOI: 10.1109/TAC.2005.851439
- **Jiang, G. S., & Shu, C. W. (2000)**. "*Weighted ENO schemes for Hamilton-Jacobi equations.*"
  SIAM Journal on Scientific Computing, 21(6), pp. 2126–2143.
  DOI: 10.1137/S106482759732455X

ModelScript Rationale & Modifications

When verifying safety of nonlinear control systems under worst-case adversarial disturbances $d \in \mathrm{D}$ and control authority $u \in \mathrm{U}$, convex set-propagation (zonotopes) can suffer from wrapping effects. `HjiLevelSetSolver` (`packages/runtime/src/analysis/wasm_hji_solver.ts`) computes the exact **Backward Reachable Tube (BRT)** by solving the zero-sum differential game **Hamilton-Jacobi-Isaacs (HJI) PDE** viscosity solution on an Eulerian level-set grid:

$$\frac{\partial V}{\partial t} + \min\{0,\ \min_{u \in \mathrm{U}} \max_{d \in \mathrm{D}} \nabla V(x,t) \cdot f(x,u,d)\} = 0, \qquad V(x,0) = \ell(x)$$

where the sub-zero level set $\mathrm{K}_{\text{unsafe}} = \{x \mid \ell(x) \le 0\}$ defines the target failure region and $\{x \mid V(x,T) \le 0\}$ captures all initial states from which the adversary d can force the trajectory into $\mathrm{K}_{\text{unsafe}}$ within horizon T despite optimal safety control u.

- **5th-Order WENO Upwind Spatial Reconstruction (`weno5Derivatives`)**: Approximates left and right spatial derivatives (p^-, p^+) along each state axis using 3-stencil smoothness indicators (s_1, s_2, s_3) and nonlinear weights $a_k = \frac{\gamma_k}{(\epsilon + s_k)^2}$, eliminating spurious oscillations near non-smooth value-function kinks.
- **Lax-Friedrichs Numerical Hamiltonian**: Evaluates the saddle-point Hamiltonian $H(x,p) = \min_{u \in \mathrm{U}} \max_{d \in \mathrm{D}} p \cdot f(x,u,d)$ at central gradients $p = \frac{1}{2}(p^- + p^+)$ augmented by artificial viscosity dissipation:

$$\hat{H}(x, p^-, p^+) = H\left(x, \frac{p^- + p^+}{2}\right) - \frac{1}{2} \sum_i \alpha_i (p_i^+ - p_i^-), \qquad \alpha_i = \max_{u,d} \left| f_i(x,u,d) \right|$$

- **Monotone BRT Accumulation**: Steps the level-set array backward in time with `nextV[idx] = Math.min(V[idx], V[idx] - dt * dVdt)` and reports the exact unsafe volume fraction across the state space.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Control-flow graphs, `delay(x, tau)` equations, and continuous vector fields from `ModelicaFlattener` .
  - Adversarial disturbance bounds D and safety level-set specifications $\ell(x) \le 0$ from SysML v2 verification cases.
- **Downstream Consumers**:
  - Feeds numeric intervals into `HC4 Contractor` and `Nelson-Oppen Coordinator` .
  - Certifies backward reachable tubes and delay flowpipes in `UnifiedVerifier` (`msx verify`).
  - Eliminates out-of-bounds array access and division-by-zero compiler warnings at compile time.

CHAPTER 59

Graph Rewriting, Pattern Matching & Metaprogramming

Implementation: `packages/dsl/src/codegen/rewriting/` and `packages/dsl/src/codegen/runtime/`

---

1. Triple Graph Grammars (TGG) & Algebraic DPO Rewriting

Academic Citations

- **Schürr, A. (1994)**. "*Specification of graph translators with triple graph grammars.*"
  Graph-Theoretic Concepts in Computer Science (WG '94), LNCS 903, pp. 151–163.
  DOI: 10.1007/3-540-59071-4_45
- **Greenyer, J., & Kindler, E. (2010)**. "*Comparing operational execution strategies for triple graph grammars.*"
  Software & Systems Modeling, 9(4), pp. 441–469.
  DOI: 10.1007/s10270-009-0140-2

ModelScript Rationale & Modifications

Synchronizes multi-domain engineering languages (e.g. Modelica 1D physics, SysML v2 requirements, STEP CAD geometry) declaratively without fragile point-to-point imperatively paired translators.

- **Modifications**: AOT-compiles declarative TGG rule sets into an in-WASM AssemblyScript dispatch kernel (`tgg_forward_dispatch`, `tgg_backward_dispatch`); tracks bidirectional correspondences in linear CST memory; achieves $\mathrm{O}(\Delta N)$ incremental propagation driven by Salsa invalidations.

---

2. Critical Pair Analysis (CPA)

Academic Citations

- **Knuth, D. E., & Bendix, P. B. (1970)**. "*Simple word problems in universal algebras.*"
  Computational Problems in Abstract Algebra, pp. 263–297. Pergamon Press.
- **Ehrig, H., Ehrig, K., de Lara, J., Taentzer, G., Varró, D., & Varró, G. (2005)**. "*Termination and confluence of graph transformation systems.*"
  Formal Methods in Software and Systems Modeling, LNCS 3440, pp. 180–195. DOI: 10.1007/978-3-540-31847-7_11
- **Heckel, R., & Taentzer, G. (2020)**. *Graph Transformation for Software Engineers: Formal Foundation and Industrial Applications*.
  Springer. DOI: 10.1007/978-3-030-43969-9

ModelScript Rationale & Modifications

In polyglot workflows, multiple rules may compete to transform the same AST elements. CPA statically detects all critical pairs (conflicting rule overlaps, target collisions, and circular translation loops) at compile time.

- **Modifications**: Tailored for heterogeneous polyglot metamodels; emits structured diagnostics with source ranges and rule conflict identifiers directly to the LSP and CLI.

---

3. Worst-Case Optimal Joins (WCOJ / Leapfrog Triejoin)

Academic Citations

- **Ngo, H. Q., Porat, E., Ré, C., & Rudra, A. (2012)**. "*Worst-case optimal join algorithms.*"
  ACM PODS '12, pp. 37–48. DOI: 10.1145/2213556.2213565
- **Veldhuizen, T. L. (2014)**. "*Leapfrog Triejoin: A simple, worst-case optimal join algorithm.*"
  ICDT '14, pp. 96–106. DOI: 10.4230/LIPIcs.ICDT.2014.96

- **Atserias, A., Grohe, M., & Marx, D. (2008)**. "*Size bounds and query plans for relational joins.*"
  IEEE FOCS '08, pp. 739–748. (The AGM Bound)

ModelScript Rationale & Modifications

Traditional binary relational joins suffer from exponential intermediate result explosions on cyclic subgraphs (e.g. triangle topologies between ports, connectors, and component definitions). WCOJ guarantees runtime strictly bounded by the Atserias-Grohe-Marx (AGM) bound, achieving $\mathrm{O}(N^{3/2})$ for triangle queries rather than $\mathrm{O}(N^2)$.

- **Modifications**: Formulates Leapfrog Triejoin iteration orders using descending vertex degree heuristics; emits execution plans directly into the AOT WebAssembly compilation engine.

---

4. Zero-GC E-Graphs & Equality Saturation

Academic Citations

- **Nelson, G., & Oppen, D. C. (1980)**. "*Fast decision procedures based on congruence closure.*"
  Journal of the ACM, 27(2), pp. 356–364. DOI: 10.1145/322186.322198
- **Tate, R., Stepp, M., Tatlock, Z., & Lerner, S. (2009)**. "*Equality saturation: A new approach to optimization.*"
  ACM POPL '09, pp. 264–276. DOI: 10.1145/1480881.1480915
- **Willsey, M., et al. (2021)**. "*egg: Fast and extensible equality saturation.*"
  ACM POPL '21, pp. 1–29. DOI: 10.1145/3434304

ModelScript Rationale & Modifications

Avoids destructive phase-ordering in symbolic equation simplification. An E-Graph maintains exponential equivalence classes of mathematically equal expressions simultaneously, extracting the globally minimal expression before numerical simulation.

- **Modifications**: WebAssembly linear memory allocation with zero GC; 32-bit Union-Find with two-pass path compression; fixed power-of-two open-addressing hash table (`HASH_MASK = 65535`) for $\mathrm{O}(1)$ deduplication.

---

5. Graph-Structured Stack (GSS) GLR Parsing

Academic Citations

- **Tomita, M. (1985)**. *Efficient Parsing for Natural Language: A Fast Generalized LR Algorithm*.
  Kluwer Academic Publishers. DOI: 10.1007/978-1-4613-2621-2
- **Scott, E., & Johnstone, A. (2006)**. "*Right Numerate Generalized LR Parsers.*"
  ACM TOPLAS, 28(4), pp. 577–618. DOI: 10.1145/1176894.1176896
- **McPeak, S., & Necula, G. C. (2004)**. "*Elkhound: A fast, practical GLR parser generator.*"
  Compiler Construction (CC 2004), LNCS 2985, pp. 73–88. DOI: 10.1007/978-3-540-24723-4_6

ModelScript Rationale & Modifications

Handles ambiguous and non-deterministic engineering grammars (such as expression vs. component declaration ambiguities in Modelica and SysML v2) by splitting and merging parse paths via the GSS without backtracking.

- **Modifications**: Active stack heads reside in contiguous unmanaged buffers (`t_activeHeads`) with power-of-two hash probing for $\mathrm{O}(1)$ stack head merging in WebAssembly.

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Raw source code parsed by the `WASM GLR Parser`.
  - Polyglot synchronization rules authored in `@modelscript/dsl`.
- **Downstream Consumers**:
  - Rewritten expressions feed directly into `DAEBuilder`.
  - TGG synchronizations feed bidirectional updates between SysML v2 and Modelica.

CHAPTER 60

Linear Memory Structures & Indexing

Implementation:

- Roaring Bitmaps: `RoaringBitmap` in `packages/runtime/src/wasm/core/roaring.ts`
- Front-Coded IRI Dictionary: `FrontCodedDictionary` in `packages/runtime/src/wasm/core/trie.ts`
- Slotted Page B+ Tree: `PagedBTree` in `packages/runtime/src/wasm/storage/paged_btree.ts`

Academic Citations

Compressed Roaring Bitmaps

- **Chambi, S., Lemire, D., Kaser, O., & Godin, R. (2016)**. "*Better bitmap performance with Roaring bitmaps.*"
  Software: Practice and Experience, 46(5), pp. 709–719.
  DOI: 10.1002/spe.2325
- **Lemire, D., Ssi-Yan-Kai, G., & Kaser, O. (2018)**. "*Consistently faster and smaller compressed bitmaps with Roaring.*"
  Software: Practice and Experience, 48(9), pp. 1615–1638.
  DOI: 10.1002/spe.2560

Front-Coded String & IRI Compression

- **Witten, I. H., Moffat, A., & Bell, T. C. (1999)**. *Managing Gigabytes: Compressing and Indexing Documents and Images* (2nd ed.). Morgan Kaufmann.
  ISBN: 1-55860-570-3
- **Martínez-Prieto, M. A., Fernández, J. D., & Cánovas, R. (2012)**. "*Compression of RDF Dictionaries.*"
  ACM Transactions on the Web (TWEB), 6(4), pp. 1–35.
  DOI: 10.1145/2382636.2382639
- **Brisaboa, N. R., Cánovas, R., Francisco, C. C., Martínez-Prieto, M. A., & Navarro, G. (2011)**. "*K2-trees for compact Web graph representation.*"
  Information Systems, 39, pp. 152–163.
  DOI: 10.1016/j.is.2012.08.003

Slotted Page B+ Tree & SPO Triple Storage

- **Bayer, R., & McCreight, E. (1972)**. "*Organization and maintenance of large ordered indexes.*"
  Acta Informatica, 1(3), pp. 173–189.
  DOI: 10.1007/BF00288683
- **Comer, D. (1979)**. "*The ubiquitous B-tree.*"
  ACM Computing Surveys, 11(2), pp. 121–137.
  DOI: 10.1145/356770.356776
- **Mohan, C., Haderle, D., Lindsay, B., Pirahesh, H., & Schwarz, P. (1992)**. "*ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging.*"
  ACM Transactions on Database Systems, 17(1), pp. 94–162.
  DOI: 10.1145/128765.128770
- **Neumann, T., & Weikum, G. (2008)**. "*RDF-3X: A RISC-style engine for RDF.*"
  Proceedings of the VLDB Endowment, 1(1), pp. 647–659.
  DOI: 10.14778/1453856.1453927

ModelScript Architectural Rationale

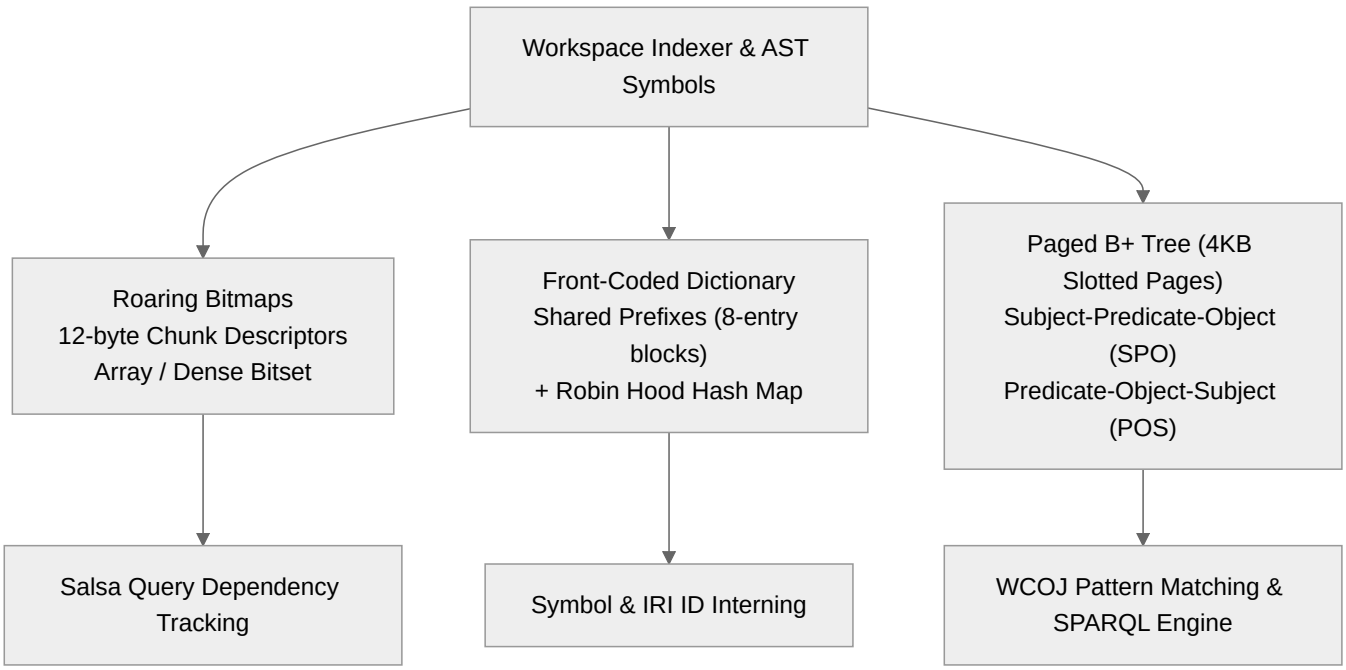
ModelScript compiles and indexes multi-million-element engineering workspaces, including the complete Modelica Standard Library (MSL), SysML v2 standard libraries (KerML), ISO 10303 STEP schemata, and OWL2 domain ontologies.

Representing these vast semantic graphs using standard JavaScript object heaps incurs:

- Extreme memory bloat (JavaScript objects typically consume 40–80 bytes of overhead per node).
- Heavy Garbage Collector (GC) pressure and unpredictable pause times during interactive IDE editing.
- Slow pointer chasing across cache lines.

To solve this, ModelScript implements **compact, zero-GC data structures** operating directly inside WebAssembly 64-bit linear memory.

Architecture of Core WASM Structures



1. Unmanaged Roaring Bitmaps

Partitions the 32-bit integer space into $2^{16} = 65{,}536$ chunks based on the upper 16 bits:

- **12-Byte Chunk Descriptor**:

```
Offset 0..3:   [key: u16] | [containerType: u16]
Offset 4..7:   [containerPtr: u32]
Offset 8..9:   [cardinality: u16]
Offset 10..11: [capacity: u16]
```

- **Adaptive Containers**:
  - **Array Container**: Used when cardinality $< 4{,}096$. Elements are stored as sorted `u16` integers in linear memory.
  - **Bitmap Container**: When cardinality reaches $4{,}096$, the container automatically converts to a dense 8,192-byte bitset ($1{,}024 \times \text{u64}$ words), allowing bitwise `AND`, `OR`, `XOR` operations at hardware SIMD speeds.

2. Front-Coded String & IRI Dictionary

Engineering identifiers and semantic URIs often share massive common prefixes:

```
http://modelscript.io/sysml2/core#PartDefinition
http://modelscript.io/sysml2/core#ItemDefinition
http://modelscript.io/sysml2/core#ActionDefinition
```

The front-coded dictionary partitions sorted strings into blocks of size $K = 8$:

- **Header String**: Stored uncompressed at the beginning of each block.
- **Suffix Entries**: Encoded as `[commonPrefixLen: u8, suffixLen: u8, suffixBytes]`.

- **Hybrid Lookups**: Combines front-coding (achieving 75–85% memory compression) with a zero-cost 64-bit Robin Hood hash map (`UnmanagedMap64`) for $O(1)$ string-to-ID interning.

3. Paged 4KB Slotted B+ Tree

Stores billions of semantic assertions and relational tuples:

- **4KB Slotted Page Layout (32-byte header + 254 entries)**:

```
[0..3]   pageId: u32
[4..5]   pageType: u16 (1 = Leaf, 2 = Internal)
[6..7]   itemCount: u16
[8..11]  prevPage: u32 (doubly-linked leaf chain for range scans)
[12..15] nextPage: u32
[16..19] parentPage: u32
[20..23] flags: u32
[24..31] lsn: u64 (Log Sequence Number for crash consistency)
```

- **RDF-3X Triples**: Triples (S, P, O) are indexed in clustered B+ tree leaf chains. Full index permutations (`SPO`, `POS`, `OSP`) permit fast range scans for multi-way graph joins without intermediate hash joins.

Upstream & Downstream Integration

Data Structure	Upstream Dependencies	Downstream Consumers
RoaringBitmap	Symbol IDs from `SymbolIndexer`	Invalidation sets in `QueryEngine`, type sets
FrontCodedDictionary	AST string literals, Qualified names, URIs	Inter-file symbol resolution, memory-mapped LSP indexing
PagedBTree	AST semantic relationships, OWL2 axioms	Leapfrog Triejoin (WCOJ), graph query evaluators, SPARQL

CHAPTER 61

Scientific Computing, Linear Algebra, Surrogates & Digital Twins

Implementation:

- Sparse Linear Algebra & Symbolic Solvers: `packages/runtime/src/wasm/solvers/`
- Reduced-Order & Dimensional Surrogates: `packages/simulate/src/surrogates/`
- Operational Digital Twin Estimation & Prognostics: `packages/simulate/src/twin/`, `packages/simulate/src/calibration/bayesian_calibrator.ts`, and `packages/simulate/src/optimizer/core/calibrator.ts`

---

1. Davis SuiteSparse Left-Looking Cholesky (LL^T)

Academic Citations

- **Davis, T. A. (2006)**. *Direct Methods for Sparse Linear Systems*. **SIAM**. DOI: 10.1137/1.9780898718881
- **Chen, Y., Davis, T. A., Hager, W. W., & Rajamanickam, S. (2008)**. "*Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate.*" **ACM Transactions on Mathematical Software**, 35(3), pp. 22:1–22:14. DOI: 10.1145/1462166.1462176

ModelScript Rationale & Modifications

Solves symmetric positive-definite sparse systems $Ax = b$ arising in finite element analysis, electrical mesh equations, and quadratic programming subproblems.

- **Modifications**: Left-looking column-by-column factorization computing non-zero patterns dynamically via elimination trees; operates inside unmanaged WebAssembly linear memory with zero heap fragmentation.

---

2. Gilbert-Peierls Left-Looking Sparse LU ($PAQ = LU$)

Academic Citations

- **Gilbert, J. R., & Peierls, T. (1988)**. "*An approach to sparse LU factorization based on graph coloring and topological sort.*" **International Conference on Supercomputing**.
- **Davis, T. A. (2006)**. *Direct Methods for Sparse Linear Systems*. **SIAM**. Chapter 5.

ModelScript Rationale & Modifications

Factorizes unsymmetric, highly irregular sparse Jacobians generated by torn physical DAE systems.

- **Modifications**: Depth-first search (DFS) topological ordering to find the exact non-zero structure of each column in L and U in time proportional to arithmetic operations; threshold partial pivoting with diagonal preference to preserve physical sparsity.

---

3. Householder QR with Column Pivoting

Academic Citations

- **Golub, G. H., & Van Loan, C. F. (2013)**. *Matrix Computations* (4th ed.). **Johns Hopkins University Press**.
- **Businger, P., & Golub, G. H. (1965)**. "*Linear least squares solutions by Householder transformations.*" **Numerische Mathematik**, 7(3), pp. 269–276. DOI: 10.1007/BF01436084

ModelScript Rationale & Modifications

Computes numerically stable orthogonal factorizations $AP = QR$ for rank-deficient least-squares calibration and parameter sensitivity studies.

---

4. Buchberger's Gröbner Basis Engine with Gebauer-Möller Criteria

Academic Citations

- **Buchberger, B. (1965)**. *An Algorithm for Finding the Basis Elements of the Residue Class Ring of a Zero Dimensional Polynomial Ideal*.
  PhD thesis, University of Innsbruck.
- **Gebauer, R., & Möller, H. M. (1988)**. "*On an installation of Buchberger's algorithm.*"
  Journal of Symbolic Computation, 6(2-3), pp. 275–286. DOI: 10.1016/S0747-7171(88)80048-8
- **Cox, D., Little, J., & O'Shea, D. (2015)**. *Ideals, Varieties, and Algorithms* (4th ed.).
  Springer. DOI: 10.1007/978-3-319-16721-3

ModelScript Rationale & Modifications

Solves small non-linear algebraic loops analytically to avoid numerical Newton iterations entirely. Computes the Gröbner basis under lexicographic monomial ordering (Lex) to triangularize polynomial systems into univariate root problems.

- **Modifications**: Implements the Gebauer-Möller criteria (B_{new} and M_{new} sets) to eliminate over 90% of redundant S-polynomial reduction checks; operates over exact rational coefficients $\mathbb{Q}$.

---

5. Proper Orthogonal Decomposition (POD) / Snapshot SVD

Academic Citations

- **Sirovich, L. (1987)**. "*Turbulence and coherence in Catherine's wheel: Method of snapshots.*"
  Quarterly of Applied Mathematics, 45(3), pp. 561–571. DOI: 10.1090/qam/910462
- **Berkooz, G., Holmes, P., & Lumley, J. L. (1993)**. "*The proper orthogonal decomposition in the analysis of turbulent flows.*"
  Annual Review of Fluid Mechanics, 25, pp. 539–575. DOI: 10.1146/annurev.fl.25.010193.002543

ModelScript Rationale & Modifications

Extracts reduced-order models (ROM) from massive 3D CFD or FEA simulation field snapshots ($N > 10^6$ nodes):

$$u(x,t) \approx \bar{u}(x) + \sum_{k=1}^{r} a_k(t)\phi_k(x)$$

- **Modifications**: Employs Sirovich's snapshot method to compute eigenvalues of the small $M \times M$ temporal correlation matrix ($M \ll N$ snapshots), retaining 99% cumulative kinetic energy in ≤ 16 spatial modes.

---

6. Buckingham π Dimensional Analysis & Similarity Engine

Academic Citations

- **Buckingham, E. (1914)**. "*On physically similar systems; illustrations of the use of dimensional equations.*"
  Physical Review, 4(4), pp. 345–376. DOI: 10.1103/PhysRev.4.345
- **Vaschy, A. (1892)**. "*Sur les lois de similitude en physique.*"
  Annales Télégraphiques, 19, pp. 25–28.
- **Barenblatt, G. I. (1996)**. *Scaling, Self-Similarity, and Intermediate Asymptotics*.
  Cambridge Texts in Applied Mathematics 14. DOI: 10.1017/CBO9781107050242

ModelScript Rationale & Modifications

Reduces parameter dimensionality in surrogate modeling by converting raw physical quantities ($q_1, \ldots, q_n$) into dimensionless similarity groups ($\Pi_1, \ldots, \Pi_{n-r}$):

- **Modifications**: Computes the null-space of the 7-dimensional SI unit exponent matrix using exact rational Gaussian elimination with coprime integer exponents; canonical fingerprint matching recognizes standard dimensionless numbers (Reynolds Re, Nusselt Nu, Mach Ma, Froude Fr).

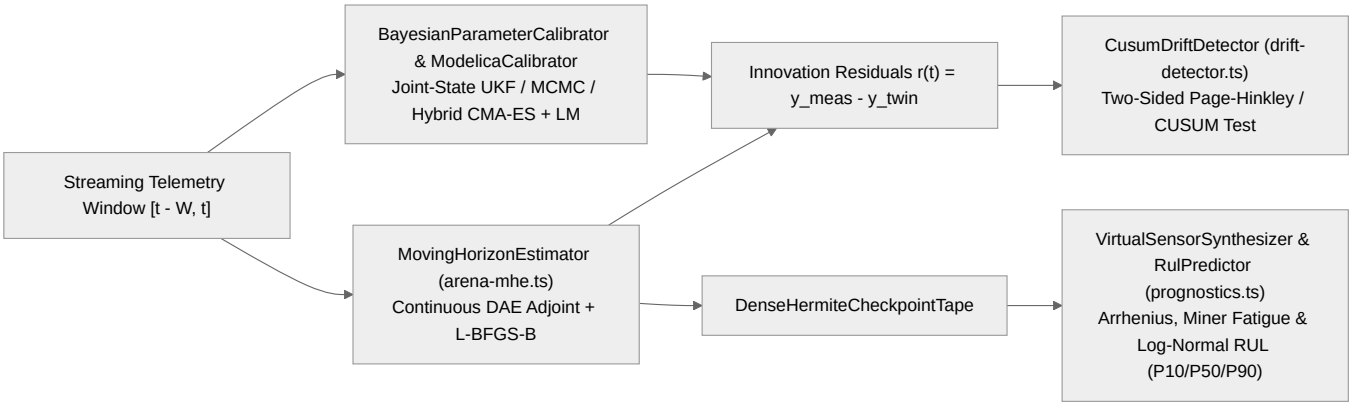
---

7. Operational Digital Twin Estimation & Prognostics (`packages/simulate/src/twin/`)

Academic Citations

- **Rao, C. V., Rawlings, J. B., & Mayne, D. Q. (2003)**. "*Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations.*"
  IEEE Transactions on Automatic Control, 48(2), pp. 246–258. DOI: 10.1109/TAC.2002.808470
- **Julier, S. J., & Uhlmann, J. K. (2004)**. "*Unscented filtering and nonlinear estimation.*"
  Proceedings of the IEEE, 92(3), pp. 401–422. DOI: 10.1109/JPROC.2003.823141
- **Page, E. S. (1954)**. "*Continuous inspection schemes.*"
  Biometrika, 41(1/2), pp. 100–115. DOI: 10.2307/2333009
- **Miner, M. A. (1945)**. "*Cumulative damage in fatigue.*"
  Journal of Applied Mechanics, 12(3), pp. A159–A164.

ModelScript Architectural Overview



1. Moving Horizon Estimation (MHE) over the Linear-Memory DAE Arena (`arena-mhe.ts`)

`MovingHorizonEstimator` reconciles streaming physical sensor windows (`TelemetryWindow` over $[t-W, t]$) against the in-memory `DAEBuilder` arena by solving a box-constrained trajectory-tracking problem:

$$\min_{\theta, w} \frac{1}{2} \sum_{k=0}^{N-1} \left\| y^{\text{sim}}(t_k; \theta, w) - y^{\text{meas}}(t_k) \right\|_2^2 + \frac{1}{2} \sum_i \lambda_i (\theta_i - \theta_i^{\text{prior}})^2 + \frac{\lambda_w}{2} \| w - w^{\text{prior}} \|_2^2$$

$$\text{s.t.} \quad \theta_{\min} \le \theta \le \theta_{\max}, \quad F(\dot{x}, x, y, \theta, w, t) = 0$$

- **Joint Physical & Neural Hybrid Calibration**: Simultaneously estimates physical DAE parameters θ and embedded `ArenaNeuralBlock` surrogate weights w inside the same optimization vector.
- **Adjoint-Driven L-BFGS-B**: Evaluates exact objective gradients via `DaeAdjointSolver` (`solveDaeAdjoint`) and converges in ≤ 25 quasi-Newton `lbfgsbSolve` iterations, automatically warm-starting initial states from window boundaries t_0.

2. Unscented Kalman Filter (UKF) & Multi-Strategy Parameter Calibration (`calibrator.ts` , `bayesian_calibrator.ts`)

ModelScript provides two complementary calibration engines for offline test-stand identification and online probabilistic tracking:

- **Joint-State Unscented Kalman Filter (`BayesianParameterCalibrator`)**: Augments the continuous state vector $x \in \mathrm{R}^{N_x}$ with unknown physical parameters $\theta \in \mathrm{R}^{N_p}$ into $z = [x^T, \theta^T]^T \in \mathrm{R}^{N_x+N_p}$. Propagates $2L+1$ deterministic sigma points ($\alpha = 10^{-3}, \beta = 2, \kappa = 0$) through the nonlinear DAE transition $z_{k+1} = f_{\text{aug}}(z_k, u_k, \Delta t)$ and measurement model $y_k = h_{\text{aug}}(z_k)$

(`UnscentedKalmanFilter`), extracting full posterior covariance $P_{k|k}$, 95% confidence intervals ($\hat{\theta}_i \pm 1.96\sigma_i$), and automated Digital Thread hypergraph write-back suggestions.

- **Deterministic & Global Metaheuristic Calibration (`ModelicaCalibrator` in `calibrator.ts`)**: Supports Levenberg-Marquardt (`lm`) with damped normal equations $(J^T J + \lambda I)\Delta\theta = -J^T r$, Sequential Quadratic Programming (`sqp`) with BFGS Hessian updates, scalable adjoint `lbfgsb` , Bayesian MCMC / global evolutionary search (`cmaes` , `de` , `pso`), and two-stage `hybrid-cmaes-lm` global-to-local polishing.

3. Page-Hinkley / CUSUM Statistical Residual Drift Detection (`drift-detector.ts`)

`CusumDriftDetector` continuously monitors multi-channel innovation residuals $r_c(t) = y_{\text{meas},c}(t) - y_{\text{twin},c}(t)$ to detect incipient actuator degradation, sensor bias, or structural drift while rejecting zero-mean Gaussian measurement noise:

- **Two-Sided Sequential Cumulative Sum**: Normalizes each channel residual $\tilde{z}_c = (r_c - \mu_c)/\sigma_c$ with minimum detectable shift slack $k_c = \frac{\delta_c}{2\sigma_c}$ (default $\delta_c = 1.5\sigma_c$) and updates positive and negative CUSUM statistics:

$$S_c^+(t) = \max(0,\ S_c^+(t-1) + \tilde{z}_c(t) - k_c)\,, \qquad S_c^-(t) = \max(0,\ S_c^-(t-1) - \tilde{z}_c(t) - k_c)$$

- **Adaptive Burn-In & Severity Classification**: Supports empirical baseline calibration (`calibrateFromData`) and triggers structured `DriftEvent` alerts when $\max(S_c^+, S_c^-) \geq h_c$, classifying severity (`low` , `medium` , `high` , `critical`) by threshold ratio S_c/h_c alongside online shift magnitude estimation.

4. Virtual Sensor Synthesis & Remaining Useful Life (RUL) Prognostics (`prognostics.ts`)

- **Virtual Sensor Synthesis (`VirtualSensorSynthesizer`)**: Evaluates unobservable internal physical states (e.g., rotor hotspot temperatures, internal valve spool stresses) at arbitrary timestamps t directly from the continuous `DenseHermiteCheckpointTape` .
- **Physics-of-Failure Damage Accumulation (`RulPredictor`)**: Computes instantaneous degradation rates $\frac{dD}{dt}$ under three selectable damage laws:
  1. **Arrhenius Thermal Aging**: $\frac{dD}{dt} = A \exp(-\frac{E_a}{k_B T})$
  2. **Miner's Power-Law / Basquin Fatigue**: $\frac{dD}{dt} = k_{\text{fat}}\, \sigma^m$
  3. **Archard Linear Mechanical Wear**: $\frac{dD}{dt} = k_{\text{wear}}\, \sigma$
- **Probabilistic RUL Quantiles**: Given current damage $D(t) \in [0, D_{\text{crit}}]$ and operational duty-cycle load variability σ_{load}, projects log-normal survival quantiles:

$$\text{RUL}_{P50} = \frac{D_{\text{crit}} - D(t)}{\dot{D}(\sigma_{\text{duty}})}, \qquad \text{RUL}_{P10} = \text{RUL}_{P50} \exp(-1.28155\,\sigma_{\text{load}}), \qquad \text{RUL}_{P90} = \text{RUL}_{P50} \exp(+1.28155\,\sigma_{\text{load}})$$

Upstream & Downstream Pipeline Connections

- **Upstream Inputs**:
  - Linear matrix systems from `TornBlock` and `TR-BDF2` .
  - 3D CFD and FEA continuum field snapshots from `CFD` and `FEA` .
  - Live telemetry streams (CSV, ASAM MDF4, MCAP) and `DAEBuilder` adjoint sensitivities.
- **Downstream Consumers**:
  - Embedded into real-time Modelica plant loops via `msx surrogate` and operational digital twins.
  - Feeds calibrated posterior parameters and RUL health metrics back into the `DigitalThreadHypergraph` .
  - Accelerates linear solves across all numerical integrators and optimal control routines.

CHAPTER 62

Semidefinite Programming, BVP, Continuum Physics & CAD/ECAD

Implementation:

- Primal-Dual Interior-Point SDP: `SdpSolver` in `packages/runtime/src/solvers/wasm_sdp_solver.ts`
- Multiple Shooting & Collocation BVP: `solveBvpShooting` in `packages/simulate/src/solvers/bvp-solver.ts`
- Constructive Solid Geometry & CFD Patches: `evaluateScad` in `languages/scad/src/evaluator.ts`
- 3D Continuum Physics & B-Rep Meshing: `WebGPULbmRunner` (`packages/simulate/src/cfd/webgpu-lbm-runner.ts`), `FeaSolver` (`packages/simulate/src/fea/fea-solver.ts`), and `StepMesher` (`packages/simulate/src/meshing/step-mesher.ts`)
- Mechanical Swept-Volume, GD&T Stack-Up & Inverse CAD: `DynamicClearanceVerifier` (`packages/cad/src/dynamic-clearance.ts`), `ToleranceStackVerifier` (`packages/cad/src/tolerance-stack.ts`), and `ParameterInversionEngine` (`packages/cad/src/parameter-inversion.ts`)
- ECAD / PCB Geometry, Autorouting & DRC: `generateManufacturingFiles` (`packages/cad/src/ecad/gerber.ts`), `PCBRouter` (`packages/cad/src/ecad/engine/router.ts`), and `runDRC` (`packages/cad/src/ecad/engine/drc.ts`)

Academic Citations

Semidefinite Programming & Sum-of-Squares (SOS)

- **Vandenberghe, L., & Boyd, S. (1996)**. "*Semidefinite Programming.*"
  SIAM Review, 38(1), pp. 49–95.
  DOI: 10.1137/1038003
- **Helmberg, C., Rendl, F., Vanderbei, R. J., & Wolkowicz, H. (1996)**. "*An interior-point method for semidefinite programming.*"
  SIAM Journal on Optimization, 6(2), pp. 342–361.
  DOI: 10.1137/0806018
- **Alizadeh, F., Haeberly, J. P. A., & Overton, M. L. (1998)**. "*Primal-dual interior-point methods for semidefinite programming: convergence rates, stability and numerical results.*"
  SIAM Journal on Optimization, 8(3), pp. 746–768.
  DOI: 10.1137/S1052623495287701X
- **Parrilo, P. A. (2003)**. "*Semidefinite programming relaxations for semialgebraic problems.*"
  Mathematical Programming, 96(2), pp. 293–320.
  DOI: 10.1007/s10107-003-0387-5 *(SOS Programming)*

Boundary Value Problems (BVP) & Multiple Shooting

- **Morrison, D. D., Riley, J. D., & Zancanaro, J. F. (1962)**. "*Multiple shooting method for two-point boundary value problems.*"
  Communications of the ACM, 5(12), pp. 613–614.
  DOI: 10.1145/368996.369028
- **Stoer, J., & Bulirsch, R. (2002)**. *Introduction to Numerical Analysis* (3rd ed.). Texts in Applied Mathematics 12, Springer.
  Section 7.3: Multiple Shooting Methods.
  ISBN: 978-0-387-95452-3
- **Ascher, U. M., Mattheij, R. M., & Russell, R. D. (1995)**. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations.*
  SIAM Classics in Applied Mathematics 13.
  DOI: 10.1137/1.9781611971231
- **Betts, J. T. (2010)**. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming* (2nd ed.).
  SIAM Advances in Design and Control 19.
  DOI: 10.1137/1.9780898718577

Constructive Solid Geometry (CSG), Continuum Physics & CAD/ECAD

- **Requicha, A. A. (1980)**. "*Representations for rigid solids: Theory, methods, and systems.*"
  ACM Computing Surveys, 12(4), pp. 437–464.
  DOI: 10.1145/356827.356833
- **Qian, Y. H., d'Humières, D., & Lallemand, P. (1992)**. "*Lattice BGK models for Navier-Stokes equation.*"
  Europhysics Letters, 17(6), pp. 479–484.
  DOI: 10.1209/0295-5075/17/6/001
- **Bouzidi, M., Firdaouss, M., & Lallemand, P. (2001)**. "*Momentum transfer of a Boltzmann-lattice fluid with boundaries.*"
  Physics of Fluids, 13(11), pp. 3452–3459.
  DOI: 10.1063/1.1399290
- **Müller, M., & Gross, M. (2004)**. "*Interactive virtual materials.*"
  Proceedings of Graphics Interface 2004, pp. 239–246. *(Corotational Finite Elements)*

ModelScript Architectural Rationale

1. **Semidefinite Programming (SDP) for Verified Safety**: Formal continuous verification of non-linear physical models requires proving that state trajectories $\mathbf{x}(t)$ never breach unsafe operating envelopes. Synthesizing polynomial Lyapunov functions $V(\mathbf{x}) > 0$ and barrier certificates $B(\mathbf{x}) \leq 0$ reduces to finding Sum-of-Squares (SOS) decompositions:

$$p(\mathbf{x}) = \mathbf{z}(\mathbf{x})^T Q\, \mathbf{z}(\mathbf{x}) \geq 0 \iff Q \succeq 0$$

   By implementing a native primal-dual interior-point SDP solver in WebAssembly, ModelScript provides mathematically verified stability proofs directly in the browser and CLI without relying on external monolithic commercial solvers.
2. **Multiple Shooting BVP for Optimal Trajectories**: Boundary Value Problems (BVPs) govern optimal trajectory planning, model predictive control (MPC), and periodic limit cycle detection with split boundary conditions:

$$g(\mathbf{y}(t_0), \mathbf{y}(t_{\text{end}}), \mathbf{p}) = \mathbf{0}$$

   Standard single shooting suffers from catastrophic numerical instability on stiff or chaotic dynamics. Multiple shooting subdivides the time horizon into M subintervals, guaranteeing numerical stability and fast Newton-Raphson convergence.
3. **Constructive Solid Geometry (CSG) for Multi-Physics 3D Packaging**: Lumped-parameter 1D physical models (Modelica) rely on 3D geometric properties (mass, inertia tensor, aerodynamic drag area, hydraulic volume). ModelScript evaluates OpenSCAD models into normalized CSG solid trees (`@modelscript/cad`), auto-tagging boundary patches (`inlet` , `outlet` , `wall` , `symmetry`) for downstream OpenCascade B-Rep evaluation, STEP export, and SU2/OpenFOAM CFD meshing.

Mathematical Formulation & Execution Architecture



1. Primal-Dual Interior Point SDP Solver

Solves standard primal and dual semidefinite programs:

$$\text{Primal: } \min_{X \in \mathbb{S}^n} \operatorname{tr}(CX) \quad \text{s.t.} \quad \operatorname{tr}(A_i X) = b_i,\ X \succeq 0$$

$$\text{Dual: } \max_{y \in \mathbb{R}^m, S \in \mathbb{S}^n} b^T y \quad \text{s.t.} \quad \sum_{i=1}^{m} y_i A_i + S = C,\ S \succeq 0$$

- **Central Path Following**: Enforces the perturbed complementarity condition $XS = \mu I$ with barrier parameter $\mu = \frac{\operatorname{tr}(XS)}{n}$.

- **WASM Linear Algebra**: Predictor-corrector search directions $(\Delta X, \Delta y, \Delta S)$ are computed using fast in-memory Cholesky factorizations (`wasm_gaussian.ts`), with backtracking line search to ensure $X + \alpha\Delta X \succ 0$ and $S + \alpha\Delta S \succ 0$.

2. Multiple Shooting BVP Solver

Subdivides the horizon $[t_0, t_{\text{end}}]$ into M subintervals with mesh points $t_0 < t_1 < \cdots < t_M = t_{\text{end}}$:

- **State Trajectory Variables**: Introduces free initial conditions $\mathbf{s}_m$ at each node t_m.
- **Subinterval Integration**: Integrates forward from t_m to t_{m+1} using adaptive `tsit5` :

$$\mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p})$$

- **Matching Residual System**: Solves the $(M \cdot n + k) \times (M \cdot n + k)$ simultaneous nonlinear system:

$$F_m(\mathbf{s}_m, \mathbf{s}_{m+1}) = \mathbf{y}(t_{m+1}; \mathbf{s}_m, \mathbf{p}) - \mathbf{s}_{m+1} = \mathbf{0}, \quad m = 0, \ldots, M-1$$

$$g(\mathbf{s}_0, \mathbf{s}_M, \mathbf{p}) = \mathbf{0}$$

- **Newton-Raphson Update**: Evaluates sensitivity Jacobians along trajectories and solves the block-banded linear system via WASM Gaussian LU elimination.

3. OpenSCAD CSG Engine & CFD Boundary Tagging

Transforms procedural CSG trees into boundary-tagged solid geometries:

- Evaluates transformations (`translate` , `rotate` , `scale` , `mirror`) and regularized Boolean operators (`union` , `difference` , `intersection`).
- Decorates faces with CFD boundary tags (`tagPatch` , `BoundaryPatchType` : `inlet` , `outlet` , `wall` , `symmetry`), for handoff to automated SU2 mesh generators.

4. 3D Continuum Physics & B-Rep Meshing (`packages/simulate`)

ModelScript embeds native 3D continuum fluid, structural, and volumetric meshing engines that couple directly to 1D DAE simulations:

1. **WebGPU D3Q19 Lattice Boltzmann Method (LBM) CFD Solver (`webgpu-lbm-runner.ts`)**:
   - Discretizes the Boltzmann transport equation over a 19-velocity 3D lattice (`D3Q19`) stored in a zero-copy Structure-of-Arrays arena (`CfdFieldArena`) with ping-pong distribution buffers (`f0` , `f1`) and WGSL compute shader execution (`LBM_D3Q19_WGSL` , workgroups $8 \times 8 \times 4$):

$$f_i^{\text{post}}(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \omega_{\text{loc}}(f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\rho, \mathbf{u})), \qquad f_i^{\text{eq}} = w_i\rho(1 + 3(\mathbf{c}_i \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_i \cdot \mathbf{u})^2 - \frac{3}{2}\|\mathbf{u}\|^2)$$

   - **Smagorinsky Subgrid LES Turbulence**: Dynamically adapts the local relaxation frequency $\omega_{\text{loc}} = (\tau_0 + \Delta\tau_{\text{LES}})^{-1}$ from the non-equilibrium momentum flux tensor $\Pi^{\text{neq}}_{\alpha\beta} = \sum_i c_{i\alpha}c_{i\beta}(f_i - f_i^{\text{eq}})$.
   - **Bouzidi Curved Boundaries & Moving-Wall FSI**: Applies sub-cell linear interpolation for curved solid obstacles ($\delta_{\text{wall}} < 0.5$ vs. $\delta_{\text{wall}} \geq 0.5$), injects moving-wall momentum $\Delta f_i = 6w_i\rho(\mathbf{c}_i \cdot \mathbf{u}_{\text{wall}})$ for two-way aeroelastic FSI, and integrates surface drag forces via momentum exchange.

2. **3D Tet4/Tet10 Linear/Nonlinear Corotational Elasticity & Thermal FEA Solver (`fea-solver.ts`)**:
   - Assembles global sparse stiffness K and consistent mass M matrices in Compressed Sparse Row (CSR) format for 4-node linear (`Tet4Element`) and 10-node quadratic (`Tet10Element`) tetrahedra.
   - **Corotational Large-Rotation Kinematics**: Extracts element rigid-body rotation $R_e \in \text{SO}(3)$ from the deformation gradient $F = e_0 E_0^{-1}$ via 6-step Higham polar decomposition iterations $R_{k+1} = \frac{1}{2}(R_k + R_k^{-T})$, rotating local element stiffness blocks $K_{\text{rot}} = R_e K_e R_e^T$ for geometric nonlinearity invariance.
   - **Jacobi-Preconditioned Conjugate Gradient (PCG) & Newmark-β Dynamics**: Solves static equilibrium $Ku = f$ and Rayleigh-damped transient elastodynamics $M\ddot{u} + (\alpha_M M + \beta_K K)\dot{u} + Ku = f$ using unconditional-stability Newmark-β ($\beta = 0.25, \gamma = 0.5$) time stepping, outputting nodal/element von Mises stresses and yield safety factors.

3. **STEP B-Rep Tetrahedral Mesher & Prismatic Boundary-Layer Extrusion (`step-mesher.ts`)**:

- `StepMesher` converts ISO 10303-21 STEP B-Rep entities and watertight surface triangulations into volumetric `Tet4` / `Tet10` meshes (`Tet4Mesher`), classifies exterior boundary patches (`PatchClassifier`), extrudes viscous prismatic inflation layers along `WALL_BODY` surfaces (`BoundaryLayerExtruder`), evaluates Jacobian/aspect-ratio quality (`MeshQualityAnalyzer`), and exports ready-to-run CalculiX (`*.inp`) and SU2 (`*.su2`) decks.

5. Mechanical Swept-Volume, GD&T Stack-Up & Inverse CAD (`packages/cad`)

1. **Dynamic Swept-Volume Collision & Clearance Envelopes (`dynamic-clearance.ts`)**:
   - `DynamicClearanceVerifier` bridges continuous reachability flowpipes (`TrajectoryStepEnclosure[]`) into 3D CAD assemblies (`Assembly`). At each time step, it propagates interval state enclosures $[x_i^{\text{lo}}, x_i^{\text{hi}}]$ through translational Minkowski sums, rotational corner-envelope expansions, and soft-tissue/compliant cavity morphing (`volume_radial` $s = (V/V_0)^{1/3}$, `anisotropic` , and `modal` skinning).
   - Evaluates exact pairwise AABB separation distances (`computeAABBDistance`) and closest witness points (`computeAABBClosestPoints`), automatically synthesizing a 3D counterexample CAD assembly at the worst-case collision timestamp t^* if any clearance rule is violated.
2. **Worst-Case, Statistical RSS & Coupled Thermal-Elastic GD&T Stack-Up (`tolerance-stack.ts`)**:
   - `ToleranceStackVerifier` verifies 1D/3D kinematic assembly gap loops across directional contributors ($d_i \in \{+1, -1\}$) combining bilateral manufacturing tolerances $[t_i^L, t_i^U]$, thermal expansion over $[T_{\min}, T_{\max}]$ ($\Delta L_{\text{th}} = L_0 \alpha_{\text{CTE}} \Delta T$), and axial elastic deflection under dynamic loads $[F_{\min}, F_{\max}]$ ($\Delta L_{\text{el}} = \frac{F L_0}{E A}$):

$$g_{\min,\,\text{WC}} = \sum_i d_i L_{0,i} + \sum_i \min(d_i t_i^L, d_i t_i^U) + \Delta g_{\text{th},\,\min} + \Delta g_{\text{el},\,\min}, \qquad \sigma_{\text{RSS}} = \sqrt{\sum_i \left(\frac{t_i^U - t_i^L}{6}\right)^2}$$

3. **Certified Bi-Directional CAD Parameter Inversion (`parameter-inversion.ts`)**:
   - `ParameterInversionEngine` closes the loop from 1D system optimization back to 3D MCAD source code. Given target mass/inertia constraints (`solveLengthForMass` , `scaleDimensionsForMass`) or optimized Modelica parameters, `patchMcadSourceCertified` verifies candidate dimensions against a multidisciplinary `SafeRegionGuard` (P_{safe} box bounds and buckling/stress/clearance predicates) before rewriting the parametric MCAD AST and regenerating STEP geometry.

6. ECAD / PCB Geometry, Autorouting & DRC (`packages/cad/src/ecad/`)

1. **RS-274X Gerber Aperture Evaluation & Excellon Drill Synthesis (`gerber.ts`)**:
   - `generateManufacturingFiles` compiles multi-layer `PCBLayout` models (`TopCopper` , `BottomCopper` , `InnerCopper1/2` , `Top/BottomSilkscreen` , `Top/BottomSolderMask` , `BoardOutline`) into IPC-compliant **RS-274X Gerber** (`%FSLAX46Y46*%` , 10^{-6} mm integer precision) with deduplicated aperture macros (`%ADD10C,...*%` , circle/rect/oval pads, 0.1 mm negative-polarity solder mask expansion `%IPNEG*%`) and **Excellon** NC drill programs (`drill.xln` , `METRIC,TZ`).
2. **Multi-Layer Interactive & Constrained PCB Trace Router (`engine/router.ts`)**:
   - `PCBRouter` (`pcb-router.ts` / `engine/router.ts`) implements a multi-layer trace routing engine supporting pad snap-targeting (`snapToPad`), 90° Manhattan (`manhattanRoute`) and 45° octilinear (`diagonal45Route` : $\text{diag} = \min(|\Delta x|, |\Delta y|)$) segment constraints, and automatic plated through-hole via insertion (`insertVia`) on layer transitions.
3. **Euclidean Trace/Pad Clearance Design Rule Checking (`engine/drc.ts`)**:
   - `runDRC` (`pcb-drc.ts` / `engine/drc.ts`) executes non-blocking geometric verification across copper layers, checking minimum trace widths, trace-to-trace, trace-to-pad, and pad-to-pad Euclidean clearances ($d(\mathbf{p}_1, \mathbf{p}_2) - \frac{w_1+w_2}{2} \geq c_{\min}$), via drill and annular ring minimums ($r_{\text{outer}} - r_{\text{drill}} \geq a_{\min}$), point-to-segment board-outline polygon distances (`minDistToPolygon`), and unconnected ratsnest net completeness.

Upstream & Downstream Integration

Component	Upstream Dependencies	Downstream Consumers
SdpSolver	Polynomial constraints from `TheoryCoordinator`	Formal Lyapunov barrier certificates, safety proofs
BvpSolver	ODE dynamics from `DaeBuilder`, split boundary specs	Model predictive control (MPC), trajectory optimization
ScadEvaluator & StepMesher	Declarative `.scad` & `.step` B-Rep models	`FeaSolver`, `WebGPULbmRunner`, CalculiX `.inp`, SU2 `.su2`
WebGPULbmRunner & FeaSolver	`StepMesher` Tet4/Tet10 meshes, FSI boundary loads	Aeroelastic FSI co-simulation, POD reduced-order models
DynamicClearance & ToleranceStack	Reachability flowpipes, GD&T specs, CAD `Assembly`	`UnifiedVerifier` (`msx verify`), counterexample STEP export
ParameterInversionEngine	Optimized 1D Modelica parameters, `SafeRegionGuard`	Automated MCAD source patching and AOT STEP regeneration
ECAD Gerber / PCBRouter / DRC	Electrical netlists, component footprints	RS-274X `.gbr` & Excellon `.xln` manufacturing outputs, WebIDE

CHAPTER 63

Diagram Auto-Layout & Orthogonal Routing

Implementation:

- Topology-Shape-Metrics Engine: `TsmLayoutEngine` in `packages/diagram/src/tsm-layout.ts`
- Port Constraint & Alignment Solver: `PortIlpSolver` in `packages/diagram/src/port-ilp-solver.ts`
- Obstacle-Avoiding Channel Router: `PortRouter` in `packages/diagram/src/port-router.ts`
- Specialized Engineering Diagram Layouts: `synthesizeFaultTreeDiagram` (`packages/diagram/src/fault-tree.ts`), `computeSequenceLayout` (`packages/diagram/src/sequence-layout.ts`), `applySwimlaneLayout` (`packages/diagram/src/swimlane-layout.ts`), and `buildPscadDiagram` (`packages/diagram/src/pscad-diagram-builder.ts`)
- Semantic Visual Diffing & Collaborative Presence: `buildVisualDiffGraph` (`packages/diagram/src/visual-diff-graph.ts`) and `CollabManager` (`packages/diagram/src/collab-manager.ts`)

Academic Citations

Topology-Shape-Metrics (TSM / Kandinsky) Orthogonal Layout

- **Tamassia, R. (1987)**. "*On embedding a grid graph with minimum number of bends.*"
  SIAM Journal on Computing, 16(3), pp. 421–444.
  DOI: 10.1137/0216030
- **Fößmeier, U., & Kaufmann, M. (1996)**. "*Drawing high-degree graphs with minimum number of bends.*"
  Graph Drawing (GD '95), Lecture Notes in Computer Science, 1027, pp. 211–225. Springer.
  DOI: 10.1007/BFb0021804
- **Di Battista, G., Eades, P., Tamassia, R., & Tollis, I. G. (1998)**. *Graph Drawing: Algorithms for the Visualization of Graphs.*
  Prentice Hall.
  ISBN: 0-13-301615-3

Port Constraint & Continuous Alignment ILP

- **Spönemann, M., von Hanxleden, R., & Fuhrmann, H. (2014)**. "*Port constraints in layout algorithms.*"
  Graph Drawing (GD 2014), Lecture Notes in Computer Science, 8871, pp. 351–362. Springer.
  DOI: 10.1007/978-3-662-45803-7_29
- **Kieffer, S., Dwyer, T., Marriott, K., & Wybrow, M. (2016)**. "*Incremental layout for dynamic schematic diagrams.*"
  IEEE Transactions on Visualization and Computer Graphics, 22(1), pp. 876–885.
  DOI: 10.1109/TVCG.2015.2467364
- **Eiglsperger, M., Fößmeier, U., & Kaufmann, M. (2001)**. "*Orthogonal graph drawing with constraints.*"
  Journal of Graph Algorithms and Applications, 5(2), pp. 1–24.
  DOI: 10.7155/jgaa.00035

Orthogonal Channel Routing & Obstacle Avoidance

- **Lee, C. Y. (1961)**. "*An algorithm for path connection and its applications.*"
  IRE Transactions on Electronic Computers, EC-10(3), pp. 346–365.
  DOI: 10.1109/TEC.1961.5219222
- **Hart, P. E., Nilsson, N. J., & Raphael, B. (1968)**. "*A formal basis for the heuristic determination of minimum cost paths.*"
  IEEE Transactions on Systems Science and Cybernetics, 4(2), pp. 100–107.
  DOI: 10.1109/TSSC.1968.300136 *(A* Search)*
- **Deutsch, D. N. (1976)**. "*A 'dogleg' channel router.*"
  Proceedings of the 13th Design Automation Conference (DAC '76), pp. 425–433.
  DOI: 10.1145/800263.810688

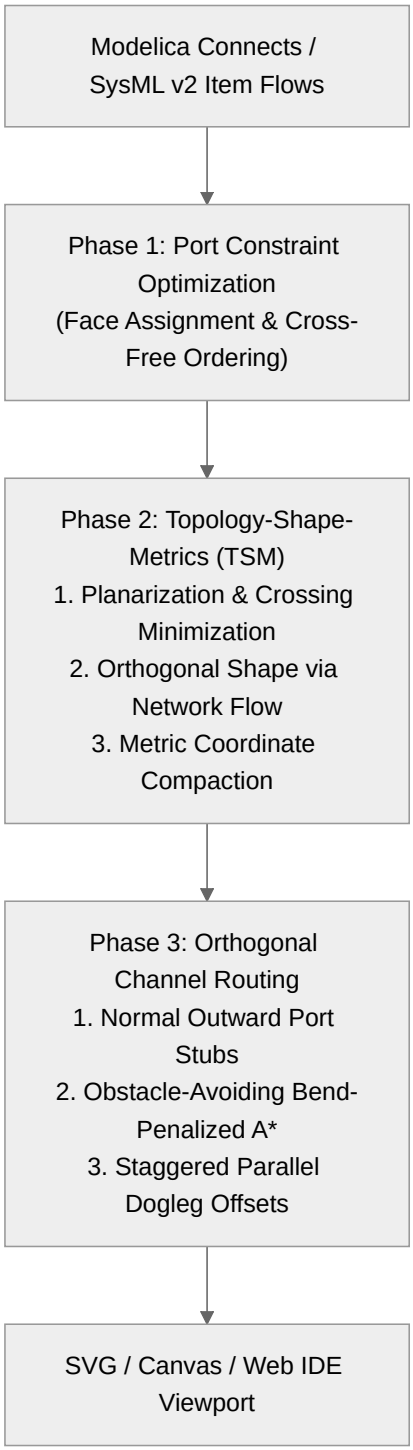
ModelScript Architectural Rationale

Physical engineering schematics (such as Modelica connection diagrams, SysML v2 Internal Block Diagrams, and hydraulic/piping schematics) require strict geometric conventions:

- **Orthogonal Edges**: Connections must run strictly along horizontal and vertical channels with clean 90° bends.
- **Port-Constraint Preservation**: High-degree engineering components have physical boundary pins (e.g. electrical resistors have pins p and n on opposite faces; hydraulic valves have ports A, B, P, T). Layout engines must place connection points on explicit component boundaries rather than abstract node centers.
- **Zero-Obstacle Penetration**: Wires must never traverse component bounding boxes.
- **Bus Disambiguation**: Multi-variable connections between blocks must not collapse onto identical collinear segments.

ModelScript implements the **Topology-Shape-Metrics (TSM)** pipeline augmented with a **Global Port ILP Solver** and **Dogleg Channel Router** natively in the browser and CLI.

Three-Phase Layout & Routing Architecture



1. Port Constraint & Continuous Alignment Solver

Resolves port assignment along component perimeters ($x, y \in \partial\text{Node}$):

- **Discrete Face Selection**: Assigns ports to Cardinal sides (`top` , `bottom` , `left` , `right`) minimizing Manhattan connection distance:

$$\min \sum_{e=(u_i,v_j)\in E} \|\mathbf{p}(u_i) - \mathbf{p}(v_j)\|_1$$

- **Causality Biasing**: Automatically places `input` ports on the left face and `output` ports on the right face for signal-flow blocks.
- **Collinear Port Snapping**: Enforces 0-bend straight lines whenever two connected ports are horizontally or vertically aligned within a configurable tolerance `straightTolerance` .

2. Topology-Shape-Metrics (TSM) Engine

Extends Tamassia's Kandinsky model for high-degree engineering vertices:

1. **Planarization**: Replaces unavoidable edge crossings with dummy vertices of degree 4.
2. **Orthogonalization (Minimum Bend Flow)**: Formulates bend minimization as a minimum-cost network flow problem over the dual graph, assigning angles $\alpha \in \{90^\circ, 180^\circ, 270^\circ, 360^\circ\}$ to faces.
3. **Compaction**: Solves two independent 1D longest path problems on horizontal and vertical constraint DAGs to compute minimal integer grid coordinates.

3. Obstacle-Avoiding Dogleg Channel Router

Renders clean multi-segment connection polylines:

- **Perpendicular Port Stubs**: Edge paths extend outward along the port normal vector by a distance `stubLength` before turning.
- **A* Grid Search with Bend Penalty**: Avoids component AABB bounding boxes using fast `segmentIntersectsRect` intersection tests while penalizing unnecessary turns:

$$\text{Cost}(p \rightarrow q) = \text{dist}(p, q) + \lambda_{\text{bend}} \cdot [\text{isTurn}(p, q)]$$

- **Staggered Parallel Channels**: Multi-wire bus connections share a common corridor but are offset by `parallelIndex * channelSpacing`, preventing overlapping lines.

4. Specialized Engineering Diagram Layouts

Beyond acausal block schematics, ModelScript provides domain-specific geometric layout engines tailored to safety, behavioral, and power-systems engineering notations:

1. **Fault Tree Gate Hierarchy Layout (`fault-tree.ts` / `fault-tree-layout.ts`)**:
   - `synthesizeFaultTreeDiagram` transforms Minimal Cut Sets ($\text{MCS} = \{C_1, \ldots, C_m\}$) into four-tier standardized Fault Tree Analysis (FTA) diagrams:
     - **Tier 1 ($y = 40$)**: Top Hazard Event rect node centered at $x_{\text{center}} = W/2$.
     - **Tier 2 ($y = 160$)**: Root Boolean `OR` gate (`createGateMarkup("OR")` SVG bezier shield) aggregating cut-set branches.
     - **Tier 3 ($y = 280$)**: Intermediate `AND` gates for multi-point failure cut sets ($\text{order} > 1$), while order-1 Single Points of Failure (SPOFs) route directly in high-visibility crimson (`#dc2626`) to basic events.
     - **Tier 4 ($y = 420$)**: Deduplicated circular Basic Event leaf nodes (`basicEventNodesMap`) distributed symmetrically across the horizontal span.
2. **UML / SysML v2 Sequence & Activity Swimlane Layouts (`sequence-layout.ts`, `swimlane-layout.ts`)**:
   - **Dual-Axis Sequence Layout (`computeSequenceLayout` / `applySequenceLayout`)**: Orders participant lifelines left-to-right ($x_i = x_{\text{left}} + i \cdot \Delta x_{\text{lifeline}} + W_{\text{hdr}}/2$), renders dashed vertical lifeline stems (`__seq_stem_*`), and stacks temporal messages top-to-bottom ($y_{k+1} = y_k + \Delta y_{\text{msg}}$) with distinct styling for synchronous (`sync`), asynchronous (`async`), dashed `reply`, and self-loop (`isSelf`, $1.5\times$ vertical spacing) messages.
   - **2D Activity Swimlane Partitioning (`applySwimlaneLayout`)**: Groups SysML v2 action nodes into vertical or horizontal allocation partitions (`SwimlanePartition`), sizing uniform lane containers (`swimlane_<id>`) to the maximum member stack height and routing cross-lane object/control flows via `port-orthogonal-astar`.
3. **PSCAD Power-System Single-Line Diagram (SLD) Busbar/Feeder Orthogonal Routing (`pscad-diagram-builder.ts` / `pscad-sld-builder.ts`)**:
   - `buildPscadDiagram` renders `.pscx` / `.pslx` electromagnetic transient models into IEEE-standard Single-Line Diagrams with voltage-coded busbar glyphs (≥ 200 kV red `#ff5252`, ≥ 69 kV amber `#ffab40`, < 69 kV cyan `#40c4ff`), specialized IEEE apparatus vector glyphs (`transformerGlyph`, `generatorGlyph`, `inverterIbrGlyph`, `breakerGlyph`, `surgeArresterGlyph`, `faultGlyph`, `converterBridgeGlyph`), solder-dot wire T-junctions, and orthogonal feeder routing.

5. Semantic Visual Diffing & Collaborative Presence

1. **AST-Aware Structural Graph Diffing & Mental-Map Stabilization (`visual-diff-graph.ts`, `visual-diff-renderer.ts` / `visual-diff.ts`)**:
   - `buildVisualDiffGraph` computes a semantic delta between `Base` and `Head` diagram revisions, classifying nodes and edges into `added`, `deleted`, `modified`, and `unchanged` states (`DiffStatus`).
   - **Breaking Interface Detection**: Inspects property and port changes (`diffProperties`) against architectural breaking keys (`direction`, `isAbstract`, `typeName`, `multiplicity`, `causality`, `variability`, `flowDirection`) and flags deleted components or

port-count mutations as `MODIFIED (BREAKING)`.

- **Anchor-Preserving Layout Stabilization (`computeIncrementalStabilization`)**: Locks `Base` node coordinates as immutable spatial anchors while iteratively relaxing newly added/modified `Head` nodes to eliminate bounding-box overlaps without destroying the reviewer's spatial mental map.

2. **CRDT / WebSocket Cursor, Selection Lock & Spatial Delta Synchronization (`collab-manager.ts`)**:
   - `CollabManager` coordinates real-time multi-user diagramming over WebSocket and VS Code Live Share transports:
     - **60 FPS Spatial Delta Streaming (`broadcastSpatialDelta`)**: Streams live component drag transformations (`PlacementItem`) with Last-Write-Wins (`LWW`) timestamp conflict resolution (`d.timestamp >= existing.timestamp`).
     - **Component Selection Locks (`SelectionLock`)**: Broadcasts per-peer component locks with distinct peer colors and automatic stale-peer heartbeat eviction (`purgeStalePeers`).
     - **Anchored Visual Review Comments (`DiagramComment`)**: Synchronizes spatially pinned sticky-note threads (`addComment`, `resolveComment`) directly on the schematic canvas.

Upstream & Downstream Integration

Pipeline Component	Upstream Dependencies	Downstream Consumers
PortIlpSolver	`SymbolIndex` connector ports, `Annotation` constraints	Port placement in `TsmLayoutEngine`
TsmLayoutEngine	Polyglot connect graphs (Modelica, SysML v2)	Node bounding box placement, routing channels
PortRouter	Component layout coordinates, Port locations	SVG polyline rendering, interactive 3D/2D IDE canvas
FaultTree / Sequence / Swimlane	Safety MCS sets, SysML v2 interactions & activities	X6 WebIDE canvas, headless SVG & PDF documentation export
PscadDiagramBuilder	PSCAD `.pscx` / `.pslx` AST & bus/feeder netlists	Power-system SLD viewer, EMT fault & breaker state overlays
VisualDiffGraph & CollabManager	Git Base/Head ASTs, WebSocket / Live Share peer streams	PR visual diff bundles (`html-diff-bundle.ts`), multi-user IDE

PART V

Command-Line Interface (msx)

CHAPTER 64

CLI Overview

The `@modelscript/cli` package provides two identical binary executables: `modelscript` and its shorthand alias `msx`. The command-line interface serves as the primary headless automation driver for compiling, simulating, verifying, linting, rendering, and orchestrating polyglot engineering systems across local workspaces, CI/CD pipelines, and cloud HPC clusters.

Installation

You can run the CLI directly from the monorepo workspace using `npx`, or install it globally for system-wide access:

```
# Run directly via npx
npx msx --help

# Install globally
npm install -g @modelscript/cli
msx --version
```

Global Options & Environment Variables

The CLI parser (`apps/cli/src/main.ts`) intercepts global execution flags prior to command dispatch and respects standard environment variables for headless CI/CD and enterprise hub integration:

Global Flags

Flag	Type	Description
`--daemon`	`boolean`	Route command execution through the persistent background compiler daemon (`~/.modelscript/daemon.sock`) to bypass cold-start WASM initialization and reuse warm Salsa caches.
`--help`	`boolean`	Display usage information and command-specific options.
`--version`	`boolean`	Print the installed `@modelscript/cli` semantic version.

Environment Variables

Variable	Default	Description
`NO_COLOR`	Unset	When set to any non-empty value, disables ANSI color sequences in terminal diagnostics, tables, and diff output.
`MODELSCRIPT_API_URL`	`http://localhost:3000`	Base URL for the ModelScript Hub, Package Registry, and Cloud HPC dispatch API (`~/.modelscriptrc` fallback).
`MODELSCRIPT_TOKEN`	Unset	Bearer token (`JWT` or `ms_bot_*` service token) used for non-interactive authentication against the Registry, Cloud HPC, and Hub APIs (also accepts `MODELSCRIPT_API_TOKEN`).

Persistent Compiler Daemon (`--daemon`)

Standard CLI invocations spin up an isolated Node.js process, compile or load `.wasm` parser binaries, and index workspace dependencies from disk. When iterating rapidly or running hundreds of checks in a pre-commit hook, pass `--daemon` to any command:

```
# Start or auto-spawn the background daemon and run lint with warm Salsa caches
msx lint model.mo --daemon

# Explicitly manage the daemon lifecycle
msx daemon status
msx daemon restart
msx daemon stop
```

When `--daemon` is passed, `msx` connects to the local Unix domain socket (`~/.modelscript/daemon.sock` on Linux/macOS or named pipe `\\.\pipe\modelscript-daemon` on Windows). If no daemon is running, the CLI automatically spawns one in the background, waits for readiness, and streams the command's `stdout`, `stderr`, and exit code back to the terminal.

Complete Command Reference by Category

The `msx` CLI registers **41 commands** (plus ergonomic aliases) organized into six functional domains. See CLI Commands for exhaustive flag tables and examples for every command.

1. Compilation, Simulation & Optimization

Command	Aliases	Summary
`compile`	`flatten`	Flatten a Modelica class hierarchy into a canonical Differential-Algebraic Equation (DAE) system.
`instantiate`	—	Instantiate a Modelica class and print its resolved component and equation structure.
`simulate`	—	Execute numerical DAE/ODE time-domain or steady-state simulations (JS, Arena, WASM, C, JIT, WebGPU, or Cloud HPC).
`optimize`	—	Solve nonlinear Dynamic Optimization and Optimal Control Problems (OCP) via direct collocation.
`grad`	—	Compute forward/adjoint parameter sensitivities ($\partial J/\partial p$) and finite-difference gradients.
`surrogate`	—	Run Design of Experiments (DoE), train Neural/Polynomial/RBF surrogates, and export ONNX/C/Modelica ROMs.
`mc`	—	Execute multi-threaded Monte Carlo uncertainty quantification and reliability sweeps.
`csg`	—	Extract Constructive Solid Geometry (CSG) primitives from Modelica models into 3D CAD meshes.

2. Linting, Formatting, Diffing & Localization

Command	Aliases	Summary
`lint`	—	Run static syntax, type, structural, and control-flow diagnostics across polyglot workspace files.
`parse`	—	Parse source files with the native WebAssembly GLR parser and inspect the concrete syntax tree (CST).
`format`	—	Format source files canonically using CST-preserving pretty-printing.
`unparse`	—	Round-trip a parsed CST back into source text to verify lossless grammar unparsing.
`diff`	—	Compute structural AST/CST and semantic DAE diffs between two files or Git revisions.
`pr-diff`	—	Generate visual SVG/HTML architectural diffs and Markdown PR review comments against a base Git ref.
`i18n`	—	Extract translatable strings, compile locale catalogs, and translate model identifiers across languages.

3. Formal Verification, Safety & Test Synthesis

Command	Aliases	Summary
`verify`	—	Verify decision tables, assume-guarantee contracts, and state machines; emit SARIF/JUnit/HTML reports.
`falsify`	—	Search for Signal Temporal Logic (STL) counterexamples by minimizing robustness $\rho(\phi, \mathbf{x}, t)$.
`export-formal`	—	Export SysML v2 models to formal verification formats (`smt2` , `nuxmv` , `ocra`).
`verify-decisions`	—	Verify architectural decision records (ADRs) and parameter trade-space bounds against formal system models.
`decompose`	—	Perform assume-guarantee contract decomposition and structural subsystem partitioning.
`generate-tests`	—	Synthesize MC/DC-complete unit test vectors and stimulus harnesses from formal model constraints.

4. Polyglot Digital Thread, Co-Simulation & Standards Interop

Command	Aliases	Summary
`render`	—	Render headless SVG diagrams for Modelica icons/schematics, SysML v2 (BDD/IBD/State/Activity), and PSCAD SLDs.
`fmu`	`export-fmu`	Package models into FMI 2.0/3.0 Functional Mock-up Units (`.fmu`) or inspect/import existing FMUs.
`import-fmu`	`fmu-import`	Unpack an external `.fmu` archive and generate typed Modelica wrapper blocks and SSP bindings.
`cosim`	—	Orchestrate and inspect distributed FMI/SSP Co-Simulation sessions, participants, and adaptive step optimizers.
`pscad`	—	Lint, render SLDs, diff, simulate EMT transients, verify IEEE 2800 ride-through, and export `.pscx` / `.pswx` projects.
`polyglot`	—	Query cross-language symbol indices, verify digital thread links, and check operational digital twin drift.
`align`	—	Reconcile Triple Graph Grammar (TGG) correspondences across SysML v2, Modelica, Spice, and STEP artifacts.
`sysml`	—	Export, import, check, inspect, and bi-directionally synchronize SysML v2 `.sysml` and Modelica `.mo` models.
`oslc`	—	Export OSLC 3.0 RDF/Turtle/JSON-LD catalogs or launch a live OSLC Tracked Resource Set (TRS 3.0) server.
`reqif`	—	Import and export OMG ReqIF 1.2 (`.reqif` / `.reqifz`) requirements exchange archives.
`ddp`	—	Inspect, pack, and convert ISO/PAS 81346 Digital Data Packages (`.ddp`) and Asset Administration Shells (`.aasx`).
`dhf`	—	Generate, audit, and inspect FDA 21 CFR Part 820 / ISO 13485 Design History File (DHF) compliance dossiers.

5. Language Workbench, Daemon & IDE Tooling

Command	Aliases	Summary
`daemon`	—	Manage the persistent background compiler daemon (`start` , `stop` , `status` , `restart`).
`build`	—	Compile a grammar DSL definition into a native WebAssembly GLR parser (`parser.wasm`) and TypeScript types.
`generate`	—	Scaffold new WASM GLR parsers or standalone VS Code language extensions from a grammar module.
`language`	`lang`	List, dynamically register, or unregister custom language plugins in the workspace registry.
`lsp`	—	Launch the multi-language ModelScript Language Server Protocol (LSP) server over `stdio` or TCP/IPC sockets.
`playground`	—	Start a local interactive web playground server for testing grammars, CSTs, and DAE compilation.
`sandbox`	—	Execute untrusted scripts or models inside a resource-capped, capability-isolated WebAssembly sandbox.

6. Package Registry, Cloud HPC & Hub Administration

Command	Aliases	Summary
`init`	—	Initialize a new `modelscript.json` package manifest in the current directory.
`login`	—	Authenticate with a ModelScript Registry / Hub server and store credentials in `~/.modelscriptrc`.
`logout`	—	Remove stored authentication tokens from `~/.modelscriptrc`.
`whoami`	—	Display the currently authenticated user, organization roles, and active registry endpoint.
`install`	`i`, `add`	Resolve, verify, and install versioned dependencies from the ModelScript Registry into `modelica_modules/`.
`pack`	—	Create a deterministic, SHA-256 content-addressed `.tgz` package tarball from the workspace.
`publish`	—	Sign (Ed25519) and publish a package tarball to the ModelScript Registry.
`unpublish`	—	Retract or deprecate a published package version from the registry.
`cloud`	—	Dispatch containerized simulation and CAE jobs to remote Slurm HPC clusters and manage compute credits.
`hub`	—	Initialize or check the health status of a self-hosted ModelScript Hub instance.
`db`	—	Inspect, migrate, and cryptographically verify the Hub PostgreSQL database schema and audit chains.
`archive`	—	Request, monitor, and download cold-storage compliance archives from the Hub.

Return Codes & CI/CD Integration

All `msx` commands return POSIX-compliant exit codes suitable for gating GitHub Actions, GitLab CI, or Jenkins pipelines:

Exit Code	Meaning
`0`	Command succeeded with zero errors (or all verification obligations proved `SAFE`).
`1`	Compilation error, static lint error (`severity: "error"`), counterexample falsification found, or runtime failure.
`2`	Invalid CLI arguments, missing required flags, or unreadable input paths.

CHAPTER 65

CLI Commands Reference

This reference documents every command, subcommand, and flag registered in `@modelscript/cli` (`apps/cli/src/main.ts` and `apps/cli/src/commands/`). All commands can be invoked via `modelscript` or its shorthand alias `msx`, and any command can be accelerated with the global `--daemon` flag.

Compilation, Simulation & Optimization

`compile` / `flatten`

Compiles and flattens a hierarchical Modelica class into a canonical Differential-Algebraic Equation (DAE) representation in linear WebAssembly memory (`DAEBuilder`). `flatten` is an exact alias for `compile`.

```
# Flatten a model using the hybrid WASM kernel with timing breakdowns
msx flatten Modelica.Electrical.Analog.Examples.ChuaCircuit -L ./libraries --timing
```

`instantiate`

Instantiates a Modelica class via the Salsa `QueryEngine` and prints the resolved component hierarchy, inherited modifications, and bindings prior to DAE index reduction.

`simulate`

Compiles a model, performs Block Lower Triangular (BLT) Pantelides index reduction and tearing, and executes a numerical time-domain or steady-state simulation locally or on a remote Slurm HPC cluster.

```
# Run a stiff DAE simulation using the 4th-order Rosenbrock solver and sparse symbolic Jacobians
msx simulate RLC_Circuit circuit.mo --stop-time 10.0 --solver rodas4p --engine arena --jacobian sparse --format csv > trajector

# Dispatch a heavy multi-domain simulation to the Cloud HPC cluster
msx simulate ThermalGrid plant.mo --cloud --profile high-cpu --stop-time 3600 --output-dir ./runs/thermal
```

`optimize`

Formulates and solves a nonlinear Dynamic Optimization / Optimal Control Problem (OCP) over a flattened DAE using direct collocation and sequential quadratic programming (SQP).

```
msx optimize EV_BatteryPack vehicle.mo \
  --objective energyLoss \
  --controls currentCommand \
  --control-bounds "currentCommand:-200:200" \
  --stop-time 600 \
  --num-intervals 100 \
  --format json > optimal_profile.json
```

`grad`

Computes forward, adjoint, or central finite-difference parameter sensitivities ($\nabla_{\mathbf{p}} J$) of scalar objective metrics with respect to model parameters.

`surrogate`

Executes a Design of Experiments (DoE) campaign over a physics model, trains a Reduced-Order Model (ROM) surrogate, attaches convex-hull/bounding-box extrapolation guardrails, and exports standalone C, ONNX, or Modelica `.mo` blocks.

```
msx surrogate HeatExchanger thermal.mo \
  --inputs "massFlow,inletTemp" \
  --input-bounds "massFlow:0.1:5.0,inletTemp:280:380" \
  --outputs "outletTemp,pressureDrop" \
  --strategy sobol \
  --num-samples 128 \
  --architecture mlp \
  --format all \
  --out-file build/hx_rom
```

`mc`

Runs high-throughput Monte Carlo uncertainty propagation across stochastic parameter distributions using worker threads and the linear-memory WASM evaluator.

```
msx mc RLC_Filter filter.mo \
  --params "R:gaussian:100:5,C:uniform:1e-6:1.2e-6" \
  --runs 5000 \
  --lhs \
  --threads 8 \
  --seed 42 \
  --output mc_summary.json
```

`csg`

Extracts Constructive Solid Geometry (CSG) primitives and kinematic frame transformations from a Modelica class, writes `build/csg/execution_graph.json`, and invokes the OpenCASCADE (`@modelscript/cad`) worker to generate 3D CAD meshes.

Linting, Formatting, Diffing & Compiler Inspection

`lint`

Executes the static analysis pipeline (WASM GLR parse validation, CST lint rules, Salsa semantic queries, unit checking, and control-flow analysis) across one or more files or directories.

```
# Lint an entire polyglot workspace and emit JSON for CI processing
msx lint ./models ./requirements --format json
```

`parse`

Parses a file using its registered native WebAssembly GLR parser (`parser.wasm`) and prints the concrete syntax tree (CST), node byte offsets, or parse errors.

`format`

Formats source files canonically using the grammar's CST-preserving pretty-printer while preserving comments and annotations.

`unparse`

Parses a source file into linear CST memory and unparses it back into text to verify lossless round-trip grammar rules.

`diff`

Computes a structural CST/AST and semantic DAE comparison between two files (`oldFile` and `newFile`), identifying added/removed components, modified parameter bindings, and altered equations regardless of whitespace formatting.

`pr-diff`

Generates visual schematic/architectural diffs (highlighting added components in green, modified in amber, and deleted in red) and PR review summaries between two Git revisions.

`i18n`

Extracts translatable descriptions, documentation strings, and identifier aliases from models (`extract`), compiles translation catalogs (`compile`), or translates a model into a target locale (`translate`).

Formal Verification, Safety & Test Synthesis

`verify`

Unified formal verification and simulation validation for SysML v2 and Modelica: decision tables, assume-guarantee contracts, state machines, and optional flowpipe reachability. Results can be emitted as terminal output or as JSON, CTRF, JUnit, SARIF, HTML, or DHF reports for CI/CD and compliance workflows.

```
# Verify a controller and emit a GitHub Code Scanning SARIF report
msx verify BrakingController brake.mo --format sarif --report results.sarif
```

`falsify`

Performs optimization-guided falsification of Signal Temporal Logic (STL) safety properties ($\Box_{[a,b]}\phi$, $\Diamond_{[a,b]}\phi$, $\phi_1 \mathrm{U}_{[a,b]}\phi_2$) by searching disturbance and parameter spaces to minimize quantitative robustness $\rho(\phi, \mathbf{x}, t) < 0$.

`export-formal`

Translates SysML v2 models into external formal verification formats: SMT-LIB2, nuXmv, and OCRA contract specifications.

```
# Export a SysML v2 controller to a nuXmv model
msx export-formal acc.sysml --format nuxmv -o acc.smv
```

`verify-decisions`

Evaluates Architectural Decision Records (ADRs) and SysML v2 trade-study constraints against flattened simulation parameters and formal bounds.

`decompose`

Performs structural subsystem partitioning and Assume-Guarantee (A/G) contract decomposition across large system models, checking whether top-level system guarantees follow compositionally from subsystem contracts ($\bigwedge_i (A_i \Rightarrow G_i) \vdash G_{\text{sys}}$).

`generate-tests`

Synthesizes deterministic test vectors and stimulus harnesses from model guard conditions and assertions to achieve Modified Condition/Decision Coverage (MC/DC) and boundary-value coverage.

Polyglot Digital Thread, Co-Simulation & Standards Interop

`render`

Renders headless, publication-grade SVG diagrams for Modelica component icons/schematics, SysML v2 architectural views, and PSCAD Single-Line Diagrams (SLDs) using `@modelscript/diagram`.

```
# Render a SysML v2 Internal Block Diagram in blueprint theme
msx render SpacecraftPower spacecraft.sysml --view IBD --theme blueprint -o docs/ibd.svg
```

`fmu`

Exports a Modelica class as a Functional Mock-up Unit (`.fmu` ZIP archive) compliant with **FMI 2.0** and/or **FMI 3.0** for Model Exchange (`me`) and Co-Simulation (`cs`), or imports/inspects an existing `.fmu` via `msx fmu import <file.fmu>`. `export-fmu` is an alias for `fmu`.

```
# Export an FMI 3.0 Co-Simulation FMU with embedded C sources and WebAssembly binary
msx fmu DC_Motor motor.mo --fmi-version 3 --type cs --wasm --output build/DC_Motor.fmu
```

`import-fmu`

Unpacks an external FMI 2.0 or FMI 3.0 `.fmu` archive, parses its `modelDescription.xml`, and generates a strongly typed Modelica wrapper `block` (and optional SSP system binding) so the FMU can be connected and simulated inside ModelScript. `fmu-import` is an alias for `import-fmu`.

`cosim`

Manages distributed FMI/SSP Co-Simulation master orchestration sessions, participant FMUs, recorded trajectory histories, and adaptive communication step-size optimization against the ModelScript Hub / Co-Simulation service.

Subcommand	Arguments	Description
`status`	—	Display health and active worker metrics of the Co-Simulation master orchestrator.
`sessions`	—	List active and completed distributed Co-Simulation sessions.
`participants`	`<sessionId>`	List connected FMU/simulator participants, port couplings, and current simulation time t in a session.
`fmus`	—	List registered FMU artifacts available in the Co-Simulation repository.
`upload`	`<path.fmu>`	Upload a local `.fmu` archive to the Co-Simulation registry.
`replay`	`<sessionId>`	Deterministically replay a recorded Co-Simulation session from its saved input/coupling log.
`history`	`<sessionId>`	Fetch step-by-step coupling residuals, rollbacks, and trajectory history for a session.
`optimize`	`<sessionId>`	Run post-mortem Richardson extrapolation analysis on a session to recommend optimal communication step sizes H_i.

`pscad`

Provides full lifecycle tooling for Manitoba Hydro **PSCAD** electromagnetic transient (EMT) workspaces (`.pscx` cases and `.pswx` workspaces), including domain-specific electrical linting, Single-Line Diagram (SLD) generation, EMT simulation, **IEEE Std 2800-2022** grid interconnection verification, and conversion to Modelica/SSP.

Action	Description
`lint`	Validate `.pscx` / `.pswx` schematics against electrical EMT rules (`M6001` – `M6011` , including floating nodes, transformer winding ratios, CSMF unconnected inputs, and traveling-wave line Δt underflow).
`diagram`	Render a publication-quality SVG Single-Line Diagram (SLD) from a `.pscx` project.
`diff`	Compute a topological and parameter diff between two `.pscx` revisions.
`sim`	Execute a fixed-step nodal admittance ($\mathbf{G}\mathbf{v}(t) = \mathbf{i}_{\text{hist}}(t - \Delta t)$) EMT simulation.
`verify-ieee2800`	Run automated **IEEE Std 2800-2022** Inverter-Based Resource (IBR) compliance checks (Low/High Voltage Ride-Through LVRT/HVRT, frequency ride-through, and reactive current injection step response).
`export-modelica`	Translate a `.pscx` schematic into an equivalent Modelica `.mo` model using `Modelica.Electrical` .
`export-ssp`	Package a `.pswx` multi-project workspace into an SSP 1.0 `.ssp` architecture container.

```
# Verify IEEE 2800-2022 LVRT/HVRT compliance on a solar plant PSCAD case
msx pscad verify-ieee2800 SolarPlant_250MW.pscx --tstop 3.0 --step 20us -o ieee2800_report.json
```

`polyglot`

Queries the unified multi-language `WorkspaceIndex` , verifies cross-domain traceability links (`@satisfies` , `@verifies` , `@allocates`), inspects digital threads, and monitors operational digital twin drift across all 12 supported languages.

Subcommand	Description
`project [dir]`	Summarize all indexed languages, files, symbols, and cross-language bindings in a workspace.
`verify [dir]`	Verify cross-language dimensional consistency, port types, requirement satisfaction, and digital thread integrity.
`thread <symbol>`	Trace the upstream/downstream polyglot digital thread for a symbol (from OWL2 ontology → SysML v2 requirement → Modelica equation → STEP CAD geometry → ReqIF/DHF dossier).
`query <expr>`	Execute a cross-language symbol or FLWOR query against the unified `WorkspaceIndex` .
`check-twins [dir]`	Compare live telemetry streams or CSV logs against nominal model predictions to detect parameter drift and anomalies.
`index [dir]`	Build and serialize the workspace-wide polyglot symbol and dependency index (`polyglot-index.json`).

`align`

Executes the Triple Graph Grammar (TGG) and Critical Pair Analysis (CPA) bidirectional synchronization engine across paired architectural, physical, and geometric models.

`sysml`

Provides dedicated interoperability workflows between **OMG SysML v2** (`.sysml`) and **Modelica** (`.mo`) models.

Subcommand	Description
`export <file.mo>`	Project a Modelica class hierarchy into a SysML v2 `.sysml` `part def` / `port def` / `connection` architecture model.
`import <file.sysml>`	Synthesize executable Modelica `.mo` skeleton classes, connectors, and `connect()` equations from a SysML v2 file.
`check <files...>`	Validate SysML v2 ↔ Modelica port compatibility, KerML quantity units, and `@satisfies` requirement constraints.
`info <file.sysml>`	Print package metrics (part definitions, ports, state machines, requirements, and constraint blocks).
`sync <file.sysml> <file.mo>`	Synchronize structural changes and parameter values bi-directionally between `.sysml` and `.mo` files.

`oslc`

Exposes workspace models and requirements as **OASIS OSLC 3.0** Linked Data resources (Requirements Management `oslc_rm` and Architecture Management `oslc_am`) with **TRS 3.0** (Tracked Resource Set) changelog feeds for IBM DOORS Next, Polarion, and Jira integration.

Subcommand	Description
`export [path]`	Serialize workspace symbols, requirements, and traceability links as static OSLC 3.0 RDF documents.
`serve [path]`	Start an HTTP OSLC 3.0 Service Provider catalog, delegated UI preview server, and TRS 3.0 change log endpoint.

`reqif`

Imports and exports **OMG ReqIF 1.2** (Requirements Interchange Format `.reqif` XML and `.reqifz` archives), translating specification hierarchies and attributes directly to and from SysML v2 `requirement def` blocks.

`ddp`

Inspects, packs, and converts **ISO/PAS 81346** Digital Data Packages (`.ddp`) and **IDTA Asset Administration Shell (AAS v3.0)** packages (`.aasx`) for industrial digital twin handover.

Subcommand	Description
`inspect <file.ddp>`	Validate and list reference designations (`=` , `+` , `-` aspect trees), models, and documents inside a `.ddp` archive.
`pack <dir>`	Bundle a workspace directory into a signed ISO/PAS 81346 `.ddp` archive.
`to-aasx <file.ddp>`	Convert a `.ddp` package or workspace manifest into an IDTA AAS v3.0 `.aasx` Digital Twin package with TechnicalData and SimulationModels submodels.

`dhf`

Generates, inspects, and cryptographically audits **Design History File (DHF)** compliance dossiers structured for **FDA 21 CFR Part 820.30**, **ISO 13485**, **ISO 26262**, and **DO-178C** regulatory submissions.

Subcommand	Description
`export [path]`	Compile requirements traceability matrices (RTM), formal verification proofs, simulation summaries, and SHA-256 artifact hashes into a DHF dossier.
`status [path]`	Report requirement verification coverage, unverified hazards, and suspect links across the workspace.
`audit [file]`	Verify the cryptographic SHA-256 hash chain and Ed25519 signatures of an exported DHF bundle.

`simulink`

Inspects, lints, compiles into linear DAEs, and projects MATLAB Simulink control block diagrams (`.slx` zip packages and `.mdl` structured text).

Action	Description
`inspect`	Dump subsystem block tree, parameters, solver settings, and line routing.
`lint`	Run pre-flight static analysis (algebraic loops `M7202` , unconnected ports `M7201` , divide-by-zero risks `M7209`).
`to-modelica`	Translate block diagram into standard acausal `Modelica.Blocks` models.
`to-dae`	Lower continuous states and transfer functions directly into the WebAssembly DAE arena.

```
msx simulink inspect CruiseControl.slx
msx simulink lint EngineGovernor.mdl
msx simulink to-modelica Suspension.slx --out=Suspension.mo
```

`autosar`

Inspects, lints, projects, and simulates AUTOSAR Classic and Adaptive (`.arxml`) software component architectures and Virtual Run-Time Environment (RTE) runnables.

Action	Description
`inspect`	Inspect software component types, port prototypes, runnables, timing events, and network PDUs.
`lint`	Detect unconnected R-ports (`M7401`), interface mismatches (`M7402`), WCET budget overflows (`M7405`), and PDU bit overlap (`M7406`).
`to-modelica`	Generate Modelica standard interface blocks for SIL plant co-simulation.
`to-sysml`	Project software architecture into OMG SysML v2 `part def` hierarchies.
`to-dae`	Lower software states and calibrated parameters into ModelScript linear-memory DAE arena.
`simulate-rte`	Execute the discrete event Virtual RTE scheduler and route signals across assembly connectors.

```
msx autosar inspect Powertrain.arxml
msx autosar lint BrakeController.arxml
msx autosar to-sysml Steering.arxml --out=Steering.sysml
msx autosar simulate-rte PowertrainSystem.arxml --duration=0.5 --step=0.01 --out=rte_trace.json
```

`olga`

Provides complete command-line tooling for Schlumberger **OLGA** transient multiphase flow models (`.opi` , `.key` , `.genkey` , `.tab` , `.tpl` , `.ppl`), including drift-flux hydrodynamic simulation, severe slugging analysis, PVT flash, autonomous history matching, and flow assurance verification.

Action	Description
`inspect`	Dump flowpath pipe discretization, valve configurations, sources, and boundary conditions.
`lint`	Run pre-flight flow assurance checks (`M7001` – `M7013` : CFL violations, severe slugging, pig stuck hazards, wax/asphaltene precipitation).
`simulate`	Execute transient 1D drift-flux simulation with Lagrangian slug tracking and scraper pig dynamics.
`verify`	Check formal regulatory limits (MAOP, hydrate subcooling margin, slug surge volume, erosion limits).
`diagram`	Render SVG schematics, seabed bathymetry elevation profiles, PVT envelopes, or slug statistics.
`tune`	Run automated history matching against field measurement CSV telemetry.
`flash`	Compute Peng-Robinson (PR) or Soave-Redlich-Kwong (SRK) thermodynamic vapor-liquid equilibrium (VLE) flash.
`network`	Solve coupled gathering network transients across multiple branches and subsea manifolds.
`cosim`	Run multi-rate SSP co-simulation coupling flowlines with topside process facilities.
`to-dae`	Lower flowline equations into ModelScript DAE arena.
`to-modelica`	Translate pipe and boundary networks into `Modelica.Fluid` models.

```
msx olga simulate subsea_tieback.opi --duration=3600 --out=traj.csv
msx olga lint deepwater_riser.genkey
msx olga diagram tieback.opi --view=profile --out=profile.svg
msx olga flash --comp="C1:0.75,C2:0.10,C3:0.08,nC4:0.04,C7:0.03" --p=50 --t=295.15 --envelope
```

`simpack`

Tooling for Dassault Systèmes **SIMPACK** multibody dynamics (`.spck` XML and `.sim` ASCII decks), including Grübler mobility analysis, 3D kinematic joint evaluation, Berg elastomeric bushings, and Craig-Bampton flexible body lowering.

Action	Description
`lint`	Check for dangling marker references, kinematic loops, and parameter inconsistencies.
`mobility`	Calculate mechanism spatial degrees of freedom ($M = 6n - \sum c_i$) and detect overconstraints.
`to-modelica`	Project the mechanism into standard `Modelica.Mechanics.MultiBody` components.
`simulate`	Lower the multibody model to a DAE and integrate it in time.
`to-fmu`	Package the multibody assembly as an FMI 2.0 Co-Simulation FMU.
`report`	Generate a homologation-style report for the model.

```
msx simpack lint DoubleWishbone.spck
msx simpack mobility Suspension.spck
msx simpack to-modelica Mechanism.sim --out=Mechanism.mo
```

`gensys`

Tooling for **GENSYS** railway vehicle dynamics (`.calc` , `.geom` , `.track`), including FASTSIM wheel-rail contact mechanics, Polach non-linear creep, ERRI B176 track irregularities, Sperling ride comfort, and **EN 14363** / **UIC 518** running safety homologation.

Action	Description
`lint`	Validate vehicle inertial and geometric parameters against railway engineering rules.
`to-modelica`	Project the vehicle model into Modelica.
`verify`	Evaluate EN 14363 derailment quotient ($Y/Q \leq 0.80$) and Prud'homme lateral limit from `--y-force` , `--q-force` , `--axle-load` .
`simulate`	Lower the vehicle model to a DAE and integrate it in time.
`to-fmu`	Package the vehicle model as an FMI Co-Simulation FMU.
`report`	Write an EN 14363 homologation report (`--format html | sarif | junit | ctrf`).

```
msx gensys lint ManchesterBenchmark.calc
msx gensys verify --y-force 40 --q-force 80 --axle-load 160
msx gensys report ManchesterBenchmark.calc --format html --out homologation.html
```

`spice`

Analog circuit simulation and netlist analysis for **SPICE** (`.cir` , `.sp` , `.net` , `.lib`), including DC operating point, transient, and AC analyses (AC sweep parameters come from the netlist `.AC` directive), and ECAD graph generation.

```
msx spice simulate LowPassFilter.cir --analysis op
msx spice simulate PowerSupply.cir --analysis tran --tstep 1e-6 --tstop 0.02 --out transient.csv
msx spice simulate AudioAmplifier.sp --analysis ac --format csv --out bode.csv
```

`adams`

Tooling for **MSC Adams** mechanism datasets (`.adm`) and command files (`.cmd`), including 3D Euler/Cardan kinematics, Grübler spatial mobility analysis, and cross-domain SIMPACK / Modelica projections.

Action	Description
`lint`	Check the dataset for dangling references and parameter inconsistencies.
`mobility`	Calculate mechanism degrees of freedom and detect overconstraints.
`to-modelica`	Project the mechanism into `Modelica.Mechanics.MultiBody` components.
`to-simpack`	Convert the Adams dataset into SIMPACK `.spck` XML.
`simulate`	Lower to a DAE (Baumgarte or algebraic stabilization via `--stabilization`) and integrate it in time.

```
msx adams mobility FourBarLinkage.adm
msx adams lint LandingGear.adm
msx adams to-simpack RobotArm.adm --out=RobotArm.spck
```

`urdf`

Robotics compiler tooling for **ROS URDF** (`.urdf`) and **Xacro** (`.xacro`) descriptions, featuring Featherstone recursive dynamics (ABA forward dynamics, RNEA inverse dynamics), joint limit verification, and 3D CAD projections.

Action	Description
`lint`	Validate kinematic tree acyclicity (`M7504`), joint limits (`M7508`), and inertia tensors (`M7506`).
`rnea`	Evaluate inverse dynamics (RNEA) at test joint coordinates.
`to-modelica`	Generate a `Modelica.Mechanics.MultiBody` robotic system.
`to-simpack`	Project the robot description into SIMPACK `.spck` XML.
`to-adams`	Project the robot description into an MSC Adams `.adm` dataset.
`to-dae`	Lower kinematic state equations directly into the `DAEBuilder` arena.

```
msx urdf rnea KukaIiwa14.urdf
msx urdf lint Manipulator.xacro
msx urdf to-modelica PandaArm.urdf --out=PandaArm.mo
```

`easy5`

Tooling for Boeing / MSC **EASY5** dynamic models (`.ez5`, `.ezm`, `.ezd`), including linear state-space analysis, User General (UG) Fortran routines, tabular map interpolation, and Modelica/SSP projections.

Action	Description
`inspect`	Dump component blocks, connection topology, and parameter tables.
`lint`	Check for dangling connections, missing table files, and rate limits.
`to-modelica`	Project continuous block diagram to Modelica 3.x.
`to-ssp`	Package model as an FMI SystemStructure.ssd container.
`to-dae`	Lower continuous and discrete equations into `DAEBuilder` arena.

`gtsuite`

Tooling for Gamma Technologies **GT-SUITE** powertrain and engine models (`.gtm` , `.gtd`), including crank-slider kinematics, Wiebe combustion, Woschni heat transfer, Zeldovich NOx emissions, and DAE lowering.

Action	Description
`inspect`	Inspect engine cylinders, intake/exhaust manifolds, valves, and turbochargers.
`lint`	Detect compressor surge (`M9002`), invalid compression ratio (`M9005`), and acoustic CFL limits.
`simulate`	Execute in-WASM Mean-Value Engine Model (MVEM) transient simulation.
`to-modelica`	Project 1D powertrain architecture to Modelica standard blocks.
`to-dae`	Lower engine thermodynamics and flow dynamics into `DAEBuilder` arena.

`sima`

Tooling for DNV / SINTEF **SIMA** marine systems (`.inp` , `.sima` , `.sys` , `.hyd`), including Cummins equation 6-DOF floater dynamics, catenary mooring solvers, and DNV-OS-E301 station-keeping verification.

Action	Description
`inspect`	Inspect marine vessels, mooring lines, catenary segments, and environmental loads.
`lint`	Check negative metacentric height (`M7001`), line underflow (`M7002`), and attachment nodes.
`summary`	Print a model summary: vessels, lines, and environment.
`verify`	Verify DNV-OS-E301 / DNV-ST-0119 line tension safety factors.
`offset`	Report vessel watch-circle and offset limits.
`to-modelica`	Project marine system into acausal Modelica physical components.
`to-ssp`	Export an SSP `SystemStructure.ssd` .
`to-sysml2`	Export a SysML v2 model.
`to-dae`	Lower hydrodynamics and mooring tension equations into `DAEBuilder` arena.

`rocets`

Tooling for NASA **ROCETS** liquid rocket engine transient decks (`.roc` , `.deck` , `.map`), including turbopump flow balance, CEA equilibrium combustion, supersonic nozzle expansion, and Bartz throat heat transfer.

Action	Description
`inspect`	Inspect turbopumps, combustion chambers, regenerative cooling channels, and valves.
`lint`	Detect extreme mixture ratios (`M8011`), turbine overspeed (`M8013`), and negative chamber pressure.
`simulate`	Solve steady-state flow balance or transient rocket engine ignition start sequence.
`to-modelica`	Project cryogenic fluid networks and turbomachinery into Modelica 3.x.
`to-dae`	Lower propulsion governing equations into `DAEBuilder` arena.

`psse`

Tooling for Siemens **PSS/E** power transmission cases (`.raw` , `.dyr` , `.sav`), including bus/branch netlist extraction, Full-Newton AC power flow solving, and Modelica / PSCAD EMT projections.

Action	Description
`inspect`	Dump bus count, generators, transmission lines, transformers, and load summary.
`lint`	Validate bus voltage bounds, check for isolated buses, and verify generator P/Q limits.
`solve`	Solve steady-state AC power flow on the nodal admittance matrix $\mathbf{Y}_{\text{bus}}$.
`summary`	Print a base-case summary.
`diagram`	Export a single-line diagram as SVG.
`netlist`	Extract the bus/branch netlist.
`to-modelica`	Project power grid into acausal Modelica transmission lines and generators.
`to-dae`	Lower bus injection equations and generator swing dynamics into `DAEBuilder` arena.

`lps`

Tooling for Energy Solutions **PipelineStudio** gas and liquid pipeline networks (`.lps` , `.tgx` , `.tlx` , `.psg`), including AGA-8 / GERG-2008 natural gas compressibility, hydrogen blending ($H_2 \leq 20\%$), open-channel slack line flow, water hammer liquid surge, and transient line pack accounting.

Action	Description
`inspect`	Dump pipeline pipes, compressors, pumps, delivery regulators, and receipt nodes.
`lint`	Check for MAOP overpressure (`M8001`), compressor surge (`M8002`), and high hydrogen blend ratios.
`simulate`	Run transient fluid network pressure propagation and inventory line pack calculations.
`solve`	Solve the steady-state network.
`linepack`	Compute pipeline inventory (line pack).
`sysml`	Project pipeline topology and control station hierarchies into OMG SysML v2.
`verify`	Run design and safety verification on the steady-state solution.
`to-modelica`	Project pipe network into `Modelica.Fluid` gas/liquid transmission models.
`to-dae`	Lower Euler gas/liquid equations directly into `DAEBuilder` arena.

Language Workbench, Daemon & IDE Tooling

`daemon`

Controls the lifecycle of the persistent background ModelScript compiler daemon (`~/.modelscript/daemon.sock`).

Subcommand	Description
`start`	Spawn the background compiler daemon if not already running.
`stop`	Gracefully shut down the running daemon and remove its socket file.
`status`	Report whether the daemon is running, its PID, uptime, and cached workspace count.
`restart`	Stop any active daemon instance and immediately start a fresh one.

`build`

Compiles a `@modelscript/dsl` grammar definition into a zero-dependency native WebAssembly GLR parser (`parser.wasm`) along with strongly typed TypeScript CST node definitions.

`generate`

Scaffolds a new WebAssembly GLR parser (`parser`) or a complete standalone VS Code language extension (`extension` including TextMate grammars, LSP wiring, and `package.json` contributions) from a `@modelscript/dsl` language definition.

`language`

Manages the dynamic polyglot language registry (`lang` is an alias for `language`).

Subcommand	Description
`list`	List all registered languages (`.mo` , `.sysml` , `.msx` , `.pscx` , `.scad` , `.stp` , `.owl` , `.ssp` , `.csv` , `.su2` , `.inp` , `.gbr`) and their loaded `.wasm` parser states.
`register <path>`	Dynamically register a custom language plugin module into `~/.modelscript/languages.json` .
`unregister <id>`	Remove a dynamically registered language plugin by language ID.

`lsp`

Starts the multi-language ModelScript Language Server Protocol (`@modelscript/lsp`) server.

`playground`

Launches a local interactive web playground server for live grammar authoring, GLR parse tree visualization, Salsa query inspection, and DAE simulation.

`sandbox`

Executes untrusted `.msx` scripts or `.mo` simulations inside a memory-capped, capability-isolated WebAssembly sandbox with no host filesystem or network access.

Package Registry, Cloud HPC & Hub Administration

`init`

Initializes a new `modelscript.json` package manifest in the current directory.

`login` , `logout` , `whoami`

Manages authentication sessions against a ModelScript Registry or self-hosted Enterprise Hub. Credentials and Ed25519 signing keys are stored in `~/.modelscriptrc` .

`install`

Resolves semantic version constraints, verifies SHA-256 content-addressable digests and Ed25519 publisher signatures, and installs Modelica/SysML/ModelScript packages into `modelica_modules/` while updating `modelscript.lock` .

`pack` , `publish` , `unpublish`

Packages, cryptographically signs, and publishes libraries to the ModelScript Registry.

`cloud`

Manages remote high-performance computing (HPC) simulation sweeps, Slurm batch jobs, and organization compute credit wallets on ModelScript Cloud.

Subcommand	Arguments	Description
`profiles`	—	List available CPU/GPU hardware execution profiles (`standard` , `high-cpu` , `gpu-a100` , `slurm-mpi`) and credit burn rates.
`balance` / `wallet`	—	Display current organization compute credit balance, active reservations, and ledger history.
`jobs` / `list`	—	List recent and running cloud simulation jobs.
`status`	`<jobId>`	Query real-time queue state (`QUEUED` , `RUNNING` , `COMPLETED` , `FAILED`) and progress percentage for a job.
`logs`	`<jobId>`	Stream stdout/stderr compiler and solver logs from a cloud worker.
`cancel`	`<jobId>`	Terminate a queued or running cloud job and refund unused reserved credits.
`download`	`<jobId> [outDir]`	Download completed trajectory CSV/Parquet artifacts and diagnostic reports.
`dispatch`	`<name> <paths...>`	Directly submit a containerized simulation or DoE sweep job to the cloud queue.

`hub` , `db` , `archive`

Administrative CLI commands for operators managing a self-hosted **ModelScript Enterprise Hub** instance (see Platform Administration Guide).

```
# Initialize Hub configuration or query cluster service health
msx hub <init|status> [--api-url <url>]

# Inspect PostgreSQL migration state, apply schema upgrades, or verify the SHA-256 audit ledger chain
msx db <status|upgrade|verify> [--connection-string <postgres://...>]

# Request, poll, or download long-term regulatory cold-storage archives
msx archive <request|status|download> <targetId> [-o <path>]
```

PART VI

Reference & Diagnostics

CHAPTER 66

title: Linter Rules Reference description: Every diagnostic rule ModelScript ships — 790 rules across 43 languages, with codes, severities and message templates generated from the source. outline: [2, 2] pageClass: lint-reference

Linter Rules Reference

ModelScript ships **790 diagnostic rules** across **43 languages**. This page is generated from each language's rule catalog (for example `languages/modelica/src/errors.ts`), so codes, severities and message templates match what the tools report. Text in `<angle brackets>` is filled in from your model.

Running the linters

Diagnostics appear as you type in VS Code and the web IDE through the language server. From the command line:

```
# Languages handled by the unified linter (detected by file extension)
msx lint ./models
msx lint ./models --format json --strict

# Domain formats with their own command
msx spice lint SensorFrontEnd.net
```

The **CLI** column below shows which command reports each language's rules.

Rule index

Language	Package	Rules	CLI
ModelScript	`@modelscript/modelscript`	10	Language server / API
Modelica	`@modelscript/modelica`	307	`msx lint`
SysML v2 / KerML	`@modelscript/sysml2`	48	`msx lint`
ReqIF	`@modelscript/reqif`	10	`msx reqif lint`
SSP	`@modelscript/ssp`	12	`msx lint`
OWL2	`@modelscript/owl2`	24	`msx lint`
STEP (ISO 10303)	`@modelscript/step`	8	`msx lint`
OpenSCAD	`@modelscript/scad`	8	`msx lint`
CFD (SU2 / OpenFOAM)	`@modelscript/cfd`	8	`msx lint`
FEA (CalculiX / Abaqus / NASTRAN)	`@modelscript/fea`	8	`msx lint`
C	`@modelscript/c`	10	`msx lint`
Fortran	`@modelscript/fortran`	10	`msx lint`
JSBSim	`@modelscript/jsbsim`	10	`msx jsbsim lint`
NASA ROCETS	`@modelscript/rocets`	14	`msx lint`
CCSDS	`@modelscript/ccsds`	8	`msx ccsds lint`
EASY5	`@modelscript/easy5`	12	`msx lint`
AUTOSAR	`@modelscript/autosar`	8	`msx autosar lint`
MATLAB Simulink	`@modelscript/simulink`	10	`msx simulink lint`
GT-SUITE	`@modelscript/gtsuite`	15	`msx lint`
ASAM OpenDRIVE	`@modelscript/opendrive`	8	`msx opendrive lint`
ASAM OpenSCENARIO	`@modelscript/openscenario`	10	`msx openscenario lint`
GENSYS	`@modelscript/gensys`	6	`msx gensys lint`
SIMPACK	`@modelscript/simpack`	6	`msx simpack lint`
PSCAD / EMTDC	`@modelscript/pscad`	17	`msx lint`
PSS/E	`@modelscript/psse`	15	`msx psse lint`
OpenDSS	`@modelscript/opendss`	8	`msx opendss lint`
OLGA	`@modelscript/olga`	13	`msx olga lint`
PipelineStudio	`@modelscript/lps`	12	`msx lint`
ECLIPSE	`@modelscript/eclipse`	10	`msx eclipse lint`
RELAP5-3D	`@modelscript/relap5`	10	`msx relap5 lint`
SIMA / SIMO / RIFLEX	`@modelscript/sima`	15	`msx lint`
SPICE	`@modelscript/spice`	10	`msx spice lint`
Verilog-A / AMS	`@modelscript/verilog-a`	10	`msx verilog-a lint`
IBIS	`@modelscript/ibis`	9	`msx ibis lint`
Touchstone	`@modelscript/touchstone`	10	`msx touchstone lint`
URDF / Xacro	`@modelscript/urdf`	8	`msx urdf lint`
PLCopen / IEC 61131-3	`@modelscript/plcopen`	10	`msx plcopen lint`
MSC Adams	`@modelscript/adams`	9	`msx adams lint`

Language	Package	Rules	CLI
EnergyPlus	`@modelscript/energyplus`	8	`msx energyplus lint`
EPANET	`@modelscript/epanet`	8	`msx epanet lint`
EPA SWMM	`@modelscript/swmm`	13	`msx swmm lint`
Aspen Plus / HYSYS	`@modelscript/aspen`	15	`msx aspen lint`
SBML	`@modelscript/sbml`	10	`msx sbml lint`

ModelScript {#modelscript}

Native systems modeling & query language. **10 rules** · `@modelscript/modelscript` · Source: `languages/modelscript/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`M1001`	`parse-error`		Parse error.
`M2001`	`undefined-variable`		Variable '`<name>`' is not declared in current scope.
`M2002`	`duplicate-declaration`		Identifier '`<name>`' is already declared in this scope.
`M2003`	`unknown-type`		Type '`<typeName>`' is not recognized.
`M3001`	`type-mismatch`		Type mismatch: cannot assign '`<sourceType>`' to '`<targetType>`'.
`M3002`	`invalid-binary-operand`		Operator '`<op>`' cannot be applied to operands of type '`<type>`'.
`M4001`	`non-iterable-collection`		Expression '`<expr>`' in 'for' clause is not iterable.
`M4002`	`flwor-unbound-variable`		Variable '`<varName>`' in FLWOR clause is not bound in preceding 'for' or 'let' clauses.
`M5001`	`argument-count-mismatch`		Function '`<fnName>`' expects `<expected>` arguments, but got `<actual>`.
`M5002`	`unknown-struct-field`		Struct '`<structName>`' has no field named '`<fieldName>`'.

Modelica {#modelica}

Acausal multi-domain physics. **307 rules** · `@modelscript/modelica` · Source: `languages/modelica/src/errors.ts`, `languages/modelica/src/lints/index.ts` · Language guide

`M1xxx` — Syntax & parse

Code	Rule	Severity	Message
`M1000`	`identity-arg-count`		Wrong number of arguments to identity.
`M1001`	`parse-error`		Parse error.
`M1002`	`parse-missing`		Parse error: '`<expected>`' expected.
`M1003`	`empty-array-constructor`		Parse error: Empty array constructors are not valid in Modelica.
`M1003`	`emptyarrayconstructor`		Empty array constructor '`<target>`' is not valid in Modelica.
`M1004`	`identity-first-arg-integer`		First argument to identity in component `<compName>` must be Integer expression.
`M1005`	`subscripted-modifier`		Subscripting modifiers is not allowed. Apply the modification on the whole identifier using an array-expression or an each-modifier.
`M1099`	`wasm-flattener-unsupported`		[ModelicaFlattener] Model '`<rootName>`' could not be flattened using strict WASM backend.

`M2xxx` — Name & scope resolution

Code	Rule	Severity	Message
M2001	duplicate-element		An element with name <name> is already declared in this scope.
M2002	variable-not-found		Variable <name> not found in scope <scope>.
M2002	variablenotfound		Variable <vName> not found in scope <className>.
M2003	class-not-found		Class <className> not found in scope <scope> (looking for a function or record).
M2003	typenotfound		Class or type '<fullType>' not found in scope.
M2004	modifier-not-found		In modifier of '<componentName>', class or component '<modName>' not found in '<className>'.
M2005	identifier-mismatch		The identifier at start and end are different.
M2005	identifiermismatch		Parse error: The identifier at start and end are different
M2006	modified-element-not-found		Modified element <modName> not found in class <className>.
M2007	modifier-class-not-found		In modifier (<modText>), class or component <modName> not found in <<className>>.
M2011	same-name-type-shadowing		Found a component with same name when looking for type <typeSpec>.
M2012	class-not-found-in-scope		Class <className> not found in scope <scope>.
M2013	base-class-not-found-in-scope		Base class <className> not found in scope <scope>.
M2014	partial-base-class		Part <partName> of base class name <extName> is not a class.
M2015	base-class-ambiguous		The base class name <partName> was found in one or more base classes: <candidateNames>.
M2016	base-class-found-in		<partName> was found in base class <baseName>.
M2038	partial-lookup-disallowed		<className> is partial, name lookup is not allowed in partial classes.
M2090	finaloverridenotification		From here:
M2090	from-here		From here:
M2091	redeclare-final-component-notification		From here:
M2091	redeclarefinalcomponentnotification		From here:
M2092	redeclare-constant-component-notification		From here:
M2092	redeclareconstantcomponentnotification		From here:
M2093	duplicate-modification-notification		From here:
M2093	duplicatemodificationnotification		From here:
M2094	missing-inner		No corresponding 'inner' declaration found for component . <typeName> <componentName> declared as 'outer'. The existing 'inner' components are: <inners> Check if you have not misspelled the 'outer' component name. Please declare an 'inner' component with the same name in the top scope. Continuing flattening by only considering the 'outer' component declaration.
M2094	missinginner		No corresponding 'inner' declaration found in scope for outer component '<target>'.
M2095	from-here-trace		From here:
M2096	redeclare-non-replaceable-notification		From here:
M2096	redeclarenonreplaceablenotification		From here:
M2097	protected-modification-notification		From here:
M2097	protectedmodificationnotification		From here:

`M3xxx` — Type system & operator / function signatures

Code	Rule	Severity	Message
`M3001`	`type-mismatch-binding`		Type mismatch in binding `<componentName> = <actualExpr>`, expected subtype of `<expectedType>`, got type `<actualType>`.
`M3001`	`typemismatchbinding`		Type mismatch in binding `<target>`, expected subtype of `<expName>`, got type `<actName>`.
`M3002`	`type-mismatch-modifier`		Type mismatch: '`<modName>`' expects type '`<expectedType>`' but got '`<actualType>`'.
`M3002`	`typemismatchmodification`		Type mismatch in binding or modification expression '`<target>`'.
`M3003`	`not-plug-compatible`		The connectors in connect(`<ref1>`, `<ref2>`) are not type compatible.
`M3004`	`not-a-connector`		`<which>` is not a valid connector.
`M3005`	`redeclare-type-mismatch`		Redeclare of '`<redeclaredName>`': type '`<newType>`' is not compatible with constraining type '`<constrainingType>`'.
`M3006`	`function-arg-type-mismatch`		Type mismatch for positional argument `<argPosition>` in `<callExpr>`. The argument has type: `<argTypeSignature>` expected type: `<expectedTypeSignature>`
`M3007`	`connect-invalid-form`		`<ref>` is not a valid form for a connector, connectors must be either c1.c2...cn or m.c (where c is a connector and m is a non-connector).
`M3008`	`extends-type-mismatch`		In extends of '`<className>`': redeclared '`<redeclaredName>`' of type '`<newType>`' is not compatible with original type '`<originalType>`'.
`M3009`	`array-index-type-mismatch`		Array index type mismatch: expected Integer or Boolean, but got '`<actualType>`'.
`M3009`	`arrayindextypemismatch`		Array index '`<target>`' has invalid type: expected Integer or Boolean.
`M3010`	`unit-mismatch`		Unit mismatch: Left-hand side has unit '`<lhsUnit>`' but right-hand side has incompatible unit '`<rhsUnit>`'.
`M3010`	`unitmismatchbinding`		Unit mismatch in equation '`<target>`'.
`M3010`	`unitmismatchequation`		Unit mismatch in equation '`<target>`'.
`M3011`	`binary-op-type-mismatch`		Cannot resolve type of expression `<expr>`. The operands have types `<type1>`, `<type2>` in component <NO_COMPONENT>.
`M3012`	`optimization-objective-type`		Optimization objective must be a scalar Real, but got type '`<typeStr>`'.
`M3013`	`optimization-constraint-type`		Optimization constraints must evaluate to scalar Real or Boolean expressions, but got type '`<typeStr>`'.
`M3014`	`builtin-attribute-type-mismatch`		Type mismatch in binding `<attrName> = <bindingText>`, expected subtype of `<expectedType>`, got type `<actualType>`.
`M3015`	`attribute-type-mismatch`		Variable `<varName>`: Wrong type on builtin attribute `<attrName>` (type was: `<actualType>`, expected: `<expectedType>`).
`M3016`	`conditional-type-mismatch`		Type error in conditional '`<condExpr>`'. Expected Boolean, got `<actualType>`.
`M3017`	`connect-types-inconsistent`		The type of variables `<r0Raw>` type: `<type0>` and `<r1Raw>` type: `<type1>` are inconsistent in connect equations.
`M3018`	`array-element-type-mismatch`		Array constructor elements have inconsistent types in '`<exprText>`'.
`M3018`	`arrayelementtypemismatch`		Array element type mismatch in '`<target>`'.
`M3019`	`no-matching-function`		No matching function found for `<callExpr>` of type `<callSig>` candidates are `<candidateSig>`
`M3020`	`type-mismatch-modifier-binding`		Type mismatch in modifier of component `<compName>`, expected type `<expectedType>`, got modifier `<exprText>` of type `<actualType>`.
`M3021`	`function-return-type-mismatch`		Function '`<funcName>`' returns type '`<actualType>`' but '`<expectedType>`' expected.

`M4xxx` — Structural, inheritance, modification & connector semantics

Code	Rule	Severity	Message
M4000	dimension-not-parameter		Expression '<exprName>' that determines the size of dimension '<dim>' of '<callText>' is not an evaluable parameter expression.
M4001	extends-cycle		extends <baseName> causes an instantiation loop.
M4001	extendscycle		Extends cycle detected for '<target>'.
M4002	duplicate-modification		Duplicate modification of element <elementName> on <kind> <componentName>.
M4002	duplicatemodification		<outerName>.<innerOrKind>
M4003	array-dimension-mismatch		Type mismatch in binding '<componentName> = <exprText>', expected array dimensions [<expectedShape>], got [<actualShape>].
M4003	arrayshapemismatch		Array shape mismatch: declared dimension does not match initializer element count in '<target>'.
M4004	unbalanced-model		The <classKind> '<name>' is not balanced: <nEquations> equation(s) and <nVariables> variable(s).
M4004	unbalancedmodel		Model '<target>' is not balanced: <value> equations for <value> variables.
M4005	protected-modification		Protected element '<elementName>' may not be modified, got '<modText>'.
M4005	protectedmodification		Protected element '<eName>' may not be modified, got '<mText>'.
M4006	external-with-algorithm		Element is not allowed in function context: algorithm
M4006	externalwithalgorithm		Element '<target>' is not allowed in function context with algorithm section.
M4007	function-public-variable		Invalid public variable <varName>, function variables that are not input/output must be protected.
M4007	functionpublicvariable		Invalid public variable <varName>, function variables that are not input/output must be protected.
M4008	array-subscript-count-mismatch		Wrong number of subscripts in <refStr> (<actualCount> subscripts for <expectedCount> dimensions).
M4009	function-default-arg-cycle		The default value of <paramName> causes a cyclic dependency.
M4010	function-invalid-var-type		Invalid type <typeName> for function component <varName>.
M4011	function-protected-io		Invalid protected variable <varName>, function variables that are input/output must be public.
M4011	functionprotectedio		Invalid protected variable <varName>, function variables that are input/output must be public.
M4012	function-arg-variability		Function argument <paramName> = <argExpr> in call to <funcName> has variability <actualVariability> which is not a <requiredVariability> expression.
M4013	nested-when-statement		Nested when statements are not allowed.
M4013	nestedwhen		Nested when statements are not allowed.
M4014	tuple-expression-context		Tuple expressions may only occur on the left side of an assignment or equation with a single function call on the right side. Got the following expression: <exprText>.
M4014	tupleexpressioncontext		Tuple expressions may only occur on the left side of an assignment or equation with a single function call on the right side.
M4015	within-in-script		The within directive is not allowed in script mode.
M4016	statement-in-standard-mode		Top-level statements are not allowed in standard Modelica mode.

Code	Rule	Severity	Message
M4017	restriction-violation		<what> are not allowed in <classKind>.
M4017	restrictionviolation		Algorithm sections are not allowed in <kindStr>.
M4018	partial-instantiation		Illegal to instantiate partial class <className>.
M4019	redeclare-non-replaceable		Redeclaration with a new type requires '<elementName>' to be replaceable.
M4019	redeclarenonreplaceable		Redeclaration with a new type requires '<eName>' to be replaceable.
M4020	builtin-time-invalid		time is not allowed in a function.
M4020	reservedbuiltintime		Identifier time is reserved for the built-in element with the same name.
M4020	timeinfunction		time is not allowed in a function.
M4021	range-step-too-small		Step size <stepSize> in range is too small.
M4022	enum-range-with-step		Range of type enumeration <enumType> may not specify a step size.
M4022	enumrangewithstep		Range of type enumeration '<target>' may not specify a step size.
M4023	connect-in-when		connect may not be used inside when-equations (found <connectExpr>).
M4023	connectinwhen		connect may not be used inside when-equations (found <target>).
M4024	connect-in-initial		Connect equations are not allowed in initial equation sections.
M4024	connectininitial		Connect equations are not allowed in initial equation sections.
M4025	connect-in-non-param-if		connect may not be used inside if-equations with non-parametric conditions (found <connectExpr>).
M4025	connectinnonparamif		connect may not be used inside if-equations with non-parametric conditions (found <target>).
M4026	final-override		Trying to override final element <elementName> with modifier '<modText>'.
M4026	finaloverride		Trying to override final element <eName> with modifier '<mText>'.
M4027	variability-binding-mismatch		Component <componentName> of variability <compVariability> has binding '<bindingExpr>' of higher variability <bindingVariability>.
M4027	variabilitybindingmismatch		Component <compName> of variability parameter has binding '<target>' of higher variability continuous.
M4028	partial-type-component		Component '<componentName>' has partial type '<typeName>'.
M4028	partialtypecomponent		Component '<target>' has partial type.
M4029	component-binding-restriction		Component '<componentName>' may not have a binding equation due to class specialization '<classKind>'.
M4030	outer-modifier		Modifier '<modText>' found on outer element <elementName>.
M4030	outermodifier		Modifier '<modNode>' found on outer element <nameNode>.
M4031	array-index-out-of-bounds		Subscript '<index>' for dimension <dimensionIndex> (size = <dimensionSize>) of <arrayName> is out of bounds.
M4031	arrayindexoutofbounds		Subscript 'index' for dimension <dimIndex> (size = <dimSize>) of array is out of bounds.
M4032	function-invalid-prefix		Invalid prefix <prefix> on formal parameter <varName>.
M4032	functioninvalidprefix		Invalid prefix '<target>' on formal parameter in function.
M4033	function-multiple-algorithm		Function <funcName> has more than one algorithm section or external declaration.

Code	Rule	Severity	Message
M4033	functionmultiplealgorithm		Function <name> has more than one algorithm section or external declaration.
M4034	replaceable-base-class		Class ' <className> ' in 'extends <extendsText> ' is replaceable, the base class name must be transitively non-replaceable.
M4034	replaceablebaseclass		Class ' <target> ' in 'extends <target> ' is replaceable, the base class name must be transitively non-replaceable.
M4035	cyclic-dimension-dependency		Dimension <dimNum> of <varName> , ' <exprText> ', could not be evaluated due to a cyclic dependency.
M4036	package-variable-not-constant		Variable <varName> in package <packageName> is not constant.
M4037	connector-variability		Invalid variability <variability> on connector ' <connectorName> '.
M4037	connectorvariability		Invalid variability <pfx> on connector ' <compName> '.
M4038	flow-outside-connector		Prefix 'flow' used outside connector declaration.
M4038	flowoutsideconnector		Prefix ' <target> ' used outside connector declaration.
M4039	discrete-not-on-lhs		Following variable is discrete, but does not appear on the LHS of a when-statement: ' <varName> '.
M4039	discretenotonlhs		Following variable is discrete, but does not appear on the LHS of a when-statement: ' <target> '.
M4040	protected-element-modification		Protected element ' <varName> ' may not be modified, got ' <modText> '.
M4041	negative-dimension		Negative dimension index (<dim>) for component <varName> .
M4041	negativedimension		Negative dimension index (-1) for component component.
M4042	constant-has-no-value		Constant ' <qualifiedName> ' has no value.
M4042	constanthasnovalue		Constant ' <target> ' has no value.
M4043	constant-variability-violation		Component <componentName> of variability constant has binding ' <bindingText> ' of higher variability <bindingVariability> .
M4043	constantvariabilityviolation		Component <compName> of variability constant has binding ' <target> ' of higher variability <varbNode> .
M4044	non-array-modification		Non-array modification ' <modText> ' for array component ' <componentName> ', possibly due to missing 'each'.
M4044	nonarraymodification		Non-array modification ' <fullMod> ' for array component, possibly due to missing 'each'.
M4045	modified-element-not-found-cst		Modified element <modName> not found in class <className> .
M4045	modifiedelementnotfound		Modified element <eText> not found in class <tText> .
M4046	constant-not-fixed		Constant ' <componentName> ' must be fixed but has 'fixed = false'
M4046	constantnotfixed		Constant declaration ' <target> ' must be fixed.
M4047	cardinality-invalid-context		cardinality may only be used in the condition of an if-statement/equation or an assert.
M4047	cardinalityinvalidcontext		Operator ' <target> ' may only be used in the condition of an if-statement/equation or an assert.
M4048	cardinality-expected-component		Expected <name> to be a component, but found class instead.
M4048	cardinalityexpectedcomponent		Expected <target> to be a component, but found class instead.
M4049	break-on-non-component		Invalid use of break on non-component ' <name> '.
M4049	breakonnoncomponent		Invalid use of break on non-component ' <ident> '.

Code	Rule	Severity	Message
M4050	break-component-invalid-type		Invalid use of break on component ' <name> ', component must be a model, block, or connector.
M4050	breakcomponentinvalidtype		Invalid use of break on component ' <ident> ', component must be a model, block, or connector.
M4051	break-connect-not-found		No matching element found for ' <connText> '.
M4051	breakconnectnotfound		No matching element found for ' <target> '.
M4052	redeclare-final-component		Redeclaration of final component <name> is not allowed.
M4052	redeclarefinalcomponent		Redeclaration of final component <eName> is not allowed.
M4053	redeclare-constant-component		Redeclaration of constant component <name> is not allowed.
M4053	redeclareconstantcomponent		Redeclaration of constant component <eName> is not allowed.
M4054	stream-unbalanced-connector		Invalid stream connector <connectorName> : A stream connector must have exactly one flow variable, this connector has <flowCount> flow variables.
M4054	streamunbalancedconnector		Invalid stream connector . <cName> : A stream connector must have exactly one flow variable, this connector has <fCount> flow variables.
M4055	cannot-connect-expandable-with-non-expandable		Cannot connect expandable connector <expName> with non-expandable connector <nonExpName> .
M4055	cannotconnectexpandablewithnonexpandable		Cannot connect expandable connector <expRef> with non-expandable connector <nonExpRef> .
M4056	duplicate-inherited-element-not-identical		Duplicate elements (due to inherited elements) not identical: first element is: <currText> second element is: <prevText>
M4057	expandable-connector-flow-element		Prefix 'flow' on component ' <compName> ' not allowed in class specialization 'expandable connector'.
M4057	expandableconnectorflowelement		Prefix 'flow' on component ' <compName> ' not allowed in class specialization 'expandable connector'.
M4058	recursive-definition		Declaration of element <elemName> causes recursive definition of class <className> .
M4059	max-recursion-depth-exceeded		The maximum recursion depth of <depth> was reached, probably due to mutually recursive definitions.
M4061	redeclare-non-inherited		Illegal redeclare of element <elementName> , no inherited element with that name exists.
M4061	redeclarenoninherited		Illegal redeclare of element <eName> , no inherited element with that name exists.
M4062	invalid-external-object		External object <className> contains invalid element ' <elemName> '.
M4063	class-extends-non-inherited		Base class targeted by class extends <className> not found in the inherited classes.
M4063	classextendsnoninherited		Invalid redeclaration of class <eName> , class extends only allowed on inherited classes.
M4064	protected-in-connector		Protected sections are not allowed in connector.
M4064	protectedinconnector		Protected sections are not allowed in <kindStr> .
M4065	equationinconnector		Equations are not allowed in connector.
M4065	equations-in-connector		Equations are not allowed in connector.
M4066	algorithm-in-connector		Algorithm sections are not allowed in connector.
M4066	algorithminconnector		Algorithm sections are not allowed in connector.
M4067	external-object-missing-constructor		External object <className> is missing a constructor.

Code	Rule	Severity	Message
M4068	redeclare-condition-attribute		Invalid redeclaration of <elementName>, a redeclare may not have a condition attribute.
M4068	redeclareconditionattribute		Invalid redeclaration of <eName>, a redeclare may not have a condition attribute.
M4069	external-object-return-restriction		Function <fnName> returns an external object, but the only function allowed to return this object is <extObjName>.constructor.
M4070	external-non-function		Class specialization violation: <className> is a <specialization>, which may not contain an external declaration.
M4070	externalnonfunction		Class specialization violation: <name> is a <kindStr>, which may not contain an external declaration.
M4071	external-invalid-language		'<lang>' is not a valid language for an external function.
M4072	external-invalid-arg-expr		Expression <expr> cannot be an external argument. Only identifiers, scalar constants, and size-expressions are allowed.
M4073	external-invalid-arg-size-const		Invalid external argument '<expr>', the dimension index must be a constant expression.
M4074	external-array-return-not-allowed		The lhs (result) of the external function declaration has array type (<typeStr>), but this is not allowed in the specification. You need to pass it as an input to the function (preferably also with a size()-expression to avoid out-of-bounds errors in the external call).
M4075	invalid-type-prefix-flow-nested		Invalid type prefix 'flow' on variable <varName>, due to existing type prefix 'flow'.
M4076	parameter-bound-contradiction		Contradictory bounds for parameter '<paramName>': lower bound '<lower>' exceeds upper bound '<upper>'.
M4076	parameterboundcontradiction		Contradictory bounds for parameter 'parameter': lower bound 'min' exceeds upper bound 'max'.
M4077	loop-termination-inconclusive		Loop over '<loopVar>' with condition '<condition>' cannot be proven to terminate.
M4078	terminate-in-function		terminate is not allowed in a function.
M4078	terminateinfunction		terminate is not allowed in a function.
M4079	return-outside-function		'return' may not be used outside function.
M4079	returnoutsidefunction		'return' may not be used outside function.
M4080	cyclic-constants-or-parameters		Cyclically dependent constants or parameters found in scope <scopeName>: { <cycleList> } (ignore with -d=ignoreCycles).
M4081	cyclic-function-components		Cyclically dependent function components found: { <cycleList> }
M4082	connector-non-parameter-subscript		Connector '<connRef>' has non-parameter subscript '<subscriptText>'.
M4083	enum-duplicate-literal		Enumeration has duplicate names: <dup> in list of names <list>.
M4084	enum-reserved-attribute-literal		Invalid use of reserved attribute name <name> as enumeration literal.
M4091	break-outside-loop		'break' must be inside a for or while loop, or enclose a connect equation.
M4091	breakoutsideloop		'break' may only be used in a while- or for-loop.
M4092	binding-dimension-mismatch		Type mismatch in binding '<componentName> = <bindingText>', expected array dimensions [<expectedDims>], got [<actualDims>].
M4093	unused-input-variable		Unused input variable <varName> in function <funcName>.
M4094	cardinality-in-condition-only		cardinality may only be used in the condition of an if-statement or if-equation.
M4095	conditional-expression-not-evaluated		The conditional expression <condText> could not be evaluated.

Code	Rule	Severity	Message
M4096	dimensions-must-be-parameter		Dimensions must be parameter or constant expression (in <callStr>).
M4097	called-function-partial		Called function ' <fnName> ' is partial.
M4098	class-specialization-violation		Class specialization violation: <className> <errorDetail> .
M4099	cannot-connect-undeclared-expandable-connectors		Cannot connect undeclared connectors <c1> with <c2> . At least one of them must be declared.
M4101	top-level-outer-element		The model can't be instantiated due to top-level outer element ' <elementName> ', it may only be used as part of a simulation model.
M4102	inner-not-found-existing-decl		An inner declaration for outer element ' <elementName> ' could not be found, and could not be automatically generated due to an existing declaration of that name.
M4103	missing-redeclare-keyword		Missing redeclare keyword on attempted redeclaration of class <className> .
M4104	invalid-redeclaration-class-as-component		Invalid redeclaration of class <className> as component.
M4105	duplicate-redeclare		<name> is already redeclared in this scope.
M4106	invalid-redeclaration-prefix		Invalid redeclaration ' <newPrefix> <name> ', original element is declared ' <origPrefix> '.
M4107	invalid-redeclaration-component-as-class		Invalid redeclaration of component <compName> as class.
M4108	redeclare-enum-not-subtype-generic		Redeclaration of enumeration ' <name> ' is not a subtype of the redeclared element (use enumeration(:) for a generic replaceable enumeration).
M4109	redeclare-enum-not-subtype		Redeclaration of enumeration ' <name> ' is not a subtype of the redeclared element.
M4110	invalid-type-prefix-conflict		Invalid type prefix ' <newPrefix> ' on component <compName> , due to existing type prefix ' <existingPrefix> '.
M4111	invalid-redeclaration-basic-type-attribute		Invalid redeclaration of <attrName> , attributes of basic types may not be redeclared.
M4112	extends-other-base-class		Found other base class for extends <baseName> after instantiating extends.
M4113	external-object-missing-destructor		External object <className> is missing a destructor.
M4114	invalid-connector-variability		Invalid variability <variability> on connector ' <compName> '.
M4115	external-object-not-replaceable		' <elemName> ' may not be replaceable.
M4116	operator-encapsulation		Operator <recordName> .' <opName> ' is not encapsulated.
M4117	operator-constructor-outputs		Operator <recordName> .'constructor' must have exactly one output.
M4118	operator-constructor-output-type		Output ' <outputName> ' in operator <recordName> .'constructor' must be of type <recordName> , got type <actualType> .
M4119	variable-in-package-not-constant		Variable <varName> in package <pkgName> is not constant.
M4120	ambiguous-operator-overload		Ambiguous matching overloaded operator functions found for <exprText> . Candidates are: <candidatesText>
M4121	no-matching-operator-function		No matching function found for <callExprText> . Candidates are: <candidatesText>
M4122	cannot-resolve-operator-type		Cannot resolve type of expression <exprText> . The operands have types <leftType> , <rightType> in component <NO_COMPONENT>.

`M5xxx` — Equations, algorithms & control-flow analysis

Code	Rule	Severity	Message
M5001	equation-type-mismatch		Type mismatch in equation <lhsExpanded> = <rhsExpanded> of type <lhsType> = <rhsType>.
M5001	equationtypemismatch		Type mismatch in equation <eqText2> of type <lName> = <rName>.
M5002	constrainedby-type-mismatch		Replaceable '<componentName>': default type '<defaultType>' is not compatible with constraining type '<constrainingType>'.
M5003	if-branch-type-mismatch		If-equation branch type mismatch: '<varName>' assigned type '<branchType>' but expected '<expectedType>'.
M5004	connect-flow-mismatch		Cannot connect flow component <flowComp> to non-flow component <nonFlowComp>.
M5004	connectflowmismatch		Cannot connect flow component <flowRef>.<elemIdent> to non-flow component <nonFlowRef>.<elemIdent>.
M5005	division-by-zero		Division by zero: <lhs> / 0.
M5005	divisionbyzero		Division by literal zero in '<target>'.
M5006	assignment-type-mismatch		Type mismatch in assignment in <target> := <source> of <targetType> := <sourceType>
M5006	assignmenttypemismatch		Type mismatch in assignment in <target> of <tName> := <vName>
M5007	for-iterator-not-1d		Iterator <iteratorName>, has type <shape>, but expected a 1D array expression.
M5007	foriteratornot1d		Iterator iterator, has type Real, but expected a 1D array expression.
M5008	assignment-to-constant		Trying to assign to constant component in <componentName> := <exprText>
M5008	assignmenttoconstant		Trying to assign to constant component <idNode>.
M5009	assignment-to-input		Trying to assign to input component '<componentName>'.
M5009	assignmenttoinput		Trying to assign to input component <idNode>.
M5010	homotopy-recommended		Equation contains steep nonlinearity '<term>' without homotopy. Consider wrapping with homotopy(actual, simplified).
M5011	assignment-to-parameter		Trying to assign to parameter component in <componentName> := <exprText>
M5012	assignment-to-iterator		Assignment to iterator '<iteratorName>'.
M5013	not-a-stream-variable		Operand '<operand>' to operator '<operator>' is not a stream variable.
M5013	notastreamvariable		Operand '<aText>' to operator '<fText>' is not a stream variable.
M5014	assignment-to-non-variable		Component '<componentName>' may not be assigned to due to class specialization '<specialization>'.
M5015	function-call-contains-subscripts		Function call <cref> contains subscripts.
M5016	tuple-subscript-not-allowed		Tuple expression can not be subscripted.
M5017	when-illegal-context		A when-statement may not be used inside a function or a while, if, or for-clause.
M5017	whenillegalcontext		A when-statement may not be used inside a function or a while, if, or for-clause.
M5018	reduction-invalid-expr-type		Invalid expression '<exprText>' of type <actualType> in <op> reduction, expected <expectedText>.
M5019	failed-to-deduce-dimension		Failed to deduce dimension <dimIdx> of <varName> due to missing binding equation.
M5020	output-not-definitely-assigned		Output variable '<varName>' in function '<funcName>' is not definitely assigned on all control flow paths.
M5021	uninitialized-variable-read		Variable '<varName>' is read before being definitely assigned.
M5022	unreachable-statement		Unreachable statement detected.

Code	Rule	Severity	Message
M5023	elsewhen-variable-mismatch		The same variables must be solved in elsewhen clause as in the when clause.
M5023	elsewhenvariablemismatch		The same variables must be solved in elsewhen clause as in the when clause.
M5024	reinit-type-mismatch		The first argument to reinit must be a subtype of Real, but <varName> has type <actualType>.
M5025	implicit-iterator-subscript-missing		Identifier <iterName> of implicit for iterator must be present as array subscript in the loop body.
M5026	implicit-iteration-dim-mismatch		Dimension <dim1> of <name1> and <dim2> of <name2> differs when trying to deduce implicit iteration range.
M5027	when-condition-not-discrete		When-condition ' <condText> ' is not a discrete-time expression.
M5028	when-if-branch-lhs-mismatch		The branches of an if-equation inside a when-equation must have the same set of component references on the left-hand side.
M5029	clocked-when-has-elsewhen		Clocked when equation can not contain elsewhen part.
M5030	clocked-when-branch-in-when		Clocked when branch in when equation.
M5031	invalid-when-lhs		Invalid left-hand side of when-equation: <lhsStr>.
M5032	when-conditional-expected-boolean		Type error in when conditional ' <condStr> '. Expected Boolean scalar or vector, got Clock.
M5033	reinit-not-continuous		The first argument to reinit must be a continuous time variable, but <varName> is <variability>.
M5034	eval-external-builtin		Internal error NFCeval.evalBuiltinCall: unimplemented case for <funcName>
M5035	argument-not-differentiable		Argument ' <argName> ' of der is not differentiable.
M5036	invalid-size-index		Invalid index <dim> in call to size of <arrName>, valid index interval is [1, <maxDim>].
M5037	function-no-named-parameter		Function <funcName> has no input parameter named <paramName>.
M5038	when-in-initial-equation		when-clause is not allowed in initial section.
M5039	size-first-arg-not-array		The first argument of size(<argText>) must be an array expression.
M5040	reinit-outside-when		Operator reinit may only be used in the body of a when equation.
M5041	reinit-in-algorithm		Operator reinit may not be used in an algorithm section (use translation flag --allowNonStandardModelica=reinitInAlgorithms to ignore).
M5042	reinit-arg1-not-variable		The first argument to reinit must be a variable of type Real or an array of such variables.
M5043	illegal-end-usage		'end' can not be used outside array subscripts.
M5044	reinit-arg2-type-mismatch		Type mismatch for positional argument 2 in reinit(= <argText>). The argument has type: <actualType> expected type: <expectedType>

M6xxx — Synchronous clocks, state machines & purity

Code	Rule	Severity	Message
M6001	mixed-clock-domains		Clocked expression '<exprText>' mixes signals from different clock domains.
M6001	mixedclockdomains		Mixed clock domains in expression '<target>' without explicit conversion operator.
M6002	sample-factor-not-positive		Sub-clock factor must be a positive integer.
M6002	samplefactornotpositive		Sample factor '<target>' must be a positive non-zero integer.
M6003	previous-outside-clocked		Operator 'previous()' may only be applied to clocked discrete-time variables in '<exprText>'.
M6003	previousoutsideclocked		previous() can only be called on clocked discrete variables, but '<target>' is unclocked.
M6004	hold-in-continuous		Operator 'hold()' converts a clocked signal to continuous-time and cannot be used inside a clocked partition: '<exprText>'.
M6004	holdincontinuous		hold() operator cannot be called in continuous equation section without boundary causality.
M6010	duplicate-initial-state		A state machine partition must have at most one initialState() call.
M6010	duplicateinitialstate		More than one state marked as initialState in state machine '<target>'.
M6020	impure-called-in-pure		Impure function '<callText>' cannot be called inside a pure function.
M6020	impurecalledinpure		Pure function cannot call impure function '<target>'.
M6021	impure-in-equation-section		Impure function '<callText>' may only be called inside a when-equation or when-statement.
M6021	impureinequationsection		Impure function '<target>' may only be called in algorithm sections or when equations.
M6040	break-connection-not-found		Connection '<connText>' in 'break' modifier does not exist in base class.

SysML v2 / KerML {#sysml2}

Systems architecture & requirements. **48 rules** · `@modelscript/sysml2` · Source: `languages/sysml2/src/errors.ts` · Language guide

`S1xxx` — Syntax, parsing & naming conventions

Code	Rule	Severity	Message
S1001	parse-error		Parse error: <detail>.
S1002	parse-missing		Expected token missing: '<expected>'.
S1003	definition-naming		Definition '<name>' should start with an uppercase letter
S1004	usage-naming		Usage '<name>' should start with a lowercase letter
S1005	package-naming		Package '<name>' should start with an uppercase letter

`S2xxx` — Scoping & name resolution

Code	Rule	Severity	Message
S2001	duplicate-member		Duplicate member '<name>' in '<scope>'
S2002	symbol-not-found		Symbol '<name>' not found in scope.
S2003	allocation-target-unresolved		Allocation target '<target>' could not be resolved
S2003	unresolved-target		Allocation target '<target>' could not be resolved
S2004	allocation-source-unresolved		Allocation source '<source>' could not be resolved
S2004	unresolved-source		Allocation source '<source>' could not be resolved
S2005	unresolved-import		Import '<pkg>' could not be resolved.
S2006	unresolved-transition-state		Transition state '<stateName>' could not be resolved in enclosing state machine.

S3xxx — Type system & static checking

Code	Rule	Severity	Message
S3001	type-mismatch		Type mismatch: cannot assign value of type '<actualType>' to feature '<featureName>' with declared type '<declaredType>'.
S3002	incompatible-port-connection		Incompatible port connection between '<p1>' (<t1>) and '<p2>' (<t2>).
S3003	dimensional-mismatch		Dimensional mismatch: cannot assign value of dimension '<actualDim>' to feature '<featureName>' with declared dimension '<targetDim>'.
S3004	invalid-multiplicity-bounds		Invalid multiplicity bounds: lower bound (<lower>) cannot be greater than upper bound (<upper>).
S3005	target-not-requirement		Target '<target>' is not a Requirement
S3006	verify-target-not-a-requirement		Verify target '<target>' is not a Requirement
S3007	dimensional-connection-mismatch		Dimensional mismatch in connection between '<sourceName>' (<sourceDim>) and '<targetName>' (<targetDim>).

S4xxx — Structural & metamodel semantics

Code	Rule	Severity	Message
S4001	circular-specialization		Circular specialization: '<name>' transitively specializes itself
S4002	circular-satisfaction		Circular satisfaction: '<name>' transitively satisfies itself
S4003	empty-definition		Definition '<name>' has no members
S4004	abstract-requirement-violation		Cannot make an existing concrete requirement abstract.
S4005	ontological-inconsistency		Ontological inconsistency detected in '<name>' <detail>
S4006	missing-subject		Requirement '<name>' should have a specified subject or satisfy relation
S4007	empty-package		Package '<name>' has no members
S4008	requirement-without-constraint		Requirement '<name>' has no constraints or textual documentation
S4009	empty-verification-case		Verification case '<name>' has no verify requirements or sub-actions
S4010	missing-verification-objective		Verification case '<name>' has no verification objective or verify statement
S4011	unverified-requirement		Requirement '<name>' is not verified by any verification case in package '<pkgName>'

S5xxx — Behavior, state machines & formal verification

Code	Rule	Severity	Message
S5001	constraint-violated		Constraint '<name>' evaluates to false
S5002	requirement-failed		Requirement '<name>' has constraint(s) that evaluate to false
S5003	broken-interface-contract		Broken interface contract on '<name>': <reason>
S5003	interface-contract-broken		Broken interface contract on '<name>': <reason>
S5004	activity-cfa-unreachable		<detail>
S5005	activity-soundness-violation		<detail>
S5006	definite-assignment-violation		Variable '<name>' may be used before definite assignment
S5007	statemachine-nondeterministic		Non-deterministic guards detected: <detail>
S5008	statemachine-deadlock		Deadlock gap detected in transition guards: <detail>
S5009	satisfying-violated-requirement		Part '<partName>' cannot satisfy requirement '<reqName>': <violations>
S5010	conflicting-requirements		Conflicting requirements in '<pkgName>': <conflictNames> cannot be simultaneously satisfied. <witnessStr>
S5011	interprocedural-contract-violation		<detail>
S5012	loop-non-terminating		<detail>
S5013	decision-table-non-exhaustive		Decision node '<decideName>' is not exhaustive: <detail>
S5014	decision-table-nondeterministic		Decision node '<decideName>' has overlapping branch guards: <detail>
S5015	decision-table-dead-branch		Decision node '<decideName>' has dead branch(es): <detail>
S5016	activity-safety-violation		<detail>

ReqIF {#reqif}

Requirements interchange & contract entailment. **10 rules** · @modelscript/reqif · Source: languages/reqif/src/lints/rules.ts · Language guide

Code	Rule	Severity	Message
M7001	dangling-requirement-relation		Requirement relation '<relId>' references missing <role> requirement '<missingId>'.
M7002	type-attribute-mismatch		Attribute '<attrName>' expected type '<expectedType>', but got value '<actualValue>'.
M7003	circular-requirement-hierarchy		Circular requirement hierarchy detected at '<reqId>' along cycle path [<cyclePath>].
M7004	unverified-leaf-requirement		Leaf requirement '<reqId>' has no outgoing 'verifies' or 'satisfies' relation.
M7005	unbounded-formal-contract		Requirement '<reqId>' is marked safety/functional but lacks a bounded mathematical contract.
M7006	duplicate-spec-object-id		Duplicate requirement identifier '<reqId>' declared in specification.
M7007	missing-mandatory-attribute		Requirement '<reqId>' is missing mandatory attribute '<attrName>'.
M7008	empty-specification		Specification '<specId>' contains no requirement hierarchies.
M7009	unresolved-type-reference		Element '<elementId>' references undefined type '<typeRef>'.
M7010	invalid-contract-comparator		Requirement '<reqId>' specifies invalid contract comparator '<comp>'.

SSP {#ssp}

FMU system structure & parameterization. **12 rules** · `@modelscript/ssp` · Source: `languages/ssp/src/errors.ts` · Language guide

Code	Rule	Severity	Message
S7001	SSP_EMPTY_CONNECTOR_NAME		Connector in <owner> has an empty name.
S7002	SSP_DUPLICATE_CONNECTOR		Duplicate connector '<name>' declared in <owner>.
S7003	SSP_INVALID_CONNECTOR_KIND		Invalid connector kind '<kind>'. Must be input, output, inout, parameter, or calculatedParameter.
S7004	SSP_EMPTY_COMPONENT_NAME		Component declared with an empty name attribute.
S7005	SSP_DUPLICATE_COMPONENT		Duplicate component '<name>' in system.
S7006	SSP_EMPTY_SOURCE		Component '<comp>' has an empty or missing source FMU URI.
S7007	SSP_UNDEFINED_COMPONENT		Referenced component '<comp>' is not defined in system elements.
S7008	SSP_UNDEFINED_CONNECTOR		Referenced connector '<conn>' not found on component '<comp>'.
S7009	SSP_CONNECTOR_TYPE_MISMATCH		Connection type mismatch: '<startType>' cannot connect to '<endType>'.
S7010	SSP_CONNECTOR_CAUSALITY_CONFLICT		Connection violates causal flow rules.
S7011	SSP_CYCLIC_CONNECTION		Connection creates an algebraic loop or cyclic dependency without delay.
S7012	SSP_INVALID_EXPERIMENT_BOUNDS		DefaultExperiment stopTime must be strictly greater than startTime.

OWL2 {#owl2}

Ontologies & description-logic reasoning. **24 rules** · `@modelscript/owl2` · Source: `languages/owl2/src/errors.ts` · Language guide

Code	Rule	Severity	Message
O1001	parse-error		Syntax error near '<snippet>'.
O1002	undeclared-prefix		Prefix '<prefix>:' used in abbreviated IRI '<iri>' is not declared.
O1003	duplicate-prefix		Prefix '<prefix>:' is declared multiple times in the ontology header.
O1004	invalid-iri		Invalid or malformed IRI '<iri>'.
O1005	insufficient-arity		OWL 2 construct '<construct>' requires at least <minArity> arguments, but got <actualArity>.
O2001	undeclared-entity		Entity '<iri>' is used as a <role> without an explicit Declaration axiom.
O2002	duplicate-declaration		Duplicate Declaration(<kind>(<iri>)) in ontology.
O2003	reserved-vocabulary-declaration		Cannot declare or redefine built-in OWL 2 / RDF / RDFS / XSD vocabulary IRI '<iri>'.
O3001	illegal-property-punning		Illegal OWL 2 DL punning: '<iri>' is used as both '<kindA>' and '<kindB>' (ObjectProperty, DataProperty, and AnnotationProperty vocabularies must be disjoint).
O3002	expected-object-property		Expected an ObjectProperty in '<construct>', but '<iri>' is declared as a DataProperty.
O3003	expected-data-property		Expected a DataProperty in '<construct>', but '<iri>' is declared as an ObjectProperty.
O3004	expected-class-expression		Expected a Class expression in '<construct>', but '<iri>' is a Datatype.
O3005	invalid-datatype		Datatype '<datatypeIri>' is not in the standard OWL 2 datatype map.
O3006	datatype-literal-mismatch		Lexical value '<literal>' is not a valid literal for datatype '<datatypeIri>'.
O4001	non-simple-property-restriction		OWL 2 DL global restriction violation: non-simple (transitive or composite) property '<propertyIri>' cannot be used in '<construct>'.
O4002	contradictory-property-characteristics		Object property '<propertyIri>' has contradictory characteristics '<charA>' and '<charB>'.
O4003	property-hierarchy-cycle		Cycle detected in property hierarchy: '<cycle>'.
O4004	class-hierarchy-cycle		SubClassOf cycle detected (equivalent classes): '<cycle>'.
O4005	self-referential-axiom		Redundant or contradictory self-referential axiom '<axiomType>' on '<iri>'.
O5001	functional-property-clash		Functional/InverseFunctional property '<propertyIri>' violation: <detail>.
O5002	negative-assertion-clash		Negative property assertion on '<propertyIri>' for '<subjectIri>' clashes with asserted or inferred value '<target>'.
O5003	unsatisfiable-class		Class '<classIri>' is unsatisfiable (equivalent to owl:Nothing): <reason>.
O5004	ontology-inconsistent		Ontology is logically inconsistent: <explanation>
O6001	profile-el-violation		Construct '<construct>' is outside the OWL 2 EL profile.

STEP (ISO 10303) {#step}

B-Rep CAD assemblies & AP242 PMI. **8 rules** · `@modelscript/step` · Source: `languages/step/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M1001	step-syntax-error		STEP file syntax error: <detail>.
M1002	missing-header-section		STEP exchange file is missing mandatory 'HEADER;' section.
M1003	missing-schema-declaration		STEP header lacks a recognized 'FILE_SCHEMA' declaration (e.g. AP203, AP214, AP242).
M1004	missing-data-section		STEP exchange file is missing mandatory 'DATA;' section.
M1005	unresolved-entity-reference		Referenced STEP entity '# <entityId> ' is not defined in the DATA section.
M1006	duplicate-entity-id		Entity ID '# <entityId> ' is declared more than once in the DATA section.
M1007	empty-data-section		STEP DATA section contains no entity records.
M1008	unsupported-ap-schema		Application Protocol schema ' <schema> ' may not support complete PMI or B-Rep extraction.

OpenSCAD {#scad}

Parametric constructive solid geometry. **8 rules** · @modelscript/scad · Source: languages/scad/src/errors.ts · Language guide

Code	Rule	Severity	Message
K1001	SCAD_UNDEFINED_VARIABLE		Undefined variable ' <name> '.
K1002	SCAD_UNDEFINED_FUNCTION		Undefined function ' <name> '.
K1003	SCAD_UNDEFINED_MODULE		Undefined module ' <name> '.
K1004	SCAD_DIVISION_BY_ZERO		Division by zero in expression.
K1005	SCAD_INVALID_DIMENSION		Invalid non-positive dimension (<val>) in ' <param> ': dimensions must be > 0.
K1006	SCAD_ARGUMENT_COUNT		Too many arguments to ' <name> ': expected at most <expected>, got <actual>.
K1007	SCAD_UNKNOWN_PARAMETER		Unknown parameter ' <param> ' in call to ' <callable> '.
K1008	SCAD_DUPLICATE_DECLARATION		Variable ' <name> ' is assigned multiple times in the same scope.

CFD (SU2 / OpenFOAM) {#cfd}

Computational fluid dynamics meshes. **8 rules** · @modelscript/cfd · Source: languages/cfd/src/errors.ts · Language guide

Code	Rule	Severity	Message
C1001	cfl-stability-violation		Courant-Friedrichs-Lewy stability number ' <cfl> ' exceeds recommended maximum limit for explicit scheme.
C1002	unresolved-mesh-file		Referenced grid mesh file ' <mesh> ' cannot be located or is invalid.
C1003	missing-inlet-boundary		CFD domain ' <name> ' lacks a declared inlet boundary condition or stagnation state.
C1004	missing-outlet-boundary		CFD domain ' <name> ' lacks a declared outlet boundary condition or pressure exit.
C1005	unphysical-freestream-mach		Freestream Mach number ' <mach> ' is non-physical or outside valid range [0, 25].
C1006	unphysical-reynolds-number		Freestream Reynolds number ' <re> ' must be strictly positive.
C1007	negative-surface-area		Aerodynamic reference area ' <area> ' must be strictly positive.
C1008	turbulent-viscosity-ratio		Turbulent eddy viscosity ratio ' <ratio> ' is non-physical.

FEA (CalculiX / Abaqus / NASTRAN) {#fea}

Finite-element structural analysis. **8 rules** · `@modelscript/fea` · Source: `languages/fea/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`F1001`	`zero-elastic-modulus`		Material '`<mat>`' has non-positive Young's modulus '`<val>`'.
`F1002`	`poisson-ratio-limit`		Poisson's ratio '`<val>`' for material '`<mat>`' violates physical stability bounds (-1.0 < nu < 0.5).
`F1003`	`unconstrained-rigid-body`		Structural mesh '`<name>`' has only '`<count>`' constrained boundary nodes; rigid-body motion may be unconstrained.
`F1004`	`empty-mesh`		FEA deck '`<name>`' does not contain any defined finite elements.
`F1005`	`missing-material-definition`		Element set '`<elset>`' lacks an assigned material specification or section.
`F1006`	`degenerate-element-nodes`		Element '`<elemId>`' of type '`<type>`' contains duplicate or degenerate node references.
`F1007`	`negative-yield-stress`		Plastic yield strength '`<val>`' for material '`<mat>`' must be strictly positive.
`F1008`	`negative-density`		Mass density '`<val>`' for material '`<mat>`' must be strictly positive for dynamic analysis.

C {#c}

Numerical & embedded C integration. **10 rules** · `@modelscript/c` · Source: `languages/c/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`M8401`	`uninitialized-variable`		Local variable '`<varName>`' may be read before being initialized.
`M8402`	`constant-division-by-zero`		Division or modulo by constant zero detected in arithmetic expression.
`M8403`	`array-bounds-overflow`		Static array subscript '`<arrName>`[`<index>`]' exceeds declared dimension bound of `<bound>`.
`M8404`	`unreachable-code`		Unreachable statements detected following unconditional 'return', 'break', or 'goto'.
`M8405`	`assignment-in-conditional`		Assignment operator '=' used inside conditional expression; did you mean '=='?
`M8406`	`algebraic-dependency-cycle`		Procedural algebraic dependency cycle detected without state lag: '`<cycle>`'.
`M8407`	`null-pointer-dereference`		Dereference of statically known NULL pointer in expression '`<expr>`'.
`M8408`	`function-prototype-mismatch`		Function '`<fnName>`' signature conflicts with declaration: expected '`<expected>`', got '`<actual>`'.
`M8409`	`missing-return-value`		Non-void function '`<fnName>`' with return type '`<returnType>`' reaches end of body without returning a value.
`M8410`	`implicit-lossy-conversion`		Implicit floating-point to integer conversion from '`<fromType>`' to '`<toType>`' may cause lossy truncation.

Fortran {#fortran}

Legacy scientific subroutines. **10 rules** · `@modelscript/fortran` · Source: `languages/fortran/src/lints/index.ts` · Language guide

Code	Rule	Severity	Message
M4070	undeclared-variable		Variable '<varName>' referenced without declaration under strict typing.
M4071	uninitialized-variable		Variable '<varName>' may be used before being assigned a value.
M4072	constant-division-by-zero		Division by constant zero detected in arithmetic expression.
M4073	storage-bounds-overflow		Storage array index <arrayName>(<index>) exceeds allocated capacity of <maxCapacity>.
M4074	dead-code-after-return		Unreachable statements detected following unconditional RETURN.
M4075	do-loop-index-mutation		DO-loop index variable '<indexVar>' is modified within its loop body.
M4076	procedural-algebraic-loop		Algebraic cycle detected among procedural assignments without time-step lag: <cycleStr>.
M4077	common-block-mismatch		COMMON block '/<blockName>/' definition conflict: <reason>.
M4078	array-dimension-underflow		Array '<arrName>' declared with non-positive dimension size (<dim> <= 0).
M4079	obsolete-arithmetic-if		Three-way arithmetic IF is obsolescent; prefer structured IF/THEN/ELSE.

JSBSim {#jsbsim}

6-DOF flight dynamics. **10 rules** · @modelscript/jsbsim · Source: languages/jsbsim/src/errors.ts · Language guide

Code	Rule	Severity	Message
M8901	missing-aircraft-name		JSBSim <fdm_config> element is missing required 'name' attribute.
M8902	non-positive-reference-area		Wing reference area (<val>) must be strictly positive.
M8903	non-positive-wingspan-chord		Wing metric '<param>' (<val>) must be strictly positive.
M8904	non-positive-empty-weight		Aircraft empty weight (<val>) must be strictly positive.
M8905	invalid-inertia-tensor		Moments of inertia tensor is invalid: <reason>.
M8906	missing-aerodynamics-section		Aircraft model does not define any aerodynamic force/moment coefficients in <aerodynamics>.
M8907	missing-landing-gear		Aircraft model defines no landing gear bogies in <ground_reactions>; ground interaction is disabled.
M8908	negative-engine-thrust		Engine '<engineName>' specifies negative maximum thrust (<val>).
M8909	undefined-fcs-input		FCS channel '<channelName>' references input property '<inputProperty>' not bound in flight control.
M8910	unrealistic-cg-location		Center of gravity (<cgX>) is significantly far from aerodynamic center (<aeroX>); aircraft may have extreme pitch instability.

NASA ROCETS {#rocets}

Liquid rocket engine transients. **14 rules** · @modelscript/rocets · Source: languages/rocets/src/lints/index.ts · Language guide

Code	Rule	Severity	Message
M8001	unconnected-fluid-port		Fluid port '<comp>.<port>' is not connected in the propellant flow circuit.
M8002	fluid-type-mismatch		Incompatible fluid stream connection '<conn>': cannot mix '<fromFluid>' into '<toFluid>' without a combustion chamber.
M8003	shaft-power-imbalance		Turbopump shaft '<shaftId>' power imbalance: <reason>.
M8004	unbalanced-flow-split		Mass conservation failure at junction '<junct>': inlet and outlet flow mismatch exceeds tolerance (|\u0394m| = <value> kg/s).
M8005	pump-cavitation-risk		Pump '<pumpId>' inlet suction pressure is below required NPSH margin (margin: <value> kPa). Cavitation risk.
M8006	negative-chamber-pressure		Non-physical pressure at '<comp>': computed pressure is <val> Pa (must be > 0).
M8007	combustion-temperature-limit		Gas temperature at '<comp>' (<value> K) exceeds throat thermal structural limit (<limitK> K).
M8008	invalid-valve-schedule		Invalid valve actuation profile for '<valveId>': <reason>.
M8009	choked-flow-inconsistency		Supersonic nozzle expansion at '<comp>' is not choked (pressure ratio Pc/Pa = <value> < 1.89).
M8010	user-fortran-routine-error		Fortran user routine in component '<comp>' has syntax or semantic error: <msg>.
M8011	extreme-mixture-ratio		Engine mixture ratio (O/F = <value>) at '<comp>' is outside stable combustion limits.
M8012	missing-fluid-table		Propellant fluid table for '<fluid>' is not loaded in the session.
M8013	turbine-overspeed-hazard		Turbopump shaft '<shaft>' speed (<value> RPM) exceeds maximum burst rating (<maxRpm> RPM).
M8014	closed-flow-circuit		Pump '<pumpId>' discharge is completely blocked with no open valve or recirculation path.

CCSDS {#ccsds}

Astrodynamics & spacecraft telemetry. **8 rules** · @modelscript/ccsds · Source: languages/ccsds/src/lints/rules.ts · Language guide

Code	Rule	Severity	Message
M8201	sub-orbital-impact-trajectory		Sub-orbital pericenter radius <rp> km is below Earth radius <earthRadius> km, resulting in atmospheric re-entry or impact.
M8202	telemetry-bit-packing-overlap		Telemetry bit packing overlap in container '<containerName>': entry '<entryName>' starts at bit <bitOffset> before previous entry '<prevEntry>' ends at bit <endBit>.
M8203	hyperbolic-unbounded-orbit		Orbit eccentricity <eccentricity> indicates a hyperbolic or parabolic trajectory.
M8204	unrecognized-reference-frame		Unrecognized or non-inertial reference frame '<refFrame>'.
M8205	missing-epoch-timestamp		Ephemeris point missing valid timestamp at index <index>.
M8206	non-monotonic-ephemeris-epochs		Ephemeris timestamp '<epoch>' is not strictly greater than previous timestamp '<prevEpoch>'.
M8207	duplicate-parameter-name		Duplicate telemetry parameter declaration for '<paramName>'.
M8208	negative-spacecraft-mass		Spacecraft mass <massKg> kg must be strictly positive.

EASY5 {#easy5}

Dynamic simulation & linear control. **12 rules** · `@modelscript/easy5` · Source: `languages/easy5/src/lints/index.ts` · Language guide

Code	Rule	Severity	Message
`M7001`	`unconnected-input-port`		Input port '`<comp>`.`<port>`' has no incoming connection.
`M7002`	`csmf-algebraic-loop`		Direct feedthrough algebraic loop detected with no continuous state or delay: `<value>`.
`M7003`	`duplicate-component-id`		Duplicate component identifier '`<id>`' in model.
`M7004`	`uninitialized-state`		Continuous state in component '`<comp>`' defaults to 0.0 with no explicit S0 specified.
`M7005`	`improper-transfer-function`		Transfer function in component '`<comp>`' is improper: numerator degree (`<degNum>`) exceeds denominator degree (`<degDen>`).
`M7006`	`parameter-out-of-bounds`		Parameter '`<param>`' in component '`<comp>`' is out of valid bounds: `<reason>`.
`M7007`	`port-dimension-mismatch`		Connection width mismatch from '`<from>`' (dim `<dim1>`) to '`<to>`' (dim `<dim2>`).
`M7008`	`state-index-out-of-bounds`		State index S(`<idx>`) in component '`<comp>`' exceeds allocated continuous states (`<max>`).
`M7009`	`sample-rate-mismatch`		Sample rate mismatch between '`<from>`' (`<rate1>`s) and '`<to>`' (`<rate2>`s) without rate transition block.
`M7010`	`user-fortran-syntax-error`		User Fortran subroutine in '`<comp>`' has defect: `<detail>`
`M7011`	`unstable-linear-pole`		Component '`<comp>`' has open-loop pole in right-half plane (Re = +`<value>`), indicating open-loop instability.
`M7012`	`table-extrapolation-hazard`		Component '`<comp>`' references table '`<tableName>`' without bounded extrapolation protection.

AUTOSAR {#autosar}

Classic & Adaptive software architecture. **8 rules** · `@modelscript/autosar` · Source: `languages/autosar/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`M7401`	`unconnected-mandatory-port`		Required port '`<portName>`' on component prototype '`<compName>`' is not connected by any assembly connector.
`M7402`	`interface-mismatch`		Connector interface mismatch: provider port '`<pPort>`' uses interface '`<pIf>`', but requirer port '`<rPort>`' expects '`<rIf>`'.
`M7403`	`invalid-runnable-period`		Timing event for runnable '`<symbol>`' defines non-positive period (`<period>`s).
`M7404`	`duplicate-short-name`		Duplicate SHORT-NAME '`<name>`' detected in package '`<pkg>`'.
`M7405`	`invalid-data-type-mapping`		Invalid data type mapping for '`<dtName>`': `<reason>`.
`M7406`	`circular-composition-dependency`		Circular software component composition dependency detected: `<path>`.
`M7407`	`missing-software-address-method`		Variable data prototype '`<variable>`' lacks a declared SwAddrMethod memory section mapping.
`M7408`	`incompatible-port-interface-direction`		Port '`<portName>`' interface direction violation: `<reason>`.

MATLAB Simulink {#simulink}

Block diagrams & control systems. **10 rules** · `@modelscript/simulink` · Source: `languages/simulink/src/lints/rules.ts` · Language guide

Code	Rule	Severity	Message
M7201	unconnected-input-port		Input port <portNumber> on block '<blockName>' (<blockType>) is unconnected. All input ports must be driven or terminated with a Ground block.
M7202	direct-feedthrough-algebraic-loop		Direct-feedthrough algebraic loop detected across blocks: [<loopNames>]. Control loops with direct algebraic feedthrough must contain at least one state delay (Integrator or UnitDelay) or will require an iterative non-linear solver.
M7203	unresolved-subsystem-link		SubSystem block '<blockName>' has no defined sub-diagram or unresolved library reference.
M7204	port-index-out-of-bounds		Block '<blockName>' has invalid port index <portNumber>. Port numbers must be positive integers (1, 2, ...).
M7205	multirate-sample-time-conflict		Block '<blockName>' has invalid SampleTime <sampleTime>. SampleTime must be >= 0 (discrete) or -1 (inherited).
M7206	non-numeric-parameter		Block '<blockName>' parameter '<paramName>' contains non-finite value <value>. Parameters must evaluate to finite Real numbers.
M7207	invalid-bus-selector		BusSelector block '<blockName>' references signal '<signalName>' absent from the incoming bus schema from '<sourceBlock>'.
M7208	duplicate-block-name		Duplicate block name '<blockName>' found in subsystem scope '<systemName>'. Block names within a system must be unique.
M7209	divide-by-zero-risk		Product/Divide block '<blockName>' performs division operations with inputs '<inputs>'; ensure denominator signal is protected against division by zero.
M7210	missing-integrator-initial-condition		Integrator block '<blockName>' does not specify an explicit 'InitialCondition'; defaulting to 0.0.

GT-SUITE {#gtsuite}

Engine, powertrain & thermal flow. **15 rules** · `@modelscript/gtsuite` · Source: `languages/gtsuite/src/lints/index.ts` · Language guide

Code	Rule	Severity	Message
M9001	cfl-acoustic-violation		Pipe '<pipeId>' cell length violates acoustic CFL condition: simulation dt (<value> \xB5s) exceeds stability limit (<value> \xB5s).
M9002	compressor-surge-hazard		Compressor '<compId>' operating point (PR = <value>, \u1E41 = <value> kg/s) falls into the aerodynamic surge region.
M9003	compressor-choke-limit		Compressor '<compId>' mass flow (<value> kg/s) exceeds maximum choke/stonewall sonic flow capacity.
M9004	valve-profile-overlap-error		Valve '<valveId>' kinematic profile defect: <reason>.
M9005	compression-ratio-invalid		Cylinder '<cylId>' compression ratio (<value>) is physically unrealistic (must be between 6.0 and 26.0).
M9006	unconnected-plenum-node		Flow split or plenum '<plenumId>' has no connected flow paths or boundary conditions.
M9007	discharge-coefficient-out-of-bounds		Valve or orifice '<valveId>' discharge coefficient (<value>) is non-physical (must satisfy 0.0 < Cd <= 1.0).
M9008	knock-margin-deficit		Cylinder '<cylId>' operates in severe autoignition/knock hazard (CR = <value>, ignition anchor = <value> \xB0 CA BTDC).
M9009	missing-boundary-pressure		Environment node '<envId>' has invalid or uninitialized static pressure (<= 0 Pa).
M9010	turbine-expansion-ratio-excess		Turbine '<turbId>' expansion ratio (<value>) exceeds single-stage choked limit (> 4.5).
M9011	intercooler-effectiveness-invalid		Intercooler '<icId>' thermal effectiveness (<value>) violates 2nd Law of Thermodynamics (must satisfy 0.0 < eff <= 1.0).
M9012	air-fuel-equivalence-out-of-bounds		Air-fuel equivalence ratio lambda (<value>) is outside combustible flammability limits (0.50 <= lambda <= 2.50).
M9013	negative-pipe-dimensions		Pipe '<pipeId>' has non-positive <dim> (<value> m).
M9014	injector-timing-conflict		Injector '<injId>' start of injection (<value> \xB0 CA) falls outside valid cycle window (-360\xB0 to +720\xB0).
M9015	isolated-shaft-inertia		Turbocharger shaft '<shaftId>' has no linked compressor or turbine rotating components.

ASAM OpenDRIVE {#opendrive}

Road network geometry. **8 rules** · `@modelscript/opendrive` · Source: `languages/opendrive/src/lints/rules.ts` · Language guide

Code	Rule	Severity	Message
M7801	road-tangency-discontinuity		Tangency discontinuity between geometry segments in road '<roadId>' at s=<s>: position gap <value> m, heading gap <value> rad.
M7802	unlinked-lane-boundary		Unlinked lane connection: lane '<laneId>' in road '<roadId>' links to non-existent <dir> lane '<targetId>'.
M7803	clothoid-curvature-divergence		Clothoid curvature rate of change in road '<roadId>' at s=<s> diverges (<value> 1/m^2).
M7804	overlapping-junction-paths		Connecting path in junction '<junctionId>' between road '<road1>' and '<road2>' conflicts without priority control.
M7805	elevation-gradient-extreme		Extreme elevation gradient in road '<roadId>' at s=<s>: slope <value> % exceeds standard 15% safety threshold.
M7806	negative-lane-width		Negative or zero lane width in lane '<laneId>' of road '<roadId>' at sOffset=<sOffset> m.
M7807	duplicate-road-id		Duplicate road ID '<roadId>' detected in roadway network.
M7808	missing-center-lane		Lane section in road '<roadId>' at s=<s> is missing mandatory center lane (ID 0).

ASAM OpenSCENARIO {#openscenario}

Driving scenarios & RSS safety margins. **10 rules** · `@modelscript/openscenario` · Source: `languages/openscenario/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8501	dangling-entity-reference		Entity '<entityName>' referenced in '<contextName>' is not defined in Entities catalog.
M8502	unsatisfiable-trigger		Trigger condition '<conditionName>' is unsatisfiable: <reason>.
M8503	unreachable-road-position		Road position specifies invalid road ID '<roadId>' or lane index '<laneId>'.
M8504	cyclic-event-trigger		Circular trigger condition detected between event '<eventA>' and event '<eventB>'.
M8505	missing-ego-entity		Scenario does not define an autonomous vehicle under test ('ego' vehicle).
M8506	negative-speed-target		Speed action '<actionName>' specifies negative target velocity '<speed>' m/s.
M8507	excessive-deceleration		Action demands deceleration '<decel>' m/s² for '<entityName>', exceeding vehicle limit '<limit>' m/s².
M8508	imminent-collision-initial-state		Initial positions for '<entityA>' and '<entityB>' violate collision envelope (distance: <value> m).
M8509	empty-maneuver-group		Maneuver group '<groupName>' specifies no actors or no maneuvers.
M8510	invalid-condition-delay		Trigger condition '<condName>' specifies negative activation delay '<delay>' s.

GENSYS {#gensys}

Railway vehicle dynamics & track. **6 rules** · `@modelscript/gensys` · Source: `languages/gensys/src/errors.ts` · Language guide

Code	Rule	Severity	Message
G1001	invalid-body-mass		Body '<name>' has non-positive mass (<mass> kg).
G1002	non-positive-inertia		Body '<name>' has non-positive principal inertia tensor components.
G1003	dangling-body1-reference		Coupling '<name>' references unknown body1 '<body1>'.
G1004	dangling-body2-reference		Coupling '<name>' references unknown body2 '<body2>'.
G1005	invalid-track-gauge		Track specification defines non-positive or unphysical gauge (<gauge> m).
G1006	unclosed-rail-profile		Wheel-rail contact profile '<profileName>' curve has discontinuous slope or is unclosed.

SIMPACK {#simpack}

Spatial multibody dynamics. **6 rules** · `@modelscript/simpack` · Source: `languages/simpack/src/errors.ts` · Language guide

Code	Rule	Severity	Message
K7001	invalid-body-mass		Body '<id>' has non-positive mass (<mass> kg).
K7002	dangling-body-reference		Marker '<markerId>' references unknown parent body '<bodyId>'.
K7003	unknown-from-marker		Joint '<jointId>' references unknown from-marker '<marker>'.
K7004	unknown-to-marker		Joint '<jointId>' references unknown to-marker '<marker>'.
K7005	overconstrained-mobility		Mechanism is overconstrained according to Grübler-Kutzbach criterion (Mobility = <mobility> < 0).
K7006	baumgarte-divergence		Joint '<jointId>' Baumgarte stabilization parameter '<param>' is non-positive or divergent (<val>).

PSCAD / EMTDC {#pscad}

Electromagnetic transients (EMT). **17 rules** · `@modelscript/pscad` · Source: `languages/pscad/src/lints/index.ts` · Language guide

Code	Rule	Severity	Message
`M6001`	`floating-electrical-node`		Electrical node '`<nodeName>`' is floating with no closed connections.
`M6002`	`transformer-winding-mismatch`		Transformer '`<xfmrName>`' `<side>` rating (`<ratedKv>` kV) does not match connected bus base (`<busKv>` kV).
`M6003`	`invalid-parameter-bounds`		Component '`<elemName>`' parameter '`<paramName>`' = `<val>` is invalid: `<reason>`.
`M6004`	`weak-grid-warning`		Point of Interconnection '`<poiBus>`' has low Short-Circuit Ratio (SCR = `<value>` < 1.5). IBR control stability risks detected.
`M6005`	`csmf-unresolved-input`		CSMF control block '`<blockId>`' references unresolved upstream signal block '`<missingInputId>`'.
`M6006`	`line-travel-time-underflow`		Transmission line '`<lineId>`' wave propagation delay (`<value>` us) is smaller than simulation time step (`<value>` us). Reduce time step or use a lumped pi-section model.
`M6007`	`sync-machine-reactance-order`		Synchronous machine '`<machineId>`' violates physical d-axis reactance hierarchy (Xd=`<xd>` >= Xd'=`<xdPrime>` >= Xd''=`<xdSub>` pu required).
`M6008`	`mov-arrester-under-rated`		MOV surge arrester '`<arresterId>`' reference voltage (`<value>` kV) is below connected bus '`<busName>`' nominal phase-to-ground peak voltage (`<value>` kV), causing continuous thermal conduction.
`M6009`	`duplicate-component-id`		Duplicate component or bus ID '`<id>`' declared in schematic module '`<modName>`'.
`M6010`	`bus-voltage-conflict`		Conductive net shorts bus '`<busA>`' (`<kvA>` kV) directly to bus '`<busB>`' (`<kvB>` kV) without an intervening transformer.
`M6011`	`port-domain-mismatch`		Wire '`<wireId>`' connects control signal port '`<sigPort>`' directly to 3-phase electrical port '`<elecPort>`'.
`M6012`	`csmf-algebraic-loop`		CSMF control blocks form an algebraic loop with no state or delay element: `<cycle>`.
`M6013`	`parallel-ideal-sources`		Ideal voltage sources `<sourceIds>` are connected in parallel at bus '`<busName>`', making the nodal matrix singular.
`M6014`	`parallel-low-impedance-sources`		Voltage sources `<sourceIds>` are connected in parallel at bus '`<busName>`' with no declared source impedance; a stiff default impedance is assumed and large circulating currents may result.
`M6015`	`limiter-permanently-saturated`		Limiter '`<blockId>`' is permanently saturated: its input stays within [`<inLo>`, `<inHi>`] but its output limits are [`<limLo>`, `<limHi>`].
`M6016`	`component-check-failed`		Component '`<cmpName>`' check failed: `<reason>`.
`M6017`	`component-check-warning`		Component '`<cmpName>`' check warning: `<reason>`.

PSS/E {#psse}

Transmission power flow. **15 rules** · `@modelscript/psse` · Source: `languages/psse/src/errors.ts` · Language guide

Code	Rule	Severity	Message
P1001	generator-bus-not-found		Generator '<genId>' references non-existent bus <busId>.
P1002	generator-negative-power		Generator '<genId>' at bus <busId> has negative active power output (<pg> MW).
P1003	duplicate-generator-id		Duplicate generator ID '<genId>' at bus <busId>.
P1004	generator-reactive-limits-inverted		Generator '<genId>' at bus <busId> has inverted reactive limits (QB = <qMin> > QT = <qMax> Mvar).
P1005	generator-power-limits-inverted		Generator '<genId>' at bus <busId> has inverted active power limits (PB = <pMin> > PT = <pMax> MW).
P2001	isolated-bus		Bus <busId> ('<busName>') has no connected branches or transformers (isolated bus).
P2002	missing-slack-bus		Power network base case contains 0 slack/swing buses (Bus Type 3). Power flow solution cannot be anchored.
P2003	multiple-slack-buses		Power network base case contains <count> swing buses without explicit multi-island partitioning.
P2004	voltage-out-of-bounds		Initial voltage magnitude at bus <busId> (<value> pu) is outside standard operational limits [0.8, 1.2 pu].
P3001	branch-from-bus-not-found		Branch from bus <fromBus> to bus <toBus> references non-existent FROM-bus <fromBus>.
P3002	branch-to-bus-not-found		Branch from bus <fromBus> to bus <toBus> references non-existent TO-bus <toBus>.
P3003	branch-zero-impedance		Branch between bus <fromBus> and bus <toBus> has zero impedance (R = 0, X = 0).
P3004	branch-negative-resistance		Branch between bus <fromBus> and bus <toBus> defines negative resistance (<r> pu).
P3005	transformer-invalid-ratio		Transformer between bus <fromBus> and bus <toBus> has non-positive turns ratio (<ratio>).
P4001	load-bus-not-found		Load '<loadId>' references non-existent bus <busId>.

OpenDSS {#opendss}

Unbalanced distribution & DER hosting. **8 rules** · `@modelscript/opendss` · Source: `languages/opendss/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8401	unconnected-bus-node		<elemType> '<elemName>' references undefined bus node '<busName>'.
M8402	phase-count-mismatch		Element '<elemName>' (<elemPhases> phases) connected to bus '<busName>' with different phase count (<busPhases>).
M8403	transformer-voltage-mismatch		Transformer '<xfmrName>' winding <winding> rated voltage (<value> kV) differs from nominal bus voltage (<value> kV).
M8404	reverse-power-hosting-risk		Total DER generation capacity on feeder '<feederName>' (<value> kW) exceeds upstream transformer thermal rating (<value> kVA).
M8405	missing-circuit-definition		OpenDSS script defines circuit elements without declaring a 'New Circuit' voltage source.
M8406	negative-load-power		Load '<loadName>' specifies negative active power (<value> kW); use Generator or Storage for net generation.
M8407	zero-or-negative-line-length		Line '<lineName>' specifies invalid non-positive length (<len>).
M8408	missing-line-code		Line '<lineName>' references undefined LineCode '<codeName>'.

OLGA {#olga}

Transient multiphase flow assurance. **13 rules** · `@modelscript/olga` · Source: `languages/olga/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`M7001`	`cfl-condition-violation`		Simulation timestep maxDt (`<maxDt>` s) violates convective/acoustic CFL stability limit (`<value>` s).
`M7002`	`invalid-pipe-dimensions`		Pipe section ' `<sectionId>` ' has invalid geometry: `<reason>` .
`M7003`	`missing-boundary-condition`		Flowpath ' `<flowpath>` ' has no defined `<missing>` boundary condition.
`M7004`	`steep-elevation-warning`		Pipe section ' `<sectionId>` ' has steep inclination (`<value>` \xB0), risking hydrodynamic slugging.
`M7005`	`fully-closed-choke-hazard`		Choke valve ' `<chokeName>` ' is fully closed (opening = 0) with no active bypass route.
`M7006`	`severe-slugging-hazard`		Severe slugging geometry detected on flowpath ' `<flowpath>` ' (seabed dip to steep riser).
`M7007`	`pig-stuck-hazard`		Pig ' `<pigName>` ' required differential pressure (`<value>` bar) exceeds maximum available (`<value>` bar).
`M7008`	`pvt-table-missing-or-invalid`		PVT fluid table validation error: `<detail>` .
`M7009`	`wax-deposition-risk`		Fluid temperature (`<value>` \xB0C) on flowpath ' `<flowpath>` ' drops below Wax Appearance Temperature (`<value>` \xB0C).
`M7010`	`sand-erosion-or-dropout-hazard`		Sand transport issue on section ' `<sectionId>` ': `<reason>` .
`M7011`	`sour-service-metallurgy-violation`		H2S partial pressure (`<value>` bar) on flowpath ' `<flowpath>` ' violates NACE MR0175 material limit (`<value>` bar).
`M7012`	`asphaltene-precipitation-hazard`		Operating pressure (`<value>` bar) on flowpath ' `<flowpath>` ' enters Asphaltene Onset Pressure envelope (`<value>` bar).
`M7013`	`heatexchanger-parameter-violation`		Heat exchanger ' `<hxName>` ' parameter violation: `<detail>` .

PipelineStudio {#lps}

Gas & liquid pipeline hydraulics. **12 rules** · `@modelscript/lps` · Source: `languages/lps/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8001	maop-exceeded		Pipe ' <pipeName> ' operating pressure (<value> bar) exceeds Maximum Allowable Operating Pressure (<value> bar).
M8002	compressor-surge-hazard		Compressor ' <compName> ' operating flow (<value> m\xB3/h) is below the aerodynamic surge limit (<value> m\xB3/h).
M8003	compressor-stonewall-limit		Compressor ' <compName> ' throughput (<value> m\xB3/h) approaches sonic choke/stonewall limit (<value> m\xB3/h).
M8004	unconnected-pipeline-node		Node ' <nodeName> ' is isolated and has no connecting pipes or equipment links.
M8005	delivery-pressure-deficit		Delivery node ' <nodeName> ' is hydraulically unreachable from any supply or storage cavern.
M8006	erosional-velocity-hazard		Gas velocity in pipe ' <pipeName> ' (<value> m/s) exceeds API RP 14E erosional limit (<value> m/s).
M8007	liquid-column-separation		Transient pressure in pipe ' <pipeName> ' (<value> bar) drops below liquid vapor pressure (<value> bar), risking cavitation.
M8008	hydrogen-embrittlement-risk		Hydrogen blend fraction in pipe ' <pipeName> ' (<value> %) exceeds 10% on steel without fracture toughness certification.
M8009	batch-transmix-excess		Batch transmix interface on pipe ' <pipeName> ' exceeds allowable contamination threshold: <detail> .
M8010	joule-thomson-freezing		Joule-Thomson throttling across regulator ' <regName> ' reduces outlet temperature to <value> \xB0C, risking ice/hydrate blockage.
M8011	pump-npsh-deficit		Pump ' <pumpName> ' net positive suction head available (<value> m) is below required NPSHr (<value> m).
M8012	line-pack-depletion-risk		Network total line pack is depleting rapidly: <detail> .

ECLIPSE {#eclipse}

Reservoir engineering. **10 rules** · @modelscript/eclipse · Source: languages/eclipse/src/errors.ts · Language guide

Code	Rule	Severity	Message
M8701	missing-reservoir-phases		RUNSPEC section does not specify any active fluid phases ('OIL', 'WATER', or 'GAS').
M8702	invalid-grid-dimensions		Reservoir grid dimensions must be strictly positive integers (got <nx> x <ny> x <nz>).
M8703	porosity-out-of-range		Reservoir porosity (<val>) must be strictly positive and less than 1.0.
M8704	negative-permeability		Reservoir permeability ' <prop> ' (<val> mD) must be strictly positive.
M8705	non-monotonic-pvt-table		PVT table ' <table> ' has non-monotonic pressure progression (<prevP> -> <curP>).
M8706	dangling-well-completion		Well completion for ' <wellName> ' at (<i> , <j> , <k>) exceeds grid boundary (<nx> , <ny> , <nz>).
M8707	non-monotonic-saturation-table		Relative permeability table ' <table> ' has non-monotonic saturation progression (<prevSat> -> <curSat>).
M8708	undefined-well-reference		Keyword ' <contextKeyword> ' references well ' <wellName> ' not defined in WELSPECS.
M8709	invalid-well-control-target		Well ' <wellName> ' specifies invalid control target for ' <mode> ' (<val>).
M8710	negative-formation-volume-factor		PVT property specifies non-positive formation volume factor B_ <phase> (<fvf>).

RELAP5-3D {#relap5}

Reactor thermal-hydraulics & kinetics. **10 rules** · `@modelscript/relap5` · Source: `languages/relap5/src/errors.ts` · Language guide

Code	Rule	Severity	Message
`M8601`	`dangling-volume-connection`		Junction '`<junctionId>`' specifies undefined `<end>`-volume '`<volId>`'.
`M8602`	`unclosed-trip-logic-cutset`		Trip logic '`<parentTripId>`' references undefined child trip ID '`<childTripId>`'.
`M8603`	`negative-heat-capacity`		Heat structure '`<hsId>`' specifies non-positive conductivity (`<k>` W/m-K) or capacity (`<cp>` J/m³-K).
`M8604`	`sonic-choked-flow-singularity`		Junction '`<junctionId>`' flagged for critical sonic choked flow transition.
`M8605`	`missing-kinetics-delayed-groups`		Point kinetics model specifies `<count>` delayed groups; standard reactor kinetics requires 6 precursor groups.
`M8606`	`negative-prompt-neutron-lifetime`		Prompt neutron generation time '`<lambda>`' s must be strictly positive.
`M8607`	`adverse-volume-elevation`		Volume '`<volId>`' elevation change dz (`<dz>` m) exceeds physical volume length (`<length>` m).
`M8608`	`negative-junction-loss-coefficient`		Junction '`<junctionId>`' specifies negative form loss coefficient (K_forw= `<kForw>`, K_rev= `<kRev>`).
`M8609`	`invalid-time-step-bounds`		Minimum time step (`<minDt>` s) exceeds maximum time step (`<maxDt>` s).
`M8610`	`negative-pressure-state`		Volume '`<volId>`' specifies non-positive thermodynamic pressure (`<p>` Pa).

SIMA / SIMO / RIFLEX {#sima}

Marine operations & riser dynamics. **15 rules** · `@modelscript/sima` · Source: `languages/sima/src/lints/error-codes.ts` · Language guide

Code	Rule	Severity	Message
M7001	negative-metacentric-height		Hydrostatic instability on vessel '<vessel>': negative <gmType> metacentric height (GM = <value> m < 0).
M7002	line-length-underflow		Mooring line '<line>' total length (<value> m) is less than water depth (<value> m), causing immediate vertical anchor snap load.
M7003	acoustic-courant-violation		Time step dt=<value> s exceeds Courant axial acoustic wave limit (<value> s) for segment '<seg>'.
M7004	unconnected-attachment-node		Mooring line '<line>' references unresolved fairlead attachment '<fairleadId>' not found on any vessel.
M7005	rao-frequency-extrapolation		Environmental wave peak frequency (<value> rad/s) is outside hydrodynamic RAO table range [<value>, <value>] rad/s.
M7006	invalid-stiffness-hierarchy		Non-physical cross-sectional stiffness on segment '<seg>': <prop> (<val>) must be strictly positive.
M7007	unphysical-seabed-friction		Unrealistic seabed friction coefficient (\u03BC = <value>): expected value between 0.0 and 2.0.
M7008	missing-hydrostatic-matrix		Vessel '<vessel>' has zero or missing hydrostatic restoring matrix C; floating body cannot equilibrate in heave/roll/pitch.
M7009	duplicate-line-identifier		Duplicate line identifier '<id>' declared in the slender system.
M7010	slack-compression-hazard		Mooring line '<line>' has negligible pre-tension under severe sea state (Hs > 8m), risking compression slack-snap loading.
M7011	morison-drag-unspecified		Segment '<seg>' in wave zone has zero Morison drag coefficient (Cd = 0); hydrodynamic damping will be neglected.
M7012	anchor-uplift-warning		Catenary line '<line>' exhibits vertical uplift angle (<value> \xB0) at anchor; drag-embedment anchors will pull out.
M7013	mass-matrix-asymmetric		Vessel '<vessel>' structural mass matrix is asymmetric at index (<r>, <c>).
M7014	qtf-symmetry-violation		Vessel '<vessel>' 2nd-order drift force QTF matrix violates conjugate symmetry.
M7015	exceeds-minimum-bend-radius		Line '<line>' minimum radius of curvature (<value> m) is below allowable MBR (<value> m).

SPICE {#spice}

Analog circuit simulation. **10 rules** · `@modelscript/spice` · Source: `languages/spice/src/lints/rules.ts` · Language guide

Code	Rule	Severity	Message
M7101	spice-floating-node		Floating node '<nodeName>' has no DC reference path to ground.
M7102	spice-voltage-loop		Voltage source '<vName>' forms a closed loop of ideal voltage sources without series resistance.
M7103	spice-current-cutset		Node '<nodeName>' is connected strictly to ideal current sources (current cutset).
M7104	spice-unresolved-subcircuit		Subcircuit '<subName>' is referenced by '<compName>' but not defined.
M7105	spice-invalid-passive-value		Passive component '<compName>' has invalid non-positive value <val>.
M7106	spice-timestep-violation		Transient analysis directive specifies invalid timestep '<tstep>' or stop time '<tstop>'.
M7107	spice-duplicate-device		Duplicate component name '<compName>' detected in circuit scope.
M7108	spice-missing-model		Active device '<compName>' references undefined model '<modelName>'.
M7109	spice-pin-count-mismatch		Subcircuit instance '<compName>' connects <nodeCount> pins but definition '<subName>' expects <portCount> ports.
M7110	spice-malformed-directive		Malformed or unsupported directive '<dirName>'.

Verilog-A / AMS {#verilog-a}

Analog compact device models. **10 rules** · `@modelscript/verilog-a` · Source: `languages/verilog-a/src/lints/rules.ts` · Language guide

Code	Rule	Severity	Message
M7601	unassigned-analog-branch		Electrical port pair '<p>' and '<n>' has no constitutive branch equation in analog block.
M7602	discontinuous-derivative-expression		Discontinuous time-derivative operator '<func>()' used inside conditional branch statement.
M7603	invalid-parameter-range		Parameter '<name>' default value <val> violates declared range '<range>'.
M7604	unbounded-exponential-growth		Unbounded exponential function 'exp()' used instead of safe 'limexp()' in '<ctx>'.
M7605	port-discipline-mismatch		Port '<port>' has no declared discipline or mismatch with module domain.
M7606	undefined-node-reference		Branch access '<branch>' references undeclared node '<node>'.
M7607	duplicate-module-declaration		Duplicate module declaration for '<name>'.
M7608	zero-resistance-branch		Potential contribution '<branch>' forms an unconstrained ideal short-circuit.
M7609	floating-internal-node		Internal node '<node>' is declared but never referenced in any analog branch equation.
M7610	unsupported-analog-primitive		Unsupported or deprecated analog primitive '<primitive>'.

IBIS {#ibis}

I/O buffer behavioral models. **9 rules** · `@modelscript/ibis` · Source: `languages/ibis/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8303	non-monotonic-iv-curve		IBIS model '<modelName>' contains non-monotonic slope in '<tableType>' table between <value> V and <value> V.
M8305	missing-pin-model-binding		Component pin '<pinName>' references undefined model '<modelName>'.
M8316	missing-component-pins		Component '<compName>' declaration has no defined pins in '[Pin]' section.
M8317	invalid-voltage-range		IBIS model '<modelName>' has invalid [Voltage Range]: min (<value> V) is greater than max (<value> V).
M8318	clamping-diode-divergence		Model '<modelName>' clamp table '<clampType>' diverges to excessive current (<value> A).
M8319	missing-pullup-pulldown		Driver model '<modelName>' of type '<modelType>' is missing both I/V tables ([Pullup]/[Pulldown]) and [Ramp] slew rates.
M8320	negative-die-capacitance		IBIS model '<modelName>' specifies non-positive die capacitance C_comp '<val>'.
M8321	negative-package-parasitic		Component '<compName>' package parasitic '<param>' has non-positive value '<val>'.
M8322	no-models-defined		IBIS file contains no buffer [Model] definitions.

Touchstone {#touchstone}

RF & signal-integrity S-parameters. **10 rules** · `@modelscript/touchstone` · Source: `languages/touchstone/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8301	passivity-violation		N-port scattering matrix violates passivity criterion (spectral norm <norm> > 1.0) at frequency <freq>.
M8302	causality-violation		Kramers-Kronig causality relation violated across frequency sweep: '<detail>'.
M8304	frequency-grid-non-monotonic		Frequency grid is not monotonically increasing: step from <prevFreq> to <currFreq> is non-increasing.
M8306	missing-option-line		Missing '#' option line; defaulting to '# GHz S MA R 50'.
M8307	invalid-parameter-format		Invalid Touchstone parameter token or format '<token>'.
M8308	incomplete-matrix-data		Incomplete N-port matrix row: expected <expected> values, received <received>.
M8309	negative-reference-resistance		Reference resistance '<r>' must be strictly positive.
M8310	asymmetric-reciprocal-network		Asymmetry in reciprocal S-parameter network: |S <i> <j> - S <j> <i> | = <diff> exceeds threshold.
M8311	no-frequency-points		Touchstone file contains no valid frequency parameter points.
M8312	duplicate-frequency-point		Duplicate frequency sample point '<freq>' encountered in sweep.

URDF / Xacro {#urdf}

Robot kinematics & dynamics. **8 rules** · `@modelscript/urdf` · Source: `languages/urdf/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M7501	unresolved-parent-link		Joint '<jointName>' references non-existent parent link '<linkName>'.
M7502	kinematic-loop		Closed kinematic chain or duplicate parent detected at link '<linkName>' (URDF requires open tree).
M7503	invalid-inertia-tensor		Link '<linkName>' defines non-physical inertia tensor: <reason>.
M7504	invalid-joint-limits		Joint '<jointName>' defines invalid limits (lower: <lower> >= upper: <upper>).
M7505	missing-mesh-resource		Visual mesh URI '<uri>' on link '<linkName>' may not resolve at runtime.
M7506	massless-link		Link '<linkName>' has zero mass and no inertial specification.
M7507	zero-norm-joint-axis		Joint '<jointName>' defines a zero-norm rotation/translation axis [0, 0, 0].
M7508	continuous-joint-with-limits		Joint '<jointName>' is declared 'continuous' but specifies upper/lower limits.

PLCopen / IEC 61131-3 {#plcopen}

PLC control software. **10 rules** · @modelscript/plcopen · Source: languages/plcopen/src/lints/rules.ts · Language guide

Code	Rule	Severity	Message
M7701	unassigned-variable-read		Variable '<name>' is read before being assigned or initialized in POU '<pouName>'.
M7702	cyclic-task-overrun		Computational complexity in task '<taskName>' (<stmtCount> statements) may exceed cycle time '<interval>'.
M7703	recursive-pou-call		Recursive invocation of POU '<calleeName>' detected in '<callerName>'. Recursion is prohibited in IEC 61131-3.
M7704	io-mapping-collision		Hardware I/O address collision at '<address>': multiple variables ('<var1>', '<var2>') assigned to same output.
M7705	type-mismatch-assignment		Type mismatch in assignment to '<varName>': expected '<expectedType>', got '<actualType>'.
M7706	timer-preset-negative		Timer preset time '<value>' for '<timerName>' cannot be negative.
M7707	unbounded-loop-detected		Loop in POU '<pouName>' does not have a deterministic bounded termination condition.
M7708	unknown-pou-type		Unknown or unresolvable POU or function block type '<typeName>' instantiated in '<instanceName>'.
M7709	unreachable-code		Unreachable code detected after '<stmtKind>' statement in POU '<pouName>'.
M7710	invalid-task-interval		Task '<taskName>' specifies invalid or non-positive periodic interval '<interval>'.

MSC Adams {#adams}

Mechanical system dynamics. **9 rules** · @modelscript/adams · Source: languages/adams/src/errors.ts · Language guide

Code	Rule	Severity	Message
M7301	overconstrained-mobility		Mechanism is overconstrained according to Grübler-Kutzbach criterion (Mobility = <mobility> < 0).
M7302	gimbal-lock-orientation		Euler orientation angles for marker '<markerId>' indicate near-gimbal-lock singularity (psi = <value>\xB0).
M7303	dangling-marker-reference		<source> references non-existent marker '<targetId>'.
M7304	non-physical-inertia-tensor		Part '<partId>' has non-physical inertia tensor: <reason>.
M7305	massless-part-floating		Moving part '<partId>' has zero or negative mass without kinematic grounding.
M7307	coincident-joint-markers		Joint '<jointId>' uses identical I and J markers ('<iMarker>').
M7308	negative-spring-damping		Spring-damper '<springId>' defines non-physical negative <param> (<value>).
M7309	missing-ground-part		Dataset does not define a ground reference body ('PART/... GROUND').
M7310	unconstrained-part		Part '<partId>' is completely disconnected from joints or force elements (floating rigid body).

EnergyPlus {#energyplus}

Building energy & HVAC. **8 rules** · @modelscript/energyplus · Source: languages/energyplus/src/lints/rules.ts · Language guide

Code	Rule	Severity	Message
M7901	unmatched-interzone-surface		Interzone surface '<surfName>' in zone '<zoneName>' specifies outside boundary object '<targetSurf>' which does not exist or has mismatched reciprocal boundary.
M7902	non-convex-surface-polygon		Surface polygon '<surfName>' in zone '<zoneName>' violates planar convexity, which may impair solar raytracing and view factor calculations.
M7903	missing-thermal-properties		Material '<matName>' has non-positive thermal property '<prop>' (<val>), violating second law of thermodynamics.
M7904	unconnected-hvac-zone		HVAC component '<hvacName>' references non-existent thermal zone '<zoneName>'.
M7905	excessive-surface-absorption		Material '<matName>' has solar absorptance coefficient <value>, which exceeds the physical maximum of 1.0.
M7906	duplicate-zone-name		Duplicate thermal zone identifier '<zoneName>' detected in building model.
M7907	unreferenced-construction		Building surface '<surfName>' references undefined construction '<constName>'.
M7908	zero-thickness-material		Opaque material '<matName>' has invalid non-positive thickness (<value> m).

EPANET {#epanet}

Pressurized water distribution. **8 rules** · @modelscript/epanet · Source: languages/epanet/src/lints/rules.ts · Language guide

Code	Rule	Severity	Message
M8101	isolated-hydraulic-node		Hydraulic node '<nodeId>' is isolated with no connecting pipes or pumps.
M8102	negative-pressure-risk		Junction '<nodeId>' elevation (<value> m) exceeds maximum network source head (<value> m), risking cavitation and dry-out.
M8103	non-monotonic-pump-curve		Pump head curve '<curveId>' violates decreasing monotonicity: head increases from <value> to <value> between flow <value> and <value>.
M8104	zero-length-pipe		Pipe link '<pipeId>' has non-positive physical length (<value> m).
M8105	zero-diameter-pipe		Pipe link '<pipeId>' has non-positive internal diameter (<value> mm).
M8106	duplicate-node-id		Duplicate node identifier '<nodeId>' detected in water network topology.
M8107	duplicate-link-id		Duplicate link identifier '<linkId>' detected in water network topology.
M8108	unreferenced-node-in-link		Link '<linkId>' references undefined endpoint node '<nodeId>'.

EPA SWMM {#swmm}

Stormwater & urban drainage. **13 rules** · `@modelscript/swmm` · Source: `languages/swmm/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8201	dangling-node-reference		Link '<linkId>' references undefined <role> node '<nodeId>'.
M8202	adverse-conduit-slope		Conduit '<conduitId>' has adverse slope: inlet elevation (<value>) is lower than outlet elevation (<value>).
M8203	missing-cross-section		Conduit '<conduitId>' has no hydraulic cross-section definition in '[XSECTIONS]'.
M8204	unsupported-outfall-type		Outfall '<outfallId>' specifies unsupported boundary condition type '<type>'.
M8205	surcharge-overflow-risk		Junction '<junctionId>' rim elevation (<value>) is lower than connected conduit crown (<value>), creating surcharge overflow risk.
M8206	duplicate-node-id		Duplicate node identifier '<nodeId>' declared in network.
M8207	duplicate-link-id		Duplicate link identifier '<linkId>' declared in network.
M8208	negative-roughness		Conduit '<conduitId>' specifies non-positive Manning's roughness coefficient (n = <n>).
M8209	zero-length-conduit		Conduit '<conduitId>' specifies non-positive length (<len>).
M8210	isolated-network-node		Junction '<nodeId>' has no connected conduits or hydraulic links (isolated node).
M8211	subcatchment-outlet-not-found		Subcatchment '<subId>' references non-existent discharge outlet '<outletId>'.
M8212	weir-offset-negative		Weir '<weirId>' specifies negative crest height offset (<offset>).
M8213	invalid-discharge-coefficient		Link '<linkId>' specifies non-positive discharge coefficient (Cd = <cd>).

Aspen Plus / HYSYS {#aspen}

Chemical process flowsheets. **15 rules** · `@modelscript/aspen` · Source: `languages/aspen/src/errors.ts` · Language guide

Code	Rule	Severity	Message
M8101	unconnected-stream-port		Stream '<streamName>' connected to block '<blockName>' inlet has no producing source block outlet.
M8102	tear-stream-unconverged		Circular flowsheet loop detected involving block '<blockName>' without designated tear stream.
M8103	invalid-eos-parameter		Component '<compName>' lacks critical properties for property method '<propertyMethod>'.
M8104	negative-component-fraction		Stream '<streamName>' specifies negative component molar fraction for '<compName>' (<val>).
M8105	column-reboiler-heat-deficit		Column '<blockName>' duty results in non-positive reboiler boil-up.
M8106	duplicate-block-id		Duplicate block ID '<blockId>' declared in flowsheet.
M8107	duplicate-component-id		Duplicate component ID '<compId>' declared in COMPONENTS.
M8108	fractions-sum-invalid		Component mole fractions in stream '<streamName>' sum to <value>, expected 1.0.
M8109	feed-stage-out-of-range		Column '<blockName>' feed stage <stage> is out of bounds (1..<nStages>).
M8110	product-stage-out-of-range		Column '<blockName>' product stage <stage> is out of bounds (1..<nStages>).
M8111	missing-flowsheet-entry		Block '<blockName>' has parameter declarations but is not connected in FLOWSHEET.
M8112	stream-temperature-negative		Stream '<streamName>' specifies non-physical absolute temperature <temp> K.
M8113	stream-pressure-negative		Stream '<streamName>' specifies non-positive pressure <pres> bar.
M8114	radfrac-stages-too-few		RadFrac column '<blockName>' has fewer than 2 stages (<nStages>).
M8115	reflux-ratio-negative		Column '<blockName>' reflux ratio (<rr>) must be non-negative.

SBML {#sbml}

Biochemical reaction networks. **10 rules** · @modelscript/sbml · Source: languages/sbml/src/errors.ts · Language guide

Code	Rule	Severity	Message
M8801	missing-model-id		SBML <model> element is missing required 'id' attribute.
M8802	undefined-compartment-reference		Species '<speciesId>' references undefined compartment '<compartmentId>'.
M8803	undefined-species-reference		Reaction '<reactionId>' references undefined <role> species '<speciesId>'.
M8804	negative-compartment-size		Compartment '<compartmentId>' defines non-positive size (<size>).
M8805	negative-initial-amount-concentration		Species '<speciesId>' specifies negative initial amount/concentration (<val>).
M8806	empty-reaction		Reaction '<reactionId>' must define at least one reactant or product.
M8807	missing-kinetic-law		Reaction '<reactionId>' does not define a kineticLaw rate law equation.
M8808	negative-stoichiometry		Species reference for '<speciesId>' in reaction '<reactionId>' specifies non-positive stoichiometry (<stoich>).
M8809	undefined-kinetic-parameter		Kinetic law formula in reaction '<reactionId>' references identifier '<id>' not found in parameters or species.
M8810	circular-species-assignment		Reaction '<reactionId>' lists '<speciesId>' as both reactant and product with identical stoichiometry; consider defining as modifier.

CHAPTER 67

Export & Exchange Formats Reference

ModelScript serves as an open interoperability hub across systems engineering, multi-physics simulation, CAD/ECAD, formal verification, and regulatory compliance. Through `@modelscript/exchange`, `@modelscript/cad`, `@modelscript/runtime`, and `@modelscript/simulate`, the platform natively imports and exports **over 30 industry-standard formats**.

---

1. Functional Mock-up Interface (FMI 2.0 & FMI 3.0)

ModelScript compiles flattened DAE systems into self-contained `.fmu` archives (and imports external third-party `.fmu` binaries) conforming to the Modelica Association **FMI 2.0** and **FMI 3.0** specifications for both **Model Exchange (ME)** and **Co-Simulation (CS)**.

```
# Export an FMI 3.0 Co-Simulation FMU with embedded C99 source and WebAssembly binary
npx msx fmu DC_Motor motor.mo --fmi-version 3 --type cs --wasm --output DC_Motor.fmu

# Export a universal FMI 2.0 + 3.0 archive supporting both ME and CS
npx msx fmu ThermalPlant plant.mo --fmi-version both --type both --target all -o ThermalPlant.fmu

# Import an external third-party FMU and synthesize a typed Modelica wrapper block
npx msx import-fmu ./vendor/BatteryController.fmu -o BatteryControllerWrapper.mo
```

Archive Layout & Capabilities

Artifact Path	Standard	Description
`modelDescription.xml`	FMI 2.0 / 3.0	Declares `ModelVariables`, causalities (`input`, `output`, `parameter`, `local`), physical units, `ModelStructure` (`Outputs`, `Derivatives`, `InitialUnknowns`), directional derivative sparsity patterns, and FMI 3.0 `Clock` / multidimensional array dimensions.
`sources/*.c`, `sources/*.h`	ANSI C99	Zero-dependency generated C99 implementation of `fmi2*` / `fmi3*` entry points, analytic Jacobians (`fmi2GetDirectionalDerivative` / `fmi3GetDirectionalDerivative`), and embedded RK4/BDF Co-Simulation steppers.
`binaries/<platform>/*.so|.dll|.dylib`	Native ABI	Pre-compiled host shared libraries generated when `--compile` is passed.
`binaries/wasm32-wasi/<model>.wasm`	WebAssembly	Portable, sandboxed WASM FMU binary generated when `--wasm` or `--target wasm` is passed, executable directly in browser workers and edge runtimes.

Universal Binary FMUs & In-WASM CPU Emulation

A major limitation of traditional FMI co-simulation is platform lock-in: an FMU containing a Windows x86 `.dll` cannot run on Linux or Apple Silicon without native source access.

Through `@modelscript/exchange/pe-emulator`, ModelScript includes **user-space CPU emulators and binary loaders executing inside WebAssembly**:

- **Cross-Architecture Execution**: Runs legacy Windows x86/x64 (`.dll`), Linux ELF (`.so`), and macOS Mach-O (`.dylib`) binary FMUs directly inside Node.js, browsers, or cloud runners without installing Wine or virtual machines.
- **CPU Translation Engines**:
  - **x86 / x86-64 Engine (`x86-cpu.ts`)**: Decodes instruction prefixes, registers, and floating-point/AVX2 vector instructions into typed array memory operations.
  - **AArch64 Engine (`aarch64-cpu.ts`)**: Decodes 64-bit ARM instruction streams and executes floating-point NEON vectors.
  - **C Runtime Shims (`c-runtime-shims.ts`)**: Traps standard C library calls (`malloc`, `free`, `sin`, `cos`, `printf`, `memcpy`) and routes them directly to host memory for native execution speed.
- **In-Browser C Compilation Worker (`wasm-compiler-worker.ts`)**:
  - Compiles FMI C99 source code into sandboxed WebAssembly modules on the fly directly inside browser WebWorkers.

- Eliminates the need for server-side GCC/Clang compilation infrastructure when simulating customer models in the browser.

---

2. System Structure and Parameterization (SSP 1.0 & SSV 1.0)

ModelScript natively parses, lints (`SSP_*` and `SSV_*` rules), simulates, and packages **Modelica Association SSP 1.0** system containers:

Extension	XML Namespace	Purpose
`.ssp`	ZIP Container	Bundles `SystemStructure.ssd` , `resources/*.fmu` components, parameter sets (`*.ssv`), and signal dictionaries (`*.ssm`).
`.ssd`	`http://ssp-standard.org/SSP1/SystemStructureDescription`	Defines hierarchical system architectures, FMU component references, typed connectors, causal signal connections, and `<ssd:DefaultExperiment>` metadata.
`.ssv`	`http://ssp-standard.org/SSP1/SystemStructureParameterValues`	Externalizes parameter sets (`Real` , `Integer` , `Boolean` , `String` , `Enumeration` , and units) for variant management without recompiling FMUs.
`.ssm`	`http://ssp-standard.org/SSP1/SystemStructureParameterMapping`	Maps abstract parameter names in `.ssv` catalogs to concrete hierarchical component paths.

```
# Lint an SSP architecture and its SSV parameter bindings
npx msx lint SystemStructure.ssd NominalParams.ssv

# Export a multi-project PSCAD workspace to an SSP 1.0 container
npx msx pscad export-ssp GridInterconnection.pswx -o GridInterconnection.ssp
```

---

3. Formal Verification Formats

The `msx export-formal` command translates SysML v2 models (constraints, requirements, and state machines) into three formal verification formats for external solvers and model checkers:

```
# Export constraints to SMT-LIB v2.6 for Z3 / cvc5 (logic auto-detected as QF_LRA or QF_NRA)
npx msx export-formal FlightControl.sysml --format smt2 -o fcs.smt2

# Export state machines to nuXmv with an extra safety invariant
npx msx export-formal FlightControl.sysml --format nuxmv --module FlightControl --invariants "altitude >= 0" -o fcs.smv

# Export assume-guarantee contracts to OCRA
npx msx export-formal FlightControl.sysml --format ocra -o fcs.oss
```

Format Flag	Extension	Target Toolchains	Encoding Semantics
`smt2`	`.smt2`	Z3, cvc5	SMT-LIB v2.6 with a `set-logic` directive (`--logic QF_LRA|QF_NRA|QF_IDL|ALL`) and optional `--minimize` / `--maximize` objectives.
`nuxmv`	`.smv`	nuXmv	Symbolic transition system module with optional `INVARSPEC` (`--invariants`) and `LTLSPEC` (`--ltl`) properties.
`ocra`	`.oss`	OCRA	Contract-based component specification; `--module` sets the top-level component name.

---

4. CI/CD Quality Gates & Regulatory Compliance Dossiers

The `msx verify` and `msx dhf` commands serialize static diagnostics, formal proof certificates, and requirement traceability matrices into industry-standard CI/CD and regulatory formats:

```
# SARIF v2.1.0 for GitHub Code Scanning / Azure DevOps Security Tab
npx msx verify system.mo --format sarif --report linter-report.sarif

# CTRF (Common Test Report Format) JSON for unified CI dashboards
npx msx verify system.mo --format ctrf --report ctrf-report.json

# JUnit XML for Jenkins, GitLab CI, and CircleCI test summaries
npx msx verify system.mo --format junit --report junit-report.xml

# FDA 21 CFR Part 820 / ISO 13485 / ISO 26262 Design History File (DHF) Dossier
npx msx verify system.mo --format dhf --report dossier.md
npx msx dhf export ./workspace --format markdown -o DHF_Dossier.md
```

Format	Schema / Standard	Key Payload Contents
SARIF (`sarif`)	OASIS SARIF v2.1.0	Rule descriptors (`M1000` – `M6040` , `S1001` – `S5008`), physical file locations (1-indexed line/column spans), severity levels, and counterexample code flows.
CTRF (`ctrf`)	CTRF v0.0.0 JSON	Standardized `results.summary` (`tests` , `passed` , `failed` , `skipped`) and per-property verification duration and witness traces.
JUnit (`junit`)	JUnit XML	`<testsuites>` and `<testcase>` elements with `<failure>` stack traces for falsified invariants or lint errors.
DHF (`dhf`)	FDA 21 CFR 820.30 / ISO 13485	End-to-end Requirements Traceability Matrix (RTM), formal proof summaries, simulation verification evidence, and SHA-256 artifact integrity digests.

---

5. Systems Engineering & Requirements Interop (SysML v2, ReqIF, OSLC, OWL2)

ModelScript bridges descriptive MBSE specifications and semantic knowledge graphs with executable physics:

Standard	Extensions / Media Types	CLI Command / API	Description
OMG SysML v2 / KerML	`.sysml` , `.kerml` , `application/json`	`msx sysml <export|import|sync>` & REST `/api/v1/sysml2`	Textual SysML v2 notation and full **OMG Systems Modeling API and Services v2.0** REST compliance (`projects` , `commits` , `branches` , `tags` , `elements` , `relationships`).
OMG ReqIF 1.2	`.reqif` , `.reqifz`	`msx reqif <import|export>`	Bidirectional mapping between DOORS/Polarion ReqIF `SPEC-OBJECT` / `SPEC-RELATION` hierarchies and SysML v2 `requirement def` / `satisfy` blocks.
OASIS OSLC 3.0 & TRS 3.0	`.ttl` , `.jsonld` , `.rdf`	`msx oslc <export|serve>`	Exposes requirements (`oslc_rm:Requirement`) and architecture resources (`oslc_am:Resource`) with Tracked Resource Set (TRS 3.0) incremental change logs and delegated UI previews.
W3C OWL 2 / RDF / SPARQL 1.1	`.owl` (Functional Syntax), `.ttl` (Turtle), `.nt` (N-Triples), `.jsonld`	`/api/v1/libraries/rdf` , `/api/v1/libraries/sparql` , MCP `query_ontology_sparql`	Extracts ontological axioms, executes OWL 2 RL forward-chaining reasoning, and answers SPARQL 1.1 queries over workspace models.

---

6. Industrial Digital Twin & Open Hardware Packaging (DDP, AASX, OKH)

For manufacturing handover, Industry 4.0 asset integration, and open-source hardware certification, ModelScript supports three complementary packaging standards:

```
# Pack a workspace into an ISO/PAS 81346 Digital Data Package (.ddp)
npx msx ddp pack ./workspace -o PumpStation.ddp

# Convert a DDP archive into an IDTA Asset Administration Shell v3.0 (.aasx) package
npx msx ddp to-aasx PumpStation.ddp -o PumpStation.aasx
```

Standard	Format / Endpoint	Description
ISO/PAS 81346 DDP	`.ddp` (ZIP + XML/JSON)	Structures multi-domain engineering deliverables by function (`=`), product (`-`), and location (`+`) reference designation aspect trees.
IDTA AAS v3.0	`.aasx` (Open Packaging Conventions) & `?lens=aas`	Packages Asset Administration Shells containing standardized `Nameplate` , `TechnicalData` , `HandoverDocumentation` , and `SimulationModels` (FMU) submodels.
DIN SPEC 3105 / OSHWA OKH	`okh.json` / `okh.toml` & `?lens=okh`	Generates Open Know-How (OKH v1.0) manifests detailing bill of materials (BOM), CAD/ECAD design files, manufacturing tolerances, and SPDX licenses.

---

7. CAD, ECAD & Multi-Physics CAE Formats (STEP, CSG, Gerber, SU2, CalculiX)

`@modelscript/cad` and the polyglot language plugins provide direct geometric and mesh interoperability:

Domain	Format / Standard	Extensions	Capabilities
3D B-Rep CAD	**ISO 10303-21 STEP** (AP203 / AP214 / AP242)	`.stp` , `.step`	Native GLR parser for `DATA` entity graphs (`ADVANCED_FACE` , `B_SPLINE_SURFACE_WITH_KNOTS` , `MANIFOLD_SOLID_BREP` , `PRODUCT_DEFINITION`) and OpenCASCADE B-Rep export.
Parametric CSG & Meshes	**OpenSCAD**, **STL**, **OBJ**, **glTF 2.0 / GLB**	`.scad` , `.stl` , `.obj` , `.gltf` , `.glb`	Evaluates `.scad` scripts and Modelica `Visualizers` (`msx csg`) into watertight boundary-representation solids and triangulated 3D web assets.
ECAD / PCB Manufacturing	**Gerber RS-274X / X2**, **Excellon**, **IPC-2581**, **ODB++**	`.gbr` , `.gtl` , `.gbl` , `.drl`	Parses aperture definitions (`%ADD...*%`), copper traces, solder mask layers, and NC drill coordinates for 3D PCB stackup rendering and thermal-electrical co-simulation.
CFD (Fluid Dynamics)	**SU2**, **OpenFOAM**, **CGNS / VTK**	`.cfg` , `.su2` , `.vtk`	Parses and validates SU2 solver configurations (`SOLVER=RANS` , boundary markers, CFL schedules) and unstructured finite-volume meshes (`NDIME=` , `NELEM=` , `NPOIN=`).
FEA (Structural / Thermal)	**Abaqus / CalculiX**, **Nastran BDF**	`.inp` , `.bdf` , `.frd`	Parses `*NODE` , `*ELEMENT` , `*MATERIAL` , `*ELASTIC` , `*STEP` , and `*BOUNDARY` decks for thermo-mechanical ROM extraction and stress verification.

---

8. Power Systems & Electronics (PSCAD & SPICE)

Format	Extensions	CLI / Tooling	Capabilities
PSCAD EMT	`.pscx` , `.pswx` , `.psmx`	`msx pscad <lint|diagram|sim|verify-ieee2800|export-modelica|export-ssp>`	Parses hierarchical XML schematics, Blackbox/CSMF control blocks, transmission line segments, and executes nodal admittance EMT simulation and IEEE Std 2800-2022 compliance checks.
SPICE Netlists	`.sp` , `.cir` , `.net` , `.sub`	TGG Synchronization (`msx align`) & `@modelscript/modelica`	Bi-directionally maps SPICE netlists (`.subckt` , `R` , `L` , `C` , `M` MOSFET, `Q` BJT, `V` , `I`) to `Modelica.Electrical.Analog` / `Spice3` components.

---

9. Surrogate Modeling & Machine Learning (ONNX, C99, Modelica ROM)

The `msx surrogate` pipeline exports trained Reduced-Order Models (MLP neural networks, multivariate polynomials, and Radial Basis Functions) with embedded domain-validity guardrails:

```
# Train an MLP surrogate and export ONNX, standalone C99, and pure Modelica (.mo) simultaneously
npx msx surrogate Compressor compressor.mo \
  --inputs "rpm,pressureRatio" \
  --input-bounds "rpm:1000:12000,pressureRatio:1.1:4.5" \
  --outputs "massFlow,efficiency" \
  --format all \
  --out-file build/compressor_rom
```

- **ONNX (`.onnx`)**: Serializes weights, biases, input normalization scalers, and activation layers (`Tanh` , `ReLU` , `GELU`) as a standard ONNX protobuf graph for deployment in ONNX Runtime, TensorRT, or PyTorch. Can also be imported via `--from-onnx` .
- **Standalone ANSI C99 (`.c` / `.h`)**: Zero-dependency static C inference function with compile-time weight arrays and bounding-box extrapolation flags for embedded MCU deployment.
- **Modelica Block (`.mo`)**: Self-contained Modelica `block` containing pure matrix/vector equations, `isExtrapolating` boolean output port, and `assert()` guardrails.

---

10. Simulation Trajectories & Scientific Data (CSV, JSON, MAT v4, HDF5, Parquet)

Simulation and Monte Carlo runners (`msx simulate` , `msx mc` , and Cloud HPC workers) produce structured time-series outputs:

```
# Export standard RFC 4180 CSV trajectory
npx msx simulate DC_Motor motor.mo --stop-time 5.0 --format csv > trajectory.csv

# Export structured JSON trajectory with variable metadata and solver statistics
npx msx simulate DC_Motor motor.mo --stop-time 5.0 --format json > trajectory.json
```

Format	Source / Consumer	Description
CSV (`.csv`)	`msx simulate --format csv` , `msx mc --format csv` , `@modelscript/csv`	Leading monotonic `time` column followed by state, derivative (`der(...)`), and algebraic variable columns; directly consumable by `Modelica.Blocks.Tables.CombiTimeTable` and `msl-comparison-runner` .
JSON (`.json`)	`msx simulate --format json` , `msx mc --format json`	Contains `time: number[]` , `signals: Record<string, number[]>` , `units: Record<string, string>` , `events` , and solver convergence telemetry.
MATLAB Level 4 (`.mat`)	OpenModelica / Dymola compatibility	Dual-matrix (`Aclass` , `name` , `description` , `dataInfo` , `data_1` parameters, `data_2` trajectories) binary format for legacy post-processing scripts.
Apache Parquet & HDF5	Cloud HPC (`msx cloud download`) & Digital Twin Archive	Columnar Zstandard-compressed trajectory storage for multi-gigabyte Monte Carlo ensembles and long-horizon operational twin logs.

---

11. Interactive Notebooks & Visual Diagrams (`.monb` , SVG, HTML)

Artifact	Producer	Description
ModelScript Notebook (`.monb`)	VS Code / Web IDE (`modelscript.monbNotebook`)	JSON document format interleaving GitHub-Flavored Markdown + KaTeX cells with executable `.mo` , `.sysml` , and `.msx` cells and inline interactive Plotly/SVG simulation outputs.
Vector Diagrams (`.svg`)	`msx render` , `msx pscad diagram` , `msx pr-diff --format visual-svg`	Standalone, CSS-styled SVG schematics with ELK orthogonal port routing, Modelica `Icon` / `Diagram` coordinate graphics, SysML v2 BDD/IBD/State/Activity compartments, and dark/blueprint theme support.
Interactive Diff & Benchmark Dashboards (`.html`)	`msx pr-diff --format visual-html` , `npm run test:msl -- --report-html`	Self-contained, dark-mode HTML reports with pan/zoom SVG diff overlays and interactive trajectory error charts.

CHAPTER 68

MCP Server & IDE Workbench Reference

ModelScript provides first-class AI agent integration via the **Model Context Protocol (`@modelscript/mcp`)** alongside a comprehensive polyglot **VS Code Extension & Web IDE (`@modelscript/ide-client` , `@modelscript/lsp`)**. Both surfaces share the same zero-allocation WebAssembly GLR parsers, Salsa `QueryEngine` , and linear-memory `DAEBuilder` .

Part I: Model Context Protocol (MCP) Server

The `@modelscript/mcp` package exposes ModelScript's compilation, simulation, formal verification, ontology reasoning, Triple Graph Grammar (TGG) synchronization, and requirements traceability engines to MCP-compatible AI coding assistants (Claude Desktop, Cursor, Windsurf, Antigravity, Zed, and custom LLM orchestrators).

1. Deployment Modes & Transports

ModelScript supports three MCP execution architectures:

A. Local `stdio` Server (`modelscript-mcp`)

For local desktop AI agents, run the `modelscript-mcp` binary over standard input/output (`StdioServerTransport`):

```
{
  "mcpServers": {
    "modelscript": {
      "command": "npx",
      "args": ["-y", "@modelscript/mcp"],
      "env": {
        "MODELICAPATH": "/path/to/ModelicaStandardLibrary"
      }
    }
  }
}
```

B. Hosted Enterprise SSE Gateway (`/api/v1/mcp`)

The ModelScript Enterprise Hub (`apps/api/src/routes/mcp.ts`) exposes a multi-tenant HTTP Server-Sent Events (SSE) MCP gateway with built-in compute credit metering:

- **`GET /api/v1/mcp/sse`** : Establishes a persistent SSE stream (`text/event-stream`), authenticates the caller via `Authorization: Bearer <JWT | ms_bot_*>` , and returns a session-scoped `endpoint` event (`/api/v1/mcp/messages?sessionId=<uuid>`).
- **`POST /api/v1/mcp/messages?sessionId=<uuid>`** : Receives JSON-RPC 2.0 `tools/call` and `resources/read` requests, checks organization compute credit balances via `BillingService` (1 credit per lightweight query, 5 credits for `modelica_simulate` / `hybrid_simulate` / `modelica_doe` / `modelscript_closed_loop_synthesize`), and streams results over the SSE channel.

C. In-WASM AOT Polyglot Dispatch (`PolyglotMcpHost`)

In browser workers and edge sandboxes (`packages/mcp/src/polyglot-server.ts`), `PolyglotMcpHost` dynamically synthesizes MCP tool schemas from every registered `LanguageDefinition` (`defineLanguage({ mcp: { tools, resources } })`). When a tool declares an `aotExport` symbol, `PolyglotMcpHost` writes the arguments directly into WebAssembly linear memory (`__newString`), invokes the compiled AssemblyScript kernel (`mcpDispatchTool(toolId, targetPtr, paramsJsonPtr)`), and reads back the JSON result with zero JS object marshaling overhead.

---

2. Complete Reference of All 31 Core MCP Tools

The canonical MCP server (`packages/mcp/src/tools.ts`) registers **31 specialized engineering tools** grouped across seven functional domains:

Domain 1: Modelica Compilation, Simulation & Sensitivity (9 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`modelica_load`	`path: string`	—	Load a `.mo` file or package directory from disk into the stateful MCP workspace session and return all discovered class FQNs.
`modelica_parse`	—	`code?: string`, `path?: string`	Parse Modelica source text or a file using the native WASM GLR parser and return CST syntax diagnostics and top-level class names.
`modelica_flatten`	`className: string`	`code?: string`, `path?: string`	Flatten a hierarchical Modelica class into a canonical DAE (`ArenaDAEPrinter` output) and return variable/equation counts and flattened source text.
`modelica_lint`	—	`code?: string`, `path?: string`	Execute all `M1xxx` – `M6xxx` static lint rules and Salsa semantic queries, returning structured diagnostics with line/column spans.
`modelica_simulate`	`className: string`	`code?: string`, `path?: string`, `tStart?: number` (`0`), `tEnd?: number` (`1`), `dt?: number` (`0.01`), `tolerance?: number` (`1e-6`), `method?: "euler" | "rk4" | "dopri5" | "bdf" | "radau5" | "cvode" | "ida"` (`"rk4"`), `variables?: string[]`	Flatten, index-reduce (Pantelides + BLT tearing), and numerically simulate a Modelica model, returning downsampled time-series trajectories and solver statistics.
`hybrid_simulate`	`className: string`	`code?: string`, `path?: string`, `tStart?: number` (`0`), `tEnd?: number` (`10`), `dt?: number` (`0.01`), `maxEvents?: number` (`500`)	Run hybrid continuous/discrete event-driven simulation with zero-crossing root finding (`when` clauses, `reinit()`, state machines, and synchronous clocks).
`modelica_doe`	`className: string`, `parameters: Record<string, number[]>`	`code?: string`, `path?: string`, `tEnd?: number` (`1`), `dt?: number` (`0.01`), `metrics?: string[]`	Execute a Cartesian/factorial Design of Experiments (DoE) sweep across parameter combinations and return terminal/summary metrics for each run.
`modelica_sensitivity`	`className: string`, `parameters: string[]`, `outputs: string[]`	`code?: string`, `path?: string`, `tEnd?: number` (`1`), `dt?: number` (`0.01`), `perturbation?: number` (`0.01`)	Compute normalized local sensitivity Jacobians ($\frac{\partial y_i}{\partial p_j} \cdot \frac{p_j}{y_i}$) via central finite differences or forward AD.
`modelica_query`	`queryType: "list_classes" | "class_info" | "components" | "equations" | "connections"`	`className?: string`	Query the active Salsa `SymbolIndex` for class specializations, typed component declarations, equations, or `connect()` topology edges.

Domain 2: Cross-Domain Consistency & Ontology Reasoning (5 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`validate_system_consistency`	`sysmlCode: string`, `modelicaCode: string`	`mapping?: Record<string, string>`	Cross-check a SysML v2 architecture definition against a Modelica physics implementation for port mismatches, unit conflicts, and unsatisfied requirement constraints.
`query_ontology_sparql`	`ontologyText: string`, `sparqlQuery: string`	`enableReasoning?: boolean` (`true`)	Parse an OWL 2 Functional Syntax or Turtle ontology, optionally run OWL 2 RL forward-chaining inference, and execute a SPARQL 1.1 `SELECT` / `ASK` query.
`query_ontology_bgp`	`ontologyText: string`, `patterns: Array<{subject?: string, predicate?: string, object?: string}>`	`enableReasoning?: boolean` (`true`)	Match Basic Graph Patterns (BGP) against the inferred RDF triple store and return variable bindings.
`trace_fault_propagation`	`ontologyText: string`, `faultSourceIri: string`	`propagationPropertyIri?: string`, `maxDepth?: number` (`10`)	Traverse transitive causal/containment relationships across the ontological knowledge graph to compute downstream failure blast radius and FMEA impact chains.
`explain_inference`	`ontologyText: string`, `subject: string`, `predicate: string`, `object: string`	—	Construct a formal proof derivation tree explaining which OWL 2 RL rules and asserted axioms entailed a specific inferred triple (s, p, o).

Domain 3: Triple Graph Grammar (TGG) Digital Thread Synchronization (3 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`tgg_query_thread`	`sysmlCode: string`, `modelicaCode: string`	`spiceCode?: string`	Build the multi-domain Triple Graph Grammar correspondence graph across SysML v2, Modelica, and SPICE, returning synchronized element triples and orphan nodes.
`tgg_diagnose_conflict`	`sysmlCode: string`, `modelicaCode: string`	`baseSysmlCode?: string`, `baseModelicaCode?: string`	Run Critical Pair Analysis (CPA) over concurrent modifications in SysML v2 and Modelica to detect conflicting edits on shared parameters or ports.
`tgg_reconcile_slot`	`sysmlCode: string`, `modelicaCode: string`, `slotId: string`, `authority: "sysml" | "modelica"`	—	Resolve a parameter or structural divergence on `slotId` by propagating the authoritative domain's value across the TGG rule engine and returning updated source code.

Domain 4: Requirements Traceability Matrix (RTM) Intelligence (3 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`rtm_query_matrix`	`files: Record<string, string>`	—	Extract all requirements, architectural elements, physics models, and verification cases across a polyglot file map and compute coverage percentages and orphan counts.
`rtm_suggest_links`	`files: Record<string, string>`	`threshold?: number` (`0.35`)	Analyze identifier lexical similarity, shared physical quantities/units, and structural proximity to recommend missing `@satisfies` and `@verifies` traceability links.
`rtm_diagnose_suspect`	`baseFiles: Record<string, string>`, `headFiles: Record<string, string>`	—	Compare two workspace snapshots via semantic AST fingerprinting to flag downstream `@satisfies` / `@verifies` links as `SUSPECT` when upstream requirement expressions change.

Domain 5: Autonomous Closed-Loop Synthesis, Calibration & Workspace Editing (5 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`modelscript_closed_loop_synthesize`	`className: string`, `modelicaCode: string`, `requirements: Array<{variable: string, condition: "lt" | "gt" | "eq" | "between", threshold: number, upperThreshold?: number}>`	`tunableParameters?: string[]`, `maxIterations?: number` (`10`), `tEnd?: number` (`5.0`)	Execute an autonomous compile → lint → simulate → STL robustness evaluation → sensitivity-guided parameter tuning loop until all formal requirements pass.
`modelscript_patch_code`	`filePath: string`, `search: string`, `replace: string`	`validateBeforeWrite?: boolean` (`true`)	Apply a surgical search-and-replace edit to a workspace file, automatically running the WASM GLR parser and linter before committing the write to disk.
`modelscript_read_file`	`filePath: string`	`startLine?: number`, `endLine?: number`	Read source lines from a workspace file within the sandboxed root directory.
`modelscript_list_workspace_files`	—	`directory?: string`, `extensions?: string[]`	Recursively list engineering source files (`.mo`, `.sysml`, `.msx`, `.pscx`, `.scad`, `.stp`, `.owl`, `.ssp`, `.csv`) in the active workspace.
`modelica_diff_calibrate`	`className: string`, `modelicaCode: string`, `parameters: string[]`, `referenceData: {time: number[], signals: Record<string, number[]>}`	`learningRate?: number` (`0.05`), `maxEpochs?: number` (`50`), `parameterBounds?: Record<string, [number, number]>`	Calibrate unknown physical parameters against experimental CSV/reference trajectories using differentiable simulation and gradient descent (Adam / L-BFGS) to minimize trajectory MSE.

Domain 6: PSCAD Power Systems & IEEE 2800 Grid Compliance (4 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`pscad_lint`	`xmlContent: string`	`fileName?: string`	Run all 11 electrical EMT lint rules (`M6001`–`M6011`) on a `.pscx` or `.pswx` XML payload.
`pscad_simulate`	`xmlContent: string`	`tStop?: number` (`1.0`), `stepUs?: number` (`20.0`)	Execute a fixed-step nodal admittance EMT simulation on a `.pscx` case and return three-phase bus voltage, branch current, and CSMF control signal waveforms.
`pscad_verify_ieee2800`	`xmlContent: string`	`tStop?: number` (`3.0`), `stepUs?: number` (`20.0`)	Run automated **IEEE Std 2800-2022** LVRT/HVRT voltage ride-through, frequency ride-through, and reactive current injection compliance verification.
`pscad_export`	`xmlContent: string`, `targetFormat: "modelica" | "ssp" | "svg"`	—	Convert a PSCAD `.pscx` schematic into an executable Modelica `.mo` model, an SSP 1.0 `.ssd` descriptor, or a rendered Single-Line Diagram SVG.

Domain 7: Social Engineering Network & Bot Collaboration (2 Tools)

Tool Name	Required Arguments	Optional Arguments	Description
`social_read_thread`	`postId: number`	—	Read a social discussion thread with its full conversation tree, parent posts, child replies, and attached engineering artifact views.
`social_post_reply`	`replyToId: number`, `content: string`	`artifactViewId?: number`	Post an engineering response or simulation synthesis back into a discussion thread on behalf of an authorized bot or automated agent.

Declarative Polyglot Language Tools (`PolyglotMcpHost`)

When using `PolyglotMcpHost` (`packages/mcp/src/polyglot-server.ts`), additional language-declared tools from `languages/*/src/language.ts` are automatically mounted:

- `sysml2_extract_topology` : Extracts hierarchical part definitions, ports, and item flow connections from a `.sysml` workspace symbol (`aotExport: "mcp_sysml2_extract_topology"`).
- `sysml2_verify_requirements` : Evaluates SysML v2 `requirement def` constraint satisfaction across the workspace (`aotExport: "mcp_sysml2_verify_requirements"`).

---

3. MCP Resources (`resources/list` & `resources/read`)

The MCP server exposes live workspace state as URI-addressable resources (`packages/mcp/src/resources.ts` and `languages/*/src/language.ts`):

Resource URI / Template	MIME Type	Description
`modelica://libraries`	`application/json`	Lists all loaded Modelica package roots and top-level libraries in the current session.
`modelica://classes`	`application/json`	Lists every indexed class FQN, specialized kind (`model` , `block` , `connector` , `record` , `function` , `package`), and description.
`modelica://classes/{name}`	`application/json`	Returns full class metadata for `{name}` , including source text, components, equations, and `connect()` clauses.
`modelica://workspace/{className}/ast`	`application/json`	(`PolyglotMcpHost`) Serializes the linear CST/AST structure of `{className}` .
`modelica://workspace/{className}/dae`	`application/json`	(`PolyglotMcpHost`) Returns the lowered DAE variables, parameters, and equations for `{className}` .
`sysml2://workspace/{partName}/topology`	`application/json`	(`PolyglotMcpHost`) Returns the resolved port and connection graph for a SysML v2 part.

Part II: VS Code Extension & Web IDE Workbench

The official ModelScript extension (`modelscript.modelscript` , source in `apps/ide/` and `packages/lsp/`) runs both in **VS Code Desktop** (Node.js extension host) and **VS Code for the Web / `ide.modelscript.org`** (100% browser WebWorker host).

1. Polyglot Language Server (`@modelscript/lsp`)

A single unified Language Server Protocol instance provides incremental language intelligence across the registered languages (`.mo` , `.sysml` , `.kerml` , `.msx` , `.pscx` , `.pswx` , `.scad` , `.stp` , `.step` , `.owl` , `.ssp` , `.ssd` , `.ssv` , `.csv` , `.su2` , `.cfg` , `.inp` , `.gbr`):

- **Real-Time Diagnostics**: Incremental GLR parsing + Salsa `lint__*` queries + cross-file unit/type checking.
- **Navigation & Refactoring**: Go-to-Definition, Find All References (across `.sysml` ↔ `.mo` ↔ `.msx` boundaries), Workspace Symbol Search, Rename Symbol, and Document Outline.
- **Code Lenses & Inlay Hints**: Inline equation/variable balance badges (`[Balanced: 14 vars / 14 eqs]`), inferred SI unit inlay hints, and one-click `▶ Simulate` , `⚡ Verify` , and `📐 Render Diagram` CodeLenses above every `model` , `part def` , and `verification def` .

2. Debug Adapter Protocol (DAP) Simulation Debugger

ModelScript registers a full **Debug Adapter Protocol** implementation (`"type": "modelscript"`) that lets engineers step through continuous-time DAE integrations, state-machine transitions, and zero-crossing events exactly like stepping through imperative code:

- **Time & Condition Breakpoints**: Set breakpoints on equation lines, `when` clauses, state transitions, or conditional simulation expressions (e.g., `time >= 2.5` or `der(omega) < 0`).
- **Execution Controls**: `Continue` , `Step Over` (advance one solver step Δt), `Step Into` (step into Newton-Raphson / event-iteration micro-steps), `Step Back` (reverse-time trajectory scrubbing), and `Restart` .
- **Live Inspection Scopes**: Inspect `States (x)` , `Derivatives (der(x))` , `Algebraic Variables (y)` , `Parameters (p)` , `Discrete / Clocked` , and `Solver Telemetry (h, localError, jacobianCond)` in the VS Code Variables and Watch panes.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "modelscript",
      "request": "launch",
      "name": "Debug Modelica Simulation",
      "model": "Modelica.Mechanics.Rotational.Examples.First",
      "program": "${file}",
      "stopTime": 5.0,
      "solver": "rodas4p",
      "breakOnZeroCrossing": true
    }
  ]
}
```

3. Virtual Filesystems & Interactive Notebooks (`.monb`)

- **Virtual FileSystem Providers**:
  - `modelica://` : Read-only browsable virtual filesystem mounting the bundled Modelica Standard Library (MSL) and installed registry packages (`modelica_modules/`).
  - `memfs://` : Ephemeral in-memory workspace used in the browser IDE (`ide.modelscript.org`).
  - `github://` : Zero-clone direct browsing and editing of remote GitHub repositories.
- **ModelScript Interactive Notebooks (`modelscript.monbNotebook`)**:
  - Native VS Code `NotebookSerializer` and `NotebookController` for `.monb` files.
  - Supports polyglot code cells (`modelica` , `sysml2` , `modelscript` , `scad` , `sparql`) that share a reactive kernel session, rendering interactive Bode plots, time-series charts, 3D CAD viewports, and SVG schematics directly below each cell.

4. Custom Document Editors & Engineering Workbench Panels

The extension contributes **9 custom visual editors** and **over 20 specialized engineering webview panels**:

Custom Editor / View ID	Bound File Types / Trigger	Capabilities
`modelscript.diagramEditor`	`.mo`, `.sysml`, `.pscx`	Bi-directional graphical schematic & SysML v2 (BDD/IBD/State/Activity) editor; dragging components or routing wires updates the underlying source CST in real time.
`modelscript.monbNotebook`	`.monb`	Literate engineering notebook combining Markdown/LaTeX narratives with live simulation and verification cells.
`modelscript.fmuViewer`	`.fmu`	Inspects FMI 2.0/3.0 `modelDescription.xml`, causal ports, dependency sparsity matrices, and runs interactive test simulations on `.fmu` files.
`modelscript.sspEditor`	`.ssp`, `.ssd`, `.ssv`	Visual block-diagram editor for wiring multi-FMU co-simulation topologies and editing `.ssv` parameter tables.
`modelscript.reqifEditor`	`.reqif`, `.reqifz`	Tabular requirements specification editor with suspect-link indicators and one-click SysML v2 synchronization.
`modelscript.cadViewer`	`.stp`, `.step`, `.scad`, `.stl`, `.glb`	Hardware-accelerated Three.js / WebGL 3D CAD viewport with section planes, B-Rep face selection, and kinematic animation playback.
`modelscript.gerberViewer`	`.gbr`, `.drl`	Multi-layer 2D/3D PCB copper, solder mask, silkscreen, and drill stackup inspector.
`modelscript.cfdViewer`	`.su2`, `.vtk`, `.cfg`	Unstructured CFD mesh wireframe, boundary marker inspector, and pressure/velocity contour viewer.
`modelscript.feaViewer`	`.inp`, `.frd`	Finite-element mesh inspector with deformed shape magnification and von Mises stress colormaps.
Simulation & Bode Plot Studio	`modelscript.simulate`	Interactive multi-axis oscilloscope, FFT spectrum analyzer, phase portrait plotter, and linearized State-Space (A, B, C, D) Bode/Nyquist root-locus analyzer.
BLT & Tearing Matrix Inspector	`modelscript.inspectBlt`	Visualizes the Tarjan Block Lower Triangular incidence matrix, Pantelides differentiated equations, and Cellier-Elmqvist tearing variables.
Formal Verification & Counterexample Center	`modelscript.verify`	Interactive dashboard for running BMC/k-induction/IC3/STL falsification and replaying falsifying counterexample traces step-by-step on the schematic.
Digital Thread & RTM Explorer	`modelscript.digitalThread`	Interactive graph explorer showing end-to-end traceability across OWL2 → SysML v2 → Modelica → CAD → Verification DHF.

5. Three-Tier AI Engineering Copilot

The IDE integrates an engineering-aware AI assistant (`modelscript.aiChatView`) that operates across three configurable inference tiers:

1. **Tier 1 — Zero-Cloud Browser WebGPU (`Qwen3-0.6B` / WebLLM)**: Runs 100% locally inside the browser or desktop GPU using quantized WebGPU weights for air-gapped syntax completion, equation explanation, and unit conversion with zero data egress.
2. **Tier 2 — Local Workstation Inference (`Ollama` / `vLLM` OpenAI-compatible)**: Connects to a local model server (`http://localhost:11434`) with automatic tool-calling bound to the in-process `@modelscript/mcp` tools.
3. **Tier 3 — Hosted Frontier Models + MCP Gateway**: Connects frontier reasoning models to the workspace's live `@modelscript/mcp` server (or `/api/v1/mcp/sse`), enabling autonomous closed-loop synthesis (`modelscript_closed_loop_synthesize`), differential calibration (`modelica_diff_calibrate`), and multi-domain TGG reconciliation (`tgg_reconcile_slot`).

Part III: Social AI Agents & Autonomous Bots

ModelScript features native autonomous bot support across the social engineering network (`apps/api` and `apps/web`), allowing automated agents to participate in engineering discussions, verify physical equations, run continuous DAE simulations, and attach interactive artifact views.

1. Bot Registration & Scopes

Users can register autonomous bots in **Settings** → **Autonomous Bots** or programmatically via `POST /api/v1/users/me/bots`:

- **Dedicated Bot Identity**: Bots receive a unique handle (e.g. `@dynamics_bot`), custom display name, bio, avatar, and system persona prompt.
- `ms_bot_*` **Bearer Tokens**: Authenticate bot calls against social endpoints, the git repository engine, and the MCP SSE gateway.
- **Granular Capability Scopes**:
  - `social:read`: Read public engineering threads, parent contexts, and mentions.
  - `social:write`: Post responses, explanations, and quotes into social threads.
  - `social:simulate`: Execute DAE solvers (SUNDIALS CVODE/IDA) and synthesize interactive trajectory plots.
  - `modelica:validate`: Compile and check Modelica CST syntax and static semantics.

2. Bring-Your-Own-Key (BYOK) Inference

Bots can run on instance-default models or configure private credentials:

- **BYOK Cloud API Keys**: Supports Gemini, OpenAI, and Anthropic API keys directly configured on the bot.
- **Custom Inference Endpoints**: Point to local `Ollama` (`http://localhost:11434/v1`), `vLLM`, or private enterprise LLM gateways.
- **Inference Models**: Select from specialized models including `gemini-1.5-flash`, `gemini-1.5-pro`, `gpt-4o-mini`, `gpt-4o`, `deepseek-r1`, or `qwen-2.5-coder`.

3. Mention Processing & Live WebSocket Indicators

When an engineering post or comment mentions an automated bot (`@dynamics_bot`):

1. **Background Queue**: The API enqueues the job into `bot_mention_queue` with anti-loop recursion safeguards (bot mentions cannot trigger nested bot replies).
2. **Real-Time WebSocket Indicators**: The backend streams live status updates over `/api/v1/collab/stream`:
   - `thinking`: The bot is parsing prompt equations and generating physical models.
   - `simulating`: The bot is running numerical integrators and preparing trajectory data.
   - `done`: The reply is posted and the UI automatically appends the completed card.
3. **Interactive Artifact Synthesis**:
   - `morsel`: Inline interactive Modelica code view with syntax highlighting and verification badges.
   - `vega-plot`: Interactive Vega-Lite timeseries charts with state variable selection and trajectory zoom.